Profiling

Signal18

April 2026

Contents

1.1.1 Goals . . . 22
1.1.2 History . . . 22
1.1.3 HA Workflow . . . 22
1.1.4 Traffic Routing . . . 22
1.3.0.1 GNU GENERAL PUBLIC LICENSE . . . 25
1.3.0.2 Preamble . . . 25
1.3.0.3 TERMS AND CONDITIONS . . . 25
2.1.1.1 Binary Flavors . . . 34
2.1.1.2 Distribution Types . . . 34
2.1.1.2.1 Package (RPM / DEB) . . . 34
2.1.1.2.2 Tarball (basedir) . . . 35
2.1.1.2.3 Embedded Binary . . . 35
2.1.1.3 Choosing a Flavor . . . 35
2.1.1.4 CLI Client . . . 35
2.1.2.1 Install from Embedded Binary . . . 36
2.1.2.2 One-Command Install . . . 36
2.1.2.2.1 Environment Variables . . . 36
2.1.2.2.2 Supported Platforms . . . 36
2.1.2.3 Manual Download from GitHub Releases . . . 36
2.1.2.3.1 Embedded binary (recommended for portable use) . . . 36
2.1.2.3.2 Tarball (basedir variant) . . . 37
2.1.2.3.3 Release Artifact Naming . . . 37
2.1.2.4 Development Builds . . . 37
2.1.4.1 Install from Repository . . . 38
2.1.4.2 CentOS / RHEL . . . 38
2.1.4.2.1 Configure the repository . . . 38
2.1.4.2.2 Install . . . 38
2.1.4.2.3 Upgrading from 2.x (epoch conflict) . . . 38
2.1.4.3 Debian / Ubuntu . . . 38
2.1.4.3.1 Configure the repository and install . . . 38
2.1.4.4 Post-Installation . . . 39
2.1.5.1.1 Setup OpenSVC Cluster Agents . . . 40
2.1.5.1.1.1 Using Overlay Networking . . . 40
2.1.5.1.1.2 Upgrade agent version . . . 41
2.1.6.1 Install from Docker . . . 42
2.1.6.2 Image Tags . . . 42
2.1.6.3 What is Included . . . 42
2.1.6.4 Quick Start . . . 43
2.1.6.4.1 Step 1 — Create directory structure . . . 43
2.1.6.4.2 Step 2 — Download configuration templates . . . 43

2.1.6.4.3 Step 3 — Configure your first cluster . . . 43
2.1.6.4.4 Step 4 — Start the container . . . 43
2.1.6.4.5 Volume Mounts . . . 43
2.1.6.4.6 Exposed Ports . . . 44
2.1.6.5 Rootless Images . . . 44
2.1.6.6 Development Container . . . 44
2.1.6.7 Logs . . . 45
2.1.9.1 Install from Source . . . 46
2.1.9.2 Clone and Build . . . 46
2.1.9.3 Build Specific Flavors . . . 46
2.1.9.4 Build Packages . . . 46
2.1.9.5 Build Tags . . . 46
2.1.9.6 Running from Source . . . 47
2.1.9.7 Continuous Integration . . . 47
2.1.10.1 Orchestrators (Pro) . . . 48
2.1.10.2 OpenSVC . . . 48
2.1.10.2.1 Install the OpenSVC Agent . . . 48
2.1.10.2.2 Configure replication-manager for OpenSVC . . . 49
2.1.10.2.3 OpenSVC Video Walkthrough . . . 49
2.1.10.3 Kubernetes . . . 49
2.1.10.3.1 Prerequisites . . . 49
2.1.10.3.2 Configure replication-manager for Kubernetes . . . 49
2.1.10.4 SlapOS . . . 49
2.1.10.4.1 Configure replication-manager for SlapOS . . . 49
2.1.10.5 On-Premise SSH . . . 50
2.1.10.5.1 Prerequisites . . . 50
2.1.10.5.2 Configure SSH access . . . 50
2.1.10.6 Local (Testing Only) . . . 50
2.1.10.6.1 Prerequisites . . . 50
2.1.10.7 Next Steps . . . 50
2.1.11.1 Install Dependencies . . . 51
2.1.11.2 Supported Database Versions . . . 51
2.1.11.3 External Tools . . . 51
2.1.11.3.1 Auto-Detected Tool Matrix . . . 51
2.1.11.3.2 Version Monitoring . . . 52
2.1.11.4 Per-Feature Dependencies . . . 52
2.1.11.4.1 Logical Backups . . . 52
2.1.11.4.2 Fast Parallel Backups (MyDumper) . . . 52
2.1.11.4.3 Snapshot Backups (Restic) . . . 52
2.1.11.4.4 HAProxy . . . 53
2.1.11.4.5 ProxySQL . . . 53
2.1.11.4.6 Sysbench (Benchmarking) . . . 53
2.1.11.4.7 Restic FUSE Mount (Backup Browsing) . . . 53
2.1.11.4.8 Remote Scripting (On-Premise Orchestrator) . . . 53
2.1.11.5 Docker Image Reference . . . 53
2.1.11.6 Overriding Detected Paths . . . 53
2.1.12.1 What Was Installed . . . 55
2.1.12.2 Package Installation (RPM / DEB) . . . 55
2.1.12.3 Tarball Installation (basedir) . . . 55
2.1.12.4 Embedded Binary . . . 55
2.1.12.5 Docker . . . 56
2.1.12.6 Configuration File Search Order . . . 56
2.3.1 First Config . . . 57
2.3.2 Config File Structure . . . 57

2.3.3 Config File Search Order . . . 57
2.3.4 Using the Include Directory . . . 57
2.3.5 Dynamic Configuration . . . 58
2.3.6 Minimal Working Config . . . 58
2.3.7 Sample Configuration Files . . . 58
2.3.8 Starting replication-manager . . . 59
2.3.9 Command Line Flags . . . 59
2.3.10 Discovering All Available Flags . . . 59
2.5.1 First Login . . . 61
2.5.2 Local Users . . . 61
2.5.3 Changing Passwords . . . 61
2.5.4 Default Password Warning — WARN0108 . . . 61
2.5.5 SSO Login via gitlab.signal18.io . . . 62
2.5.6 Summary . . . 62
2.7.1 Instance Registration & SSO . . . 63
2.7.2 Registering Your Instance . . . 63
2.7.3 Community Plugin Access . . . 63
2.7.4 Configuration Backup and Restore . . . 63
2.7.5 Cluster Role Sharing with External Users . . . 64
2.7.5.1 Roles . . . 64
2.7.5.2 External Partner Lifecycle . . . 64
2.7.6 Direct Chat with Signal18 and Partners . . . 65
2.7.7 GitLab Object Mapping . . . 65
2.7.8 Secret Storage . . . 65
2.7.9 Team Management . . . 65
2.7.10 Configuration Reference . . . 66
2.7.1.1 Plugins Overview . . . 67
2.7.1.2 Enabling Plugins . . . 67
2.7.1.3 What Community Plugins Do . . . 67
2.7.1.3.1 Workload Plugins . . . 67
2.7.1.3.2 Security Plugins . . . 68
2.7.1.3.3 Score Plugins . . . 68
2.7.1.4 Full Plugin Reference . . . 68
2.8.1 Pre-Migration Checklist . . . 69
2.8.2 Major Changes in 3.x . . . 69
2.8.2.1 Configuration File Location Changes . . . 69
2.8.2.2 Docker Volume Mapping Changes . . . 69
2.8.3 Deprecated Parameters . . . 69
2.8.3.1 Parameter Renames . . . 70
2.8.3.2 Migration Script for Deprecated Parameters . . . 71
2.8.4 Default Value Changes . . . 72
2.8.5 New Features Requiring Configuration . . . 72
2.8.5.1 Active-Passive Topology (3.1) . . . 72
2.8.5.2 Enhanced Backup Management (3.1) . . . 72
2.8.5.3 Transaction Monitoring (3.1) . . . 73
2.8.5.4 Per-Module Log Levels (3.1) . . . 73
2.8.6 Breaking Changes . . . 73
2.8.6.1 API Changes . . . 73
2.8.6.2 SSL/TLS Mode Changes . . . 73
2.8.6.3 Backup Tool Changes . . . 73
2.8.7 Migration Procedure . . . 74
2.8.7.1 Step 1: Prepare Environment . . . 74
2.8.7.2 Step 2: Update Configuration Files . . . 74
2.8.7.3 Step 3: Install replication-manager 3.x . . . 74

2.8.7.4 Step 4: Verify Configuration . . . 74
2.8.7.5 Step 5: Start and Validate . . . 75
2.8.7.6 Step 6: Monitor for Issues . . . 75
2.8.8 Post-Migration Tasks . . . 75
2.8.8.1 Update Backup Scripts . . . 75
2.8.8.2 Update Monitoring . . . 75
2.8.8.3 Documentation Updates . . . 76
2.8.9 Rollback Procedure . . . 76
2.8.10 Troubleshooting . . . 76
2.8.10.1 Configuration Not Loading . . . 76
2.8.10.2 Deprecated Parameter Warnings . . . 76
2.8.10.3 Performance Schema Overhead . . . 76
2.8.10.4 Docker Container Not Persisting Config . . . 77
2.8.10.5 Backup Tools Not Found . . . 77
2.8.11 Getting Help . . . 77
2.8.12 Version-Specific Notes . . . 77
2.8.12.1 Migrating from 2.0 to 3.x . . . 77
2.8.12.2 Migrating from 2.1/2.2 to 3.x . . . 77
2.8.12.3 Migrating from 2.3 to 3.x . . . 77
2.8.12.4 Upgrading from 3.0 to 3.1 . . . 78
3.1.1 Start & Stop Monitoring . . . 79
3.1.2 Start . . . 79
3.1.3 Stop . . . 79
3.1.4 Check Status . . . 79
3.1.5 Logs . . . 80
3.2.1 Command Line Client . . . 81
3.2.2 Interactive Console . . . 81
3.2.2.1 Keyboard Shortcuts . . . 81
3.2.3 Non-Interactive Commands . . . 81
3.2.3.1 Switchover . . . 81
3.2.3.2 Failover . . . 81
3.2.3.3 Bootstrap Replication . . . 81
3.2.3.4 Cluster Status . . . 82
3.2.3.5 Topology . . . 82
3.2.3.6 Server Maintenance . . . 82
3.3.1 Web Interface . . . 83
3.3.1.1 Sample nginx configuration providing secure access to the dashboard . . . 83
3.4.1.1 API Client Usage . . . 84

Calling API via the client **84**

Calling API via curl or wget **84**
3.4.1.2 API documentation . . . 84
3.4.2.1 All endpoints . . . 85
3.4.2.1.1 auth . . . 85
3.4.2.1.2 cloud18 . . . 85
3.4.2.1.3 cluster . . . 85
3.4.2.1.4 cluster_actions . . . 86
3.4.2.1.5 cluster_backup . . . 86
3.4.2.1.6 cluster_backups . . . 86
3.4.2.1.7 cluster_certificates . . . 86
3.4.2.1.8 cluster_graphite . . . 87
3.4.2.1.9 cluster_maintenance . . . 87
3.4.2.1.10 cluster_monitor . . . 87

3.4.2.1.11 cluster_provision . . . 88
3.4.2.1.12 cluster_replication . . . 88
3.4.2.1.13 cluster_schema . . . 88
3.4.2.1.14 cluster_settings . . . 89
3.4.2.1.15 cluster_tags . . . 90
3.4.2.1.16 cluster_test . . . 90
3.4.2.1.17 cluster_topology . . . 90
3.4.2.1.18 cluster_traffics . . . 91
3.4.2.1.19 cluster_vault . . . 91
3.4.2.1.20 database . . . 91
3.4.2.1.21 database_actions . . . 93
3.4.2.1.22 database_backup . . . 94
3.4.2.1.23 database_logs . . . 95
3.4.2.1.24 database_maintenance . . . 95
3.4.2.1.25 database_provision . . . 96
3.4.2.1.26 database_queries . . . 96
3.4.2.1.27 database_replication . . . 97
3.4.2.1.28 database_schema . . . 98
3.4.2.1.29 database_tasks . . . 98
3.4.2.1.30 database_topology . . . 98
3.4.2.1.31 global_setting . . . 99
3.4.2.1.32 proxies . . . 99
3.4.2.1.33 public . . . 99
3.4.2.1.34 user . . . 100
3.4.2.2 Paths . . . 100
3.4.2.2.1 Delete a cluster (*DeleteAPIClustersActionsDeleteClusterName*) . . . 100
3.4.2.2.2 Fetch clusters (*GetAPIClusters*) . . . 101
3.4.2.2.3 Retrieve details of a cluster (*GetAPIClustersClusterName*) . . . 101
3.4.2.2.4 Retrieve backups for a specific cluster (*GetAPIClustersClusterNameBackups*) . . . 102
3.4.2.2.5 Retrieve client certificates for a specific cluster (*GetAPIClustersClusterNameCertificates*) . . . 103
3.4.2.2.6 Retrieve variable differences for a specific cluster (*GetAPIClustersClusterNameDiffvariables*) . . . 104
3.4.2.2.7 Retrieve Graphite filter list for a specific cluster (*GetAPIClustersClusterNameGraphiteFilterlist*) . . . 104
3.4.2.2.8 Retrieve job entries for a specific cluster (*GetAPIClustersClusterNameJobs*) . . . 105
3.4.2.2.9 Check if a cluster needs a rolling reprovision (*GetAPIClustersClusterNameNeedRollingReprov*) . . . 106
3.4.2.2.10 Check if a cluster needs a rolling restart (*GetAPIClustersClusterNameNeedRollingRestart*) . . . 106
3.4.2.2.11 Check if Proxy Needs Reprovision (*GetAPIClustersClusterNameProxiesProxyNameActionsNeedReprov*) . . . 107
3.4.2.2.12 Check if Proxy Needs Restart (*GetAPIClustersClusterNameProxiesProxyNameActionsNeedRestart*) . . . 108
3.4.2.2.13 Retrieve query rules for a specific cluster (*GetAPIClustersClusterNameQueryrules*) 109
3.4.2.2.14 Retrieve schema information for a specific cluster (*GetAPIClustersClusterNameSchema*) . . . 109
3.4.2.2.15 Perform a backup of the error log on a server (*GetAPIClustersClusterNameServersServerNameActionsBackupErrorLog*) . . . 110
3.4.2.2.16 Perform a logical backup on a server (*GetAPIClustersClusterNameServersServerNameActionsBackupLogical*) . . . 111
3.4.2.2.17 Perform a physical backup on a server (*GetAPIClustersClusterNameServersServerNameActionsBackupPhysical*) . . . 112

3.4.2.2.18 Perform a backup of the slow query log on a server (*GetAPIClustersClusterNameServersServerNameActionsBackupSlowqueryLog*) . . . 112
3.4.2.2.19 Delete maintenance mode on a server (*GetAPIClustersClusterNameServersServerNameActionsDelMaintenance*) . . . 113
3.4.2.2.20 Flush logs on a server (*GetAPIClustersClusterNameServersServerNameActionsFlushLogs*) . . . 114
3.4.2.2.21 Cancel a task on a server (*GetAPIClustersClusterNameServersServerNameActionsJobCancelTask*) . . . 114
3.4.2.2.22 Toggle maintenance mode on a server (*GetAPIClustersClusterNameServersServerNameActionsMaintenance*) . . . 115
3.4.2.2.23 Optimize a server (*GetAPIClustersClusterNameServersServerNameActionsOptimize*) 116
3.4.2.2.24 Provision a server (*GetAPIClustersClusterNameServersServerNameActionsProvision*) 117
3.4.2.2.25 Reseed a server (*GetAPIClustersClusterNameServersServerNameActionsReseedBackupMethod*) . . . 117
3.4.2.2.26 Cancel a reseed task on a server (*GetAPIClustersClusterNameServersServerNameActionsReseedCancelTask*) . . . 118
3.4.2.2.27 Reset the master on a server (*GetAPIClustersClusterNameServersServerNameActionsResetMaster*) . . . 119
3.4.2.2.28 Reset PFS queries on a server (*GetAPIClustersClusterNameServersServerNameActionsResetPfsQueries*) . . . 120
3.4.2.2.29 Reset all slaves on a server (*GetAPIClustersClusterNameServersServerNameActionsResetSlaveAll*) . . . 120
3.4.2.2.30 Run jobs on a server (*GetAPIClustersClusterNameServersServerNameActionsRunJobs*) . . . 121
3.4.2.2.31 Set a server as ignored (*GetAPIClustersClusterNameServersServerNameActionsSetIgnored*) . . . 122
3.4.2.2.32 Set long query time on a server (*GetAPIClustersClusterNameServersServerNameActionsSetLongQueryTimeQueryTime*) . . . 123
3.4.2.2.33 Set a server to maintenance mode (*GetAPIClustersClusterNameServersServerNameActionsSetMaintenance*) . . . 123
3.4.2.2.34 Set a server as preferred (*GetAPIClustersClusterNameServersServerNameActionsSetPrefered*) . . . 124
3.4.2.2.35 Set a server as unrated (*GetAPIClustersClusterNameServersServerNameActionsSetUnrated*) . . . 125
3.4.2.2.36 Skip a replication event on a server (*GetAPIClustersClusterNameServersServerNameActionsSkipReplicationEvent*) . . . 126
3.4.2.2.37 Start a server (*GetAPIClustersClusterNameServersServerNameActionsStart*) . . . 126
3.4.2.2.38 Start the slave on a server (*GetAPIClustersClusterNameServersServerNameActionsStartSlave*) . . . 127
3.4.2.2.39 Stop a server (*GetAPIClustersClusterNameServersServerNameActionsStop*) . . . 128
3.4.2.2.40 Stop the slave on a server (*GetAPIClustersClusterNameServersServerNameActionsStopSlave*) . . . 128
3.4.2.2.41 Perform a switchover on a server (*GetAPIClustersClusterNameServersServerNameActionsSwitchover*) . . . 129
3.4.2.2.42 Toggle InnoDB monitor on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleInnodbMonitor*) . . . 130
3.4.2.2.43 Toggle metadata locks on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleMetaDataLocks*) . . . 131
3.4.2.2.44 Toggle PFS slow query capture on a server (*GetAPIClustersClusterNameServersServerNameActionsTooglePfsSlowQuery*) . . . 131
3.4.2.2.45 Toggle query response time on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleQueryResponseTime*) . . . 132
3.4.2.2.46 Toggle read-only mode on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleReadOnly*) . . . 133

3.4.2.2.47 Toggle SQL error log on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleSQLErrorLog*) . . . 134
3.4.2.2.48 Toggle slow query on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleSlowQuery*) . . . 134
3.4.2.2.49 Toggle slow query capture on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleSlowQueryCapture*) . . . 135
3.4.2.2.50 Toggle slow query table mode on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleSlowQueryTable*) . . . 136
3.4.2.2.51 Unprovision a server (*GetAPIClustersClusterNameServersServerNameActionsUnprovision*) . . . 136
3.4.2.2.52 Wait for InnoDB purge on a server (*GetAPIClustersClusterNameServersServerNameActionsWaitInnodbPurge*) . . . 137
3.4.2.2.53 Get status of all slaves of a server (*GetAPIClustersClusterNameServersServerNameAllSlavesStatus*) . . . 138
3.4.2.2.54 Get PFS statements of a server (*GetAPIClustersClusterNameServersServerNameDigestStatementsPfs*) . . . 139
3.4.2.2.55 Get PFS statements from the slow log of a server (*GetAPIClustersClusterNameServersServerNameDigestStatementsSlow*) . . . 140
3.4.2.2.56 Get error log of a server (*GetAPIClustersClusterNameServersServerNameErrorlog*) 140
3.4.2.2.57 Check if a server is a master (*GetAPIClustersClusterNameServersServerNameIsMaster*) . . . 141
3.4.2.2.58 Check if a server is a slave (*GetAPIClustersClusterNameServersServerNameIsSlave*) 142
3.4.2.2.59 Get master status of a server (*GetAPIClustersClusterNameServersServerNameMasterStatus*) . . . 143
3.4.2.2.60 Get metadata locks of a server (*GetAPIClustersClusterNameServersServerNameMetaDataLocks*) . . . 143
3.4.2.2.61 Get process list of a server (*GetAPIClustersClusterNameServersServerNameProcesslist*) . . . 144
3.4.2.2.62 Analyze a query using PFS on a server (*GetAPIClustersClusterNameServersServerNameQueriesQueryDigestActionsAnalyzePfs*) . . . 145
3.4.2.2.63 Analyze a query using the slow log on a server (*GetAPIClustersClusterNameServersServerNameQueriesQueryDigestActionsAnalyzeSlowlog*) . . . 146
3.4.2.2.64 Explain a query using PFS on a server (*GetAPIClustersClusterNameServersServerNameQueriesQueryDigestActionsExplainPfs*) . . . 147
3.4.2.2.65 Explain a query using the slow log on a server (*GetAPIClustersClusterNameServersServerNameQueriesQueryDigestActionsExplainSlowlog*) . . . 147
3.4.2.2.66 Kill a query on a server (*GetAPIClustersClusterNameServersServerNameQueriesQueryDigestActionsKillQuery*) . . . 148
3.4.2.2.67 Kill a thread on a server (*GetAPIClustersClusterNameServersServerNameQueriesQueryDigestActionsKillThread*) . . . 149
3.4.2.2.68 Get query response time of a server (*GetAPIClustersClusterNameServersServerNameQueryResponseTime*) . . . 150
3.4.2.2.69 Get schemas of a server (*GetAPIClustersClusterNameServersServerNameSchemas*) 150
3.4.2.2.70 Perform a physical backup on a server port (*GetAPIClustersClusterNameServersServerNameServerPortBackup*) . . . 151
3.4.2.2.71 Get server port configuration (*GetAPIClustersClusterNameServersServerNameServerPortConfig*) . . . 152
3.4.2.2.72 Check if a server port is a master (*GetAPIClustersClusterNameServersServerNameServerPortIsMaster*) . . . 153
3.4.2.2.73 Check if a server port is a slave (*GetAPIClustersClusterNameServersServerNameServerPortIsSlave*) . . . 154
3.4.2.2.74 Check if a server needs a config change (*GetAPIClustersClusterNameServersServerNameServerPortNeedConfigChange*) . . . 154

3.4.2.2.75 Check if a server needs provisioning (*GetAPIClustersClusterNameServersServerNameServerPortNeedProv*) . . . 155
3.4.2.2.76 Check if a server needs re-provisioning (*GetAPIClustersClusterNameServersServerNameServerPortNeedReprov*) . . . 156
3.4.2.2.77 Check if a server needs a restart (*GetAPIClustersClusterNameServersServerNameServerPortNeedRestart*) . . . 157
3.4.2.2.78 Check if a server needs to start (*GetAPIClustersClusterNameServersServerNameServerPortNeedStart*) . . . 157
3.4.2.2.79 Check if a server needs to stop (*GetAPIClustersClusterNameServersServerNameServerPortNeedStop*) . . . 158
3.4.2.2.80 Check if a server needs unprovisioning (*GetAPIClustersClusterNameServersServerNameServerPortNeedUnprov*) . . . 159
3.4.2.2.81 Get database service configuration of a server (*GetAPIClustersClusterNameServersServerNameServiceOpensvc*) . . . 159
3.4.2.2.82 Get slow log of a server (*GetAPIClustersClusterNameServersServerNameSlowQueries*) 160
3.4.2.2.83 Get status of a server (*GetAPIClustersClusterNameServersServerNameStatus*) . . . 161
3.4.2.2.84 Get status delta of a server (*GetAPIClustersClusterNameServersServerNameStatusDelta*) . . . 162
3.4.2.2.85 Get InnoDB status of a server (*GetAPIClustersClusterNameServersServerNameStatusInnodb*) . . . 163
3.4.2.2.86 Get tables of a server (*GetAPIClustersClusterNameServersServerNameTables*) . . . 163
3.4.2.2.87 Get variables of a server (*GetAPIClustersClusterNameServersServerNameVariables*) 164
3.4.2.2.88 Get virtual tables of a server (*GetAPIClustersClusterNameServersServerNameVtables*) 165
3.4.2.2.89 Retrieve settings for a specific cluster (*GetAPIClustersClusterNameSettings*) . . . 166
3.4.2.2.90 Retrieve shard clusters for a specific cluster (*GetAPIClustersClusterNameShardclusters*) . . . 167
3.4.2.2.91 Get Sphinx Indexes (*GetAPIClustersClusterNameSphinxIndexes*) . . . 167
3.4.2.2.92 Retrieve status of a cluster (*GetAPIClustersClusterNameStatus*) . . . 168
3.4.2.2.93 Retrieve tags for a specific cluster (*GetAPIClustersClusterNameTags*) . . . 169
3.4.2.2.94 Retrieve top metrics for a specific cluster (*GetAPIClustersClusterNameTop*) . . . 169
3.4.2.2.95 Shows the alerts for that specific named cluster (*GetAPIClustersClusterNameTopologyAlerts*) . . . 171
3.4.2.2.96 Retrieve crashes for a specific cluster (*GetAPIClustersClusterNameTopologyCrashes*) 172
3.4.2.2.97 Retrieve logs for a specific cluster (*GetAPIClustersClusterNameTopologyLogs*) . . . 172
3.4.2.2.98 Retrieve master of a cluster (*GetAPIClustersClusterNameTopologyMaster*) . . . 173
3.4.2.2.99 Shows the proxies for that specific named cluster (*GetAPIClustersClusterNameTopologyProxies*) . . . 174
3.4.2.2.100 Retrieve servers for a specific cluster (*GetAPIClustersClusterNameTopologyServers*) 174
3.4.2.2.101 Shows the slaves for that specific named cluster (*GetAPIClustersClusterNameTopologySlaves*) . . . 175
3.4.2.2.102 Retrieve peer clusters for sale (*GetAPIClustersForSale*) . . . 175
3.4.2.2.103 Retrieve peer clusters for a user (*GetAPIClustersPeers*) . . . 176
3.4.2.2.104 List Grafana files (*GetAPIConfigsGrafana*) . . . 177
3.4.2.2.105 Monitor Heartbeat (*GetAPIHeartbeat*) . . . 177
3.4.2.2.106 Handles replication manager requests (*GetAPIMonitor*) . . . 178
3.4.2.2.107 Fetch Prometheus metrics (*GetAPIPrometheus*) . . . 179
3.4.2.2.108 Retrieve current repository component (*GetAPIRepocompCurrent*) . . . 179
3.4.2.2.109 Get Replication Manager Status (*GetAPIStatus*) . . . 180
3.4.2.2.110 Retrieves terms (*GetAPITerms*) . . . 180
3.4.2.2.111 Check if the replication manager is running (*GetAPITimeout*) . . . 180
3.4.2.2.112 Add a new cluster (*PostAPIClustersActionsAddClusterName*) . . . 182
3.4.2.2.113 Add a sharding cluster to an existing cluster (*PostAPIClustersClusterNameActionsAddClusterShardingName*) . . . 183

3.4.2.2.114 Add a server to a specific cluster (*PostAPIClustersClusterNameActionsAddserverHostPort*) . . . 183
3.4.2.2.115 Add a server to a specific cluster (*PostAPIClustersClusterNameActionsAddserverHostPortType*) . . . 184
3.4.2.2.116 Cancel rolling reprovision for a specific cluster (*PostAPIClustersClusterNameActionsCancelRollingReprov*) . . . 185
3.4.2.2.117 Cancel rolling restart for a specific cluster (*PostAPIClustersClusterNameActionsCancelRollingRestart*) . . . 186
3.4.2.2.118 Rotate keys for a specific cluster (*PostAPIClustersClusterNameActionsCertificatesRotate*) . . . 187
3.4.2.2.119 Calculate checksum for all tables in a specific cluster (*PostAPIClustersClusterNameActionsChecksumAllTables*) . . . 187
3.4.2.2.120 Handles the failover process for a given cluster. (*PostAPIClustersClusterNameActionsFailover*) . . . 188
3.4.2.2.121 Perform a physical backup for the master of a specific cluster (*PostAPIClustersClusterNameActionsMasterPhysicalBackup*) . . . 189
3.4.2.2.122 Optimize a specific cluster (*PostAPIClustersClusterNameActionsOptimize*) . . . 190
3.4.2.2.123 Bootstrap replication for a specific cluster (*PostAPIClustersClusterNameActionsReplicationBootstrapTopology*) . . . 190
3.4.2.2.124 Cleanup replication bootstrap for a specific cluster (*PostAPIClustersClusterNameActionsReplicationCleanup*) . . . 191
3.4.2.2.125 Reset failover control for a specific cluster (*PostAPIClustersClusterNameActionsResetFailoverControl*) . . . 192
3.4.2.2.126 Reset SLA for a specific cluster (*PostAPIClustersClusterNameActionsResetSLA*) . 193
3.4.2.2.127 Handles the rolling restart process for a given cluster. (*PostAPIClustersClusterNameActionsRolling*) . . . 193
3.4.2.2.128 Rotate passwords for a specific cluster (*PostAPIClustersClusterNameActionsRotatePasswords*) . . . 194
3.4.2.2.129 Start traffic for a specific cluster (*PostAPIClustersClusterNameActionsStartTraffic*) 195
3.4.2.2.130 Stop traffic for a specific cluster (*PostAPIClustersClusterNameActionsStopTraffic*) 196
3.4.2.2.131 Handles the switchover process for a given cluster. (*PostAPIClustersClusterNameActionsSwitchover*) . . . 196
3.4.2.2.132 Run sysbench for a specific cluster (*PostAPIClustersClusterNameActionsSysbench*) 197
3.4.2.2.133 Wait for databases to be ready for a specific cluster (*PostAPIClustersClusterNameActionsWaitdatabases*) . . . 198
3.4.2.2.134 Provision Proxy Service (*PostAPIClustersClusterNameProxiesProxyNameActionsProvision*) . . . 199
3.4.2.2.135 Start Proxy Service (*PostAPIClustersClusterNameProxiesProxyNameActionsStart*) 200
3.4.2.2.136 Stop Proxy Service (*PostAPIClustersClusterNameProxiesProxyNameActionsStop*) 200
3.4.2.2.137 Unprovision Proxy Service (*PostAPIClustersClusterNameProxiesProxyNameActionsUnprovision*) . . . 201
3.4.2.2.138 Accept a subscription for a specific cluster (*PostAPIClustersClusterNameSalesAcceptSubscription*) . . . 202
3.4.2.2.139 Remove a sponsor from a specific cluster (*PostAPIClustersClusterNameSalesEndSubscription*) . . . 203
3.4.2.2.140 Reject a subscription for a specific cluster (*PostAPIClustersClusterNameSalesRefuseSubscription*) . . . 204
3.4.2.2.141 Calculate checksum for a specific table in a specific cluster (*PostAPIClustersClusterNameSchemaSchemaNameTableNameActionsChecksumTable*) . . . 204
3.4.2.2.142 Move a table to a different shard cluster (*PostAPIClustersClusterNameSchemaSchemaNameTableNameActionsMoveTableClusterShard*) . . . 205
3.4.2.2.143 Reshard a table for a specific cluster (*PostAPIClustersClusterNameSchemaSchemaNameTableNameActionsReshardTable*) . . . 206

3.4.2.2.144 Reshard a table for a specific cluster (*PostAPIClustersClusterNameSchemaSchemaNameTableNameActionsReshardTableClusterList*) . . . 207
3.4.2.2.145 Set a universal table for a specific cluster (*PostAPIClustersClusterNameSchemaSchemaNameTableNameActionsUniversalTable*) . . . 208
3.4.2.2.146 Send Vault token to a specific cluster (*PostAPIClustersClusterNameSendVaultToken*) 209
3.4.2.2.147 Perform a point-in-time recovery on a server (*PostAPIClustersClusterNameServersServerNameActionsPitr*) . . . 209
3.4.2.2.148 Write logs for a server (*PostAPIClustersClusterNameServersServerNameServerPortWriteLogTask*) . . . 210
3.4.2.2.149 Provision services for a specific cluster (*PostAPIClustersClusterNameServicesActionsProvision*) . . . 211
3.4.2.2.150 Unprovision services for a specific cluster (*PostAPIClustersClusterNameServicesActionsUnprovision*) . . . 212
3.4.2.2.151 Add a tag to a specific cluster (*PostAPIClustersClusterNameSettingsActionsAddDbTagTagValue*) . . . 213
3.4.2.2.152 Add a proxy tag to a specific cluster (*PostAPIClustersClusterNameSettingsActionsAddProxyTagTagValue*) . . . 213
3.4.2.2.153 Apply dynamic configuration for a specific cluster (*PostAPIClustersClusterNameSettingsActionsApplyDynamicConfig*) . . . 214
3.4.2.2.154 Reload client certificates for a specific cluster (*PostAPIClustersClusterNameSettingsActionsCertificatesReload*) . . . 215
3.4.2.2.155 Discover settings for a specific cluster (*PostAPIClustersClusterNameSettingsActionsDiscover*) . . . 216
3.4.2.2.156 Remove a tag from a specific cluster (*PostAPIClustersClusterNameSettingsActionsDropDbTagTagValue*) . . . 217
3.4.2.2.157 Remove a proxy tag from a specific cluster (*PostAPIClustersClusterNameSettingsActionsDropProxyTagTagValue*) . . . 217
3.4.2.2.158 Reload cluster settings (*PostAPIClustersClusterNameSettingsActionsReload*) . . . 218
3.4.2.2.159 Reload Graphite filter list for a specific cluster (*PostAPIClustersClusterNameSettingsActionsReloadGraphiteFilterlist*) . . . 219
3.4.2.2.160 Reset Graphite filter list for a specific cluster (*PostAPIClustersClusterNameSettingsActionsResetGraphiteFilterlistTemplate*) . . . 220
3.4.2.2.161 Set cron jobs for a specific cluster (*PostAPIClustersClusterNameSettingsActionsSetCronSettingNameSettingValue*) . . . 220
3.4.2.2.162 Set Graphite filter list for a specific cluster (*PostAPIClustersClusterNameSettingsActionsSetGraphiteFilterlistFilterType*) . . . 221
3.4.2.2.163 Set settings for a specific cluster (*PostAPIClustersClusterNameSettingsActionsSetSettingNameSettingValue*) . . . 222
3.4.2.2.164 Switch settings for a specific cluster (*PostAPIClustersClusterNameSettingsActionsSwitchSettingName*) . . . 223
3.4.2.2.165 Subscribe a user to a cluster (*PostAPIClustersClusterNameSubscribe*) . . . 224
3.4.2.2.166 Run all tests for a given cluster (*PostAPIClustersClusterNameTestsActionsRunAll*) 225
3.4.2.2.167 Run a specific test for a given cluster (*PostAPIClustersClusterNameTestsActionsRunTestName*) . . . 226
3.4.2.2.168 Add a new user to a cluster (*PostAPIClustersClusterNameUsersAdd*) . . . 226
3.4.2.2.169 Drop a cluster user (*PostAPIClustersClusterNameUsersDrop*) . . . 227
3.4.2.2.170 Send credentials to a specific user (*PostAPIClustersClusterNameUsersSendCredentials*) . . . 228
3.4.2.2.171 Update a cluster user (*PostAPIClustersClusterNameUsersUpdate*) . . . 229
3.4.2.2.172 Reload cluster plans (*PostAPIClustersSettingsActionsReloadClustersPlans*) . . . 230
3.4.2.2.173 Set global settings for the server (*PostAPIClustersSettingsActionsSetSettingNameSettingValue*) . . . 230
3.4.2.2.174 Switch global settings for the server (*PostAPIClustersSettingsActionsSwitchSettingName*) . . . 231

3.4.2.2.175 User login (*PostAPILogin*) . . . 232
3.4.2.2.176 Drop a server monitor from a cluster (*PostClusterClusterNameActionsDropserverHostPort*) . . . 233
3.4.2.2.177 Drop a server monitor from a cluster (*PostClusterClusterNameActionsDropserverHostPortType*) . . . 234
3.4.2.3 Models . . . 234
3.4.2.3.1 cluster.APIUser . . . 234
3.4.2.3.2 cluster.Agent . . . 235
3.4.2.3.3 cluster.Alerts . . . 235
3.4.2.3.4 cluster.Cluster . . . 235
3.4.2.3.5 cluster.ClusterForm . . . 237
3.4.2.3.6 cluster.Crash . . . 238
3.4.2.3.7 cluster.DelayStat . . . 238
3.4.2.3.8 cluster.DelayStatHistory . . . 238
3.4.2.3.9 cluster.Diff . . . 238
3.4.2.3.10 cluster.GraphiteFilterList . . . 238
3.4.2.3.11 cluster.ServerBackupMeta . . . 239
3.4.2.3.12 cluster.ServerDelayStat . . . 239
3.4.2.3.13 cluster.ServerMonitor . . . 239
3.4.2.3.14 cluster.SlaveVariables . . . 242
3.4.2.3.15 cluster.SlavesOldestMasterFile . . . 243
3.4.2.3.16 cluster.Test . . . 243
3.4.2.3.17 cluster.UserForm . . . 243
3.4.2.3.18 cluster.VariableDiff . . . 243
3.4.2.3.19 config.BackupMetaMap . . . 244
3.4.2.3.20 config.BackupMetadata . . . 244
3.4.2.3.21 config.ConfigVariableType . . . 244
3.4.2.3.22 config.DockerRepo . . . 244
3.4.2.3.23 config.DockerTag . . . 244
3.4.2.3.24 config.Grant . . . 245
3.4.2.3.25 config.Partner . . . 245
3.4.2.3.26 config.PeerCluster . . . 245
3.4.2.3.27 config.PointInTimeMeta . . . 246
3.4.2.3.28 config.Role . . . 246
3.4.2.3.29 config.ServicePlan . . . 246
3.4.2.3.30 config.StringsMap . . . 247
3.4.2.3.31 config.TagResult . . . 247
3.4.2.3.32 config.Tarball . . . 247
3.4.2.3.33 config.TasksMap . . . 248
3.4.2.3.34 config.VersionsMap . . . 248
3.4.2.3.35 config.WorkLoad . . . 248
3.4.2.3.36 config.WorkLoadsMap . . . 248
3.4.2.3.37 configurator.Configurator . . . 248
3.4.2.3.38 dbhelper.BinaryLogMetaMap . . . 248
3.4.2.3.39 dbhelper.Event . . . 248
3.4.2.3.40 dbhelper.MasterStatus . . . 249
3.4.2.3.41 dbhelper.SlaveStatus . . . 249
3.4.2.3.42 github_com_signal18_replication-manager_config.Config . . . 249
3.4.2.3.43 gtid.Gtid . . . 262
3.4.2.3.44 opensvc.Addr . . . 262
3.4.2.3.45 opensvc.Host . . . 263
3.4.2.3.46 opensvc.Service . . . 263
3.4.2.3.47 os.Process . . . 263
3.4.2.3.48 regexp.Regexp . . . 263

3.4.2.3.49 repmanv3.BackupStat . . . 263
3.4.2.3.50 repmanv3.Tag . . . 264
3.4.2.3.51 s18log.HttpLog . . . 264
3.4.2.3.52 s18log.HttpMessage . . . 264
3.4.2.3.53 server.ApiResponse . . . 264
3.4.2.3.54 server.CredentialMailForm . . . 264
3.4.2.3.55 server.DecodedData . . . 265
3.4.2.3.56 server.Heartbeat . . . 265
3.4.2.3.57 server.ReplicationManager . . . 265
3.4.2.3.58 server.token . . . 266
3.4.2.3.59 server.userCredentials . . . 266
3.4.2.3.60 sql.NullInt64 . . . 267
3.4.2.3.61 sql.NullString . . . 267
3.4.2.3.62 state.Sla . . . 267
3.4.2.3.63 state.StateHttp . . . 267
3.4.2.3.64 state.StateMachine . . . 267
3.4.2.3.65 user.User . . . 268
3.4.2.3.66 version.Version . . . 268
3.4.3.0.1 Monitor Unprotected Endpoints . . . 270

Version 2.0 & 2.1 **270**
3.4.3.0.2 API Protected Endpoints . . . 270
3.4.3.1 Cluster Protected Endpoint . . . 276
3.4.3.1.1 2.0 . . . 276
3.4.3.2 2.1 . . . 278
3.4.3.3 Database Server Protected Endpoint . . . 278
3.4.3.3.1 2.0 . . . 278
3.4.3.3.2 2.1 . . . 278
3.4.3.4 Proxy Protected Endpoint . . . 279
3.6.1 Provisioning Agent . . . 280
3.6.1.1 Get services manage by the agent . . . 280
3.6.1.2 Check service status . . . 280
3.6.1.3 Get service configuration . . . 280
3.6.1.4 Get logs of a docker image . . . 282
3.6.1.5 Entering a docker image . . . 282
3.6.1.6 Secret and config services . . . 282
3.7.1 Logs . . . 284
3.7.2 Log Files Written . . . 284
3.7.2.1 Main Log . . . 284
3.7.2.2 Derived Logs . . . 284
3.7.2.3 Per-Cluster SQL Logs . . . 284
3.7.2.4 Panic Log . . . 284
3.7.2.5 Carbon / Graphite API Log . . . 284
3.7.3 Log Levels . . . 285
3.7.4 Per-Module Log Levels . . . 285
3.7.5 Log Message Format . . . 286
3.7.6 Log Rotation . . . 287
3.7.7 Syslog . . . 287
3.7.8 Viewing Logs on a Modern Linux System . . . 287
3.7.8.1 journalctl — follow in real time . . . 287
3.7.8.2 journalctl — query by time . . . 288
3.7.8.3 journalctl — filter by priority . . . 288
3.7.8.4 journalctl — output formats . . . 288
3.7.8.5 journalctl — search inside messages . . . 288

3.7.8.6 Tailing the log file directly . . . 288
3.7.8.7 Searching historical logs . . . 289
3.7.9 Accessing Logs via the REST API . . . 289
3.7.10 Temporarily Increasing Verbosity . . . 289
3.7.11 Diagnostic Quick Reference . . . 290
4.1.1 High Availability . . . 291
4.1.2 Topology Discovery . . . 291
4.1.3 False Positive Protection . . . 291
4.1.4 Default Mode — Alert, Don't Act . . . 291
4.1.5 Switchover — Planned Handoff . . . 292
4.1.6 Failover — Automatic Recovery . . . 292
4.1.7 Rejoin — Reintroducing an Old Primary . . . 292
4.1.8 Reseeding — Adding a New Server . . . 293
4.1.9 Summary . . . 293
4.2.0.1 Switchover Workflow . . . 294
4.2.0.2 Switchover Events Graph . . . 294
4.3.0.1 Failover . . . 295
4.4.1.0.1 GTID . . . 297
4.4.1.0.2 Positional Replication . . . 297
4.4.12.1 Staging config important setup : . . . 309
4.4.12.9 Lesson learns: . . . 309
4.4.12.10 Todo : . . . 309
4.4.12.11 To be removed after integration . . . 312
4.4.12.12 Todo . . . 312
4.4.13.1 Configuration . . . 313
4.4.13.2 Topology Characteristics . . . 313
4.4.13.2.1 Behavior . . . 313
4.4.13.2.2 Automatic Detection . . . 313
4.4.13.3 Use Cases . . . 313
4.4.13.4 Features Available in Active-Passive Mode . . . 313
4.4.13.5 Features Disabled in Active-Passive Mode . . . 314
4.4.13.6 Configuration Example . . . 314
4.4.13.6.1 Minimal Configuration . . . 314
4.4.13.6.2 Full Monitoring Configuration . . . 314
4.4.13.6.3 Multi-Server Active-Passive (No Replication) . . . 315
4.4.13.7 Transitioning Between Modes . . . 315
4.4.13.7.1 From Master-Slave to Active-Passive . . . 315
4.4.13.7.2 From Active-Passive to Master-Slave . . . 315
4.4.13.8 API Behavior . . . 315
4.4.13.9 Comparison with Other Topologies . . . 315
4.4.13.10 Troubleshooting . . . 315
4.4.13.11 Best Practices . . . 316
4.5.1.1 Database servers configuration (`db-servers`) . . . 317
4.5.2.0.1 Deprecated . . . 322
4.5.2.2.1 Crash recovery . . . 325
4.5.2.2.2 All cluster down . . . 327
4.5.3.1 Switchover Configuration . . . 329
4.5.4.1 Replication Configuration . . . 331
4.5.4.2 Topology Configuration . . . 331
4.5.4.2.1 Master Slaves Configuration . . . 331
4.5.4.3 Multi Tier Slaves Configuration . . . 331
4.5.4.4 Multi Master Configuration . . . 332
4.5.4.5 Multi Source or Channel Configuration . . . 332
4.5.4.6 Multi Master Ring Configuration . . . 334

4.5.4.7 Multi Master Galera Configuration . . . 334
4.5.5.1.0.1 HaProxy Configuration . . . 335
4.5.5.2.0.1 ProxySQL Configuration . . . 340
4.5.5.4.1 Testing Configuration . . . 348
4.5.5.6.0.1 External Scripts . . . 352
4.5.5.6.0.2 External Proxy & Route . . . 352
4.5.6.1 Checks . . . 355
4.5.6.2 Enforce . . . 355
4.6.1 Troubleshooting High Availability . . . 359
4.6.2 Data Directory Layout . . . 359
4.6.3 Key Files Explained . . . 359
4.6.3.1 `clusterstate.json` . . . 359
4.6.3.2 `failover.YYYYMMDDHHMMSS.json` . . . 359
4.6.3.3 `sql_error.log` and `sql_general.log` . . . 360
4.6.3.4 `panic.log` . . . 360
4.6.4 Log Files . . . 360
4.6.5 REST API Endpoints . . . 360
4.6.6 Increasing Verbosity for an Investigation . . . 361
4.6.7 Common First Steps . . . 361
5.1.1 Monitoring Overview . . . 363
5.1.2 What the Monitor Tracks . . . 363
5.1.2.1 Replication topology . . . 363
5.1.2.2 Server health . . . 363
5.1.2.3 Workload . . . 363
5.1.2.4 Proxies . . . 363
5.1.2.5 Metrics . . . 363
5.1.3 Monitoring Loop . . . 363
5.1.4 Accessing Monitoring Data . . . 364
5.1.4.1 GUI . . . 364
5.1.4.2 REST API . . . 364
5.1.4.3 Prometheus . . . 364
5.1.4.4 CLI . . . 364
5.1.5 Configuration . . . 364
5.2.1.1 Server daemon configuration (`monitoring,log`) . . . 365
5.2.1.1.1 Deprecated . . . 369
5.2.3.1 Basic Processlist Monitoring . . . 374
5.2.3.2 Transaction Monitoring . . . 374
5.2.3.3 Query Monitoring . . . 375
5.2.3.4 Usage Example . . . 375
5.2.3.5 API Access . . . 375
5.2.3.6 Performance Considerations . . . 376
5.2.4.1 Basic Performance Schema Monitoring . . . 377
5.2.4.2 Mutex Wait Monitoring . . . 377
5.2.4.3 Latch (RWLock) Wait Monitoring . . . 377
5.2.4.4 Performance Schema Memory Monitoring . . . 378
5.2.4.5 Configuration Example . . . 378
5.2.4.6 Metrics Export . . . 379
5.2.4.7 Performance Considerations . . . 379
5.2.4.8 Database Configuration . . . 379
5.2.4.9 Troubleshooting . . . 379
5.2.5.0.1 Active standby with external arbitrator . . . 380
5.2.5.0.2 Internals . . . 381
5.2.5.0.3 Run a private arbitrator . . . 381
5.3.1 Logs and Troubleshooting . . . 383

5.3.2 replication-manager Daemon Log . . . 383
5.3.2.1 Typical log-level guide . . . 383
5.3.3 Per-Cluster Diagnostic Files . . . 383
5.3.3.1 sql_general.log . . . 384
5.3.3.2 Capture files . . . 384
5.3.4 Memory Profiling . . . 384
5.3.5 Common Troubleshooting Steps . . . 384
5.3.5.1 Replication-manager won't connect to a server . . . 384
5.3.5.2 Failover triggered unexpectedly . . . 384
5.3.5.3 Alert firing but no notification received . . . 385
5.3.5.4 High log volume . . . 385
6.1.1 Maintenance . . . 386
6.1.2 How the Job System Works . . . 386
6.1.2.1 Jobs Table . . . 386
6.1.3 Scheduled Tasks . . . 386
6.1.4 How the dbjobs Script Is Delivered . . . 387
6.1.4.1 replication-manager-pro (container mode) . . . 387
6.1.4.2 replication-manager-osc (SSH mode) . . . 387
6.1.5 Environment Variables Injected by replication-manager . . . 388
6.1.6 How Results Are Streamed Back . . . 388
6.2.1.1 Scheduler . . . 390
6.2.1.2 CRON Expression Format . . . 390
6.2.1.3 CRON jobs configuration . . . 390
6.2.2.1 Backups . . . 393
6.2.2.1.0.1 Binlogs Backups . . . 396
6.2.2.2.0.1 Archiving Backups . . . 397
6.2.2.2.1 Disk Space Management . . . 399
6.2.2.2.2 Configuration Examples . . . 400
6.2.2.2.2.1 Local Repository Configuration . . . 400
6.2.2.2.2.2 S3/Minio Configuration . . . 400
6.2.2.2.2.3 Retention Policy . . . 401
6.2.2.2.3 Restic Task Management . . . 401
6.2.2.2.4 Automatic Cleanup Workflow . . . 401
6.2.2.2.5 Performance Considerations . . . 401
6.2.2.3.0.1 Streaming Backups . . . 402

Schema and Data Monitoring **404**
6.2.3.1 Schema Monitoring . . . 404
6.2.3.1.1 How Collection Works . . . 404
6.2.3.1.2 Enabling the Scheduler . . . 404
6.2.3.1.3 Collection Scope . . . 404
6.2.3.1.4 Change Detection . . . 405
6.2.3.1.5 Special Case: Shard Proxy . . . 405
6.2.3.1.6 Persisting the Table Dictionary . . . 405
6.2.3.2 Data Checksumming . . . 405
6.2.3.2.1 Triggering a Checksum . . . 405
6.2.3.2.2 How Checksumming Works . . . 406
6.2.3.2.3 Table Sync States . . . 406
6.2.3.2.4 Excluding Expected Divergence . . . 406
6.2.3.2.5 Supported Primary Key Types . . . 406
6.2.3.3 Repairing Divergent Tables . . . 407
6.2.3.4 GUI: Shards and Schema Graph . . . 407
6.2.3.5 Failover Policy and Data Divergence . . . 407
6.2.3.5.1 How Divergence Affects Replica Election . . . 407

6.2.3.5.2 Controlling Failover Behaviour . . . 408
6.2.3.5.3 Recommended Workflow for Strict Mode . . . 408
6.2.3.5.4 GUI Setting . . . 408
6.2.4.1 Donor and joiner cron script . . . 409
7.1.1 Security . . . 410
7.1.2 Feature Summary . . . 410
7.1.2.1 Process Privilege Drop (`--user`) . . . 410
7.1.2.2 TLS — API and Web Interface . . . 410
7.1.2.3 TLS — Database Connections . . . 410
7.1.2.4 Encrypted Configuration . . . 410
7.1.2.5 HashiCorp Vault Integration . . . 410
7.1.2.6 API Credentials and ACLs . . . 410
7.1.2.7 Provisioning TLS . . . 411
7.1.3 HTTPS Bastion — Browser Terminal Access . . . 411
7.1.4 Supply Chain and Regulatory Compliance . . . 411
7.1.5 Configuration Guide . . . 411
7.2.1 Process User and Privilege Drop . . . 412
7.2.1.1 `--user` . . . 412
7.2.2 Configuration File Security . . . 413
7.2.3 API Security Configuration . . . 413
7.2.4 Database Security Configuration . . . 414
7.2.5 Vault Security Configuration . . . 415
7.2.5.1 Config_store_v2 . . . 416
7.2.5.2 Database_engine . . . 417
7.2.5.3 Rotation of Database Credentials . . . 418
7.3.1 SBOM and EU Cyber Resilience Act . . . 419
7.3.2 The Cyber Resilience Act . . . 419
7.3.2.1 What the CRA requires . . . 419
7.3.2.2 Open-source software and the CRA . . . 419
7.3.3 Software Bill of Materials . . . 419
7.3.3.1 Format . . . 419
7.3.3.2 What is included . . . 420
7.3.3.3 How to get the SBOM . . . 420
7.3.3.4 Generating the SBOM locally . . . 420
7.3.4 Using the SBOM . . . 420
7.3.4.1 Vulnerability scanning . . . 420
7.3.4.2 Software composition analysis . . . 421
7.3.4.3 License compliance . . . 421
7.3.5 Vulnerability Disclosure . . . 421
7.3.5.1 Reporting a vulnerability . . . 421
7.3.5.2 Response commitment . . . 421
7.3.5.3 Security advisories . . . 421
7.3.6 Signed Releases . . . 421
7.3.7 Regulatory References . . . 421
7.4.1 HTTPS Bastion and Terminal . . . 423
7.4.2 Terminal Types . . . 423
7.4.3 WebSocket API Endpoints . . . 423
7.4.3.1 OpenSVC container selector (bash only) . . . 423
7.4.3.2 List active sessions . . . 424
7.4.4 Authentication Flow . . . 424
7.4.5 Role-Based Database Credential Selection . . . 424
7.4.6 Back-End Transport . . . 424
7.4.6.1 Non-OpenSVC clusters (on-premise / SSH) . . . 424
7.4.6.2 OpenSVC clusters . . . 425

7.4.7 Session Resume . . . 425
7.4.8 Configuration Reference . . . 425
7.4.8.1 Enabling the feature . . . 425
7.4.8.2 All terminal settings . . . 425
7.4.9 ACL and Grants . . . 427
7.4.10 Security Notes . . . 427
8.1.1 Alerting . . . 429
8.1.2 Alert Channels . . . 429
8.1.3 What Triggers an Alert . . . 429
8.1.4 Alert Lifecycle . . . 429
8.1.5 Configuration Guide . . . 429
8.2.1 Alerting Configuration Guide . . . 430
8.2.2 Alert Triggering . . . 430
8.2.3 Alert Blackout Window . . . 430
8.2.4 External Script . . . 431
8.2.5 Email . . . 431
8.2.6 Slack . . . 432
8.2.7 Pushover . . . 433
8.2.8 Microsoft Teams . . . 433
8.2.9 Syslog . . . 434
8.2.10 Cloud18 Integration . . . 434
8.2.11 Checking Alert Status via API . . . 435
9.1.1 Provisioning . . . 436
9.1.2 Orchestration Backends . . . 436
9.1.3 What Can Be Provisioned . . . 436
9.1.3.1 Database Services . . . 436
9.1.3.2 Proxy Services . . . 436
9.1.3.3 Service Plans . . . 437
9.1.4 Bootstrap Workflow . . . 437
9.1.5 Configuration Guide . . . 437
9.2.1.3.1 Disks . . . 439
9.2.1.3.2 network . . . 440
9.2.4.1 Sample MariaDB from source deployed in /usr/local . . . 445
9.3.1.1 Software Configurator . . . 449
9.3.1.2 How It Works . . . 449
9.3.1.3 Config Discovery . . . 449
9.3.1.4 Config Delivery . . . 450
9.3.1.4.1 Container mode (pro / OpenSVC / Kubernetes) . . . 450
9.3.1.4.2 SSH mode (osc / onpremise) . . . 451
9.3.1.4.3 Secure download . . . 451
9.3.1.5 Starting the Database . . . 451
9.3.1.5.1 Bootstrap vs Start . . . 451
9.3.1.5.2 Conditional Config Fetch on Start . . . 452
9.3.1.5.3 Controlling Config Fetch . . . 452
9.3.1.5.4 Preserving an Existing my.cnf . . . 452
9.3.1.5.5 The `.system` Directory . . . 453
9.3.1.5.6 nosplitpath — Flat Datadir Layout . . . 453
9.3.1.6 Regenerating Config . . . 454
9.3.1.7 Filesystem Layout . . . 454
9.3.2.1 Database Tags . . . 455
9.3.2.1.1 Storage Engines . . . 455
9.3.2.1.2 Disk and I/O . . . 455
9.3.2.1.3 Logging . . . 455
9.3.2.1.4 Network and Threads . . . 456

9.3.2.1.5 Replication . . . 456
9.3.2.1.6 Security . . . 456
9.3.2.1.7 Optimizer . . . 457
9.3.2.1.8 Character Set . . . 457
9.3.2.1.9 Platform / Orchestrator . . . 457
9.3.2.2 Proxy Tags . . . 458
9.3.2.3 Compliance Module . . . 458
9.3.3.1 Config Tracking . . . 459
9.3.3.2 Three-Layer Config System . . . 459
9.3.3.2.1 01_preserved.cnf — Server-specific locked values . . . 459
9.3.3.2.2 02_delta.cnf — Calculated drift . . . 459
9.3.3.2.3 03_agreed.cnf — Manually accepted deviations . . . 459
9.3.3.3 Cluster-Wide Preserved Variables . . . 460
9.3.3.4 Configuration Keys . . . 460
9.3.3.5 Monitoring Config Changes . . . 461
9.3.3.6 Reading Variables from Config Files . . . 461
9.3.4.1 Configuration Guide . . . 462
9.3.4.2 Database Resource Settings . . . 462
9.3.4.3 Tag Settings . . . 463
9.3.4.4 Config Preservation . . . 463
9.3.4.5 Compliance Module Override . . . 464
9.3.4.6 Replication and Domain . . . 464
9.3.4.7 Binary Distribution . . . 465
9.3.4.8 Start Behavior . . . 465
9.3.4.9 Bootstrap Scripts . . . 465
9.3.4.10 Security . . . 466
9.3.4.11 Proxy Resources . . . 466
10.1.1 Building from source . . . 469
10.1.2 Compilation flags . . . 469
10.2.0.1 Non-regression tests . . . 470
10.3.0.1 Report Bugs . . . 471
10.4.0.1 Profiling CPU usage . . . 472
11.1.1 Plugin Architecture . . . 473
11.1.2 Lifecycle — External Plugins . . . 473
11.1.3 Wire Protocol — JSON stdin/stdout . . . 473
11.1.3.1 Request . . . 474
11.1.3.2 Response . . . 474
11.1.4 Plugin Configuration . . . 475
11.1.5 Plugin Discovery and Loading . . . 475
11.1.5.1 Plugin Signing . . . 475
11.1.6 Security Considerations . . . 476
11.1.7 Global Plugin Settings . . . 476
11.2.1 Built-in Log Plugins . . . 477
11.2.2 errorlog . . . 477
11.2.3 sqlerrorlog . . . 477
11.2.4 slowlog . . . 478
11.2.5 auditlog . . . 478
11.2.6 Spike Detection Algorithm . . . 478
11.3.1 Workload Plugins . . . 480
11.3.2 plugin-innodb-corruption . . . 480
11.3.3 plugin-connection-storm . . . 480
11.3.4 plugin-error-storm . . . 481
11.3.5 plugin-full-table-scan-spike . . . 481
11.3.6 plugin-metadata-lock-contention . . . 482

11.3.7 plugin-replication-lag-predictor . . . 482
11.3.8 plugin-slow-query-regression . . . 483
11.3.9 plugin-tmp-table-storm . . . 483
11.3.10 plugin-off-hours-access . . . 484
11.3.11 plugin-privilege-escalation . . . 485
11.3.12 plugin-binlog-cleartext-password . . . 485
11.3.13 plugin-binlog-creditcard-leak . . . 486
11.3.14 WARN Code Reference . . . 486
11.4.1 Security Plugins . . . 488
11.4.2 plugin-security-no-password-user . . . 488
11.4.3 plugin-security-weak-auth . . . 488
11.4.4 plugin-security-local-infile . . . 489
11.4.5 plugin-security-hardening . . . 489
11.4.5.1 SEC0103 — require_secure_transport=OFF . . . 489
11.4.5.2 SEC0104 — general_log=ON . . . 489
11.4.5.3 SEC0105 — secure_file_priv=’’ . . . 490
11.4.5.4 SEC0106 — skip_name_resolve=OFF . . . 490
11.4.5.5 SEC0107 — Anonymous user account . . . 490
11.4.5.6 SEC0108 — Wildcard-host privileged user . . . 490
11.4.5.7 SEC0109 / SEC0110 / SEC0111 — At-rest encryption (via plugin-score-encryption) . 490
11.4.5.8 SEC0112 — Audit logging not active . . . 491
11.4.5.9 SEC0113 / SEC0114 / SEC0115 — Password validation plugins *(MariaDB only)* . . 491
11.4.5.10 SEC0118 — Hostname grants with skip_name_resolve=ON . . . 491
11.4.6 Remediation Engine . . . 491
11.4.6.1 Remediation Plan . . . 491
11.4.6.2 Apply a Fix . . . 492
11.4.6.3 Compliance Tags . . . 493
11.4.7 SEC Code Reference . . . 494
11.4.8 Security Considerations . . . 495
11.5.1 Score Plugins . . . 496
11.5.2 plugin-score-ssl . . . 496
11.5.3 plugin-score-encryption . . . 496
11.5.4 plugin-score-auth . . . 496
11.5.5 plugin-score-network . . . 497
11.5.6 plugin-score-proxy . . . 497
11.5.7 plugin-score-passwords . . . 497
11.5.8 plugin-score-lts . . . 498
11.5.9 plugin-score-audit . . . 498
11.5.10 SecurityScore Dashboard . . . 498
11.6.1 Developing Plugins . . . 499
11.6.2 Quick Start . . . 499
11.6.2.1 Create the plugin directory . . . 499
11.6.2.2 Write the plugin . . . 499
11.6.2.3 Build the plugin . . . 500
11.6.2.4 Sign the plugin (if signature verification is enabled) . . . 500
11.6.3 Wire Protocol Reference . . . 500
11.6.3.1 wire.Request . . . 500
11.6.3.2 wire.DBUser . . . 500
11.6.3.3 wire.Finding . . . 500
11.6.3.4 wire.ScoreCheck . . . 501
11.6.3.5 wire.Response . . . 501
11.6.4 Configuration Helpers . . . 501
11.6.5 Error Code Assignment . . . 502
11.6.6 Registering GUI Configuration . . . 502

11.6.7 Adding an Automated Remediation . . . 502
11.6.8 Testing a Plugin Locally . . . 502
11.6.9 Checklist . . . 503
12.1.1 Replication best practices . . . 504
12.1.1.1 Parallel replication . . . 504
12.1.1.2 Semi-synchronous replication . . . 504
12.2.0.1 Enforcing best practices . . . 505
12.3.0.1 Troubleshooting failover switchover . . . 506
12.3.0.2 Troubleshooting internal status . . . 506
12.3.0.3 Troubleshoot Crashes . . . 506
13.1.0.1 Features . . . 508
13.2.0.1 Features . . . 509
13.3.0.1 Features . . . 510
13.4.0.1 Features . . . 511
13.5.0.1 Features . . . 513
13.6.0.1 Features . . . 514
13.7.0.1 Features . . . 515
13.8.0.1 Features . . . 516
14.1 Installation & Setup . . . 519
14.1.1 Why does connection fail when using “localhost” as the database host? . . . 519
14.1.2 What are the minimum database versions required? . . . 519
14.1.3 Why isn’t HAProxy showing statistics correctly? . . . 519
14.1.4 Why are my configuration changes not persisting after restart? . . . 519
14.1.5 How do I migrate from version 2.x to 3.x? . . . 520
14.1.6 What’s the minimum configuration needed to start monitoring? . . . 520
14.2 Replication & Synchronization . . . 521
14.2.1 Does semi-sync replication guarantee no data loss after a master crash? . . . 521
14.2.2 Why does my switchover fail after long write inactivity? . . . 521
14.2.3 What’s the difference between AFTER_SYNC and AFTER_COMMIT? . . . 521
14.2.4 Can SUPER privileged users write during switchover on MariaDB? . . . 522
14.2.5 Why does MySQL hang during shutdown with GTID enabled? . . . 522
14.2.6 What happens when semi-sync reaches timeout? . . . 522
14.2.7 Why can’t relay slaves reconnect after their master dies in multi-tier topology? . . . 523
14.2.8 What does “server-id 1000” reserved mean? . . . 523
14.3 Failover & Switchover . . . 523
14.3.1 How does replication-manager prevent false positive failovers? . . . 523
14.3.2 What happens when the entire cluster goes down? . . . 524
14.3.3 Why did my failover get rejected? . . . 524
14.3.4 How long should I wait between failover attempts? . . . 524
14.3.5 What does “failover-at-sync” mean for data safety? . . . 525
14.3.6 Why is my switchover stuck or timing out? . . . 525
14.3.7 When will replication-manager NOT perform automatic failover? . . . 526
14.4 Topology & Deployment . . . 526
14.4.1 Why are two-node clusters not recommended? . . . 526
14.4.2 What are the limitations of Galera cluster support? . . . 526
14.4.3 How does multi-master prevent split-brain? . . . 527
14.4.4 What happens if a relay slave crashes in multi-tier topology? . . . 527
14.5 Security & Configuration . . . 527
14.5.1 How do I change the default API credentials? . . . 527
14.5.2 How do I encrypt passwords in configuration files? . . . 528
14.5.3 How do I integrate with HashiCorp Vault? . . . 528
14.5.4 Do I need to replace the self-signed certificates? . . . 529
14.6 Proxies & Routing . . . 530
14.6.1 Why aren’t ProxySQL users syncing automatically? . . . 530

14.6.2 What's the difference between ProxySQL bootstrap-users and manual configuration? . 530
14.6.3 Why does MaxScale have a monitoring delay? 530
14.6.4 Which proxy should I use? 531
14.7 Recovery & Data Loss 531
14.7.1 How do I recover a failed master? 531
14.7.2 When should I use flashback vs mysqldump recovery? 532
14.7.3 Where can I find crash information after failover? 532
14.8 Monitoring & Troubleshooting 533
14.8.1 How do I increase logging for debugging? 533
14.8.2 Why is performance schema monitoring causing overhead? 533
14.8.3 How do I check internal status for support tickets? 534
14.9 Operational Best Practices 535
14.9.1 What are the recommended parallel replication settings? 535
14.9.2 What are the recommended semi-sync settings? 535
14.9.3 Should I enable dynamic best practice enforcement? 535
14.9.4 What backup strategy should I use? 536

1.1.1 Goals

replication-manager is an open source database cluster orchestrator that embeds best practices for configuration, deployment, HA operations, maintenance, monitoring, and troubleshooting for MySQL, MariaDB, and Percona. The design hides database clustering complexity while maintaining simplicity similar to Amazon RDS.

replication-manager is configurable using nix-style configuration, supports multi-tenancy, provides security through encryption and ACL, and offers API, command line, and web interfaces for database and proxy management.

replication-manager delivers multi-cluster sharding and routing solutions to address scalability and high availability challenges.

1.1.2 History

replication-manager was initially written to close the gap between Galera Cluster and MySQL Master HA. While Galera Cluster addresses many clustering needs, it has limitations around performance and cluster-wide locking. **replication-manager** leverages newer MySQL and MariaDB features including Global Transaction ID, Semi-Synchronous replication, and binary log flashback to provide high availability and node switchover with minimal performance impact.

replication-manager evolved to manage full-stack database cluster deployments for testing and production. Clusters can be provisioned, tested, and unprovisioned across different topologies and product releases. The solution supports both existing deployments and architecture-as-a-service driven by external orchestrators.

Integration support includes: - **OpenSVC**: Using collector API and direct cluster node VIP API - **Kubernetes**: Cluster management and orchestration - **SlapOS**: Open source hyperconverged and edge computing infrastructure where the API checks and performs actions on database infrastructure components

1.1.3 HA Workflow

To perform switchover on leader replicas while preserving data consistency, **replication-manager** uses an improved workflow similar to common MySQL failover tools such as MHA:

- ☒ Verify replication settings
- ☒ Check replication on the slaves (configurable)
- ☒ Check for long running queries and transactions on master
- ☒ Elect a new master (defaults to most up-to-date replica, or a designated candidate)
- ☒ Call optional script to remove IP address from master
- ☒ Notify proxies about the state change
- ☒ Reject writes on master by calling FLUSH TABLES WITH READ LOCK
- ☒ Reject writes on master by setting READ_ONLY flag
- ☒ Reject writes on master by decreasing MAX_CONNECTIONS
- ☒ Kill pending connections on master if any remain
- ☒ Watch for all slaves to catch up to the current GTID position
- ☒ Promote the candidate slave to be the new master
- ☒ Call optional script to assign IP address to new master
- ☒ Notify proxies about the new state
- ☒ Switch other slaves and old master to replicate from the new master
- ☒ Set slave read-only

1.1.4 Traffic Routing

replication-manager is commonly used as an arbitrator with a routing layer that directs write database traffic to a single leader database node.

- ☒ Layer 3: DNS routing updates discovery services like Consul or other API-based DNS to enable direct routing without proxying
- ☒ Layer 4: HAProxy configuration generation or reconfiguration via API calls
- ☒ Layer 4: HAProxy external checks via **replication-manager** API as a resource manager reporting cluster node states
- ☒ Layer 4: External scripts to reuse existing routes like VIP failover, Puppet/Ansible scenarios, or ldirector reconfiguration
- ☒ Layer 7: ProxySQL or MariaDB MaxScale receive instant notifications of topology changes without relying on their own monitoring
- ☒ Layer 7: MariaDB Spider layer for database traffic splitting or table sharding across different database clusters

Multiple strategies can be combined depending on feature maturity, security, or performance requirements.

Leader election clusters are recommended in these cases:

- ☒ Dysfunctional nodes have no impact on cluster performance
- ☒ Heterogeneous configuration and hardware have no impact on cluster performance
- ☒ Network speed has no impact on cluster performance
- ☒ Read scalability does not impact write scalability
- ☒ Network quality fluctuation has no impact on cluster performance
- ☒ Benefits from human expertise on false positive failure detection
- ☒ Supports minimum cluster size of two data nodes
- ☒ Supports different database storage engines

Trade-offs include:

- ☒ Overloading the leader can lead to data loss on failover or prevent failover depending on setup
- ☒ Read operations on replicas are eventually consistent
- ☒ ACID properties can be preserved by routing all operations to the leader
- ☒ Read operations on replicas can be committed reads when using MariaDB 10.2 semi-sync no-slave-behind feature

Leader election asynchronous clusters can guarantee continuity of service at no performance cost for the leader and in specific conditions achieve zero data loss.

In production deployments, hardware failures occur infrequently. When failures happen, repairing the replication stream quickly is critical. **replication-manager** facilitates rapid recovery and tracks failover SLA (Service Level Availability) to provide historical view of replication stream latency. This enables routine switchovers for database maintenance.

Automatic failover in asynchronous clusters is not always desirable. **replication-manager** provides tunable settings to constrain the architecture state in which failover can occur.

Automatic failover scenarios can be classified into three SLA states:

- ☒ Replica stream in sync
- ☒ Replica stream not in sync but state allows automatic failover
- ☒ Replica stream not in sync and state does not allow automatic failover

! Staying in sync ! ! When replication can be monitored in sync, failover can be performed without data loss, provided that **replication-manager** waits for all replicated events to be applied to the elected replica before re-opening traffic. ! To maintain this state, use the settings described in the next section.

replication-manager is licensed under the **GNU Public License version 3**.

1.3.0.1 GNU GENERAL PUBLIC LICENSE

Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. http://fsf.org/

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

1.3.0.2 Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program–to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

1.3.0.3 TERMS AND CONDITIONS

1.3.0.3.1 Definitions. "This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1.3.0.3.2 Source Code. The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

1.3.0.3.3 Basic Permissions. All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

1.3.0.3.4 Protecting Users' Legal Rights From Anti-Circumvention Law. No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

1.3.0.3.5 Conveying Verbatim Copies. You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

1.3.0.3.6 Conveying Modified Source Versions. You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

- a) The work must carry prominent notices stating that you modified it, and giving a relevant date.
- b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".
- c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.
- d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

1.3.0.3.7 Conveying Non-Source Forms. You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

- a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.
- b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.
- c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.
- d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.
- e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide

support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

1.3.0.3.8 Additional Terms. "Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

- a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or
- b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or
- c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or
- d) Limiting the use for publicity purposes of names of licensors or authors of the material; or
- e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or
- f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

1.3.0.3.9 Termination. You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

1.3.0.3.10 Acceptance Not Required for Having Copies. You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

1.3.0.3.11 Automatic Licensing of Downstream Recipients. Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

1.3.0.3.12 Patents. A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

1.3.0.3.13 No Surrender of Others' Freedom. If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

1.3.0.3.14 Use with the GNU Affero General Public License. Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

1.3.0.3.15 Revised Versions of this License. The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's publie welcome to redistribute it under certain conditions; type 'show c' for details.

The hypothetical commands 'show w' and 'show c' should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see http://www.gnu.org/licenses/.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read http://www.gnu.org/philosophy/why-not-lgpl.html.

replication-manager ships as a self-contained binary with no external library dependencies. Choose the method that fits your environment — all paths lead to the same monitoring daemon.

The current stable version is **3.1**. Check GitHub Releases for the latest release or review the changelog.

Section	Description
Overview	Binary flavors (osc, pro, arb) and distribution types
Embedded Binary	One-command install, no package manager required
Repository	RPM and DEB packages via Signal18 apt/yum repo
Docker	Official container images with all dependencies bundled
Source	Build from Go source for custom flavors
Orchestrators	Install OpenSVC, Kubernetes, and SlapOS for Pro provisioning
What Was Installed	Files and directories created by each install method
Install Dependencies	External tools — auto-detected, with optional path overrides

replication-manager is built from a single codebase with compile-time feature flags that produce several distinct **flavors**. Choose the flavor that matches your deployment model.

2.1.1.1 Binary Flavors

Flavor	Binary name	Description
osc	`replication-manager-osc`	Open Source Community — full HA monitoring, failover, switchover, proxy integration, backups, alerting. No cluster provisioning. Default orchestrator: `onpremise`.
pro	`replication-manager-pro`	Provisioning edition — everything in `osc` plus orchestrated cluster provisioning via OpenSVC, Kubernetes, or SlapOS. Default orchestrator: `opensvc`.
arb	`replication-manager-arb`	Arbitrator — lightweight split-brain arbitration service for replication-manager clustering. No monitoring, no GUI.
tst	`replication-manager-tst`	Testing edition — `osc` plus local bootstrap, benchmarking with Sysbench, and regression testing tooling.
embedded	`replication-manager`	Standalone binary with the full React dashboard and all static assets embedded. Portable — runs from any directory.

For most on-premise deployments start with **osc**. Use **pro** when you need replication-manager to provision and manage the lifecycle of database containers through an orchestrator.

2.1.1.2 Distribution Types

Each flavor is available in three distribution forms:

2.1.1.2.1 Package (RPM / DEB)

Installed via the Signal18 repository. Uses system directories:

Path	Contents
`/etc/replication-manager/`	Configuration files
`/usr/share/replication-manager/`	Static assets, dashboard, templates
`/var/lib/replication-manager/`	Runtime data, metrics, backups
`$HOME/.config/replication-manager/`	Dynamic config changes (v3+)
`/var/log/replication-manager.log`	Log file

2.1.1.2.2 Tarball (basedir)

Tarball releases use **basedir variants** (`-basedir` suffix) with static assets embedded in the archive. Self-contained under a single directory:

Path	Contents
`/usr/local/replication-manager/etc/`	Configuration files
`/usr/local/replication-manager/share/`	Static assets
`/usr/local/replication-manager/data/`	Runtime data

2.1.1.2.3 Embedded Binary

The `replication-manager` binary (no suffix) bundles the dashboard, templates, and all assets directly inside the binary. Suitable for portable or container deployments — no separate asset directory required.

2.1.1.3 Choosing a Flavor

```
Need cluster provisioning (Docker/Podman via OpenSVC or Kubernetes)?
    replication-manager-pro

Monitoring and HA of existing clusters only?
    replication-manager-osc

Running replication-manager in HA with split-brain protection?
    replication-manager-arb  (one instance per availability zone)

Local testing, benchmarking, regression tests?
    replication-manager-tst

Portable single-file deployment or Docker?
    replication-manager  (embedded)
```

2.1.1.4 CLI Client

`replication-manager-cli` is the command-line client. It communicates with the monitoring daemon over HTTPS using JWT authentication. It is bundled with all server packages but can be installed independently.

2.1.2.1 Install from Embedded Binary

The embedded binary (`replication-manager`) includes the full React dashboard and all static assets compiled into a single executable. No package manager, no asset directory, no system paths required.

2.1.2.2 One-Command Install

Install the latest release with a single command:

```
curl -fsSL https://signal18.io/get-repman | bash
```

The script auto-detects your operating system and architecture, downloads the binary from GitHub Releases, and places it in `/usr/local/bin` (or `~/.local/bin` if sudo is not available).

System-wide install (with sudo):

```
curl -fsSL https://signal18.io/get-repman | sudo bash
```

Install specific version:

```
curl -fsSL https://signal18.io/get-repman | REPMAN_VERSION=v3.1.16 bash
```

Install server + CLI client together:

```
curl -fsSL https://signal18.io/get-repman | REPMAN_INSTALL_CLI=true bash
```

Custom install directory:

```
curl -fsSL https://signal18.io/get-repman | REPMAN_INSTALL_DIR=/opt/repman bash
```

2.1.2.2.1 Environment Variables

Variable	Default	Description
`REPMAN_VERSION`	latest	Specific release version to install (e.g. `v3.1.16`)
`REPMAN_INSTALL_DIR`	`/usr/local/bin` or `~/.local/bin`	Installation directory
`REPMAN_INSTALL_CLI`	`false`	Also install `replication-manager-cli`
`REPMAN_SKIP_VERIFY`	`false`	Skip post-installation verification

2.1.2.2.2 Supported Platforms

OS	Architectures
Linux	amd64, arm64
macOS (Darwin)	amd64 (Intel), arm64 (Apple Silicon)

2.1.2.3 Manual Download from GitHub Releases

All release artifacts are available at github.com/signal18/replication-manager/releases.

2.1.2.3.1 Embedded binary (recommended for portable use)

`replication-manager-{version}.{arch}`

Example — download and install manually:

```
VERSION=3.1.24
ARCH=x86_64

curl -LO https://github.com/signal18/replication-manager/releases/download/v${VERSION}/replication-manag
chmod +x replication-manager-${VERSION}.${ARCH}
sudo mv replication-manager-${VERSION}.${ARCH} /usr/local/bin/replication-manager
```

2.1.2.3.2 Tarball (basedir variant)

Tarballs include the binary plus the `share/` asset directory. Use these when you want a versioned directory layout under `/usr/local/`:

```
replication-manager-{flavor}-{version}.tar.gz
```

```
sudo tar zxvf replication-manager-osc-3.1.24.tar.gz -C /usr/local/
sudo ln -s /usr/local/replication-manager-osc-3.1.24 /usr/local/replication-manager

# Copy systemd unit file
sudo cp /usr/local/replication-manager/share/replication-manager.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now replication-manager
```

2.1.2.3.3 Release Artifact Naming

Artifact	Description
`replication-manager-{version}.{arch}`	Embedded binary (dashboard + assets built in)
`replication-manager-{flavor}-{version}.tar.gz`	Tarball (basedir variant)
`replication-manager-{flavor}-{version}.{arch}.rpm`	RPM package
`replication-manager-{flavor}_{version}_{arch}.deb`	DEB package
`replication-manager-cli-{version}`	Standalone CLI client
`sbom-{version}.json`	Software Bill of Materials (CycloneDX)

Replace `{flavor}` with: `osc`, `tst`, `pro`, or `arb`

2.1.2.4 Development Builds

Nightly builds from the `develop` branch are available on the Continuous Integration Server.

2.1.4.1 Install from Repository

Signal18 provides signed package repositories for Debian/Ubuntu and CentOS/RHEL. Package installations deploy binaries, configuration templates, and static assets to standard system paths.

> **Upgrading from 2.x?** Version 3.x introduced configuration file location changes and parameter renames. See the Migration Guide before upgrading.

2.1.4.2 CentOS / RHEL

2.1.4.2.1 Configure the repository

```
cat > /etc/yum.repos.d/signal18.repo <<'EOF'
[signal18]
name=Signal18 repositories
baseurl=http://repo.signal18.io/centos/${releasever}/${basearch}/3.1/
gpgcheck=0
enabled=1
EOF
```

2.1.4.2.2 Install

```
yum install replication-manager-osc
```

Replace `replication-manager-osc` with your chosen flavor (`replication-manager-pro`, `replication-manager-arb`).

2.1.4.2.3 Upgrading from 2.x (epoch conflict)

If you encounter this error when upgrading from 2.x:

```
file /usr/bin/replication-manager-cli from install of replication-manager-client-3.1.xx-1.x86_64
conflicts with file from package replication-manager-pro-1730721861:2.3.53-1.x86_64
```

This is caused by an epoch removed in 3.x packages. Resolve it as follows:

1. Stop the replication-manager service
2. Back up `/var/lib/replication-manager/` and `/etc/replication-manager/`
3. Remove all existing replication-manager packages:

   ```
   yum remove replication-manager-pro replication-manager-osc replication-manager-client
   ```

4. Clear cached repository metadata:

   ```
   dnf clean metadata --disablerepo="*" --enablerepo=signal18
   ```

5. Install the 3.x package
6. Review and restore your configuration

2.1.4.3 Debian / Ubuntu

2.1.4.3.1 Configure the repository and install

```
# Set version
version="3.1"

# Import GPG signing key
```

```
gpg --recv-keys --keyserver keyserver.ubuntu.com FAE20E50
gpg --export FAE20E50 > /etc/apt/trusted.gpg.d/signal18.gpg

# Add repository
echo "deb [signed-by=/etc/apt/trusted.gpg.d/signal18.gpg] http://repo.signal18.io/deb $(lsb_release -sc)
  > /etc/apt/sources.list.d/signal18.list

# Install
apt-get update
apt-get install replication-manager-osc
```

Replace `replication-manager-osc` with your chosen flavor (`replication-manager-pro`, `replication-manager-arb`).

2.1.4.4 Post-Installation

After package installation:

- Configuration template: `/etc/replication-manager/config.toml`
- Cluster configuration directory: `/etc/replication-manager/cluster.d/`
- Static assets: `/usr/share/replication-manager/`
- Data directory: `/var/lib/replication-manager/`
- Log file: `/var/log/replication-manager.log`

Enable and start the service:

```
systemctl enable --now replication-manager
```

Proceed to Configuration to set up your first cluster.

2.1.5.1.1 Setup OpenSVC Cluster Agents

Install the agents on the nodes of your provisioning cluster.

A minimal set of tools are advices for better file system support, virtualization of micro services and network debugging purpose.

Download

Supposing the package is installed :

Since Agent version 1.9 the cluster need to be joined

Setup Heartbeat on the donor:

```
# nodemgr set --param hb#1.type --value unicast
# nodemgr get --kw cluster.secret
3665ab8630e011e8ab20525400e412aa
```

On the joiner:

```
# nodemgr daemon join --node=node-1-1.vdc.opensvc.com --secret 3665ab8630e011e8ab20525400e412aa
```

###Debian, Ubuntu:

```
apt-get install -y python net-tools docker-io psmisc zfsutils-linux system-config-lvm xfsprogs wget
```

###Centos, RHEL:

```
sudo yum install -y net-tools docker bridge-utils git runc wget bind-utils

# bugfixes
cd /usr/libexec/docker/
sudo ln -s docker-runc-current docker-runc
echo "PATH=\$PATH:/usr/libexec/docker" >>/etc/sysconfig/opensvc
sed -i -e "s/: false/: true/" /etc/oci-register-machine.conf
```

Installing ZFS

Agent may require to communicate via root together setup a no password ssh access

```
sudo sed -i -e "s/^PasswordAuthentication no/PasswordAuthentication yes/" /etc/ssh/sshd_config
sudo systemctl restart sshd
```

You can verify that the agent is discovered by going to the web interface of replication-manager and check the agents tab.

2.1.5.1.1.1 Using Overlay Networking

Centos & Rehdat requires EPEL

```
yum install epel-release
yum install containernetworking-cni
```

Binary Install

```
cd /tmp
wget https://github.com/containernetworking/cni/releases/download/v0.6.0/cni-amd64-v0.6.0.tgz
wget https://github.com/containernetworking/plugins/releases/download/v0.6.0/cni-plugins-amd64-v0.6.0.tg
sudo mkdir -p /opt/cni/bin
cd  /opt/cni/bin
tar xvf /tmp/cni-amd64-v0.6.0.tgz
tar xvf /tmp/cni-plugins-amd64-v0.6.0.tgz
```

```
mkdir -p /opt/cni/net.d
```

Install one overlay CNI plugin like weave network

```
curl -L git.io/weave -o /usr/local/bin/weave
chmod a+x /usr/local/bin/weave
```

Make sure Docker daemon is started at boot and disable MountFlags:

```
sed -i s/^MountFlags=slave/#MountFlags=slave/ /lib/systemd/system/docker.service
systemctl enable docker
systemctl start docker
```

For each node

```
weave setup
weave launch $(nodemgr get --kw cluster.nodes)

mkdir -p /var/lib/opensvc/cni/net.d/
cat > /var/lib/opensvc/cni/net.d/repman.conf <<EOF
{
    "cniVersion": "0.2.0",
    "name": "repman",
    "type": "weave-net"
}
EOF
```

Make sure the OpenSVC Cluster is defined and joined before the next step Instruct the OpenSVC agent about cni path:

```
nodemgr set --kw cni.plugins=/usr/libexec/cni
nodemgr set --kw cni.config=/var/lib/opensvc/cni/net.d
```

Check cni from agent

```
$ nodemgr network ls
repman
$ nodemgr network show --id  repman
node-1-2.vdc.opensvc.com
`- repman
   |- cniVersion                0.2.0
   |- type                      weavenet
   `- name                      repman
```

Check from weave

```
$ weave report
```

Setup the DNS as a docker service on each agent

```
svcmgr create -s odns  --config http://www.opensvc.com/init/static/templates/odns.conf --provision
```

2.1.5.1.1.2 Upgrade agent version

```
nodemgr set --kw node.repopkg=https://repo.opensvc.com
nodemgr updatepkg
```

2.1.6.1 Install from Docker

Official Signal18 container images are available on Docker Hub. Images bundle all external dependencies so no separate tool installation is required on the host.

2.1.6.2 Image Tags

Tag	Flavor	Description
`3.1`	osc	Open Source edition. Minimal dependencies, `prov-orchestrator = "onpremise"`.
`3.1-pro`	pro	Production edition with full dependencies. `prov-orchestrator = "opensvc"`. Includes MariaDB client tools, ProxySQL, MyDumper, HAProxy, Restic, Sysbench, Grafana.
`nightly`	pro	Latest develop-branch build. For testing new features before release.
`dev`	—	Build environment with Go toolchain. No default entrypoint — used for building from source inside a container.

Rootless variants (run as non-root user `repman`, UID/GID 10001:10001) are available with the `-rootless` suffix: `3.1-rootless`, `3.1-pro-rootless`.

2.1.6.3 What is Included

Standard image (`3.1`) — minimal: - `replication-manager-osc` binary - `replication-manager-cli` - `gotty-client` (terminal-in-browser) - MariaDB repository configured (client tools not installed)

Pro image (`3.1-pro`) — full dependency set:

Tool	Version	Purpose
`mariadb` / `mariadb-dump` / `mariadb-binlog`	11.4 LTS	Database client, logical backups, binlog processing
`mariadb-plugin-spider`	11.4	Spider sharding proxy
`mydumper` / `myloader`	0.17	Fast parallel logical backup
`restic`	latest	Snapshot-based backup archiving
`haproxy`	distro	TCP/HTTP load-balancing proxy
`proxysql`	2.7.3	Advanced MySQL-protocol proxy
`sysbench`	distro	Benchmarking and load injection
`grafana`	8.1	Metrics dashboards
`gotty-client`	1.10	Browser terminal sessions
`openssh-client`	distro	Remote scripting via SSH

Tool	Version	Purpose
`fuse`	distro	Restic FUSE mount for backup browsing

2.1.6.4 Quick Start

2.1.6.4.1 Step 1 — Create directory structure

```
mkdir -p ~/repman/etc/cluster.d ~/repman/data ~/repman/config
```

2.1.6.4.2 Step 2 — Download configuration templates

```
# Global configuration
curl -o ~/repman/etc/config.toml \
  https://raw.githubusercontent.com/signal18/replication-manager/refs/heads/3.1/etc/config.toml

# Cluster configuration sample
curl -o ~/repman/etc/cluster.d/cluster1.toml.sample \
  https://raw.githubusercontent.com/signal18/replication-manager/refs/heads/3.1/etc/cluster.d/cluster1.t
```

2.1.6.4.3 Step 3 — Configure your first cluster

Edit `~/repman/etc/cluster.d/cluster1.toml.sample`:

```
[my-cluster]
title = "my-cluster"
prov-orchestrator = "onpremise"
db-servers-hosts = "db1:3306,db2:3306"
db-servers-prefered-master = "db1:3306"
db-servers-credential = "root:my-password"
replication-credential = "repl:repl-password"
```

Rename the file to match the cluster name:

```
mv ~/repman/etc/cluster.d/cluster1.toml.sample ~/repman/etc/cluster.d/my-cluster.toml
```

2.1.6.4.4 Step 4 — Start the container

```
docker run \
  -v ~/repman/etc:/etc/replication-manager:rw \
  -v ~/repman/data:/var/lib/replication-manager:rw \
  -v ~/repman/config:/root/.config/replication-manager:rw \
  -p 443:10005 \
  -p 80:10001 \
  --name replication-manager \
  --detach \
  signal18/replication-manager:3.1
```

> The third volume mount (`/root/.config/replication-manager`) is required in v3.x to persist dynamic configuration changes made via the API or GUI. Without it, changes are lost on container restart.

2.1.6.4.5 Volume Mounts

Host path	Container path	Contents
`~/repman/etc`	`/etc/replication-manager`	`config.toml` and `cluster.d/`
`~/repman/data`	`/var/lib/replication-manager`	Runtime data, metrics, backups
`~/repman/config`	`/root/.config/replication-manager`	Dynamic config (v3+ required)

2.1.6.4.6 Exposed Ports

Port	Protocol	Description
`10001`	HTTP	Web interface (unencrypted)
`10005`	HTTPS	Web interface and REST API
`10002-10004`	TCP	Graphite & pickle API (optional)

2.1.6.5 Rootless Images

Rootless images run as the `repman` user (UID/GID 10001:10001). Before starting a rootless container, set ownership of the mounted directories:

```
sudo chown -R 10001:10001 ~/repman/etc ~/repman/data ~/repman/config
```

```
docker run \
  --user repman \
  -v ~/repman/etc:/etc/replication-manager:rw \
  -v ~/repman/data:/var/lib/replication-manager:rw \
  -v ~/repman/config:/root/.config/replication-manager:rw \
  -p 443:10005 \
  -p 80:10001 \
  --name replication-manager \
  --detach \
  signal18/replication-manager:3.1-rootless
```

2.1.6.6 Development Container

The `dev` image includes Go 1.24 and build tooling. Use it to build replication-manager from source inside a container:

```
docker run \
  --user=124:132 \
  -e HOME=/go/src/github.com/signal18/replication-manager \
  --detach --interactive --tty \
  --name=repman-dev \
  -v /home/dev/replication-manager:/go/src/github.com/signal18/replication-manager:rw \
  -v /home/dev/etc/replication-manager:/etc/replication-manager:rw \
  -v /home/dev/data:/var/lib/replication-manager:rw \
  signal18/replication-manager:dev \
  /bin/bash
```

2.1.6.7 Logs

Container logs are written to `/var/log/replication-manager.log` inside the container. To follow them:

```
docker exec replication-manager tail -f /var/log/replication-manager.log
```

2.1.9.1 Install from Source

Building from source requires Go 1.24 or later and Node.js 22 (for the React dashboard).

2.1.9.2 Clone and Build

```
git clone https://github.com/signal18/replication-manager.git \
  ~/go/src/github.com/signal18/replication-manager
```

```
cd ~/go/src/github.com/signal18/replication-manager
```

Build all binary variants:

```
make bin
```

Build the React dashboard (required for `osc`, `pro`, and embedded binaries):

```
make react
```

2.1.9.3 Build Specific Flavors

```
make osc        # Open Source Community - no provisioning
make pro        # Pro - with OpenSVC provisioning
make tst        # Test - with Sysbench and regression testing
make arb        # Arbitrator
make emb        # Embedded binary (dashboard built in)
make cli        # CLI client only
```

2.1.9.4 Build Packages

```
make package
```

This runs `package_linux.sh` and produces RPM and DEB packages in `build/`.

2.1.9.5 Build Tags

The build system uses Go compile-time feature flags (`-X` linker flags) to control which features are compiled into each binary. Key flags:

Flag	Effect
`WithProvisioning`	Enable OpenSVC/Kubernetes cluster provisioning
`WithArbitration`	Enable arbitration client
`WithProxysql`	Enable ProxySQL integration
`WithHaproxy`	Enable HAProxy integration
`WithMaxscale`	Enable MaxScale integration
`WithOpenSVC`	Enable OpenSVC collector API
`WithEmbed`	Embed dashboard and assets in binary

See the `Makefile` for the full flag sets used per flavor.

2.1.9.6 Running from Source

After building, binaries are written to `build/binaries/`. The embedded binary can be run directly:

```
build/binaries/replication-manager monitor \
  --config ./etc/local/config.toml \
  --http-server
```

2.1.9.7 Continuous Integration

Development builds from the `develop` branch are available on the Signal18 CI Server.

2.1.10.1 Orchestrators (Pro)

> **Requires:** `replication-manager-pro` flavor

The **pro** flavor of replication-manager can provision and manage the full lifecycle of database clusters — bootstrapping containers, deploying configuration, starting/stopping services — by delegating to an external orchestrator.

Set the active orchestrator with:

```
prov-orchestrator = "opensvc"   # opensvc | kube | slapos | onpremise | local
```

Orchestrator	`prov-orchestrator` value	Description
OpenSVC	`opensvc`	Service orchestration cluster — the primary supported orchestrator. Default for `pro`.
Kubernetes	`kube`	Container orchestration via a K8s cluster.
SlapOS	`slapos`	Distributed service mesh from Nexedi.
On-Premise SSH	`onpremise`	SSH-based bootstrap onto existing bare-metal or VM hosts. No orchestrator agent required.
Local	`local`	Bootstrap processes directly on the same host as replication-manager. For testing only.

`osc` supports `onpremise` and `local` only. All five modes are available in `pro`.

2.1.10.2 OpenSVC

OpenSVC is a service orchestration cluster that manages Docker/Podman micro-services across a pool of agents. replication-manager uses the OpenSVC API to push service definitions and trigger provisioning.

2.1.10.2.1 Install the OpenSVC Agent

On each database host, install the OpenSVC agent:

```
# Debian/Ubuntu
curl -o /tmp/opensvc.latest.deb https://repo.opensvc.com/deb/current
dpkg -i /tmp/opensvc.latest.deb

# RHEL/CentOS
curl -o /tmp/opensvc.latest.rpm https://repo.opensvc.com/rpm/current
rpm -ivh /tmp/opensvc.latest.rpm
```

Start and join the cluster:

```
om daemon start
om cluster set --param cluster.secret --value <shared-secret>
om daemon join --node <first-node-ip>
```

Full OpenSVC documentation: docs.opensvc.com

2.1.10.2.2 Configure replication-manager for OpenSVC

replication-manager can use either the **Cluster API** (recommended, direct agent HTTP2 API) or the legacy **Collector API** (Signal18 SaaS or on-premise collector).

2.1.10.2.2.1 Cluster API (recommended) Point replication-manager at any OpenSVC cluster node:

```
prov-orchestrator           = "opensvc"
opensvc-cluster-host        = "opensvc-node1:1215"
opensvc-cluster-admin-user  = "root@localhost.localdomain:opensvc"
```

2.1.10.2.2.2 Collector API (legacy)

```
prov-orchestrator    = "opensvc"
opensvc-host         = "collector.mycompany.com:9443"
opensvc-admin-user   = "root@mycompany.com:secret"
```

The Signal18 SaaS collector (`ci.signal18.io:9443`) can be used for evaluation — contact Signal18 to obtain your `account.yaml` and place it in the `share/opensvc/` directory of replication-manager-pro.

2.1.10.2.3 OpenSVC Video Walkthrough

plugin:youtube

2.1.10.3 Kubernetes

replication-manager can deploy database services into a Kubernetes cluster using a standard kubeconfig file.

2.1.10.3.1 Prerequisites

- A running Kubernetes cluster (1.20+)
- `kubectl` configured and working on the replication-manager host
- Sufficient RBAC permissions to create namespaces, deployments, services, and PersistentVolumeClaims

2.1.10.3.2 Configure replication-manager for Kubernetes

```
prov-orchestrator = "kube"
kube-config       = "/etc/replication-manager/kube/config"
```

Copy your cluster's kubeconfig to the path referenced above. replication-manager uses the default context from the file.

```
cp ~/.kube/config /etc/replication-manager/kube/config
chmod 600 /etc/replication-manager/kube/config
```

2.1.10.4 SlapOS

SlapOS is a distributed service mesh used in the Nexedi ecosystem. replication-manager-pro can deploy database instances as SlapOS services.

2.1.10.4.1 Configure replication-manager for SlapOS

```
prov-orchestrator = "slapos"
```

Contact Signal18 for SlapOS deployment templates specific to your environment.

2.1.10.5 On-Premise SSH

The `onpremise` mode bootstraps database configuration and services on existing hosts via SSH — no orchestrator agent required. replication-manager connects as a sudoer user and deploys configuration files, starts/stops services using the host's init system.

2.1.10.5.1 Prerequisites

- SSH access from the replication-manager host to each database host
- A user with passwordless sudo on the database hosts
- MariaDB or MySQL server packages already installed on the database hosts

2.1.10.5.2 Configure SSH access

```
prov-orchestrator         = "onpremise"
onpremise-ssh             = true
onpremise-ssh-port        = 22
onpremise-ssh-credential  = "deploy"
```

replication-manager uses its own SSH private key (`~/.ssh/id_rsa` by default). Add the corresponding public key to the `authorized_keys` of the `deploy` user on each database host.

2.1.10.6 Local (Testing Only)

The `local` orchestrator boots MariaDB or MySQL processes directly on the same host as replication-manager. It is intended for local regression testing and benchmarking — not for production use.

2.1.10.6.1 Prerequisites

- MariaDB or MySQL server binaries installed locally
- HAProxy and/or ProxySQL if proxy testing is needed
- Sysbench for load injection

```
prov-orchestrator       = "local"
prov-db-binary-basedir  = "/usr/bin"
haproxy-binary-path     = "/usr/sbin/haproxy"
proxysql-binary-path    = "/usr/bin/proxysql"
```

2.1.10.7 Next Steps

After installing and configuring the orchestrator, proceed to Provisioning Configuration to define service templates and bootstrap your first managed cluster.

2.1.11.1 Install Dependencies

replication-manager is a self-contained binary — no system libraries are required at the OS level. However, several external tools are needed to enable specific features such as backup, proxy management, load testing, and terminal sessions.

replication-manager **auto-detects** all external tools at startup by searching `PATH`. If a tool is found, its version is logged and used automatically. If a tool is not found, the related feature is disabled and a `WARN` state is set in the cluster dashboard. Paths can always be overridden explicitly in the cluster configuration.

2.1.11.2 Supported Database Versions

Database	Minimum version	Notes
MariaDB	10.0	GTID recommended (10.0.2+)
MySQL	5.6	GTID recommended (5.6+)
Percona Server	5.6	Treated as MySQL-compatible

GTID-based replication is strongly recommended for reliable failover. Non-GTID topologies are supported with limitations.

2.1.11.3 External Tools

2.1.11.3.1 Auto-Detected Tool Matrix

Tool	Feature	Auto-detect order	Config key to override
`mariadb` / `mysql`	SQL client, connection checks	`mariadb` → `mysql` → embedded	`backup-mysqlclient-path`
`mariadb-dump` / `mysqldump`	Logical backups	`mariadb-dump` → `mysqldump` → embedded	`backup-mysqldump-path`
`mariadb-binlog` / `mysqlbinlog`	Binlog processing and pitr	`mariadb-binlog` → `mysqlbinlog` → embedded	`backup-mysqlbinlog-path`
`mydumper`	Fast parallel logical backup	`mydumper` → embedded	`backup-mydumper-path`
`myloader`	Fast parallel restore	`myloader` → embedded	`backup-myloader-path`
`restic`	Snapshot backup archiving	configured path only	`backup-restic-binary-path`
`haproxy`	TCP/HTTP proxy	configured path	`haproxy-binary-path`
`proxysql`	MySQL-protocol proxy	configured path	`proxysql-binary-path`
`maxscale`	MaxScale proxy	configured path	`maxscale-binary-path`
`sysbench`	Benchmarking / load injection	`sysbench` in PATH	`sysbench-binary-path`
`gotty-client`	Browser terminal sessions	`gotty-client` → embedded	`backup-gotty-client-path`
`fusermount` / `fusermount3`	Restic FUSE mount for backup browsing	`fusermount3` → `fusermount`	—
`ssh`	Remote scripting (onpremise orchestrator)	system `openssh-client`	—

Embedded fallback: For `mariadb`/`mysql`, `mariadb-dump`/`mysqldump`, `mariadb-binlog`/`mysqlbinlog`, `mydumper`, `myloader`, and `gotty-client`, replication-manager includes compiled-in versions under `<share>/<arch>/<os>/`. These are used automatically when no system binary is found — no installation required for basic operation.

2.1.11.3.2 Version Monitoring

replication-manager actively tracks tool versions at runtime. On every monitoring cycle it runs `--version` on each configured tool and logs any version change as `discovered` or `changed`. If a tool is missing or its version cannot be parsed, the corresponding `WARN` state is raised:

State	Tool
`WARN0117`	Database client (`mariadb` / `mysql`) not found or version unreadable
`WARN0118`	Dump client (`mariadb-dump` / `mysqldump`) not found or version unreadable
`WARN0119`	Binlog client (`mariadb-binlog` / `mysqlbinlog`) not found or version unreadable
`WARN0120`	MyDumper not found or version unreadable
`WARN0121`	Restic not found or version unreadable
`WARN0167`	Sysbench not found or version unreadable

These warnings appear in the cluster dashboard and clear automatically when the tool becomes available.

2.1.11.4 Per-Feature Dependencies

2.1.11.4.1 Logical Backups

```
# MariaDB client tools (provides mariadb, mariadb-dump, mariadb-binlog)
apt-get install mariadb-client      # Debian/Ubuntu
yum install MariaDB-client          # RHEL/CentOS

# Or MySQL client tools
apt-get install mysql-client
```

2.1.11.4.2 Fast Parallel Backups (MyDumper)

```
# Debian/Ubuntu - from MariaDB repository or GitHub releases
apt-get install mydumper

# Or download from https://github.com/mydumper/mydumper/releases
```

2.1.11.4.3 Snapshot Backups (Restic)

```
apt-get install restic      # Debian/Ubuntu
yum install restic          # RHEL/CentOS
# Or: https://restic.net/
```

Configure Restic in the cluster TOML:

```
backup-restic               = true
backup-restic-binary-path   = "/usr/bin/restic"
backup-restic-repository    = "/var/lib/replication-manager/backup/restic"
backup-restic-password      = "your-repo-password"
```

2.1.11.4.4 HAProxy

```
apt-get install haproxy     # Debian/Ubuntu
yum install haproxy         # RHEL/CentOS
```

```
haproxy-binary-path = "/usr/sbin/haproxy"
```

2.1.11.4.5 ProxySQL

Download from github.com/sysown/proxysql/releases:

```
# Debian/Ubuntu example
curl -LO https://github.com/sysown/proxysql/releases/download/v2.7.3/proxysql_2.7.3-debian12_amd64.deb
dpkg -i proxysql_2.7.3-debian12_amd64.deb
```

```
proxysql-binary-path = "/usr/bin/proxysql"
```

2.1.11.4.6 Sysbench (Benchmarking)

```
apt-get install sysbench    # Debian/Ubuntu
yum install sysbench        # RHEL/CentOS
```

```
sysbench-binary-path = "/usr/bin/sysbench"
```

2.1.11.4.7 Restic FUSE Mount (Backup Browsing)

```
apt-get install fuse        # provides fusermount / fusermount3
```

2.1.11.4.8 Remote Scripting (On-Premise Orchestrator)

```
apt-get install openssh-client
```

2.1.11.5 Docker Image Reference

The **pro** Docker image (`signal18/replication-manager:3.1-pro`) bundles all of the above. Use it as a reference for what a fully-equipped deployment looks like:

```
mariadb-client 11.4  (mariadb, mariadb-dump, mariadb-binlog, mariadb-plugin-spider)
mydumper 0.17 + myloader
restic (latest apt)
haproxy (distro)
proxysql 2.7.3
sysbench (distro)
grafana 8.1
gotty-client 1.10
openssh-client
fuse (fusermount3)
```

The **standard** image (`signal18/replication-manager:3.1`) installs none of these — it relies on the embedded fallback binaries for basic SQL client operations and `prov-orchestrator = "onpremise"` for any provisioning needs.

2.1.11.6 Overriding Detected Paths

If a tool is installed in a non-standard location, or you want to pin a specific version, set the path explicitly in the cluster TOML:

```
[mycluster]
backup-mysqlclient-path   = "/opt/mariadb/bin/mariadb"
backup-mysqldump-path     = "/opt/mariadb/bin/mariadb-dump"
backup-mysqlbinlog-path   = "/opt/mariadb/bin/mariadb-binlog"
backup-mydumper-path      = "/opt/mydumper/bin/mydumper"
backup-myloader-path      = "/opt/mydumper/bin/myloader"
backup-restic-binary-path = "/opt/restic/restic"
haproxy-binary-path       = "/opt/haproxy/sbin/haproxy"
proxysql-binary-path      = "/opt/proxysql/bin/proxysql"
sysbench-binary-path      = "/opt/sysbench/bin/sysbench"
```

When an explicit path is set, auto-detection is bypassed for that tool.

2.1.12.1 What Was Installed

This page describes the files and directories created by each installation method.

2.1.12.2 Package Installation (RPM / DEB)

Installing via the Signal18 repository creates the following layout:

Path	Contents
`/etc/replication-manager/config.toml`	Main configuration file (template on first install)
`/etc/replication-manager/cluster.d/`	Per-cluster TOML files (included via `include =` directive)
`/usr/share/replication-manager/`	Static assets, dashboard, templates, and test files
`/usr/share/replication-manager/dashboard/`	HTTP server root — the React web GUI
`/usr/share/replication-manager/tests/`	Non-regression test files and example MariaDB config files
`/var/lib/replication-manager/`	Runtime data: metrics, backup state, bootstrap data
`$HOME/.config/replication-manager/`	Dynamic config changes saved via API or GUI (v3+)
`/var/log/replication-manager.log`	Log file
`/usr/bin/replication-manager`	Server binary
`/usr/bin/replication-manager-cli`	CLI client binary

The `/etc/replication-manager/` directory is the **template** source. On first startup, v3 copies it to `$HOME/.config/replication-manager/` which becomes the **active** configuration. Changes made via the API or GUI are written to the active directory, not back to `/etc/`.

2.1.12.3 Tarball Installation (basedir)

Tarball releases extract to a self-contained directory tree:

Path	Contents
`/usr/local/replication-manager/etc/`	Configuration files and samples
`/usr/local/replication-manager/share/`	Static assets and dashboard
`/usr/local/replication-manager/share/dashboard/`	React web GUI
`/usr/local/replication-manager/data/`	Runtime data
`/usr/local/replication-manager/bin/replication-manager`	Server binary
`/usr/local/replication-manager/bin/replication-manager-cli`	CLI client binary

A symlink is typically created:

```
ln -s /usr/local/replication-manager-osc-3.1.24 /usr/local/replication-manager
```

2.1.12.4 Embedded Binary

The embedded binary installs as a single file. All assets (dashboard, templates, scripts) are compiled into the binary itself — no `share/` directory is created. The working directories are created at first run:

Path	Contents
`/usr/local/bin/replication-manager`	The binary (assets embedded)
`$HOME/.config/replication-manager/`	Dynamic configuration (created at first start)

Configuration and data directory locations can be overridden:

```
[Default]
monitoring-datadir  = "/var/lib/replication-manager"
monitoring-sharedir = "/usr/share/replication-manager"
```

2.1.12.5 Docker

Inside the container the layout matches the package installation:

Container path	Description
`/etc/replication-manager/config.toml`	Template configuration (from image)
`/etc/replication-manager/cluster.d/`	Cluster configs (mount from host)
`/var/lib/replication-manager/`	Runtime data (mount from host)
`/root/.config/replication-manager/`	Active dynamic config (mount from host, v3+)
`/var/log/replication-manager.log`	Log file
`/usr/bin/replication-manager`	Server binary
`/usr/bin/replication-manager-cli`	CLI client binary
`/usr/share/replication-manager/`	Static assets

All three directories should be mounted from the host so that data survives container restarts. See Install from Docker for the full `docker run` command.

2.1.12.6 Configuration File Search Order

At startup replication-manager searches for `config.toml` in this order:

1. Path specified by `--config` flag
2. `$HOME/.config/replication-manager/config.toml`
3. `/etc/replication-manager/config.toml`
4. `/usr/local/replication-manager/etc/config.toml`
5. `./config.toml` (current directory)

The first file found wins. The `--config` flag is the recommended way to specify a non-default location.

2.3.1 First Config

replication-manager uses a TOML configuration file. The default file is `config.toml`, split into a mandatory `[Default]` section and one or more named cluster sections.

2.3.2 Config File Structure

```toml
[Default]
# Global settings - applied to all clusters
monitoring-save-config = true
include = "/etc/replication-manager/cluster.d"

[cluster1]
# Settings for this cluster only
title                        = "cluster1"
prov-orchestrator            = "onpremise"
db-servers-hosts             = "10.0.1.10:3306,10.0.1.11:3306"
db-servers-prefered-master   = "10.0.1.10:3306"
db-servers-credential        = "root:password"
replication-credential       = "repl_user:repl_password"
```

`[Default]` is required. Variables placed here are enforced across all clusters — use it for instance-level settings like data directories, log paths, API credentials, and the `include` pointer. Most operational settings should go in a named cluster section so they can differ per cluster.

A single `[Default]` section with no separate cluster section is valid for single-cluster deployments.

2.3.3 Config File Search Order

replication-manager searches for `config.toml` in this order:

1. Path specified by `--config` flag
2. `$HOME/.config/replication-manager/config.toml`
3. `/etc/replication-manager/config.toml`
4. `/usr/local/replication-manager/etc/config.toml`
5. `./config.toml` (current directory)

The first file found wins.

2.3.4 Using the Include Directory

For multi-cluster setups, split configuration across individual files:

`/etc/replication-manager/config.toml` — global settings only:

```toml
[Default]
monitoring-save-config = true
include = "/etc/replication-manager/cluster.d"
http-server = true
http-bind-address = "0.0.0.0"
```

`/etc/replication-manager/cluster.d/production.toml` — one file per cluster:

```toml
[production]
title                        = "Production"
prov-orchestrator            = "onpremise"
db-servers-hosts             = "db1:3306,db2:3306,db3:3306"
db-servers-prefered-master   = "db1:3306"
db-servers-credential        = "root:prod-password"
replication-credential       = "repl:repl-password"
```

All `*.toml` files in the include directory are merged with the main config at startup.

2.3.5 Dynamic Configuration

When `monitoring-save-config = true` is set in `[Default]`, changes made via the API or GUI are persisted to the **active configuration directory**:

`$HOME/.config/replication-manager/<cluster-name>/config.toml`

This is separate from `/etc/replication-manager/` so the template config is never overwritten by runtime changes.

2.3.6 Minimal Working Config

```toml
[Default]
monitoring-save-config = true
http-server            = true
http-bind-address      = "0.0.0.0"
api-credentials        = "admin:change-me,dba:change-me"

[mycluster]
title                        = "mycluster"
prov-orchestrator            = "onpremise"
db-servers-hosts             = "db1:3306,db2:3306"
db-servers-prefered-master   = "db1:3306"
db-servers-credential        = "root:db-password"
replication-credential       = "repl:repl-password"
```

Change `api-credentials` before first start to avoid the WARN0108 default password warning.

2.3.7 Sample Configuration Files

Package installations include ready-to-use samples:

```bash
# Package installation - list available samples
ls /etc/replication-manager/*.sample*

# Copy a master-replica + HAProxy sample
cp /etc/replication-manager/config.toml.sample.masterslave-haproxy \
   /etc/replication-manager/config.toml

# Tarball installation
cp /usr/local/replication-manager/etc/config.toml.sample.masterslave-haproxy \
   /usr/local/replication-manager/etc/config.toml
```

2.3.8 Starting replication-manager

```
# Package / systemd
systemctl enable --now replication-manager

# Tarball
/usr/local/replication-manager/bin/replication-manager monitor \
  --config /usr/local/replication-manager/etc/config.toml \
  --http-server

# Embedded binary - reads config.toml from current directory or ~/.config
replication-manager monitor --http-server
```

2.3.9 Command Line Flags

Every configuration key has a corresponding command line flag. Flags on the command line take priority over the config file.

The `--cluster` flag restricts monitoring to a subset of the clusters defined in the config file:

```
replication-manager monitor --cluster production,staging
```

Key startup flags:

Flag	Default	Description
`--config`	`/etc/replication-manager/config.toml`	Path to the configuration file
`--cluster`	*(all)*	Comma-separated list of cluster sections to load
`--http-server`	`false`	Start the HTTP/HTTPS web interface
`--monitoring-datadir`	`/var/lib/replication-manager`	Runtime data directory
`--monitoring-confdir`	`/etc/replication-manager`	Configuration directory
`--monitoring-sharedir`	`/usr/share/replication-manager`	Static assets directory
`--monitoring-save-config`	`true`	Persist API/GUI changes to datadir
`--api-credentials`	`admin:repman`	REST API users in `user:password` format
`--api-port`	`10005`	HTTPS API and web GUI port

> Command line flags are overridden by variables found in the `config.toml` file — the config file takes precedence over flags.

2.3.10 Discovering All Available Flags

> **Tip:** `replication-manager monitor --help` is the authoritative reference — it prints all 400+ flags with defaults and descriptions for the exact version you have installed, including anything added after this documentation was written.

```
replication-manager monitor --help
```

Each flag name maps directly to a TOML key — strip the `--` prefix:

```
# Flag form:         --monitoring-ticker 2
# TOML equivalent:  monitoring-ticker = 2
```

Filter by feature area with `grep`:

```
replication-manager monitor --help | grep backup
replication-manager monitor --help | grep proxy
replication-manager monitor --help | grep failover
replication-manager monitor --help | grep ssl
```

2.5.1 First Login

After starting replication-manager for the first time, access the web GUI at `https://<host>:10005` (or `https://localhost:10005` during local setup).

2.5.2 Local Users

replication-manager ships with two built-in local accounts:

Username	Default password	Default role
`admin`	`repman`	Full access — all cluster, proxy, provisioning, and global settings
`dba`	`repman`	Database operations — no provisioning, no global settings, no user management

These accounts are defined by the `api-credentials` configuration key:

```
api-credentials = "admin:repman,dba:repman"
```

or via command-line flag:

```
replication-manager monitor --api-credentials "admin:repman,dba:repman"
```

Change the default passwords immediately after first login. Accounts using the default password `repman` trigger a cluster warning (see below).

2.5.3 Changing Passwords

Passwords can be changed from the **Settings → Users** panel in the GUI, or by updating `api-credentials` in the cluster TOML file and reloading.

Each entry in `api-credentials` follows the format `username:password`. Multiple accounts are separated by commas.

2.5.4 Default Password Warning — WARN0108

replication-manager continuously monitors whether the built-in accounts still use their default passwords. When either `admin` or `dba` retains the password `repman`, the cluster enters state:

```
WARN0108: Default users still use default password.
          Please change the credentials for users: (admin,dba)
```

This warning appears in the cluster dashboard and in the main HA log. It clears automatically once the passwords are updated.

The check is performed by `CheckDefaultUser()` on every monitoring tick — no restart is required after changing passwords.

2.5.5 SSO Login via gitlab.signal18.io

Once your instance is registered, users can log in using their **gitlab.signal18.io** identity instead of a local password. SSO login does not require a local `api-credentials` entry — authentication is handled by the Signal18 GitLab OAuth flow.

To log in with SSO:

1. Click **Sign in with GitLab** on the replication-manager login page
2. Authenticate at gitlab.signal18.io (your Signal18 SSO account)
3. replication-manager validates your identity against the GitLab group membership for your registered instance namespace

The role and grants assigned to an SSO user are managed entirely through the replication-manager GUI and API — not through GitLab group roles. See Cluster Role Sharing for the full role and grant model.

2.5.6 Summary

Step	Action
1	Open `https://<host>:10005`
2	Log in as `admin` / `repman`
3	Change the `admin` and `dba` passwords — clears WARN0108
4	(Optional) Register the instance at gitlab.signal18.io to enable SSO and community plugins

2.7.1 Instance Registration & SSO

Community and Enterprise plugins require your replication-manager instance to be **registered** with the Signal18 SSO at gitlab.signal18.io.

Registration is free. It takes less than two minutes and unlocks:

- **Community plugins** — the full library of workload, security, and score plugins, kept up to date automatically
- **Config backup & restore** — all cluster configurations versioned in GitLab, recoverable on any new replication-manager instance in one command
- **Cluster role sharing** — grant scoped access to any cluster to other registered SSO users without sharing credentials or VPN
- **Direct chat** — a built-in chat panel in the replication-manager GUI for real-time conversation with the Signal18 team and marketplace partners
- **Cloud18 marketplace** — consume clusters provided by other participants or publish your own; become an active Signal18 partner or customer

---

2.7.2 Registering Your Instance

```
# 1. Create an account at gitlab.signal18.io if you don't have one

# 2. Register the instance and obtain your SSO token
replication-manager instance-register \
    --gitlab-token <your-gitlab-personal-access-token> \
    --subdomain <your-chosen-subdomain>

# 3. replication-manager writes the issued token automatically to:
#   monitoring-instance-token = "eyJ..."
```

After registration, restart replication-manager. Community plugins are downloaded on the first startup and updated automatically on subsequent reloads.

2.7.3 Community Plugin Access

Once registered, replication-manager downloads the community plugin manifest on startup and keeps the plugin library up to date automatically. No manual binary management is required.

See Plugins for the full plugin reference — architecture, workload plugins, security plugins, score plugins, and how to write your own.

2.7.4 Configuration Backup and Restore

Every cluster configuration is continuously pushed to a GitLab repository in your SSO namespace:

- **Full audit history** — every configuration change is a Git commit with author, timestamp, and diff
- **Config restore** — on a fresh host, replication-manager clones the entire working directory from GitLab and reconstructs all cluster definitions automatically
- **Disaster recovery** — topology, routing configuration, and provisioning settings all recovered from GitLab

2.7.4.0.1 Starting fresh from GitLab

```
replication-manager monitor \
  --cloud18 \
  --monitoring-restore-config-on-start \
  --cloud18-domain          <your-organisation> \
  --cloud18-sub-domain      <your-instance-subdomain> \
  --cloud18-sub-domain-zone <your-geo-zone> \
  --cloud18-gitlab-user     <your-gitlab-user> \
  --cloud18-gitlab-password <your-gitlab-password>
```

When `--monitoring-restore-config-on-start` is set, replication-manager:

1. Authenticates to gitlab.signal18.io and obtains a personal access token
2. Clones `gitlab.signal18.io/<domain>/<subdomain>-<zone>.git` into the working directory — replacing any existing local config
3. Clones the read-only pull mirror `<subdomain>-<zone>-pull.git` into `<working-dir>/.pull`
4. Clears the flag so the restore does not repeat on the next restart
5. Reads `cloud18.toml` and reconstructs all cluster definitions

Configuration files stored in GitLab contain no plaintext secrets — all sensitive values are stored in AES-encrypted form. The encryption key is generated locally and **never leaves the host**.

2.7.5 Cluster Role Sharing with External Users

Registered instances can share cluster access with other registered SSO users — Signal18 partners, support engineers, or trusted operators. Access is managed entirely through the **replication-manager API and GUI**.

2.7.5.1 Roles

Role	Who it is for	Default grant scope
`sysops`	Local infrastructure operator (owner)	All grants — full access
`dbops`	Local database operator	DB operations — no provisioning, no global settings
`extsysops`	External SysOps partner (Cloud18)	Same as `sysops` minus sales and global settings
`extdbops`	External DBOps partner (Cloud18)	DB, show, proxy, grant operations
`sponsor`	Marketplace subscriber	DB operations, show, proxy, grant, app access
`visitor`	Read-only observer	Show grants only

2.7.5.2 External Partner Lifecycle

```
Register  →  pending  →  quote  →  active  →  unsubscribed
              (waiting     (price     (full       (access
               approval)   proposed)  access)     revoked)
```

2.7.6 Direct Chat with Signal18 and Partners

replication-manager integrates a **Chat** tab in the GUI powered by **Mattermost** at meet.signal18.io. All three connection methods use your gitlab.signal18.io SSO identity — no separate Mattermost account needed:

- **replication-manager GUI** — the Chat tab authenticates automatically via your active GitLab SSO session
- **Browser** — go to meet.signal18.io and click **Sign in with GitLab**
- **Desktop / mobile client** — add `https://meet.signal18.io` as a server in the Mattermost app and sign in with GitLab OAuth

Channel	Who you reach
Support	Signal18 engineering and support team
Community	Other registered replication-manager operators
Partners	Marketplace partners you are connected to

2.7.7 GitLab Object Mapping

Each registered instance is assigned a unique namespace:

```
Instance  →  GitLab Group      gitlab.signal18.io/<subdomain>/
Cluster   →  GitLab Project    gitlab.signal18.io/<subdomain>/<cluster-name>/
Config    →  Git repository    versioned TOML files inside the project
Team      →  Group members     GitLab group membership with role assignments
```

Your subdomain maps one-to-one to your GitLab group and is the routing key for plugin delivery, configuration sync, and marketplace identity.

2.7.8 Secret Storage

All secrets (passwords, encryption keys) are **AES-encrypted** at the replication-manager host before being written to GitLab. The encryption key is generated locally (`replication-manager keygen`) and never leaves the host.

```
replication-manager host
      AES encryption key (local, never synced)


     encrypted secret → GitLab repository (ciphertext only)
```

2.7.9 Team Management

To manage who can access your instance and its clusters:

1. Go to `gitlab.signal18.io/<subdomain>` — your instance group
2. Open **Manage → Members**
3. Add, remove, or change the role of any member

Changes take effect on the next replication-manager authentication sync — no restart required.

To share a **single cluster** without instance-wide access, manage membership at the project level:

```
gitlab.signal18.io/<subdomain>/<cluster-name>/ → Manage → Members
```

2.7.10 Configuration Reference

Key	Default	Description
`monitoring-instance-token`	`""`	JWT issued at registration — identifies this instance to the Signal18 SSO
`plugin-registry-url`	`https://registry.signal18.io`	Community plugin manifest endpoint
`plugin-auto-update`	`true`	Automatically download updated community plugins on reload
`gitlab-url`	`https://gitlab.signal18.io`	GitLab instance used for config sync and team management

2.7.1.1 Plugins Overview

Available since: replication-manager **v3.1.24**

replication-manager extends its monitoring capabilities through a plugin system. On every monitoring tick each enabled plugin is invoked, receives a full snapshot of the monitored server state as JSON on stdin, and writes findings and score checks as JSON on stdout.

Plugins are organised in three tiers:

Tier	Who can use it	Requires	Examples
Static	Everyone	Nothing — bundled in the binary	`errorlog`, `slowlog`, `auditlog`
Community	Registered instances	Free account at gitlab.signal18.io	All workload, security, and score plugins
Enterprise	Support contract customers	Signal18 Support Contract	`plugin-critical-alerts`

Static plugins run in-process as Go functions. They use Graphite-backed spike detection and need no registration.

Community plugins are external binaries distributed to registered instances. Registering your instance unlocks the full community library — workload anomaly detection, security auditing, and scoring — and keeps them updated automatically. See Registration & SSO for how to register.

Enterprise plugins are developed from Signal18's field experience and are available to customers with an active Support Contract.

2.7.1.2 Enabling Plugins

The plugin subsystem is controlled by two global settings:

```
[Default]
log-plugin       = true   # master switch - enable external plugin evaluation
log-level-plugin = 0      # verbosity: 0=off … 4=debug
```

Individual plugins are enabled per cluster:

```
[mycluster.plugin-config.plugin-connection-storm]
enabled = true
sleep-ratio-threshold = 0.75
```

2.7.1.3 What Community Plugins Do

2.7.1.3.1 Workload Plugins

Detect operational anomalies — performance problems, resource saturation, and regressions — by analysing the real-time server state. Findings carry `WARN` codes routed to the main HA log.

Plugin	Finds
`plugin-innodb-corruption`	InnoDB corruption indicators in error log (WARN0300)
`plugin-slow-query-regression`	Query latency regressed vs PFS baseline (WARN0301)

Plugin	Finds
`plugin-error-storm`	Repeated error template spike (WARN0302)
`plugin-replication-lag-predictor`	DML write burst predicting future lag (WARN0303)
`plugin-full-table-scan-spike`	Full-scan ratio exceeded (WARN0304)
`plugin-metadata-lock-contention`	MDL wait duration or count exceeded (WARN0305)
`plugin-tmp-table-storm`	On-disk temporary table spike (WARN0306)
`plugin-connection-storm`	Sleep ratio or lock-wait count exceeded (WARN0307)
`plugin-privilege-escalation`	Privilege DDL by non-admin account (WARN0308)
`plugin-off-hours-access`	DB access outside business hours (WARN0309)
`plugin-binlog-cleartext-password`	Cleartext password in binlog (WARN0310)
`plugin-binlog-creditcard-leak`	Credit card PAN detected in binlog (WARN0311)

2.7.1.3.2 Security Plugins

Audit database configuration, user accounts, and activity logs for security weaknesses. Findings carry `SEC` codes and feed the **remediation engine** — many can be auto-fixed.

Plugin	Finds
`plugin-security-no-password-user`	Account with empty password (SEC0100)
`plugin-security-weak-auth`	Weak or deprecated auth plugin (SEC0101)
`plugin-security-local-infile`	`local_infile=ON` (SEC0102)
`plugin-security-hardening`	CIS Benchmark controls SEC0103–SEC0118

2.7.1.3.3 Score Plugins

Compute binary pass/fail checks that feed the **SecurityScore** gauge in the cluster dashboard.

Plugin	Scores
`plugin-score-ssl`	TLS/SSL enabled and configured
`plugin-score-encryption`	Table, binlog, and tmp encryption at rest
`plugin-score-auth`	Strong authentication plugins in use
`plugin-score-passwords`	Password validation plugins active
`plugin-score-audit`	Audit logging active
`plugin-score-network`	Network security controls
`plugin-score-lts`	Running a supported LTS version
`plugin-score-proxy`	Proxy layer present

2.7.1.4 Full Plugin Reference

For wire protocol, configuration, prerequisites, remediation engine, and developer guide see the **Plugins chapter**.

This guide covers migrating from **replication-manager** 2.x to 3.x. Version 3.0 introduced significant changes to configuration management, file locations, and default behaviors. Version 3.1 builds on these changes with additional features.

2.8.1 Pre-Migration Checklist

Before upgrading:

- ☐ Backup your configuration files from `/etc/replication-manager/`
- ☐ Document current cluster states and topologies
- ☐ Review custom scripts that interact with **replication-manager**
- ☐ Check API client compatibility if using REST API
- ☐ Verify database versions are supported (MariaDB 10.x+, MySQL 5.7+, Percona 5.7+)
- ☐ Plan maintenance window for upgrade and testing

2.8.2 Major Changes in 3.x

2.8.2.1 Configuration File Location Changes

Version 3.0 introduced a new configuration storage model:

Old (2.x):

```
/etc/replication-manager/config.toml          # Global config
/etc/replication-manager/cluster.d/*.toml     # Cluster configs
```

New (3.x):

```
/etc/replication-manager/config.toml          # Global config (read-only reference)
/root/.config/replication-manager/            # Active configuration directory
/root/.config/replication-manager/config.toml # Merged global config
/root/.config/replication-manager/clusters.d/ # Active cluster configs
```

Behavior: - On first start, 3.x copies `/etc/replication-manager/` to `~/.config/replication-manager/` - Dynamic configuration changes via API/UI are saved to `~/.config/replication-manager/` - The `/etc/` directory becomes a template for initial configuration - Git integration stores encrypted secrets in `~/.config/` directory

Migration action: - Ensure `~/.config/replication-manager/` directory exists and is writable - Review file ownership (runs as `replication-manager` user by default) - Update backup scripts to include `~/.config/replication-manager/`

2.8.2.2 Docker Volume Mapping Changes

Old (2.x):

```
docker run -v/home/repman/etc:/etc/replication-manager:rw \
           -v/home/repman/data:/var/lib/replication-manager:rw
```

New (3.x):

```
docker run -v/home/repman/etc:/etc/replication-manager:rw \
           -v/home/repman/data:/var/lib/replication-manager:rw \
           -v/home/repman/config:/root/.config/replication-manager:rw
```

Add the third volume mount for dynamic configuration persistence.

2.8.3 Deprecated Parameters

replication-manager 3.x includes automatic detection of deprecated configuration parameters and will log warnings (WARN0159, WARN0160) when found.

2.8.3.1 Parameter Renames

The following parameters were renamed. **replication-manager** supports both old and new names, but you should migrate to the new names:

Old Parameter (2.x)	New Parameter (3.x)	Notes
`monitoring-config-rewrite`	`monitoring-save-config`	Control dynamic config saves
`api-user`	`api-credentials`	Changed to credential format
`replication-master-connection`	`replication-source-name`	Multi-source replication name
`master-connection`	`replication-source-name`	Alias for above
`logfile`	`log-file`	Log file path
`wait-kill`	`switchover-wait-kill`	Switchover kill timeout
`hosts`	`db-servers-hosts`	Database server list
`hosts-tls-ca-cert`	`db-servers-tls-ca-cert`	TLS CA certificate
`hosts-tls-client-key`	`db-servers-tls-client-key`	TLS client key
`hosts-tls-client-cert`	`db-servers-tls-client-cert`	TLS client certificate
`connect-timeout`	`db-servers-connect-timeout`	Connection timeout
`rpluser`	`replication-credential`	Replication user credentials
`prefmaster`	`db-servers-prefered-master`	Preferred master server
`ignore-servers`	`db-servers-ignored-hosts`	Ignored servers list
`master-connect-retry`	`replication-master-connection-retry`	Connection retry count
`readonly`	`failover-readonly-state`	Read-only enforcement
`mdbshardproxy-hosts`	`shardproxy-servers`	Shard proxy servers
`shardproxy-hosts`	`shardproxy-servers`	Shard proxy servers
`multimaster`	`replication-multi-master`	Multi-master topology

Old Parameter (2.x)	New Parameter (3.x)	Notes
`multi-tier-slave`	`replication-multi-tier-slave`	Multi-tier topology
`pre-failover-script`	`failover-pre-script`	Pre-failover hook
`post-failover-script`	`failover-post-script`	Post-failover hook
`rejoin-script`	`autorejoin-script`	Rejoin hook script
`share-directory`	`monitoring-sharedir`	Shared files directory
`working-directory`	`monitoring-datadir`	Working data directory
`interactive`	`failover-mode`	Failover mode setting

2.8.3.2 Migration Script for Deprecated Parameters

Use this bash script to update your configuration files:

```bash
#!/bin/bash
# migrate-config-3x.sh

CONFIG_FILE="$1"

if [ -z "$CONFIG_FILE" ]; then
    echo "Usage: $0 <config-file>"
    exit 1
fi

# Create backup
cp "$CONFIG_FILE" "$CONFIG_FILE.backup-$(date +%Y%m%d)"

# Apply renames
sed -i 's/^monitoring-config-rewrite/monitoring-save-config/g' "$CONFIG_FILE"
sed -i 's/^api-user/api-credentials/g' "$CONFIG_FILE"
sed -i 's/^replication-master-connection/replication-source-name/g' "$CONFIG_FILE"
sed -i 's/^master-connection/replication-source-name/g' "$CONFIG_FILE"
sed -i 's/^logfile/log-file/g' "$CONFIG_FILE"
sed -i 's/^wait-kill/switchover-wait-kill/g' "$CONFIG_FILE"
sed -i 's/^hosts\s/db-servers-hosts /g' "$CONFIG_FILE"
sed -i 's/^hosts-tls-ca-cert/db-servers-tls-ca-cert/g' "$CONFIG_FILE"
sed -i 's/^hosts-tls-client-key/db-servers-tls-client-key/g' "$CONFIG_FILE"
sed -i 's/^hosts-tls-client-cert/db-servers-tls-client-cert/g' "$CONFIG_FILE"
sed -i 's/^connect-timeout/db-servers-connect-timeout/g' "$CONFIG_FILE"
sed -i 's/^rpluser/replication-credential/g' "$CONFIG_FILE"
sed -i 's/^prefmaster/db-servers-prefered-master/g' "$CONFIG_FILE"
sed -i 's/^ignore-servers/db-servers-ignored-hosts/g' "$CONFIG_FILE"
sed -i 's/^master-connect-retry/replication-master-connection-retry/g' "$CONFIG_FILE"
sed -i 's/^readonly\s/failover-readonly-state /g' "$CONFIG_FILE"
sed -i 's/^mdbshardproxy-hosts/shardproxy-servers/g' "$CONFIG_FILE"
```

```
sed -i 's/^shardproxy-hosts/shardproxy-servers/g' "$CONFIG_FILE"
sed -i 's/^multimaster/replication-multi-master/g' "$CONFIG_FILE"
sed -i 's/^multi-tier-slave/replication-multi-tier-slave/g' "$CONFIG_FILE"
sed -i 's/^pre-failover-script/failover-pre-script/g' "$CONFIG_FILE"
sed -i 's/^post-failover-script/failover-post-script/g' "$CONFIG_FILE"
sed -i 's/^rejoin-script/autorejoin-script/g' "$CONFIG_FILE"
sed -i 's/^share-directory/monitoring-sharedir/g' "$CONFIG_FILE"
sed -i 's/^working-directory/monitoring-datadir/g' "$CONFIG_FILE"
sed -i 's/^interactive/failover-mode/g' "$CONFIG_FILE"

echo "Configuration migrated. Backup saved to $CONFIG_FILE.backup-$(date +%Y%m%d)"
echo "Review changes and test before deploying to production"
```

2.8.4 Default Value Changes

Several parameters have new default values in 3.x:

Parameter	2.x Default	3.x Default	Impact
`monitoring-performance-schema-mutex`	false	true	Mutex monitoring enabled by default
`monitoring-performance-schema-latch`	false	true	Latch monitoring enabled by default
`monitoring-performance-schema-memory`	false	true	PFS memory monitoring enabled
`backup-restic-purge-older-on-disk-space`	N/A	true	Automatic backup purge enabled
`backup-disk-threshold-warn`	N/A	85	Warning at 85% disk usage
`backup-disk-threshold-crit`	N/A	95	Critical at 95% disk usage

Migration action: - Review monitoring overhead if performance schema monitoring causes issues - Verify disk space management aligns with your backup strategy

2.8.5 New Features Requiring Configuration

2.8.5.1 Active-Passive Topology (3.1)

If you want monitoring without automatic failover:

```
replication-active-passive = true
```

See Active-Passive Topology documentation.

2.8.5.2 Enhanced Backup Management (3.1)

Configure disk space management for restic backups:

```
backup-restic-purge-oldest-on-disk-space = true
backup-restic-purge-oldest-on-disk-threshold = 90
backup-disk-threshold-warn = 85
backup-disk-threshold-crit = 95
backup-restic-timeout = 3600
```

2.8.5.3 Transaction Monitoring (3.1)

Enable sleeping transaction detection:

```
monitoring-processlist = true
monitoring-processlist-transactions = true
monitoring-processlist-information-schema = true
```

2.8.5.4 Per-Module Log Levels (3.1)

Fine-grained logging control:

```
log-level = 2                    # Global default
log-level-backup-stream = 3      # More verbose for backups
log-level-proxy = 1              # Less verbose for proxies
log-level-heartbeat = 4          # Debug heartbeat issues
```

2.8.6 Breaking Changes

2.8.6.1 API Changes

Endpoint modifications in 3.x:

- OAuth authentication support added (optional)
- Compact JSON response format available
- Some endpoint response structures changed for consistency

Migration action: - Test API clients against 3.x before upgrading production - Update API client libraries if available - Review custom scripts using REST API

2.8.6.2 SSL/TLS Mode Changes

MySQL 8.4 compatibility required adjustments to SSL mode handling:

- Validate your `db-servers-tls-mode` setting
- Ensure SSL certificates are properly configured
- Test SSL connections after upgrade

2.8.6.3 Backup Tool Changes

Default paths for backup tools changed:

Old (2.x):

```
backup-mydumper-path = "/usr/bin/mydumper"
backup-myloader-path = "/usr/bin/myloader"
```

New (3.x):

```
backup-mydumper-path = ""  # Auto-detect from PATH
backup-myloader-path = ""  # Auto-detect from PATH
```

MariaDB tool names: - `maria-backup` renamed to `mariadb-backup` - `maria-import` renamed to `mariadb-import`

Migration action: - Update `backup-*-path` parameters if using non-standard locations - Verify backup tools are in PATH or specify full paths

2.8.7 Migration Procedure

2.8.7.1 Step 1: Prepare Environment

```
# Stop replication-manager 2.x
systemctl stop replication-manager

# Backup configuration
tar czf /tmp/repman-2x-config-$(date +%Y%m%d).tar.gz /etc/replication-manager/
tar czf /tmp/repman-2x-data-$(date +%Y%m%d).tar.gz /var/lib/replication-manager/

# Backup database states
replication-manager-cli status --cluster=all > /tmp/repman-2x-status.txt
```

2.8.7.2 Step 2: Update Configuration Files

```
# Run migration script on each config file
for file in /etc/replication-manager/cluster.d/*.toml; do
    ./migrate-config-3x.sh "$file"
done

# Review and update config.toml
./migrate-config-3x.sh /etc/replication-manager/config.toml

# Add new 3.x parameters as needed
```

2.8.7.3 Step 3: Install replication-manager 3.x

Repository installation:

```
# Update repository to 3.1
# CentOS/RHEL
echo "[replication-manager]
name=Replication-Manager
baseurl=http://repo.signal18.io/centos/\$releasever/\$basearch/3.1/
gpgcheck=0
enabled=1" > /etc/yum.repos.d/replication-manager.repo

yum update replication-manager
```

Docker:

```
docker pull signal18/replication-manager:3.1
```

2.8.7.4 Step 4: Verify Configuration

```
# Test configuration syntax
replication-manager-cli test --cluster=all

# Check for deprecated parameters
journalctl -u replication-manager | grep -i "WARN0159\|WARN0160"
```

2.8.7.5 Step 5: Start and Validate

```
# Start replication-manager 3.x
systemctl start replication-manager

# Verify all clusters are discovered
replication-manager-cli status --cluster=all

# Check web UI
# https://your-server:10005

# Verify monitoring is active
replication-manager-cli api /api/clusters

# Test failover capability (non-production first!)
# replication-manager-cli switchover --cluster=test_cluster
```

2.8.7.6 Step 6: Monitor for Issues

```
# Watch logs for warnings
tail -f /var/log/replication-manager.log

# Check for deprecated parameter warnings
grep "WARN0159\|WARN0160" /var/log/replication-manager.log

# Monitor system resources (new PFS monitoring may increase load)
top
iostat -x 1

# Verify backups still work
# Trigger manual backup via API or CLI
```

2.8.8 Post-Migration Tasks

2.8.8.1 Update Backup Scripts

If backing up configuration:

```
#!/bin/bash
# New backup locations for 3.x

# Backup active configs
tar czf /backup/repman-config-$(date +%Y%m%d).tar.gz \
    /root/.config/replication-manager/

# Backup data directory
tar czf /backup/repman-data-$(date +%Y%m%d).tar.gz \
    /var/lib/replication-manager/
```

2.8.8.2 Update Monitoring

- Grafana dashboards may need updates for new metric names
- Alert rules may need adjustment for new warning codes
- Prometheus exporters should work without changes

2.8.8.3 Documentation Updates

- Update internal documentation with new parameter names
- Document new 3.x features being used
- Update runbooks with new CLI commands and API endpoints

2.8.9 Rollback Procedure

If issues occur, rollback to 2.x:

```
# Stop 3.x
systemctl stop replication-manager

# Remove 3.x
yum remove replication-manager  # or apt remove

# Restore 2.x configuration
rm -rf /etc/replication-manager/
tar xzf /tmp/repman-2x-config-*.tar.gz -C /

# Install 2.x
yum install replication-manager-2.x  # specify exact version

# Restore data if needed
rm -rf /var/lib/replication-manager/
tar xzf /tmp/repman-2x-data-*.tar.gz -C /

# Start 2.x
systemctl start replication-manager
```

2.8.10 Troubleshooting

2.8.10.1 Configuration Not Loading

Symptom: Clusters not discovered after upgrade

Solution:

```
# Check file permissions
ls -la /root/.config/replication-manager/
chown -R replication-manager:replication-manager /root/.config/replication-manager/

# Verify configuration syntax
replication-manager-cli test --cluster=all
```

2.8.10.2 Deprecated Parameter Warnings

Symptom: WARN0159 or WARN0160 in logs

Solution: - Use migration script to update parameter names - Check `/root/.config/replication-manager/clusters.d/` for old parameter names - Review and update manually if script missed any

2.8.10.3 Performance Schema Overhead

Symptom: Increased database load after upgrade

Solution:

```
# Disable new monitoring features temporarily
monitoring-performance-schema-mutex = false
monitoring-performance-schema-latch = false
monitoring-performance-schema-memory = false
```

2.8.10.4 Docker Container Not Persisting Config

Symptom: Configuration changes lost on restart

Solution:

```
# Ensure third volume is mounted
docker run -v/home/repman/etc:/etc/replication-manager:rw \
           -v/home/repman/data:/var/lib/replication-manager:rw \
           -v/home/repman/config:/root/.config/replication-manager:rw \
           signal18/replication-manager:3.1
```

2.8.10.5 Backup Tools Not Found

Symptom: Backup jobs failing with "command not found"

Solution:

```
# Specify full paths
backup-mydumper-path = "/usr/local/bin/mydumper"
backup-myloader-path = "/usr/local/bin/myloader"
backup-restic-binary-path = "/usr/bin/restic"
```

2.8.11 Getting Help

If you encounter issues during migration:

- Review logs: `/var/log/replication-manager.log`
- Check deprecated parameter warnings (WARN0159, WARN0160)
- Consult documentation: https://docs.signal18.io
- Report issues: https://github.com/signal18/replication-manager/issues
- Community support: Signal18 Slack/Discord channels

2.8.12 Version-Specific Notes

2.8.12.1 Migrating from 2.0 to 3.x

- Largest configuration changes
- Requires most careful planning
- Test thoroughly before production

2.8.12.2 Migrating from 2.1/2.2 to 3.x

- Fewer breaking changes
- Most parameter renames still apply
- Configuration location change is primary concern

2.8.12.3 Migrating from 2.3 to 3.x

- Minimal breaking changes
- Configuration location change
- Some default value changes
- Docker volume mapping addition

2.8.12.4 Upgrading from 3.0 to 3.1

- No breaking changes
- New features added
- New parameters with safe defaults
- Backward compatible with 3.0 configurations

3.1.1 Start & Stop Monitoring

[Figure: clientserver]

replication-manager runs as a daemon that continuously monitors your clusters. It exposes a REST API, a web GUI, and accepts commands from the CLI client.

3.1.2 Start

systemd (package install):

```
systemctl start replication-manager
```

Enable on boot:

```
systemctl enable replication-manager
```

init.d (legacy):

```
/etc/init.d/replication-manager start
```

Manual (tarball or embedded binary):

```
replication-manager monitor --config /etc/replication-manager/config.toml --http-server
```

3.1.3 Stop

```
systemctl stop replication-manager
```

3.1.4 Check Status

Service status:

```
systemctl status replication-manager
```

Cluster status via CLI:

```
replication-manager-cli status
```

All clusters:

```
replication-manager-cli --cluster="mycluster" status
```

Sample output:

```
| Group: mycluster |  Mode: Automatic
         Id            Host   Port    Status   Failures   Using GTID         Current GTID    Delay  RO
5641630519400684578  10.0.1.10  3306    Master          0    Slave_Pos     0-3306-4210            0 OFF
9624235790336213315  10.0.1.11  3306     Slave          0    Slave_Pos     0-3306-4210            0  ON
3944708846436490796  10.0.1.12  3306     Slave          0    Slave_Pos     0-3306-4210            0  ON
```

Cluster topology:

```
replication-manager-cli topology
```

Web GUI:

Open `https://<host>:10005` in a browser — the dashboard shows live cluster state, replication health, and server metrics.

3.1.5 Logs

See Logs for a full reference — log files written, per-module verbosity settings, rotation, and how to query logs via journalctl and the REST API.

3.2.1 Command Line Client

replication-manager ships with `replication-manager-cli`, a command line client that connects to the running daemon over the REST API. It can be used interactively via the terminal console or non-interactively for scripting and automation.

3.2.2 Interactive Console

Start the interactive console:

```
replication-manager-cli console
```

For a specific cluster:

```
replication-manager-cli console --cluster=mycluster
```

[Figure: mrmconsole]

3.2.2.1 Keyboard Shortcuts

Key	Action
`Ctrl-D`	Print debug information
`Ctrl-F`	Manual failover
`Ctrl-I`	Toggle automatic / manual failover mode
`Ctrl-R`	Set replicas read-only
`Ctrl-S`	Switchover
`Ctrl-W`	Set replicas read-write
`Ctrl-P` / `Ctrl-N`	Switch between clusters
`Ctrl-Q`	Quit

3.2.3 Non-Interactive Commands

3.2.3.1 Switchover

Trigger a controlled primary handoff:

```
replication-manager-cli switchover --cluster=mycluster
```

3.2.3.2 Failover

Trigger a failover without waiting for the primary to recover:

```
replication-manager-cli failover --cluster=mycluster
```

3.2.3.3 Bootstrap Replication

Initialize replication on existing database nodes that have no replication configured. Supported topologies:

- `master-slave`
- `master-slave-no-gtid`
- `maxscale-binlog`
- `multi-master`
- `multi-tier-slave`

```
replication-manager-cli --cluster=mycluster bootstrap --clean-all --topology=multi-tier-slave
```

3.2.3.4 Cluster Status

```
replication-manager-cli --cluster=mycluster status
```

Sample output:

```
| Group: mycluster |  Mode: Manual
                 Id            Host   Port          Status   Failures   Using GTID         Current GTID
5641630519400684578       127.0.0.1   3317          Master          0           No          0-3317-3124
9624235790336213315       127.0.0.1   3318     Maintenance          0    Slave_Pos          0-3317-3124
3944708846436490796       127.0.0.1   3319           Slave          0    Slave_Pos          0-3317-3124
```

3.2.3.5 Topology

```
replication-manager-cli topology
```

3.2.3.6 Server Maintenance

Place a specific server in maintenance mode using its numeric ID:

```
replication-manager-cli server --id=9624235790336213315 --maintenance
```

3.3.1 Web Interface

Once **replication-manager** is started in monitor mode, it provides an internal HTTP server to oversee your clusters.

Access it via pointing a web browser to http://localhost:10001.

By default it is bound to `localhost`. This can be changed via **http-bind-address** variable or command line flag to get remote access on the network.

Using **replication-manager 2.1**, secured connection is open via pointing a web browser to https://replication-manager-host:10005/.

It looks like this:

[Figure: mrmdash]

> Starting with **replication-manager 2.1** the http client can be secured via navigation to the https api port https://localhost:10005., this offer encryption on the wire and JWT tokens for identity tracking. The default credential is the apiUser credential so admin:repman but can be change per cluster.
>
> Previously to **replication-manager 2.1** The http dashboard has no protected access, use creativity to restrict access to it. Some login protection using http-auth = true is deprecate. We advice not to used it but to protect via a web proxying authentication instead.

3.3.1.1 Sample nginx configuration providing secure access to the dashboard

```
server {
  server_name repman.dashboard;
  location / {
    auth_basic           "Login required";
    auth_basic_user_file conf/htpasswd;
    proxy_pass http://localhost:10001;
  }
}
```

3.4.1.1 API Client Usage

The rest API is using JWT TLS and is served by default on port 10005 by the replication-manager monitor/

Credentials can be customized by setting your own user and password in configuration file.

```
api-port ="10005"
api-credentials = "admin:repman"
```

At startup of the monitor, X509 certificates are loaded from the replication-manager share directory or from keys defined from config files to ensure TLS https secure communication.

Replace those files with your own certificate signed or self signed to make sure your deployment is secured.

```
# Key considerations for algorithm "RSA"  2048-bit
openssl genrsa -out server.key 2048

# Key considerations for algorithm "ECDSA"  secp384r1
# List ECDSA the supported curves (openssl ecparam -list_curves)
openssl ecparam -genkey -name secp384r1 -out server.key
openssl req -new -x509 -sha256 -key server.key -out server.crt -days 3650
```

At startup **replication-manager** monitor will generate in memory extra self-signed RSA certificate to ensure later token encryption exchange for JWT.

Calling API via the client

API can be called via command line client to simplify curl syntax with JWT token.

```
./replication-manager-cli api  --url="https://127.0.0.1:10005/api/clusters/ux_dck_zpool_loop/servers/act
```

Calling API via curl or wget

```
TOKEN=$(curl -s -k -X POST -H 'Accept: application/json' -H 'Content-Type: application/json' --data '{"u
```

or without json response

```
TOKEN=$(curl -s -k -X POST -H 'Accept: text/html' -H 'Content-Type: application/json' --data '{"username
```

follow by your request:

```
curl -k -H 'Accept: application/json' -H "Authorization: Bearer ${TOKEN}" https://demo.signal18.io/api/c
```

Or via wget

```
TOKEN=$(wget  -qO- --no-check-certificate --post-data '{"username":"admin","password":"repman"}' --heade

wget -qO- --no-check-certificate --header 'Accept: application/json' --header "Authorization: Bearer ${T
```

3.4.1.2 API documentation

Can be browsed online Or by installing the product and point to http://127.0.0.1:10001/api-docs

3.4.2.1 All endpoints

3.4.2.1.1 auth

Method	URI	Name	Summary
POST	/api/login	post API login	User login

3.4.2.1.2 cloud18

Method	URI	Name	Summary
GET	/api/clusters/for-sale	get API clusters for sale	Retrieve peer clusters for sale
GET	/api/clusters/peers	get API clusters peers	Retrieve peer clusters for a user
GET	/api/terms	get API terms	Retrieves terms
POST	/api/clusters/{clusterName}/sales/accept-subscription	post API clusters cluster name sales accept subscription	Accept a subscription for a specific cluster
POST	/api/clusters/{clusterName}/sales/end-subscription	post API clusters cluster name sales end subscription	Remove a sponsor from a specific cluster
POST	/api/clusters/{clusterName}/sales/refuse-subscription	post API clusters cluster name sales refuse subscription	Reject a subscription for a specific cluster
POST	/api/clusters/{clusterName}/subscribe	post API clusters cluster name subscribe	Subscribe a user to a cluster

3.4.2.1.3 cluster

Method	URI	Name	Summary
DELETE	/api/clusters/actions/delete/{clusterName}	delete API clusters actions delete cluster name	Delete a cluster
GET	/api/clusters	get API clusters	Fetch clusters
GET	/api/clusters/{clusterName}	get API clusters cluster name	Retrieve details of a cluster
GET	/api/clusters/{clusterName}/diffvariables	get API clusters cluster name diffvariables	Retrieve variable differences for a specific cluster
GET	/api/clusters/{clusterName}/jobs	get API clusters cluster name jobs	Retrieve job entries for a specific cluster
GET	/api/clusters/{clusterName}/queryrules	get API clusters cluster name queryrules	Retrieve query rules for a specific cluster
GET	/api/clusters/{clusterName}/status	get API clusters cluster name status	Retrieve status of a cluster
GET	/api/clusters/{clusterName}/top	get API clusters cluster name top	Retrieve top metrics for a specific cluster
GET	/api/clusters/{clusterName}/topology/crashes	get API clusters cluster name topology crashes	Retrieve crashes for a specific cluster

Method	URI	Name	Summary
POST	/api/clusters/actions/add/{clusterName}	post API clusters actions add cluster name	Add a new cluster
POST	/api/clusters/{clusterName}/actions/waitdatabases	post API clusters cluster name actions waitdatabases	Wait for databases to be ready for a specific cluster

3.4.2.1.4 cluster_actions

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/actions/failover	post API clusters cluster name actions failover	Handles the failover process for a given cluster.
POST	/api/clusters/{clusterName}/actions/optimize	post API clusters cluster name actions optimize	Optimize a specific cluster
POST	/api/clusters/{clusterName}/actions/reset-failover-control	post API clusters cluster name actions reset failover control	Reset failover control for a specific cluster
POST	/api/clusters/{clusterName}/actions/reset-sla	post API clusters cluster name actions reset SLA	Reset SLA for a specific cluster
POST	/api/clusters/{clusterName}/actions/rotate-passwords	post API clusters cluster name actions rotate passwords	Rotate passwords for a specific cluster
POST	/api/clusters/{clusterName}/actions/switchover	post API clusters cluster name actions switchover	Handles the switchover process for a given cluster.
POST	/api/clusters/settings/actions/reload-clusters-plans	post API clusters settings actions reload clusters plans	Reload cluster plans

3.4.2.1.5 cluster_backup

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/actions/master-physical-backup	post API clusters cluster name actions master physical backup	Perform a physical backup for the master of a specific cluster

3.4.2.1.6 cluster_backups

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/backups	get API clusters cluster name backups	Retrieve backups for a specific cluster

3.4.2.1.7 cluster_certificates

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/certificates	get API clusters cluster name certificates	Retrieve client certificates for a specific cluster
POST	/api/clusters/{clusterName}/certificates-rotate	post API clusters cluster name actions certificates rotate	Rotate keys for a specific cluster

3.4.2.1.8 cluster_graphite

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/graphite-filterlist	get API clusters cluster name graphite filterlist	Retrieve Graphite filter list for a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/reload-graphite-filterlist	post API clusters cluster name settings actions reload graphite filterlist	Reload Graphite filter list for a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/reset-graphite-filterlist/{template}	post API clusters cluster name settings actions reset graphite filterlist template	Reset Graphite filter list for a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/set-graphite-filterlist/{filterType}	post API clusters cluster name settings actions set graphite filterlist filter type	Set Graphite filter list for a specific cluster

3.4.2.1.9 cluster_maintenance

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/actions/cancel-rolling-restart	post API clusters cluster name actions cancel rolling restart	Cancel rolling restart for a specific cluster
POST	/api/clusters/{clusterName}/actions/rolling	post API clusters cluster name actions rolling	Handles the rolling restart process for a given cluster.

3.4.2.1.10 cluster_monitor

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/actions/addserver/{hostAndPort}	post API clusters cluster name actions addserver host port	Add server to a specific cluster
POST	/api/clusters/{clusterName}/actions/addserver/{hostAndPort}/{type}	post API clusters cluster name actions addserver host port type	Add server to a specific cluster

Method	URI	Name	Summary
POST	/cluster/{clusterName}/actions/dropserver/{host}/{port}	post cluster clusters cluster name actions dropserver host port	Drop a server monitor from a cluster
POST	/cluster/{clusterName}/actions/dropserver/{host}/{port}/{type}	post cluster clusters cluster name actions dropserver host port type	Drop a server monitor from a cluster

3.4.2.1.11 cluster_provision

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/actions/cancel-rolling-reprov	post API clusters cluster name actions cancel rolling reprov	Cancel rolling reprovision for a specific cluster
POST	/api/clusters/{clusterName}/services/actions/provision	post API clusters cluster name services actions provision	Provision services for a specific cluster
POST	/api/clusters/{clusterName}/services/actions/unprovision	post API clusters cluster name services actions unprovision	Unprovision services for a specific cluster

3.4.2.1.12 cluster_replication

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/actions/replication/bootstrap/{topology}	post API clusters cluster name actions replication bootstrap topology	Bootstrap replication for a specific cluster
POST	/api/clusters/{clusterName}/actions/replication/cleanup	post API clusters cluster name actions replication cleanup	Cleanup replication bootstrap for a specific cluster

3.4.2.1.13 cluster_schema

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/schema	get API clusters cluster name schema	Retrieve schema information for a specific cluster
POST	/api/clusters/{clusterName}/actions/checksum-all-tables	post API clusters cluster name actions checksum all tables	Calculate checksum for all tables in a specific cluster
POST	/api/clusters/{clusterName}/schema/{schemaName}/{tableName}/actions/checksum-table	post API clusters cluster name schema schema name table name actions checksum table	Calculate checksum for a specific table in a specific cluster

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/schema/{schemaName}/{tableName}/actions/move-table/{clusterShard}	post API clusters cluster name schema schema name table name actions move table cluster shard	Move table to a different shard cluster
POST	/api/clusters/{clusterName}/schema/{schemaName}/{tableName}/actions/reshard-table	post API clusters cluster name schema schema name table name actions reshard table	Reshard a table for a specific cluster
POST	/api/clusters/{clusterName}/schema/{schemaName}/{tableName}/actions/reshard-table/{clusterList}	post API clusters cluster name schema schema name table name actions reshard table cluster list	Reshard a table for a specific cluster
POST	/api/clusters/{clusterName}/schema/{schemaName}/{tableName}/actions/universal-table	post API clusters cluster name schema schema name table name actions universal table	Set a table universal for a specific cluster

3.4.2.1.14 cluster_settings

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/settings	get API clusters cluster name settings	Retrieve settings for a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/certificates/reload	post API clusters cluster name settings actions certificates reload	Reload client certificates for a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/discover	post API clusters cluster name settings actions discover	Discover settings for a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/reload	post API clusters cluster name settings actions reload	Reload cluster settings
POST	/api/clusters/{clusterName}/settings/actions/set-cron/{settingName}/{settingValue}	post API clusters cluster name settings actions set cron setting name setting value	Set cron jobs for a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/set/{settingName}/{settingValue}	post API clusters cluster name settings actions set setting name setting value	Set settings for a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/switch/{settingName}	post API clusters cluster name settings actions switch setting name	Switch settings for a specific cluster

Method	URI	Name	Summary
POST	/api/clusters/settings/actions/set/{settingName}/{settingValue}	post /api clusters settings actions set setting name setting value	Set global settings for the server

3.4.2.1.15 cluster_tags

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/tags	get /api clusters cluster name tags	Retrieve tags for a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/add-db-tag/{tagValue}	post /api clusters cluster name settings actions add db tag tag value	Add a tag to a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/add-proxy-tag/{tagValue}	post /api clusters cluster name settings actions add proxy tag tag value	Add a proxy tag to a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/apply-dynamic-config	post /api clusters cluster name settings actions apply dynamic config	Apply dynamic configuration for a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/drop-db-tag/{tagValue}	post /api clusters cluster name settings actions drop db tag tag value	Remove a tag from a specific cluster
POST	/api/clusters/{clusterName}/settings/actions/drop-proxy-tag/{tagValue}	post /api clusters cluster name settings actions drop proxy tag tag value	Remove a proxy tag from a specific cluster

3.4.2.1.16 cluster_test

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/actions/sysbench	post /api clusters cluster name actions sysbench	Run sysbench for a specific cluster
POST	/api/clusters/{clusterName}/tests/actions/run/all	post /api clusters cluster name tests actions run all	Run all tests for a given cluster
POST	/api/clusters/{clusterName}/tests/actions/run/{testName}	post /api clusters cluster name tests actions run test name	Run a specific test for a given cluster

3.4.2.1.17 cluster_topology

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/shardclusters	get API clusters cluster name shardclusters	Retrieve shard clusters for a specific cluster
GET	/api/clusters/{clusterName}/topology/alerts	get API clusters cluster name topology alerts	Shows the alerts for that specific named cluster
GET	/api/clusters/{clusterName}/topology/logs	get API clusters cluster name topology logs	Retrieve logs for a specific cluster
GET	/api/clusters/{clusterName}/topology/master	get API clusters cluster name topology master	Retrieve master of a cluster
GET	/api/clusters/{clusterName}/topology/proxies	get API clusters cluster name topology proxies	Shows the proxies for that specific named cluster
GET	/api/clusters/{clusterName}/topology/servers	get API clusters cluster name topology servers	Retrieve servers for a specific cluster
GET	/api/clusters/{clusterName}/topology/slaves	get API clusters cluster name topology slaves	Shows the slaves for that specific named cluster
POST	/api/clusters/{clusterName}/actions/add/{clusterShardingName}	post API clusters cluster name actions add cluster sharding name	Add a sharding cluster to an existing cluster

3.4.2.1.18 cluster_traffics

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/actions/start-traffic	post API clusters cluster name actions start traffic	Start traffic for a specific cluster
POST	/api/clusters/{clusterName}/actions/stop-traffic	post API clusters cluster name actions stop traffic	Stop traffic for a specific cluster

3.4.2.1.19 cluster_vault

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/send-vault-token	post API clusters cluster name send vault token	Send Vault token to a specific cluster

3.4.2.1.20 database

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/need-rolling-reprov	get API clusters cluster name need rolling reprov	Check if a cluster needs a rolling reprovision
GET	/api/clusters/{clusterName}/need-rolling-restart	get API clusters cluster name need rolling restart	Check if a cluster needs a rolling restart
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-innodb-monitor	get API clusters cluster name servers server name actions toogle innodb monitor	Toogle InnoDB monitor on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/all-slaves-status	get API clusters cluster name servers server name all slaves status	Get status of all slaves of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/is-master	get API clusters cluster name servers server name is master	Check if a server is a master
GET	/api/clusters/{clusterName}/servers/{serverName}/is-slave	get API clusters cluster name servers server name is slave	Check if a server is a slave
GET	/api/clusters/{clusterName}/servers/{serverName}/master-status	get API clusters cluster name servers server name master status	Get master status of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/meta-data-locks	get API clusters cluster name servers server name meta data locks	Get metadata locks of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/processlist	get API clusters cluster name servers server name processlist	Get process list of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/config	get API clusters cluster name servers server name server port config	Get server port configuration
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/is-master	get API clusters cluster name servers server name server port is master	Check if a server port is a master
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/is-slave	get API clusters cluster name servers server name server port is slave	Check if a server port is a slave
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-config-change	get API clusters cluster name servers server name server port need config change	Check if a server needs a config change
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-prov	get API clusters cluster name servers server name server port need prov	Check if a server needs provisioning

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-reprov	get API clusters cluster name servers server name server port need reprov	Check if server needs re-provisioning
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-restart	get API clusters cluster name servers server name server port need restart	Check if server needs a restart
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-start	get API clusters cluster name servers server name server port need start	Check if server needs to start
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-stop	get API clusters cluster name servers server name server port need stop	Check if server needs to stop
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-unprov	get API clusters cluster name servers server name server port need unprov	Check if server needs unprovisioning
GET	/api/clusters/{clusterName}/servers/{serverName}/service/opensvc	get API clusters cluster name servers server name service opensvc	Get database service configuration of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/status	get API clusters cluster name servers server name status	Get status of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/status/delta	get API clusters cluster name servers server name status delta	Get status delta of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/status/innodb	get API clusters cluster name servers server name status innodb	Get InnoDB status of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/tables	get API clusters cluster name servers server name tables	Get tables of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/variables	get API clusters cluster name servers server name variables	Get variables of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/vtables	get API clusters cluster name servers server name vtables	Get virtual tables of a server

3.4.2.1.21 database_actions

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/optimize	get API clusters cluster name servers server name actions optimize	Optimize server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/start	get API clusters cluster name servers server name actions start	Starts a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/stop	get API clusters cluster name servers server name actions stop	Stops a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-meta-data-locks	get API clusters cluster name servers server name actions toogle meta data locks	Toogle toogle meta data locks on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-query-response-time	get API clusters cluster name servers server name actions toogle query response time	Toogle toogle query response time on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-read-only	get API clusters cluster name servers server name actions toogle read only	Toogle toogle read only mode on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/wait-for-innodb-purge	get API clusters cluster name servers server name actions wait innodb purge	Wait for InnoDB purge on a server

3.4.2.1.22 database_backup

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/backup-logical	get API clusters cluster name servers server name actions backup logical	Perform backup logical on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/backup-physical	get API clusters cluster name servers server name actions backup physical	Perform backup physical backup on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/reseed/{backupMethod}	get API clusters cluster name servers server name actions reseed backup method	Reseed reseed
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/reseed-cancel/{task}	get API clusters cluster name servers server name actions reseed cancel task	Cancel reseed task on a server

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/backup	get API /{clusters/ cluster name servers server name server port backup	Perform a physical backup on a server port
POST	/api/clusters/{clusterName}/servers/{serverName}/actions/pitr	post API /{clusters/ cluster name servers server name actions pitr	Perform point-in-time recovery on a server

3.4.2.1.23 database_logs

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/backup-error-log	get API /{clusters/ cluster name servers server name actions backup error log	Perform a backup of the error log on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/backup-slowquery-log	get API /{clusters/ cluster name servers server name actions backup slowquery log	Perform a backup of the slow query log on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/flush-logs	get API /{clusters/ cluster name servers server name actions flush logs	Flush logs on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-sql-error-log	get API /{clusters/ cluster name servers server name actions toogle SQL error log	Toogle SQL error log on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/errorlog	get API /{clusters/ cluster name servers server name errorlog	Get error log of a server

3.4.2.1.24 database_maintenance

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/del-maintenance	get API /{clusters/ cluster name servers server name actions del maintenance	Delete maintenance mode on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-maintenance	get API /{clusters/ cluster name servers server name actions maintenance	Toogle maintenance mode on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/set-maintenance	get API /{clusters/ cluster name servers server name actions set maintenance	Set a server to maintenance mode

3.4.2.1.25 database_provision

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/provision	get API /{version} cluster name servers server name actions provision	Provision a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/unprovision	get API /{version} cluster name servers server name actions unprovision	Unprovision a server

3.4.2.1.26 database_queries

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/reset-pfs-queries	get API /{version} cluster name servers server name actions reset pfs queries	Reset PFS queries on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/set-long-query-time/{queryTime}	get API /{version} cluster name servers server name actions set long query time query time	Set long query time on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-pfs-slow-query	get API /{version} cluster name servers server name actions toogle pfs slow query	Toogle PFS slow query capture on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-slow-query	get API /{version} cluster name servers server name actions toogle slow query	Toogle slow query on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-slow-query-capture	get API /{version} cluster name servers server name actions toogle slow query capture	Toogle slow query capture on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-slow-query-table	get API /{version} cluster name servers server name actions toogle slow query table	Toogle slow query table mode on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/digest/statements-pfs	get API /{version} cluster name servers server name digest statements pfs	Get PFS statements of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/digest/statements-slow	get API /{version} cluster name servers server name digest statements slow	Get PFS statements from the slow log of a server

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/analyze-pfs	get API /{version} cluster name servers server name queries query digest actions analyze pfs	Analyze query digest PFS on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/analyze-slowlog	get API /{version} cluster name servers server name queries query digest actions analyze slowlog	Analyze query digest the slow log on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/explain-pfs	get API /{version} cluster name servers server name queries query digest actions explain pfs	Explain query digest PFS on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/explain-slowlog	get API /{version} cluster name servers server name queries query digest actions explain slowlog	Explain query digest the slow log on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/kill-query	get API /{version} cluster name servers server name queries query digest actions kill query	Kill a query on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/kill-thread	get API /{version} cluster name servers server name queries query digest actions kill thread	Kill a thread on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/queries/response-time	get API /{version} cluster name servers server name query response time	Get query response time of a server
GET	/api/clusters/{clusterName}/servers/{serverName}/slow-queries	get API /{version} cluster name servers server name slow queries	Get slow log of a server

3.4.2.1.27 database_replication

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/reset-master	get API /{version} cluster name servers server name actions reset master	Reset the master on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/reset-slave-all	get API /{version} cluster name servers server name actions reset slave all	Reset all slaves on a server

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/skip-replication-event	get API /{version}/ cluster name servers server name actions skip replication event	Skips replication event on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/start-slave	get API /{version}/ cluster name servers server name actions start slave	Starts the slave on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/stop-slave	get API /{version}/ cluster name servers server name actions stop slave	Stops the slave on a server

3.4.2.1.28 database_schema

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/schemas	get API /{version}/ cluster name servers server name schemas	Get schemas of a server

3.4.2.1.29 database_tasks

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/job-cancel/{task}	get API /{version}/ cluster name servers server name actions job cancel task	Cancel job task on a server
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/run-jobs	get API /{version}/ cluster name servers server name actions run jobs	Run jobs on a server
POST	/api/clusters/{clusterName}/servers/{serverName}/port-write-log/{task}	post API /{version}/ cluster name servers server name server port write log task	Write log to server

3.4.2.1.30 database_topology

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/set-ignored	get API /{version}/ cluster name servers server name actions set ignored	Sets server as ignored
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/set-prefered	get API /{version}/ cluster name servers server name actions set prefered	Sets server as preferred

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/set-unrated	get API clusters {clusterName} servers cluster name servers server name actions set unrated	Set a server as unrated
GET	/api/clusters/{clusterName}/servers/{serverName}/actions/switchover	get API clusters {clusterName} servers cluster name servers server name actions switchover	Perform switchover on a server

3.4.2.1.31 global_setting

Method	URI	Name	Summary
POST	/api/clusters/settings/actions/switch/{settingName}	post API clusters settings actions switch setting name	Switch global settings for the server

3.4.2.1.32 proxies

Method	URI	Name	Summary
GET	/api/clusters/{clusterName}/proxies/{proxyName}/actions/need-reprov	get API clusters {clusterName} proxies cluster name proxies proxy name actions need reprov	Check if Proxy Needs Reprovision
GET	/api/clusters/{clusterName}/proxies/{proxyName}/actions/need-restart	get API clusters {clusterName} proxies cluster name proxies proxy name actions need restart	Check if Proxy Needs Restart
GET	/api/clusters/{clusterName}/sphinx/indexes	get API clusters {clusterName} sphinx cluster name sphinx indexes	Get Sphinx Indexes
POST	/api/clusters/{clusterName}/proxies/{proxyName}/actions/provision	post API clusters {clusterName} proxies cluster name proxies proxy name actions provision	Provision Proxy Service
POST	/api/clusters/{clusterName}/proxies/{proxyName}/actions/start	post API clusters {clusterName} proxies cluster name proxies proxy name actions start	Start Proxy Service
POST	/api/clusters/{clusterName}/proxies/{proxyName}/actions/stop	post API clusters {clusterName} proxies cluster name proxies proxy name actions stop	Stop Proxy Service
POST	/api/clusters/{clusterName}/proxies/{proxyName}/actions/unprovision	post API clusters {clusterName} proxies cluster name proxies proxy name actions unprovision	Unprovision Proxy Service

3.4.2.1.33 public

Method	URI	Name	Summary
GET	/api/configs/grafana	get API configs grafana	List Grafana files
GET	/api/heartbeat	get API heartbeat	Monitor Heartbeat
GET	/api/monitor	get API monitor	Handles replication manager requests
GET	/api/prometheus	get API prometheus	Fetch Prometheus metrics
GET	/api/repocomp/current	get API repocomp current	Retrieve current repository component
GET	/api/status	get API status	Get Replication Manager Status
GET	/api/timeout	get API timeout	Check if the replication manager is running

3.4.2.1.34 user

Method	URI	Name	Summary
POST	/api/clusters/{clusterName}/users/add	post API clusters cluster name users add	Add a new user to a cluster
POST	/api/clusters/{clusterName}/users/drop	post API clusters cluster name users drop	Drop a cluster user
POST	/api/clusters/{clusterName}/users/send-credentials	post API clusters cluster name users send credentials	Send credentials to a specific user
POST	/api/clusters/{clusterName}/users/update	post API clusters cluster name users update	Update a cluster user

3.4.2.2 Paths

3.4.2.2.1 Delete a cluster (*DeleteAPIClustersActionsDeleteClusterName*)

`DELETE /api/clusters/actions/delete/{clusterName}`

Deletes a cluster identified by its name.

3.4.2.2.1.1 Produces

- application/json

3.4.2.2.1.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.1.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Cluster deleted successfully		schema
400	Bad Request	Invalid cluster name		schema
500	Internal Server Error	Internal server error		schema

3.4.2.2.1.4 Responses

200 - Cluster deleted successfully Status: OK

Schema

400 - Invalid cluster name Status: Bad Request

Schema

500 - Internal server error Status: Internal Server Error

Schema

3.4.2.2.2 Fetch clusters (*GetAPIClusters*)

`GET /api/clusters`

Fetches the list of clusters that the user has access to based on ACL.

3.4.2.2.2.1 Produces

- application/json

3.4.2.2.2.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of clusters		schema
401	Unauthorized	Unauthenticated resource		schema
500	Internal Server Error	Internal server error		schema

3.4.2.2.2.3 Responses

200 - List of clusters Status: OK

Schema

[][ClusterCluster](#cluster-cluster)

401 - Unauthenticated resource Status: Unauthorized

Schema

500 - Internal server error Status: Internal Server Error

Schema

3.4.2.2.3 Retrieve details of a cluster (*GetAPIClustersClusterName*)

`GET /api/clusters/{clusterName}`

This endpoint retrieves the details of a specified cluster and returns it in JSON format.

3.4.2.2.3.1 Produces

- application/json

3.4.2.2.3.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.3.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Cluster details		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.3.4 Responses

200 - Cluster details Status: OK

Schema

ClusterCluster

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.4 Retrieve backups for a specific cluster (*GetAPIClustersClusterNameBackups*)

`GET /api/clusters/{clusterName}/backups`

This endpoint retrieves the backups for the specified cluster.

3.4.2.2.4.1 Produces

- application/json

3.4.2.2.4.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.4.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of backups		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.4.4 Responses

200 - List of backups Status: OK

Schema

[][interface{}](#interface)

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.5 Retrieve client certificates for a specific cluster (*GetAPIClustersClusterNameCertificates*)

`GET /api/clusters/{clusterName}/certificates`

This endpoint retrieves the client certificates for the specified cluster.

3.4.2.2.5.1 Produces

- application/json

3.4.2.2.5.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.5.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of client certificates		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.5.4 Responses

200 - List of client certificates Status: OK

Schema

[][interface{}](#interface)

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.6 Retrieve variable differences for a specific cluster (*GetAPIClustersClusterNameDiffvariables*)

`GET /api/clusters/{clusterName}/diffvariables`

This endpoint retrieves the variable differences for the specified cluster.

3.4.2.2.6.1 Produces

- application/json

3.4.2.2.6.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.6.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of variable differences		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.6.4 Responses

200 - List of variable differences Status: OK

Schema

[][ClusterVariableDiff](#cluster-variable-diff)

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.7 Retrieve Graphite filter list for a specific cluster (*GetAPIClustersClusterNameGraphiteFilterlist*)

`GET /api/clusters/{clusterName}/graphite-filterlist`

This endpoint retrieves the Graphite filter list for the specified cluster.

3.4.2.2.7.1 Produces

- application/json

3.4.2.2.7.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.7.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of Graphite filters		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.7.4 Responses

200 - List of Graphite filters Status: OK

Schema

[]string

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.8 Retrieve job entries for a specific cluster (*GetAPIClustersClusterNameJobs*)

`GET /api/clusters/{clusterName}/jobs`

This endpoint retrieves the job entries for the specified cluster.

3.4.2.2.8.1 Produces

- application/json

3.4.2.2.8.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.8.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of job entries		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.8.4 Responses

200 - List of job entries Status: OK

Schema

[][interface{}](#interface)

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.9 Check if a cluster needs a rolling reprovision (*GetAPIClustersClusterNameNeedRollingReprov*)

`GET /api/clusters/{clusterName}/need-rolling-reprov`

Checks if a specified cluster needs a rolling reprovision.

3.4.2.2.9.1 Produces

- text/plain

3.4.2.2.9.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.9.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Need rolling reprov!		schema
500	Internal Server Error	503 -No rolling reprov needed!" or "500 -No cluster		schema

3.4.2.2.9.4 Responses

200 - 200 -Need rolling reprov! Status: OK

Schema

500 - 503 -No rolling reprov needed!" or "500 -No cluster Status: Internal Server Error

Schema

3.4.2.2.10 Check if a cluster needs a rolling restart (*GetAPIClustersClusterNameNeedRollingRestart*)

`GET /api/clusters/{clusterName}/need-rolling-restart`

Checks if a specified cluster needs a rolling restart.

3.4.2.2.10.1 Produces

- text/plain

3.4.2.2.10.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.10.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Need rolling restart!		schema
500	Internal Server Error	503 -No rolling restart needed!" or "500 -No cluster		schema

3.4.2.2.10.4 Responses

200 - 200 -Need rolling restart! Status: OK

Schema

500 - 503 -No rolling restart needed!" or "500 -No cluster Status: Internal Server Error

Schema

3.4.2.2.11 Check if Proxy Needs Reprovision (*GetAPIClustersClusterNameProxiesProxyNameActionsNeedReprov*)

`GET /api/clusters/{clusterName}/proxies/{proxyName}/actions/need-reprov`

Check if the proxy service for a given cluster and proxy needs reprovisioning

3.4.2.2.11.1 Consumes

- application/json

3.4.2.2.11.2 Produces

- application/json

3.4.2.2.11.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
proxyName	`path`	string	`string`				Proxy Name

3.4.2.2.11.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Need reprov!		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema
503	Service Unavailable	No reprov needed!" "Not a Valid Server!		schema

3.4.2.2.11.5 Responses

200 - Need reprov! Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

503 - No reprov needed!" "Not a Valid Server! Status: Service Unavailable

Schema

3.4.2.2.12 Check if Proxy Needs Restart (*GetAPIClustersClusterNameProxiesProxyNameActionsNeedRestart*)

`GET /api/clusters/{clusterName}/proxies/{proxyName}/actions/need-restart`

Check if the proxy service for a given cluster and proxy needs a restart

3.4.2.2.12.1 Consumes

- application/json

3.4.2.2.12.2 Produces

- application/json

3.4.2.2.12.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
proxyName	`path`	string	`string`				Proxy Name

3.4.2.2.12.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Need restart!		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema
503	Service Unavailable	No restart needed!" "Not a Valid Server!		schema

3.4.2.2.12.5 Responses

200 - Need restart! Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

503 - No restart needed!" "Not a Valid Server! Status: Service Unavailable

Schema

3.4.2.2.13 Retrieve query rules for a specific cluster (*GetAPIClustersClusterNameQueryrules*)

`GET /api/clusters/{clusterName}/queryrules`

This endpoint retrieves the query rules for the specified cluster.

3.4.2.2.13.1 Produces

- application/json

3.4.2.2.13.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.13.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of query rules		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.13.4 Responses

200 - List of query rules Status: OK

Schema

[][interface{}](#interface)

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.14 Retrieve schema information for a specific cluster (*GetAPIClustersClusterNameSchema*)

`GET /api/clusters/{clusterName}/schema`

This endpoint retrieves the schema information for the specified cluster.

3.4.2.2.14.1 Produces

- application/json

3.4.2.2.14.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.14.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Schema information		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.14.4 Responses

200 - Schema information Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.15 Perform a backup of the error log on a server (*GetAPIClustersClusterName-ServersServerNameActionsBackupErrorLog*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/backup-error-log`

Initiates a backup of the error log on a specified server within a cluster.

3.4.2.2.15.1 Produces

- application/json

3.4.2.2.15.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.15.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Backup initiated successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.15.4 Responses

200 - Backup initiated successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.16 Perform a logical backup on a server (*GetAPIClustersClusterNameServersServerNameActionsBackupLogical*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/backup-logical`

Initiates a logical backup on a specified server within a cluster.

3.4.2.2.16.1 Produces

- application/json

3.4.2.2.16.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.16.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Backup initiated successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.16.4 Responses

200 - Backup initiated successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.17 Perform a physical backup on a server (*GetAPIClustersClusterNameServersServerNameActionsBackupPhysical*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/backup-physical`

Initiates a physical backup on a specified server within a cluster.

3.4.2.2.17.1 Produces

- application/json

3.4.2.2.17.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.17.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Backup initiated successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.17.4 Responses

200 - Backup initiated successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.18 Perform a backup of the slow query log on a server (*GetAPIClustersClusterNameServersServerNameActionsBackupSlowqueryLog*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/backup-slowquery-log`

Initiates a backup of the slow query log on a specified server within a cluster.

3.4.2.2.18.1 Produces

- application/json

3.4.2.2.18.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.18.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Backup initiated successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.18.4 Responses

200 - Backup initiated successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.19 Delete maintenance mode on a server (*GetAPIClustersClusterNameServersServerNameActionsDelMaintenance*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/del-maintenance`

Deletes the maintenance mode on a specified server within a cluster.

3.4.2.2.19.1 Produces

- application/json

3.4.2.2.19.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.19.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Maintenance mode deleted successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.19.4 Responses

200 - Maintenance mode deleted successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.20 Flush logs on a server (*GetAPIClustersClusterNameServersServerNameActionsFlushLogs*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/flush-logs`

Flushes the logs on a specified server within a cluster.

3.4.2.2.20.1 Produces

- application/json

3.4.2.2.20.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.20.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Logs flushed successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.20.4 Responses

200 - Logs flushed successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.21 Cancel a task on a server (*GetAPIClustersClusterNameServersServerNameActionsJobCancelTask*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/job-cancel/{task}`

Cancels a task identified by its name on a specified server within a cluster.

3.4.2.2.21.1 Produces

- application/json

3.4.2.2.21.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
task	`path`	string	`string`				Task Name

3.4.2.2.21.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Task canceled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Error canceling task		schema

3.4.2.2.21.4 Responses

200 - Task canceled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Error canceling task Status: Internal Server Error

Schema

3.4.2.2.22 Toggle maintenance mode on a server (*GetAPIClustersClusterNameServersServerNameActionsMaintenance*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/maintenance`

Toggles the maintenance mode on a specified server within a cluster.

3.4.2.2.22.1 Produces

- application/json

3.4.2.2.22.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.22.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Maintenance mode toggled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.22.4 Responses

200 - Maintenance mode toggled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.23 Optimize a server (*GetAPIClustersClusterNameServersServerNameActionsOptimize*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/optimize`

Optimizes a specified server within a cluster.

3.4.2.2.23.1 Produces

- application/json

3.4.2.2.23.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.23.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Server optimized successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.23.4 Responses

200 - Server optimized successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.24 Provision a server (*GetAPIClustersClusterNameServersServerNameActionsProvision*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/provision`

Provisions a specified server within a cluster.

3.4.2.2.24.1 Produces

- application/json

3.4.2.2.24.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.24.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Server provisioned successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.24.4 Responses

200 - Server provisioned successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.25 Reseed a server (*GetAPIClustersClusterNameServersServerNameActionsReseedBackupMethod*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/reseed/{backupMethod}`

Reseeds a specified server within a cluster using the specified backup method.

3.4.2.2.25.1 Produces

- application/json

3.4.2.2.25.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
backupMethod	`path`	string	`string`				Backup Method
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.25.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Reseed initiated successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Error reseed logical backup" or "Error reseed physical backup		schema

3.4.2.2.25.4 Responses

200 - Reseed initiated successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Error reseed logical backup" or "Error reseed physical backup Status: Internal Server Error

Schema

3.4.2.2.26 Cancel a reseed task on a server (*GetAPIClustersClusterNameServersServerName-ActionsReseedCancelTask*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/reseed-cancel/{task}`

Cancels a reseed task identified by its name on a specified server within a cluster.

3.4.2.2.26.1 Produces

- application/json

3.4.2.2.26.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
task	`path`	string	`string`				Task Name

3.4.2.2.26.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Task canceled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Error canceling task		schema

3.4.2.2.26.4 Responses

200 - Task canceled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Error canceling task Status: Internal Server Error

Schema

3.4.2.2.27 Reset the master on a server (*GetAPIClustersClusterNameServersServerNameActionsResetMaster*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/reset-master`

Resets the master on a specified server within a cluster.

3.4.2.2.27.1 Produces

- application/json

3.4.2.2.27.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.27.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Master reset successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.27.4 Responses

200 - Master reset successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.28 Reset PFS queries on a server (*GetAPIClustersClusterNameServersServerNameActionsResetPfsQueries*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/reset-pfs-queries`

Resets PFS queries on a specified server within a cluster.

3.4.2.2.28.1 Produces

- application/json

3.4.2.2.28.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.28.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	PFS queries reset successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.28.4 Responses

200 - PFS queries reset successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.29 Reset all slaves on a server (*GetAPIClustersClusterNameServersServerNameActionsResetSlaveAll*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/reset-slave-all`

Resets all slaves on a specified server within a cluster.

3.4.2.2.29.1 Produces

- application/json

3.4.2.2.29.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.29.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Slaves reset successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.29.4 Responses

200 - Slaves reset successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.30 Run jobs on a server (*GetAPIClustersClusterNameServersServerNameActionsRunJobs*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/run-jobs`

Runs jobs on a specified server within a cluster.

3.4.2.2.30.1 Produces

- application/json

3.4.2.2.30.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.30.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Jobs run successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Error running job		schema

3.4.2.2.30.4 Responses

200 - Jobs run successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Error running job Status: Internal Server Error

Schema

3.4.2.2.31 Set a server as ignored (*GetAPIClustersClusterNameServersServerNameActionsSetIgnored*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/set-ignored`

Sets a specified server within a cluster as ignored.

3.4.2.2.31.1 Produces

- application/json

3.4.2.2.31.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.31.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Server set as ignored successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.31.4 Responses

200 - Server set as ignored successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.32 Set long query time on a server (*GetAPIClustersClusterNameServersServerName-ActionsSetLongQueryTimeQueryTime*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/set-long-query-time/{queryTime}`

Sets the long query time on a specified server within a cluster.

3.4.2.2.32.1 Produces

- application/json

3.4.2.2.32.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
queryTime	`path`	string	`string`				Query Time
serverName	`path`	string	`string`				Server Name

3.4.2.2.32.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Long query time set successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.32.4 Responses

200 - Long query time set successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.33 Set a server to maintenance mode (*GetAPIClustersClusterNameServersServer-NameActionsSetMaintenance*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/set-maintenance`

Sets a specified server within a cluster to maintenance mode.

3.4.2.2.33.1 Produces

- application/json

3.4.2.2.33.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.33.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Server set to maintenance mode successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.33.4 Responses

200 - Server set to maintenance mode successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.34 Set a server as preferred (*GetAPIClustersClusterNameServersServerNameActionsSetPrefered*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/set-prefered`

Sets a specified server within a cluster as preferred.

3.4.2.2.34.1 Produces

- application/json

3.4.2.2.34.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.34.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Server set as preferred successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.34.4 Responses

200 - Server set as preferred successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.35 Set a server as unrated (*GetAPIClustersClusterNameServersServerNameActions-SetUnrated*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/set-unrated`

Sets a specified server within a cluster as unrated.

3.4.2.2.35.1 Produces

- application/json

3.4.2.2.35.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.35.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Server set as unrated successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.35.4 Responses

200 - Server set as unrated successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.36 Skip a replication event on a server (*GetAPIClustersClusterNameServersServerNameActionsSkipReplicationEvent*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/skip-replication-event`

Skips a replication event on a specified server within a cluster.

3.4.2.2.36.1 Produces

- application/json

3.4.2.2.36.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.36.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Replication event skipped successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.36.4 Responses

200 - Replication event skipped successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.37 Start a server (*GetAPIClustersClusterNameServersServerNameActionsStart*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/start`

Starts a specified server within a cluster.

3.4.2.2.37.1 Produces

- application/json

3.4.2.2.37.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.37.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Server started successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.37.4 Responses

200 - Server started successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.38 Start the slave on a server (*GetAPIClustersClusterNameServersServerNameActionsStartSlave*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/start-slave`

Starts the slave on a specified server within a cluster.

3.4.2.2.38.1 Produces

- application/json

3.4.2.2.38.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.38.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Slave started successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.38.4 Responses

200 - Slave started successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.39 Stop a server (*GetAPIClustersClusterNameServersServerNameActionsStop*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/stop`

Stops a specified server within a cluster.

3.4.2.2.39.1 Produces

- application/json

3.4.2.2.39.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.39.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Server stopped successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.39.4 Responses

200 - Server stopped successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.40 Stop the slave on a server (*GetAPIClustersClusterNameServersServerNameActionsStopSlave*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/stop-slave`

Stops the slave on a specified server within a cluster.

3.4.2.2.40.1 Produces

- application/json

3.4.2.2.40.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.40.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Slave stopped successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.40.4 Responses

200 - Slave stopped successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.41 Perform a switchover on a server (*GetAPIClustersClusterNameServersServerName-ActionsSwitchover*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/switchover`

Initiates a switchover on a specified server within a cluster.

3.4.2.2.41.1 Produces

- application/json

3.4.2.2.41.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.41.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Switchover initiated successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Master failed, cannot initiate switchover		schema

3.4.2.2.41.4 Responses

200 - Switchover initiated successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Master failed, cannot initiate switchover Status: Internal Server Error

Schema

3.4.2.2.42 Toggle InnoDB monitor on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleInnodbMonitor*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/toogle-innodb-monitor`

Toggles the InnoDB monitor on a specified server within a cluster.

3.4.2.2.42.1 Produces

- application/json

3.4.2.2.42.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.42.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	InnoDB monitor toggled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.42.4 Responses

200 - InnoDB monitor toggled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.43 Toggle metadata locks on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleMetaDataLocks*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/toogle-meta-data-locks`

Toggles the metadata locks on a specified server within a cluster.

3.4.2.2.43.1 Produces

- application/json

3.4.2.2.43.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.43.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Metadata locks toggled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.43.4 Responses

200 - Metadata locks toggled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.44 Toggle PFS slow query capture on a server (*GetAPIClustersClusterNameServersServerNameActionsToogglePfsSlowQuery*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/toogle-pfs-slow-query`

Toggles the PFS slow query capture on a specified server within a cluster.

3.4.2.2.44.1 Produces

- application/json

3.4.2.2.44.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.44.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	PFS slow query capture toggled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.44.4 Responses

200 - PFS slow query capture toggled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.45 Toggle query response time on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleQueryResponseTime*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/toogle-query-response-time`

Toggles the query response time on a specified server within a cluster.

3.4.2.2.45.1 Produces

- application/json

3.4.2.2.45.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.45.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Query response time toggled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.45.4 Responses

200 - Query response time toggled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.46 Toggle read-only mode on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleReadOnly*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/toogle-read-only`

Toggles the read-only mode on a specified server within a cluster.

3.4.2.2.46.1 Produces

- application/json

3.4.2.2.46.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.46.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Read-only mode toggled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.46.4 Responses

200 - Read-only mode toggled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.47 Toggle SQL error log on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleSQLErrorLog*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/toogle-sql-error-log`

Toggles the SQL error log on a specified server within a cluster.

3.4.2.2.47.1 Produces

- application/json

3.4.2.2.47.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.47.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	SQL error log toggled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.47.4 Responses

200 - SQL error log toggled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.48 Toggle slow query on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleSlowQuery*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/toogle-slow-query`

Toggles the slow query on a specified server within a cluster.

3.4.2.2.48.1 Produces

- application/json

3.4.2.2.48.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.48.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Slow query toggled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.48.4 Responses

200 - Slow query toggled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.49 Toggle slow query capture on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleSlowQueryCapture*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/toogle-slow-query-capture`

Toggles the slow query capture on a specified server within a cluster.

3.4.2.2.49.1 Produces

- application/json

3.4.2.2.49.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.49.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Slow query capture toggled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.49.4 Responses

200 - Slow query capture toggled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.50 Toggle slow query table mode on a server (*GetAPIClustersClusterNameServersServerNameActionsToogleSlowQueryTable*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/toogle-slow-query-table`

Toggles the slow query table mode on a specified server within a cluster.

3.4.2.2.50.1 Produces

- application/json

3.4.2.2.50.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.50.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Slow query table mode toggled successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.50.4 Responses

200 - Slow query table mode toggled successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.51 Unprovision a server (*GetAPIClustersClusterNameServersServerNameActionsUnprovision*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/unprovision`

Unprovisions a specified server within a cluster.

3.4.2.2.51.1 Produces

- application/json

3.4.2.2.51.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.51.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Server unprovisioned successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.51.4 Responses

200 - Server unprovisioned successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.52 Wait for InnoDB purge on a server (*GetAPIClustersClusterNameServersServerNameActionsWaitInnodbPurge*)

`GET /api/clusters/{clusterName}/servers/{serverName}/actions/wait-innodb-purge`

Waits for InnoDB purge on a specified server within a cluster.

3.4.2.2.52.1 Produces

- application/json

3.4.2.2.52.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.52.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	InnoDB purge completed successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Error waiting for InnoDB purge		schema

3.4.2.2.52.4 Responses

200 - InnoDB purge completed successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Error waiting for InnoDB purge Status: Internal Server Error

Schema

3.4.2.2.53 Get status of all slaves of a server (*GetAPIClustersClusterNameServersServerNameAllSlavesStatus*)

`GET /api/clusters/{clusterName}/servers/{serverName}/all-slaves-status`

Retrieves the status of all slaves of a specified server within a cluster.

3.4.2.2.53.1 Produces

- application/json

3.4.2.2.53.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.53.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Status of all slaves retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.53.4 Responses

200 - Status of all slaves retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.54 Get PFS statements of a server (*GetAPIClustersClusterNameServersServerNameDigestStatementsPfs*)

`GET /api/clusters/{clusterName}/servers/{serverName}/digest-statements-pfs`

Retrieves the PFS statements of a specified server within a cluster.

3.4.2.2.54.1 Produces

- application/json

3.4.2.2.54.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.54.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	PFS statements retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.54.4 Responses

200 - PFS statements retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.55 Get PFS statements from the slow log of a server (*GetAPIClustersClusterNameServersServerNameDigestStatementsSlow*)

`GET /api/clusters/{clusterName}/servers/{serverName}/digest-statements-slow`

Retrieves the PFS statements from the slow log of a specified server within a cluster.

3.4.2.2.55.1 Produces

- application/json

3.4.2.2.55.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.55.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	PFS statements from slow log retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.55.4 Responses

200 - PFS statements from slow log retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.56 Get error log of a server (*GetAPIClustersClusterNameServersServerNameErrorlog*)

`GET /api/clusters/{clusterName}/servers/{serverName}/errorlog`

Retrieves the error log of a specified server within a cluster.

3.4.2.2.56.1 Produces

- application/json

3.4.2.2.56.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.56.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Error log retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.56.4 Responses

200 - Error log retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.57 Check if a server is a master (*GetAPIClustersClusterNameServersServerNameIsMaster*)

`GET /api/clusters/{clusterName}/servers/{serverName}/is-master`

Checks if a specified server within a cluster is a master.

3.4.2.2.57.1 Produces

- text/plain

3.4.2.2.57.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.57.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Valid Master!		schema
500	Internal Server Error	No cluster		schema
503	Service Unavailable	503 -Not a Valid Master!		schema

3.4.2.2.57.4 Responses

200 - 200 -Valid Master! Status: OK

Schema

500 - No cluster Status: Internal Server Error

Schema

503 - 503 -Not a Valid Master! Status: Service Unavailable

Schema

3.4.2.2.58 Check if a server is a slave (*GetAPIClustersClusterNameServersServerNameIsSlave*)

`GET /api/clusters/{clusterName}/servers/{serverName}/is-slave`

Checks if a specified server within a cluster is a slave.

3.4.2.2.58.1 Produces

- text/plain

3.4.2.2.58.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.58.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Valid Slave!		schema
500	Internal Server Error	No cluster		schema
503	Service Unavailable	503 -Not a Valid Slave!		schema

3.4.2.2.58.4 Responses

200 - 200 -Valid Slave! Status: OK

Schema

500 - No cluster Status: Internal Server Error

Schema

503 - 503 -Not a Valid Slave! Status: Service Unavailable

Schema

3.4.2.2.59 Get master status of a server (*GetAPIClustersClusterNameServersServerNameMasterStatus*)

`GET /api/clusters/{clusterName}/servers/{serverName}/master-status`

Retrieves the master status of a specified server within a cluster.

3.4.2.2.59.1 Produces

- application/json

3.4.2.2.59.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.59.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Master status retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.59.4 Responses

200 - Master status retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.60 Get metadata locks of a server (*GetAPIClustersClusterNameServersServerNameMetaDataLocks*)

`GET /api/clusters/{clusterName}/servers/{serverName}/meta-data-locks`

Retrieves the metadata locks of a specified server within a cluster.

3.4.2.2.60.1 Produces

- application/json

3.4.2.2.60.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.60.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Metadata locks retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.60.4 Responses

200 - Metadata locks retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.61 Get process list of a server (*GetAPIClustersClusterNameServersServerNameProcesslist*)

`GET /api/clusters/{clusterName}/servers/{serverName}/processlist`

Retrieves the process list of a specified server within a cluster.

3.4.2.2.61.1 Produces

- application/json

3.4.2.2.61.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.61.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Process list retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.61.4 Responses

200 - Process list retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.62 Analyze a query using PFS on a server (*GetAPIClustersClusterNameServersServerNameQueriesQueryDigestActionsAnalyzePfs*)

`GET /api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/analyze-pfs`

Analyzes a query identified by its digest on a specified server within a cluster using PFS.

3.4.2.2.62.1 Produces

- application/json

3.4.2.2.62.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
queryDigest	`path`	string	`string`				Query Digest
serverName	`path`	string	`string`				Server Name

3.4.2.2.62.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Query analyzed successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.62.4 Responses

200 - Query analyzed successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.63 Analyze a query using the slow log on a server (*GetAPIClustersClusterName-ServersServerNameQueriesQueryDigestActionsAnalyzeSlowlog*)

`GET /api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/analyze-slowlog`

Analyzes a query identified by its digest on a specified server within a cluster using the slow log.

3.4.2.2.63.1 Produces

- application/json

3.4.2.2.63.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
queryDigest	`path`	string	`string`				Query Digest
serverName	`path`	string	`string`				Server Name

3.4.2.2.63.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Query analyzed successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.63.4 Responses

200 - Query analyzed successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.64 Explain a query using PFS on a server (*GetAPIClustersClusterNameServersServerNameQueriesQueryDigestActionsExplainPfs*)

`GET /api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/explain-pfs`

Explains a query identified by its digest on a specified server within a cluster using PFS.

3.4.2.2.64.1 Produces

- application/json

3.4.2.2.64.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
queryDigest	`path`	string	`string`				Query Digest
serverName	`path`	string	`string`				Server Name

3.4.2.2.64.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Query explained successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.64.4 Responses

200 - Query explained successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.65 Explain a query using the slow log on a server (*GetAPIClustersClusterNameServersServerNameQueriesQueryDigestActionsExplainSlowlog*)

`GET /api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/explain-slowlog`

Explains a query identified by its digest on a specified server within a cluster using the slow log.

3.4.2.2.65.1 Produces

- application/json

3.4.2.2.65.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
queryDigest	`path`	string	`string`				Query Digest
serverName	`path`	string	`string`				Server Name

3.4.2.2.65.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Query explained successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.65.4 Responses

200 - Query explained successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.66 Kill a query on a server (*GetAPIClustersClusterNameServersServerNameQueries-QueryDigestActionsKillQuery*)

`GET /api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/kill-query`

Kills a query identified by its digest on a specified server within a cluster.

3.4.2.2.66.1 Produces

- application/json

3.4.2.2.66.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
queryDigest	`path`	string	`string`				Query Digest
serverName	`path`	string	`string`				Server Name

3.4.2.2.66.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Query killed successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.66.4 Responses

200 - Query killed successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.67 Kill a thread on a server (*GetAPIClustersClusterNameServersServerNameQueries-QueryDigestActionsKillThread*)

`GET /api/clusters/{clusterName}/servers/{serverName}/queries/{queryDigest}/actions/kill-thread`

Kills a thread identified by its digest on a specified server within a cluster.

3.4.2.2.67.1 Produces

- application/json

3.4.2.2.67.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
queryDigest	`path`	string	`string`				Query Digest
serverName	`path`	string	`string`				Server Name

3.4.2.2.67.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Query killed successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.67.4 Responses

200 - Query killed successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.68 Get query response time of a server (*GetAPIClustersClusterNameServersServerNameQueryResponseTime*)

`GET /api/clusters/{clusterName}/servers/{serverName}/query-response-time`

Retrieves the query response time of a specified server within a cluster.

3.4.2.2.68.1 Produces

- application/json

3.4.2.2.68.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.68.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Query response time retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.68.4 Responses

200 - Query response time retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.69 Get schemas of a server (*GetAPIClustersClusterNameServersServerNameSchemas*)

`GET /api/clusters/{clusterName}/servers/{serverName}/schemas`

Retrieves the schemas of a specified server within a cluster.

3.4.2.2.69.1 Produces

- application/json

3.4.2.2.69.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.69.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Schemas retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.69.4 Responses

200 - Schemas retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.70 Perform a physical backup on a server port (*GetAPIClustersClusterNameServersServerNameServerPortBackup*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/backup`

Initiates a physical backup on a specified server port within a cluster.

3.4.2.2.70.1 Produces

- application/json

3.4.2.2.70.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.70.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Backup initiated successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.70.4 Responses

200 - Backup initiated successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.71 Get server port configuration (*GetAPIClustersClusterNameServersServerNameServerPortConfig*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/config`

Retrieves the configuration of a specified server port within a cluster.

3.4.2.2.71.1 Produces

- application/octet-stream

3.4.2.2.71.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.71.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Configuration file		schema
403	Forbidden	No valid ACL		schema
404	Not Found	File not found		schema
500	Internal Server Error	No cluster" or "No server		schema

3.4.2.2.71.4 Responses

200 - Configuration file Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

404 - File not found Status: Not Found

Schema

500 - No cluster" or "No server Status: Internal Server Error

Schema

3.4.2.2.72 Check if a server port is a master (*GetAPIClustersClusterNameServersServerNameServerPortIsMaster*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/is-master`

Checks if a specified server port within a cluster is a master.

3.4.2.2.72.1 Produces

- text/plain

3.4.2.2.72.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.72.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Valid Master!		schema
500	Internal Server Error	No cluster		schema
503	Service Unavailable	503 -Not a Valid Master!		schema

3.4.2.2.72.4 Responses

200 - 200 -Valid Master! Status: OK

Schema

500 - No cluster Status: Internal Server Error

Schema

503 - 503 -Not a Valid Master! Status: Service Unavailable

Schema

3.4.2.2.73 Check if a server port is a slave (*GetAPIClustersClusterNameServersServerNameServerPortIsSlave*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/is-slave`

Checks if a specified server port within a cluster is a slave.

3.4.2.2.73.1 Produces

- text/plain

3.4.2.2.73.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.73.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Valid Slave!		schema
500	Internal Server Error	No cluster		schema
503	Service Unavailable	503 -Not a Valid Slave!		schema

3.4.2.2.73.4 Responses

200 - 200 -Valid Slave! Status: OK

Schema

500 - No cluster Status: Internal Server Error

Schema

503 - 503 -Not a Valid Slave! Status: Service Unavailable

Schema

3.4.2.2.74 Check if a server needs a config change (*GetAPIClustersClusterNameServersServerNameServerPortNeedConfigChange*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-config-change`

Checks if a specified server within a cluster needs a config change.

3.4.2.2.74.1 Produces

- text/plain

3.4.2.2.74.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.74.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Need config change!		schema
500	Internal Server Error	500 -No config change needed!" or "500 -No valid server!" or "500 -No cluster!		schema

3.4.2.2.74.4 Responses

200 - 200 -Need config change! Status: OK

Schema

500 - 500 -No config change needed!" or "500 -No valid server!" or "500 -No cluster! Status: Internal Server Error

Schema

3.4.2.2.75 Check if a server needs provisioning (*GetAPIClustersClusterNameServersServerNameServerPortNeedProv*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-prov`

Checks if a specified server within a cluster needs provisioning.

3.4.2.2.75.1 Produces

- text/plain

3.4.2.2.75.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.75.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Need provisioning!		schema
500	Internal Server Error	503 -Not a Valid Server!		schema
503	Service Unavailable	503 -No provisioning needed!		schema

3.4.2.2.75.4 Responses

200 - 200 -Need provisioning! Status: OK

Schema

500 - 503 -Not a Valid Server! Status: Internal Server Error

Schema

503 - 503 -No provisioning needed! Status: Service Unavailable

Schema

3.4.2.2.76 Check if a server needs re-provisioning (*GetAPIClustersClusterNameServersServerNameServerPortNeedReprov*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-reprov`

Checks if a specified server within a cluster needs re-provisioning.

3.4.2.2.76.1 Produces

- text/plain

3.4.2.2.76.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.76.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Need reprov!		schema
500	Internal Server Error	503 -Not a Valid Server!		schema
503	Service Unavailable	503 -No reprov needed!		schema

3.4.2.2.76.4 Responses

200 - 200 -Need reprov! Status: OK

Schema

500 - 503 -Not a Valid Server! Status: Internal Server Error

Schema

503 - 503 -No reprov needed! Status: Service Unavailable

Schema

3.4.2.2.77 Check if a server needs a restart (*GetAPIClustersClusterNameServersServerNameServerPortNeedRestart*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-restart`

Checks if a specified server within a cluster needs a restart.

3.4.2.2.77.1 Produces

- text/plain

3.4.2.2.77.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.77.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Need restart!		schema
500	Internal Server Error	503 -Not a Valid Server!		schema
503	Service Unavailable	503 -No restart needed!		schema

3.4.2.2.77.4 Responses

200 - 200 -Need restart! Status: OK

Schema

500 - 503 -Not a Valid Server! Status: Internal Server Error

Schema

503 - 503 -No restart needed! Status: Service Unavailable

Schema

3.4.2.2.78 Check if a server needs to start (*GetAPIClustersClusterNameServersServerNameServerPortNeedStart*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-start`

Checks if a specified server within a cluster needs to start.

3.4.2.2.78.1 Produces

- text/plain

3.4.2.2.78.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.78.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Need start!		schema
500	Internal Server Error	500 -No start needed!" or "500 -No valid server!" or "500 -No cluster!		schema

3.4.2.2.78.4 Responses

200 - 200 -Need start! Status: OK

Schema

500 - 500 -No start needed!" or "500 -No valid server!" or "500 -No cluster! Status: Internal Server Error

Schema

3.4.2.2.79 Check if a server needs to stop (*GetAPIClustersClusterNameServersServerNameServerPortNeedStop*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-stop`

Checks if a specified server within a cluster needs to stop.

3.4.2.2.79.1 Produces

- text/plain

3.4.2.2.79.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.79.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Need stop!		schema
500	Internal Server Error	500 -No stop needed!" or "500 -No valid server!" or "500 -No cluster!		schema

3.4.2.2.79.4 Responses

200 - 200 -Need stop! Status: OK

Schema

500 - 500 -No stop needed!" or "500 -No valid server!" or "500 -No cluster! Status: Internal Server Error

Schema

3.4.2.2.80 Check if a server needs unprovisioning (*GetAPIClustersClusterNameServersServerNameServerPortNeedUnprov*)

`GET /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/need-unprov`

Checks if a specified server within a cluster needs unprovisioning.

3.4.2.2.80.1 Produces

- text/plain

3.4.2.2.80.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name
serverPort	`path`	string	`string`				Server Port

3.4.2.2.80.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	200 -Need unprov!		schema
500	Internal Server Error	503 -Not a Valid Server!		schema
503	Service Unavailable	503 -No unprov needed!		schema

3.4.2.2.80.4 Responses

200 - 200 -Need unprov! Status: OK

Schema

500 - 503 -Not a Valid Server! Status: Internal Server Error

Schema

503 - 503 -No unprov needed! Status: Service Unavailable

Schema

3.4.2.2.81 Get database service configuration of a server (*GetAPIClustersClusterNameServersServerNameServiceOpensvc*)

`GET /api/clusters/{clusterName}/servers/{serverName}/service-opensvc`

Retrieves the database service configuration of a specified server within a cluster.

3.4.2.2.81.1 Produces

- application/json

3.4.2.2.81.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.81.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Database service configuration retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found		schema

3.4.2.2.81.4 Responses

200 - Database service configuration retrieved successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.82 Get slow log of a server (*GetAPIClustersClusterNameServersServerNameSlowQueries*)

`GET /api/clusters/{clusterName}/servers/{serverName}/slow-queries`

Retrieves the slow log of a specified server within a cluster.

3.4.2.2.82.1 Produces

- application/json

3.4.2.2.82.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.82.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Slow log retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.82.4 Responses

200 - Slow log retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.83 Get status of a server (*GetAPIClustersClusterNameServersServerNameStatus*)

`GET /api/clusters/{clusterName}/servers/{serverName}/status`

Retrieves the status of a specified server within a cluster.

3.4.2.2.83.1 Produces

- application/json

3.4.2.2.83.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.83.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Status retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.83.4 Responses

200 - Status retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.84 Get status delta of a server (*GetAPIClustersClusterNameServersServerNameStatusDelta*)

`GET /api/clusters/{clusterName}/servers/{serverName}/status-delta`

Retrieves the status delta of a specified server within a cluster.

3.4.2.2.84.1 Produces

- application/json

3.4.2.2.84.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.84.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Status delta retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.84.4 Responses

200 - Status delta retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.85 Get InnoDB status of a server (*GetAPIClustersClusterNameServersServerNameStatusInnodb*)

`GET /api/clusters/{clusterName}/servers/{serverName}/status-innodb`

Retrieves the InnoDB status of a specified server within a cluster.

3.4.2.2.85.1 Produces

- application/json

3.4.2.2.85.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.85.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	InnoDB status retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.85.4 Responses

200 - InnoDB status retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.86 Get tables of a server (*GetAPIClustersClusterNameServersServerNameTables*)

`GET /api/clusters/{clusterName}/servers/{serverName}/tables`

Retrieves the tables of a specified server within a cluster.

3.4.2.2.86.1 Produces

- application/json

3.4.2.2.86.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.86.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Tables retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.86.4 Responses

200 - Tables retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.87 Get variables of a server (*GetAPIClustersClusterNameServersServerNameVariables*)

`GET /api/clusters/{clusterName}/servers/{serverName}/variables`

Retrieves the variables of a specified server within a cluster.

3.4.2.2.87.1 Produces

- application/json

3.4.2.2.87.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.87.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Variables retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.87.4 Responses

200 - Variables retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.88 Get virtual tables of a server (*GetAPIClustersClusterNameServersServerNameVtables*)

`GET /api/clusters/{clusterName}/servers/{serverName}/vtables`

Retrieves the virtual tables of a specified server within a cluster.

3.4.2.2.88.1 Produces

- application/json

3.4.2.2.88.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.88.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Virtual tables retrieved successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Encoding error		schema

3.4.2.2.88.4 Responses

200 - Virtual tables retrieved successfully Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Encoding error Status: Internal Server Error

Schema

3.4.2.2.89 Retrieve settings for a specific cluster (*GetAPIClustersClusterNameSettings*)

`GET /api/clusters/{clusterName}/settings`

This endpoint retrieves the settings for the specified cluster.

3.4.2.2.89.1 Produces

- application/json

3.4.2.2.89.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.89.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Cluster settings		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.89.4 Responses

200 - Cluster settings Status: OK

Schema

GithubComSignal18ReplicationManagerConfigConfig

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.90 Retrieve shard clusters for a specific cluster (*GetAPIClustersClusterNameShardclusters*)

`GET /api/clusters/{clusterName}/shardclusters`

This endpoint retrieves the shard clusters for the specified cluster.

3.4.2.2.90.1 Produces

- application/json

3.4.2.2.90.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.90.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of shard clusters		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.90.4 Responses

200 - List of shard clusters Status: OK

Schema

[][interface{}](#interface)

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.91 Get Sphinx Indexes (*GetAPIClustersClusterNameSphinxIndexes*)

`GET /api/clusters/{clusterName}/sphinx/indexes`

Get the Sphinx indexes for a given cluster

3.4.2.2.91.1 Consumes

- application/json

3.4.2.2.91.2 Produces

- application/json

3.4.2.2.91.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.91.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Sphinx Indexes		schema
403	Forbidden	No valid ACL		schema
404	Not Found	Something went wrong		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.91.5 Responses

200 - Sphinx Indexes Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

404 - Something went wrong Status: Not Found

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.92 Retrieve status of a cluster (*GetAPIClustersClusterNameStatus*)

`GET /api/clusters/{clusterName}/status`

This endpoint retrieves the status of a specified cluster and returns it in JSON format.

3.4.2.2.92.1 Produces

- application/json

3.4.2.2.92.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.92.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Cluster status		schema
400	Bad Request	No cluster found		schema

3.4.2.2.92.4 Responses

200 - Cluster status Status: OK

Schema

map of string

400 - No cluster found Status: Bad Request

Schema

3.4.2.2.93 Retrieve tags for a specific cluster (*GetAPIClustersClusterNameTags*)

`GET /api/clusters/{clusterName}/tags`

This endpoint retrieves the tags for the specified cluster.

3.4.2.2.93.1 Produces

- application/json

3.4.2.2.93.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.93.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of tags		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.93.4 Responses

200 - List of tags Status: OK

Schema

[]string

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.94 Retrieve top metrics for a specific cluster (*GetAPIClustersClusterNameTop*)

`GET /api/clusters/{clusterName}/top`

This endpoint retrieves the top metrics for the specified cluster.

3.4.2.2.94.1 Produces

- application/json

3.4.2.2.94.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`query`	string	`string`				Server Name

3.4.2.2.94.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Top metrics		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.94.4 Responses

200 - Top metrics Status: OK

Schema

any

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.95 Shows the alerts for that specific named cluster (*GetAPIClustersClusterNameTopologyAlerts*)

`GET /api/clusters/{clusterName}/topology/alerts`

Shows the alerts for that specific named cluster

3.4.2.2.95.1 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.95.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	A list of alerts		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.95.3 Responses

200 - A list of alerts Status: OK

Schema

ClusterAlerts

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.96 Retrieve crashes for a specific cluster (*GetAPIClustersClusterNameTopology-Crashes*)

`GET /api/clusters/{clusterName}/topology/crashes`

This endpoint retrieves the crashes for the specified cluster.

3.4.2.2.96.1 Produces

- application/json

3.4.2.2.96.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.96.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of crashes		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.96.4 Responses

200 - List of crashes Status: OK

Schema

[]string

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.97 Retrieve logs for a specific cluster (*GetAPIClustersClusterNameTopologyLogs*)

`GET /api/clusters/{clusterName}/topology/logs`

This endpoint retrieves the logs for the specified cluster.

3.4.2.2.97.1 Produces

- application/json

3.4.2.2.97.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.97.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of logs		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.97.4 Responses

200 - List of logs Status: OK

Schema

[]string

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.98 Retrieve master of a cluster (*GetAPIClustersClusterNameTopologyMaster*)

`GET /api/clusters/{clusterName}/topology/master`

This endpoint retrieves the master of a specified cluster and returns it in JSON format.

3.4.2.2.98.1 Produces

- application/json

3.4.2.2.98.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.98.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Master server		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.98.4 Responses

200 - Master server Status: OK

Schema

ClusterServerMonitor

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.99 Shows the proxies for that specific named cluster (*GetAPIClustersClusterName-TopologyProxies*)

`GET /api/clusters/{clusterName}/topology/proxies`

Shows the proxies for that specific named cluster

3.4.2.2.99.1 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.99.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	A list of proxies		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.99.3 Responses

200 - A list of proxies Status: OK

Schema

[][interface{}](#interface)

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.100 Retrieve servers for a specific cluster (*GetAPIClustersClusterNameTopology-Servers*)

`GET /api/clusters/{clusterName}/topology/servers`

This endpoint retrieves the servers for the specified cluster.

3.4.2.2.100.1 Produces

- application/json

3.4.2.2.100.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.100.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of servers		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.100.4 Responses

200 - List of servers Status: OK

Schema

[][interface{}](#interface)

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.101 Shows the slaves for that specific named cluster (*GetAPIClustersClusterName-TopologySlaves*)

`GET /api/clusters/{clusterName}/topology/slaves`

Shows the slaves for that specific named cluster

3.4.2.2.101.1 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.101.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	A list of slaves		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.101.3 Responses

200 - A list of slaves Status: OK

Schema

[][interface{}](#interface)

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.102 Retrieve peer clusters for sale (*GetAPIClustersForSale*)

`GET /api/clusters/for-sale`

This endpoint returns a list of peer clusters that are available for sale, excluding those that are booked by the requesting user.

3.4.2.2.102.1 Produces

- application/json

3.4.2.2.102.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
Authorization	`header`	string	`string`				JWT token

3.4.2.2.102.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of peer clusters available for sale		schema
401	Unauthorized	Unauthenticated resource		schema
500	Internal Server Error	Failed to get token claims or Error Marshal		schema

3.4.2.2.102.4 Responses

200 - List of peer clusters available for sale Status: OK

Schema

[][ConfigPeerCluster](#config-peer-cluster)

401 - Unauthenticated resource Status: Unauthorized

Schema

500 - Failed to get token claims or Error Marshal Status: Internal Server Error

Schema

3.4.2.2.103 Retrieve peer clusters for a user (*GetAPIClustersPeers*)

`GET /api/clusters/peers`

This endpoint retrieves the peer clusters that a user has access to.

3.4.2.2.103.1 Produces

- application/json

3.4.2.2.103.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
Authorization	`header`	string	`string`				Bearer token

3.4.2.2.103.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of peer clusters		schema
401	Unauthorized	Unauthenticated resource		schema
500	Internal Server Error	Failed to get token claims or Error Marshal		schema

3.4.2.2.103.4 Responses

200 - List of peer clusters Status: OK

Schema

[][ConfigPeerCluster](#config-peer-cluster)

401 - Unauthenticated resource Status: Unauthorized

Schema

500 - Failed to get token claims or Error Marshal Status: Internal Server Error

Schema

3.4.2.2.104 List Grafana files (*GetAPIConfigsGrafana*)

`GET /api/configs/grafana`

Returns a list of Grafana files from the specified directory.

3.4.2.2.104.1 Produces

- application/json

3.4.2.2.104.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of Grafana files		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.104.3 Responses

200 - List of Grafana files Status: OK

Schema

[]string

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.105 Monitor Heartbeat (*GetAPIHeartbeat*)

`GET /api/heartbeat`

Returns the heartbeat status of the replication manager.

3.4.2.2.105.1 Consumes

- application/json

3.4.2.2.105.2 Produces

- application/json

3.4.2.2.105.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	OK		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.105.4 Responses

200 - OK Status: OK

Schema

ServerHeartbeat

500 - Internal Server Error Status: Internal Server Error

Schema

map of string

3.4.2.2.106 Handles replication manager requests (*GetAPIMonitor*)

`GET /api/monitor`

This endpoint processes the replication manager requests, validates cluster ACLs, and returns the cluster list in JSON format.

3.4.2.2.106.1 Consumes

- application/json

3.4.2.2.106.2 Produces

- application/json

3.4.2.2.106.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successful response with replication manager details		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.106.4 Responses

200 - Successful response with replication manager details Status: OK

Schema

ServerReplicationManager

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.107 Fetch Prometheus metrics (*GetAPIPrometheus*)

`GET /api/prometheus`

Fetches Prometheus metrics for all servers in all clusters managed by the replication manager.

3.4.2.2.107.1 Produces

- text/plain

3.4.2.2.107.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	Prometheus metrics		schema
500	Internal Server Error	Internal Server Error		schema

3.4.2.2.107.3 Responses

200 - Prometheus metrics Status: OK

Schema

500 - Internal Server Error Status: Internal Server Error

Schema

3.4.2.2.108 Retrieve current repository component (*GetAPIRepocompCurrent*)

`GET /api/repocomp/current`

Reads the current repository component from the specified directory and returns its content.

3.4.2.2.108.1 Produces

- text/plain

3.4.2.2.108.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	Current repository component content		schema
404	Not Found	404 Something went wrong - Not Found		schema

3.4.2.2.108.3 Responses

200 - Current repository component content Status: OK

Schema

404 - 404 Something went wrong - Not Found Status: Not Found

Schema

3.4.2.2.109 Get Replication Manager Status (*GetAPIStatus*)

`GET /api/status`

Returns the status of the replication manager indicating whether it is running or starting.

3.4.2.2.109.1 Produces

- application/json

3.4.2.2.109.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	{"alive": "running"} or {"alive": "starting"}		schema

3.4.2.2.109.3 Responses

200 - {"alive": "running"} or {"alive": "starting"} Status: OK

Schema

map of string

3.4.2.2.110 Retrieves terms (*GetAPITerms*)

`GET /api/terms`

This endpoint returns the terms managed by the replication manager.

3.4.2.2.110.1 Produces

- text/plain

3.4.2.2.110.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	Terms		schema

3.4.2.2.110.3 Responses

200 - Terms Status: OK

Schema

3.4.2.2.111 Check if the replication manager is running (*GetAPITimeout*)

`GET /api/timeout`

This endpoint is used to check if the replication manager is running. It will respond with a JSON object after a delay of 1200 seconds.

3.4.2.2.111.1 Produces

- application/json

3.4.2.2.111.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	OK		schema

3.4.2.2.111.3 Responses

200 - OK Status: OK

Schema

map of string

3.4.2.2.112 Add a new cluster (*PostAPIClustersActionsAddClusterName*)

`POST /api/clusters/actions/add/{clusterName}`

Adds a new cluster to the replication manager. If the cluster already exists, it returns an error.

3.4.2.2.112.1 Consumes

- application/json

3.4.2.2.112.2 Produces

- application/json

3.4.2.2.112.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Descriptio
clusterName	`path`	string	`string`				Cluster Na
cluster	`body`	ClusterClusterForm	`models.ClusterClusterForm`				Cluster Fo

3.4.2.2.112.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Cluster added successfully		schema
400	Bad Request	Error in request or Cluster already exists		schema
500	Internal Server Error	User is not valid		schema

3.4.2.2.112.5 Responses

200 - Cluster added successfully Status: OK

Schema

ClusterCluster

400 - Error in request or Cluster already exists Status: Bad Request

Schema

500 - User is not valid Status: Internal Server Error

Schema

3.4.2.2.113 Add a sharding cluster to an existing cluster (*PostAPIClustersClusterNameActionsAddClusterShardingName*)

`POST /api/clusters/{clusterName}/actions/add/{clusterShardingName}`

This endpoint adds a sharding cluster to an existing cluster and triggers a rolling restart.

3.4.2.2.113.1 Consumes

- application/json

3.4.2.2.113.2 Produces

- application/json

3.4.2.2.113.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
clusterShardingName	`path`	string	`string`				Cluster Sharding Name

3.4.2.2.113.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Sharding cluster added successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.113.5 Responses

200 - Sharding cluster added successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.114 Add a server to a specific cluster (*PostAPIClustersClusterNameActionsAddserverHostPort*)

`POST /api/clusters/{clusterName}/actions/addserver/{host}/{port}`

This endpoint adds a server to the specified cluster.

3.4.2.2.114.1 Consumes

- application/json

3.4.2.2.114.2 Produces

- application/json

3.4.2.2.114.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
host	`path`	string	`string`				Host
port	`path`	string	`string`				Port

3.4.2.2.114.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Monitor added		schema
403	Forbidden	No valid ACL		schema
409	Conflict	Error adding new monitor		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.114.5 Responses

200 - Monitor added Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

409 - Error adding new monitor Status: Conflict

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.115 Add a server to a specific cluster (*PostAPIClustersClusterNameActionsAddserverHostPortType*)

`POST /api/clusters/{clusterName}/actions/addserver/{host}/{port}/{type}`

This endpoint adds a server to the specified cluster.

3.4.2.2.115.1 Consumes

- application/json

3.4.2.2.115.2 Produces

- application/json

3.4.2.2.115.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
host	`path`	string	`string`				Host
port	`path`	string	`string`				Port
type	`path`	string	`string`				Type

3.4.2.2.115.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Monitor added		schema
403	Forbidden	No valid ACL		schema
409	Conflict	Error adding new monitor		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.115.5 Responses

200 - Monitor added Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

409 - Error adding new monitor Status: Conflict

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.116 Cancel rolling reprovision for a specific cluster (*PostAPIClustersClusterNameActionsCancelRollingReprov*)

`POST /api/clusters/{clusterName}/actions/cancel-rolling-reprov`

This endpoint cancels the rolling reprovision for the specified cluster.

3.4.2.2.116.1 Consumes

- application/json

3.4.2.2.116.2 Produces

- application/json

3.4.2.2.116.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.116.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully cancelled rolling reprovision		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.116.5 Responses

200 - Successfully cancelled rolling reprovision Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.117 Cancel rolling restart for a specific cluster (*PostAPIClustersClusterNameActionsCancelRollingRestart*)

`POST /api/clusters/{clusterName}/actions/cancel-rolling-restart`

This endpoint cancels the rolling restart for the specified cluster.

3.4.2.2.117.1 Consumes

- application/json

3.4.2.2.117.2 Produces

- application/json

3.4.2.2.117.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.117.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully cancelled rolling restart		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.117.5 Responses

200 - Successfully cancelled rolling restart Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.118 Rotate keys for a specific cluster (*PostAPIClustersClusterNameActionsCertificatesRotate*)

`POST /api/clusters/{clusterName}/actions/certificates-rotate`

Rotate the keys for the specified cluster

3.4.2.2.118.1 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.118.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	Keys rotated successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.118.3 Responses

200 - Keys rotated successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.119 Calculate checksum for all tables in a specific cluster (*PostAPIClustersClusterNameActionsChecksumAllTables*)

`POST /api/clusters/{clusterName}/actions/checksum-all-tables`

This endpoint triggers the checksum calculation for all tables in the specified cluster.

3.4.2.2.119.1 Consumes

- application/json

3.4.2.2.119.2 Produces

- application/json

3.4.2.2.119.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.119.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully triggered checksum calculation for all tables		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.119.5 Responses

200 - Successfully triggered checksum calculation for all tables Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.120 Handles the failover process for a given cluster. (*PostAPIClustersClusterName-ActionsFailover*)

`POST /api/clusters/{clusterName}/actions/failover`

This endpoint triggers a master failover for the specified cluster.

3.4.2.2.120.1 Consumes

- application/json

3.4.2.2.120.2 Produces

- application/json

3.4.2.2.120.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.120.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully triggered failover		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.120.5 Responses

200 - Successfully triggered failover Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.121 Perform a physical backup for the master of a specific cluster (*PostAPIClustersClusterNameActionsMasterPhysicalBackup*)

`POST /api/clusters/{clusterName}/actions/master-physical-backup`

This endpoint triggers a physical backup for the master of the specified cluster.

3.4.2.2.121.1 Consumes

- application/json

3.4.2.2.121.2 Produces

- application/json

3.4.2.2.121.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.121.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully triggered physical backup		schema
400	Bad Request	No cluster found		schema
403	Forbidden	No valid ACL		schema

3.4.2.2.121.5 Responses

200 - Successfully triggered physical backup Status: OK

Schema

400 - No cluster found Status: Bad Request

Schema

403 - No valid ACL Status: Forbidden

Schema

3.4.2.2.122 Optimize a specific cluster (*PostAPIClustersClusterNameActionsOptimize*)

`POST /api/clusters/{clusterName}/actions/optimize`

This endpoint triggers the optimization process for the specified cluster.

3.4.2.2.122.1 Consumes

- application/json

3.4.2.2.122.2 Produces

- application/json

3.4.2.2.122.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.122.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully triggered optimization		schema
400	Bad Request	No cluster found		schema
403	Forbidden	No valid ACL		schema

3.4.2.2.122.5 Responses

200 - Successfully triggered optimization Status: OK

Schema

400 - No cluster found Status: Bad Request

Schema

403 - No valid ACL Status: Forbidden

Schema

3.4.2.2.123 Bootstrap replication for a specific cluster (*PostAPIClustersClusterNameActionsReplicationBootstrapTopology*)

`POST /api/clusters/{clusterName}/actions/replication/bootstrap/{topology}`

This endpoint triggers the bootstrap replication process for the specified cluster.

3.4.2.2.123.1 Consumes

- application/json

3.4.2.2.123.2 Produces

- application/json

3.4.2.2.123.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
topology	`path`	string	`string`				Topology

3.4.2.2.123.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully bootstrapped replication		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.123.5 Responses

200 - Successfully bootstrapped replication Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.124 Cleanup replication bootstrap for a specific cluster (*PostAPIClustersClusterName-ActionsReplicationCleanup*)

`POST /api/clusters/{clusterName}/actions/replication/cleanup`

This endpoint triggers the cleanup process for replication bootstrap for the specified cluster.

3.4.2.2.124.1 Consumes

- application/json

3.4.2.2.124.2 Produces

- application/json

3.4.2.2.124.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.124.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully cleaned up replication bootstrap		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.124.5 Responses

200 - Successfully cleaned up replication bootstrap Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.125 Reset failover control for a specific cluster (*PostAPIClustersClusterNameActionsResetFailoverControl*)

`POST /api/clusters/{clusterName}/actions/reset-failover-control`

This endpoint resets the failover control for the specified cluster.

3.4.2.2.125.1 Consumes

- application/json

3.4.2.2.125.2 Produces

- application/json

3.4.2.2.125.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.125.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully reset failover control		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.125.5 Responses

200 - Successfully reset failover control Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.126 Reset SLA for a specific cluster (*PostAPIClustersClusterNameActionsResetSLA*)

`POST /api/clusters/{clusterName}/actions/reset-sla`

Reset the SLA for the specified cluster

3.4.2.2.126.1 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.126.2 All responses

Code	Status	Description	Has headers	Schema
200	OK	SLA reset successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.126.3 Responses

200 - SLA reset successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.127 Handles the rolling restart process for a given cluster. (*PostAPIClustersClusterNameActionsRolling*)

`POST /api/clusters/{clusterName}/actions/rolling`

This endpoint triggers a rolling restart for the specified cluster.

3.4.2.2.127.1 Consumes

- application/json

3.4.2.2.127.2 Produces

- application/json

3.4.2.2.127.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.127.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully triggered rolling restart		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.127.5 Responses

200 - Successfully triggered rolling restart Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.128 Rotate passwords for a specific cluster (*PostAPIClustersClusterNameActionsRotatePasswords*)

`POST /api/clusters/{clusterName}/actions/rotate-passwords`

This endpoint rotates the passwords for the specified cluster.

3.4.2.2.128.1 Consumes

- application/json

3.4.2.2.128.2 Produces

- application/json

3.4.2.2.128.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.128.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully rotated passwords		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.128.5 Responses

200 - Successfully rotated passwords Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.129 Start traffic for a specific cluster (*PostAPIClustersClusterNameActionsStartTraffic*)

`POST /api/clusters/{clusterName}/actions/start-traffic`

This endpoint starts traffic for the specified cluster.

3.4.2.2.129.1 Consumes

- application/json

3.4.2.2.129.2 Produces

- application/json

3.4.2.2.129.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.129.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully started traffic		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.129.5 Responses

200 - Successfully started traffic Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.130 Stop traffic for a specific cluster (*PostAPIClustersClusterNameActionsStopTraffic*)

`POST /api/clusters/{clusterName}/actions/stop-traffic`

This endpoint stops traffic for the specified cluster.

3.4.2.2.130.1 Consumes

- application/json

3.4.2.2.130.2 Produces

- application/json

3.4.2.2.130.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.130.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully stopped traffic		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.130.5 Responses

200 - Successfully stopped traffic Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.131 Handles the switchover process for a given cluster. (*PostAPIClustersClusterNameActionsSwitchover*)

`POST /api/clusters/{clusterName}/actions/switchover`

This endpoint triggers a master switchover for the specified cluster.

3.4.2.2.131.1 Consumes

- application/json

3.4.2.2.131.2 Produces

- application/json

3.4.2.2.131.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
prefmaster	`formData`	string	`string`				Preferred Master

3.4.2.2.131.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully triggered switchover		schema
400	Bad Request	Master failed		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.131.5 Responses

200 - Successfully triggered switchover Status: OK

Schema

400 - Master failed Status: Bad Request

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.132 Run sysbench for a specific cluster (*PostAPIClustersClusterNameActionsSysbench*)

`POST /api/clusters/{clusterName}/actions/sysbench`

This endpoint runs sysbench for the specified cluster.

3.4.2.2.132.1 Consumes

- application/json

3.4.2.2.132.2 Produces

- application/json

3.4.2.2.132.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
threads	`query`	string	`string`				Number of threads

3.4.2.2.132.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully triggered sysbench		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.132.5 Responses

200 - Successfully triggered sysbench Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.133 Wait for databases to be ready for a specific cluster (*PostAPIClustersClusterNameActionsWaitdatabases*)

`POST /api/clusters/{clusterName}/actions/waitdatabases`

This endpoint waits for the databases to be ready for the specified cluster.

3.4.2.2.133.1 Consumes

- application/json

3.4.2.2.133.2 Produces

- application/json

3.4.2.2.133.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.133.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Databases are ready		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.133.5 Responses

200 - Databases are ready Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.134 Provision Proxy Service (*PostAPIClustersClusterNameProxiesProxyNameActionsProvision*)

`POST /api/clusters/{clusterName}/proxies/{proxyName}/actions/provision`

Provision the proxy service for a given cluster and proxy

3.4.2.2.134.1 Consumes

- application/json

3.4.2.2.134.2 Produces

- application/json

3.4.2.2.134.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
proxyName	`path`	string	`string`				Proxy Name

3.4.2.2.134.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Proxy Service Provisioned		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" "Server Not Found		schema

3.4.2.2.134.5 Responses

200 - Proxy Service Provisioned Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.135 Start Proxy Service (*PostAPIClustersClusterNameProxiesProxyNameActionsStart*)

`POST /api/clusters/{clusterName}/proxies/{proxyName}/actions/start`

Start the proxy service for a given cluster and proxy

3.4.2.2.135.1 Consumes

- application/json

3.4.2.2.135.2 Produces

- application/json

3.4.2.2.135.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
proxyName	`path`	string	`string`				Proxy Name

3.4.2.2.135.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Proxy Service Started		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" "Server Not Found		schema

3.4.2.2.135.5 Responses

200 - Proxy Service Started Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.136 Stop Proxy Service (*PostAPIClustersClusterNameProxiesProxyNameActionsStop*)

`POST /api/clusters/{clusterName}/proxies/{proxyName}/actions/stop`

Stop the proxy service for a given cluster and proxy

3.4.2.2.136.1 Consumes

- application/json

3.4.2.2.136.2 Produces

- application/json

3.4.2.2.136.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
proxyName	`path`	string	`string`				Proxy Name

3.4.2.2.136.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Proxy Service Stopped		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" "Server Not Found		schema

3.4.2.2.136.5 Responses

200 - Proxy Service Stopped Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.137 Unprovision Proxy Service (*PostAPIClustersClusterNameProxiesProxyNameActionsUnprovision*)

`POST /api/clusters/{clusterName}/proxies/{proxyName}/actions/unprovision`

Unprovision the proxy service for a given cluster and proxy

3.4.2.2.137.1 Consumes

- application/json

3.4.2.2.137.2 Produces

- application/json

3.4.2.2.137.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
proxyName	`path`	string	`string`				Proxy Name

3.4.2.2.137.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Proxy Service Unprovisioned		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" "Server Not Found		schema

3.4.2.2.137.5 Responses

200 - Proxy Service Unprovisioned Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" "Server Not Found Status: Internal Server Error

Schema

3.4.2.2.138 Accept a subscription for a specific cluster (*PostAPIClustersClusterNameSalesAcceptSubscription*)

`POST /api/clusters/{clusterName}/sales/accept-subscription`

This endpoint accepts a subscription for the specified cluster.

3.4.2.2.138.1 Consumes

- application/json

3.4.2.2.138.2 Produces

- application/json

3.4.2.2.138.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
body	`body`	ClusterUserForm	`models.ClusterUserForm`				User Form

3.4.2.2.138.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Email sent to sponsor!		schema
403	Forbidden	No valid ACL		schema

Code	Status	Description	Has headers	Schema
500	Internal Server Error	Error accepting subscription		schema

3.4.2.2.138.5 Responses

200 - Email sent to sponsor! Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Error accepting subscription Status: Internal Server Error

Schema

3.4.2.2.139 Remove a sponsor from a specific cluster (*PostAPIClustersClusterNameSalesEndSubscription*)

`POST /api/clusters/{clusterName}/sales/end-subscription`

This endpoint removes a sponsor from the specified cluster.

3.4.2.2.139.1 Consumes

- application/json

3.4.2.2.139.2 Produces

- application/json

3.4.2.2.139.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
body	`body`	ClusterUserForm	`models.ClusterUserForm`				User Form

3.4.2.2.139.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Sponsor subscription removed!		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Error removing sponsor subscription		schema

3.4.2.2.139.5 Responses

200 - Sponsor subscription removed! Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Error removing sponsor subscription Status: Internal Server Error

Schema

3.4.2.2.140 Reject a subscription for a specific cluster (*PostAPIClustersClusterNameSalesRefuseSubscription*)

`POST /api/clusters/{clusterName}/sales/refuse-subscription`

This endpoint rejects a subscription for the specified cluster.

3.4.2.2.140.1 Consumes

- application/json

3.4.2.2.140.2 Produces

- application/json

3.4.2.2.140.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
body	`body`	ClusterUserForm	`models.ClusterUserForm`				User Form

3.4.2.2.140.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Subscription removed!		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Error removing subscription		schema

3.4.2.2.140.5 Responses

200 - Subscription removed! Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Error removing subscription Status: Internal Server Error

Schema

3.4.2.2.141 Calculate checksum for a specific table in a specific cluster (*PostAPIClustersClusterNameSchemaSchemaNameTableNameActionsChecksumTable*)

`POST /api/clusters/{clusterName}/schema/{schemaName}/{tableName}/actions/checksum-table`

This endpoint triggers the checksum calculation for a specific table in the specified cluster.

3.4.2.2.141.1 Consumes

- application/json

3.4.2.2.141.2 Produces

- application/json

3.4.2.2.141.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
schemaName	`path`	string	`string`				Schema Name
tableName	`path`	string	`string`				Table Name

3.4.2.2.141.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully triggered checksum calculation for the table		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.141.5 Responses

200 - Successfully triggered checksum calculation for the table Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.142 Move a table to a different shard cluster (*PostAPIClustersClusterNameSchemaSchemaNameTableNameActionsMoveTableClusterShard*)

`POST /api/clusters/{clusterName}/schema/{schemaName}/{tableName}/actions/move-table/{clusterShard}`

This endpoint moves a table to a different shard cluster for the specified cluster.

3.4.2.2.142.1 Consumes

- application/json

3.4.2.2.142.2 Produces

- application/json

3.4.2.2.142.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
clusterShard	`path`	string	`string`				Cluster Shard
schemaName	`path`	string	`string`				Schema Name
tableName	`path`	string	`string`				Table Name

3.4.2.2.142.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully moved table		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.142.5 Responses

200 - Successfully moved table Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.143 Reshard a table for a specific cluster (*PostAPIClustersClusterNameSchemaSchemaNameTableNameActionsReshardTable*)

`POST /api/clusters/{clusterName}/schema/{schemaName}/{tableName}/actions/reshard-table`

This endpoint triggers the resharding of a table for the specified cluster.

3.4.2.2.143.1 Consumes

- application/json

3.4.2.2.143.2 Produces

- application/json

3.4.2.2.143.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
schemaName	`path`	string	`string`				Schema Name
tableName	`path`	string	`string`				Table Name

3.4.2.2.143.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully triggered resharding of the table		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.143.5 Responses

200 - Successfully triggered resharding of the table Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.144 Reshard a table for a specific cluster (*PostAPIClustersClusterNameSchemaSchemaNameTableNameActionsReshardTableClusterList*)

`POST /api/clusters/{clusterName}/schema/{schemaName}/{tableName}/actions/reshard-table/{clusterList}`

This endpoint triggers the resharding of a table for the specified cluster.

3.4.2.2.144.1 Consumes

- application/json

3.4.2.2.144.2 Produces

- application/json

3.4.2.2.144.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterList	`path`	string	`string`				Cluster List
clusterName	`path`	string	`string`				Cluster Name
schemaName	`path`	string	`string`				Schema Name
tableName	`path`	string	`string`				Table Name

3.4.2.2.144.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully triggered resharding of the table		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.144.5 Responses

200 - Successfully triggered resharding of the table Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.145 Set a universal table for a specific cluster (*PostAPIClustersClusterNameSchemaSchemaNameTableNameActionsUniversalTable*)

`POST /api/clusters/{clusterName}/schema/{schemaName}/{tableName}/actions/universal-table`

This endpoint sets a universal table for the specified cluster.

3.4.2.2.145.1 Consumes

- application/json

3.4.2.2.145.2 Produces

- application/json

3.4.2.2.145.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
schemaName	`path`	string	`string`				Schema Name
tableName	`path`	string	`string`				Table Name

3.4.2.2.145.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully set universal table		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.145.5 Responses

200 - Successfully set universal table Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.146 Send Vault token to a specific cluster (*PostAPIClustersClusterNameSendVaultToken*)

`POST /api/clusters/{clusterName}/send-vault-token`

This endpoint sends the Vault token to the specified cluster via email.

3.4.2.2.146.1 Consumes

- application/json

3.4.2.2.146.2 Produces

- application/json

3.4.2.2.146.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.146.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Vault token sent successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.146.5 Responses

200 - Vault token sent successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.147 Perform a point-in-time recovery on a server (*PostAPIClustersClusterNameServersServerNameActionsPitr*)

`POST /api/clusters/{clusterName}/servers/{serverName}/actions/pitr`

Initiates a point-in-time recovery on a specified server within a cluster.

3.4.2.2.147.1 Produces

- application/json

3.4.2.2.147.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
serverName	`path`	string	`string`				Server Name

3.4.2.2.147.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	PITR initiated successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found" or "Server Not Found" or "Decode error" or "PITR error		schema

3.4.2.2.147.4 Responses

200 - PITR initiated successfully Status: OK

Schema

ServerAPIResponse

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found" or "Server Not Found" or "Decode error" or "PITR error Status: Internal Server Error

Schema

3.4.2.2.148 Write logs for a server (*PostAPIClustersClusterNameServersServerNameServerPortWriteLogTask*)

`POST /api/clusters/{clusterName}/servers/{serverName}/{serverPort}/write-log/{task}`

Writes logs for a specified server within a cluster.

3.4.2.2.148.1 Produces

- application/json

3.4.2.2.148.2 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Nar
serverName	`path`	string	`string`				Server Nam
serverPort	`path`	string	`string`				Server Port
task	`path`	string	`string`				Task
data	`body`	ServerDecodedData	`models.ServerDecodedData`				Log Data

3.4.2.2.148.3 All responses

Code	Status	Description	Has headers	Schema
200	OK	Message logged		schema
400	Bad Request	Bad request: Task is not registered" or "Decode reading body" or "Decode body		schema
500	Internal Server Error	No cluster" or "No server" or "Error decrypting data		schema

3.4.2.2.148.4 Responses

200 - Message logged Status: OK

Schema

ServerAPIResponse

400 - Bad request: Task is not registered" or "Decode reading body" or "Decode body Status: Bad Request

Schema

500 - No cluster" or "No server" or "Error decrypting data Status: Internal Server Error

Schema

3.4.2.2.149 Provision services for a specific cluster (*PostAPIClustersClusterNameServicesActionsProvision*)

`POST /api/clusters/{clusterName}/services/actions/provision`

This endpoint provisions services for the specified cluster.

3.4.2.2.149.1 Consumes

- application/json

3.4.2.2.149.2 Produces

- application/json

3.4.2.2.149.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.149.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully provisioned services		schema
403	Forbidden	No valid ACL		schema

Code	Status	Description	Has headers	Schema
500	Internal Server Error	No cluster		schema

3.4.2.2.149.5 Responses

200 - Successfully provisioned services Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.150 Unprovision services for a specific cluster (*PostAPIClustersClusterNameServicesActionsUnprovision*)

`POST /api/clusters/{clusterName}/services/actions/unprovision`

This endpoint unprovisions services for the specified cluster.

3.4.2.2.150.1 Consumes

- application/json

3.4.2.2.150.2 Produces

- application/json

3.4.2.2.150.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.150.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully unprovisioned services		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.150.5 Responses

200 - Successfully unprovisioned services Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.151 Add a tag to a specific cluster (*PostAPIClustersClusterNameSettingsActionsAddDbTagTagValue*)

`POST /api/clusters/{clusterName}/settings/actions/add-db-tag/{tagValue}`

This endpoint adds a tag to the specified cluster.

3.4.2.2.151.1 Consumes

- application/json

3.4.2.2.151.2 Produces

- application/json

3.4.2.2.151.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
tagValue	`path`	string	`string`				Tag Value

3.4.2.2.151.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Tag added successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.151.5 Responses

200 - Tag added successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.152 Add a proxy tag to a specific cluster (*PostAPIClustersClusterNameSettingsActionsAddProxyTagTagValue*)

`POST /api/clusters/{clusterName}/settings/actions/add-proxy-tag/{tagValue}`

This endpoint adds a proxy tag to the specified cluster.

3.4.2.2.152.1 Consumes

- application/json

3.4.2.2.152.2 Produces

- application/json

3.4.2.2.152.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
tagValue	`path`	string	`string`				Tag Value

3.4.2.2.152.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Tag added successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.152.5 Responses

200 - Tag added successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.153 Apply dynamic configuration for a specific cluster (*PostAPIClustersClusterName-SettingsActionsApplyDynamicConfig*)

`POST /api/clusters/{clusterName}/settings/actions/apply-dynamic-config`

This endpoint applies dynamic configuration for the specified cluster.

3.4.2.2.153.1 Consumes

- application/json

3.4.2.2.153.2 Produces

- application/json

3.4.2.2.153.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.153.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully applied dynamic configuration		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.153.5 Responses

200 - Successfully applied dynamic configuration Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.154 Reload client certificates for a specific cluster (*PostAPIClustersClusterNameSettingsActionsCertificatesReload*)

`POST /api/clusters/{clusterName}/settings/actions/certificates-reload`

This endpoint reloads the client certificates for the specified cluster.

3.4.2.2.154.1 Consumes

- application/json

3.4.2.2.154.2 Produces

- application/json

3.4.2.2.154.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.154.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully reloaded client certificates		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.154.5 Responses

200 - Successfully reloaded client certificates Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.155 Discover settings for a specific cluster (*PostAPIClustersClusterNameSettingsActionsDiscover*)

`POST /api/clusters/{clusterName}/settings/actions/discover`

This endpoint triggers the discovery of settings for the specified cluster.

3.4.2.2.155.1 Consumes

- application/json

3.4.2.2.155.2 Produces

- application/json

3.4.2.2.155.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.155.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully discovered settings		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.155.5 Responses

200 - Successfully discovered settings Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.156 Remove a tag from a specific cluster (*PostAPIClustersClusterNameSettingsActionsDropDbTagTagValue*)

`POST /api/clusters/{clusterName}/settings/actions/drop-db-tag/{tagValue}`

This endpoint removes a tag from the specified cluster.

3.4.2.2.156.1 Consumes

- application/json

3.4.2.2.156.2 Produces

- application/json

3.4.2.2.156.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
tagValue	`path`	string	`string`				Tag Value

3.4.2.2.156.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Tag removed successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.156.5 Responses

200 - Tag removed successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.157 Remove a proxy tag from a specific cluster (*PostAPIClustersClusterNameSettingsActionsDropProxyTagTagValue*)

`POST /api/clusters/{clusterName}/settings/actions/drop-proxy-tag/{tagValue}`

This endpoint removes a proxy tag from the specified cluster.

3.4.2.2.157.1 Consumes

- application/json

3.4.2.2.157.2 Produces

- application/json

3.4.2.2.157.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
tagValue	`path`	string	`string`				Tag Value

3.4.2.2.157.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Tag removed successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.157.5 Responses

200 - Tag removed successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.158 Reload cluster settings (*PostAPIClustersClusterNameSettingsActionsReload*)

`POST /api/clusters/{clusterName}/settings/actions/reload`

This endpoint reloads the settings for the specified cluster.

3.4.2.2.158.1 Consumes

- application/json

3.4.2.2.158.2 Produces

- application/json

3.4.2.2.158.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.158.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully reloaded settings		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.158.5 Responses

200 - Successfully reloaded settings Status: OK

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.159 Reload Graphite filter list for a specific cluster (*PostAPIClustersClusterNameSettingsActionsReloadGraphiteFilterlist*)

`POST /api/clusters/{clusterName}/settings/actions/reload-graphite-filterlist`

This endpoint reloads the Graphite filter list for the specified cluster.

3.4.2.2.159.1 Consumes

- application/json

3.4.2.2.159.2 Produces

- application/json

3.4.2.2.159.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.159.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully reloaded Graphite filter list		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.159.5 Responses

200 - Successfully reloaded Graphite filter list Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.160 Reset Graphite filter list for a specific cluster (*PostAPIClustersClusterNameSettingsActionsResetGraphiteFilterlistTemplate*)

`POST /api/clusters/{clusterName}/settings/actions/reset-graphite-filterlist/{template}`

This endpoint resets the Graphite filter list for the specified cluster.

3.4.2.2.160.1 Consumes

- application/json

3.4.2.2.160.2 Produces

- application/json

3.4.2.2.160.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
template	`path`	string	`string`				Template

3.4.2.2.160.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully reset Graphite filter list		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.160.5 Responses

200 - Successfully reset Graphite filter list Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.161 Set cron jobs for a specific cluster (*PostAPIClustersClusterNameSettingsActionsSetCronSettingNameSettingValue*)

`POST /api/clusters/{clusterName}/settings/actions/set-cron/{settingName}/{settingValue}`

This endpoint sets the cron jobs for the specified cluster.

3.4.2.2.161.1 Consumes

- application/json

3.4.2.2.161.2 Produces

- application/json

3.4.2.2.161.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
settingName	`path`	string	`string`				Setting Name
settingValue	`path`	string	`string`				Setting Value

3.4.2.2.161.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully set cron job		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.161.5 Responses

200 - Successfully set cron job Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.162 Set Graphite filter list for a specific cluster (*PostAPIClustersClusterNameSettingsActionsSetGraphiteFilterlistFilterType*)

`POST /api/clusters/{clusterName}/settings/actions/set-graphite-filterlist/{filterType}`

This endpoint sets the Graphite filter list for the specified cluster.

3.4.2.2.162.1 Consumes

- application/json

3.4.2.2.162.2 Produces

- application/json

3.4.2.2.162.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
filterType	`path`	string	`string`				Filter Type
body	`body`	ClusterGraphiteFilterList	`models.ClusterGraphiteFilterList`				Graphite Filter List

3.4.2.2.162.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Filterlist updated		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.162.5 Responses

200 - Filterlist updated Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.163 Set settings for a specific cluster (*PostAPIClustersClusterNameSettingsActionsSetSettingNameSettingValue*)

`POST /api/clusters/{clusterName}/settings/actions/set/{settingName}/{settingValue}`

This endpoint sets the settings for the specified cluster.

3.4.2.2.163.1 Consumes

- application/json

3.4.2.2.163.2 Produces

- application/json

3.4.2.2.163.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
settingName	`path`	string	`string`				Setting Name
settingValue	`path`	string	`string`				Setting Value

3.4.2.2.163.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully set setting		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.163.5 Responses

200 - Successfully set setting Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.164 Switch settings for a specific cluster (*PostAPIClustersClusterNameSettingsActionsSwitchSettingName*)

`POST /api/clusters/{clusterName}/settings/actions/switch/{settingName}`

This endpoint switches the settings for the specified cluster.

3.4.2.2.164.1 Consumes

- application/json

3.4.2.2.164.2 Produces

- application/json

3.4.2.2.164.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
settingName	`path`	string	`string`				Setting Name

3.4.2.2.164.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully switched setting		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.164.5 Responses

200 - Successfully switched setting Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.165 Subscribe a user to a cluster (*PostAPIClustersClusterNameSubscribe*)

`POST /api/clusters/{clusterName}/subscribe`

This endpoint allows a user to subscribe to a specified cluster.

3.4.2.2.165.1 Consumes

- application/json

3.4.2.2.165.2 Produces

- application/json

3.4.2.2.165.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
userform	`body`	ClusterUserForm	`models.ClusterUserForm`				User Form

3.4.2.2.165.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Email sent to admin!		schema
400	Bad Request	Error in request		schema
401	Unauthorized	Error logging in to gitlab: Token credentials is not valid		schema
403	Forbidden	Error parsing JWT" / "Current user is not logged in via Gitlab!		schema
409	Conflict	User already subscribed on peer cluster!" / "Another user already subscribed on peer cluster!		schema
500	Internal Server Error	No valid cluster" / "Peer does not have cloud18 setup!" / "Error sending email		schema

3.4.2.2.165.5 Responses

200 - Email sent to admin! Status: OK

Schema

400 - Error in request Status: Bad Request

Schema

401 - Error logging in to gitlab: Token credentials is not valid Status: Unauthorized

Schema

403 - Error parsing JWT" / "Current user is not logged in via Gitlab! Status: Forbidden

Schema

409 - User already subscribed on peer cluster!" / "Another user already subscribed on peer cluster! Status: Conflict

Schema

500 - No valid cluster" / "Peer does not have cloud18 setup!" / "Error sending email Status: Internal Server Error

Schema

3.4.2.2.166 Run all tests for a given cluster (*PostAPIClustersClusterNameTestsActionsRunAll*)

`POST /api/clusters/{clusterName}/tests/actions/run/all`

This endpoint runs all tests for the specified cluster.

3.4.2.2.166.1 Consumes

- application/json

3.4.2.2.166.2 Produces

- application/json

3.4.2.2.166.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name

3.4.2.2.166.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	List of test results		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.166.5 Responses

200 - List of test results Status: OK

Schema

[][ClusterTest](#cluster-test)

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.167 Run a specific test for a given cluster (*PostAPIClustersClusterNameTestsActionsRunTestName*)

`POST /api/clusters/{clusterName}/tests/actions/run/{testName}`

This endpoint runs a specific test for the specified cluster.

3.4.2.2.167.1 Consumes

- application/json

3.4.2.2.167.2 Produces

- application/json

3.4.2.2.167.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
testName	`path`	string	`string`				Test Name
provision	`formData`	string	`string`				Provision the cluster before running the test
unprovision	`formData`	string	`string`				Unprovision the cluster after running the test

3.4.2.2.167.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Test result		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.167.5 Responses

200 - Test result Status: OK

Schema

ClusterTest

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.168 Add a new user to a cluster (*PostAPIClustersClusterNameUsersAdd*)

`POST /api/clusters/{clusterName}/users/add`

Adds a new user to the specified cluster if the request is valid and the user has the necessary permissions.

3.4.2.2.168.1 Consumes

- application/json

3.4.2.2.168.2 Produces

- application/json

3.4.2.2.168.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
userform	`body`	ClusterUserForm	`models.ClusterUserForm`				User Form

3.4.2.2.168.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	User added successfully		schema
400	Bad Request	Error in request		schema
403	Forbidden	No Valid ACL		schema
500	Internal Server Error	Error adding new user" or "No valid cluster		schema

3.4.2.2.168.5 Responses

200 - User added successfully Status: OK

Schema

400 - Error in request Status: Bad Request

Schema

403 - No Valid ACL Status: Forbidden

Schema

500 - Error adding new user" or "No valid cluster Status: Internal Server Error

Schema

3.4.2.2.169 Drop a cluster user (*PostAPIClustersClusterNameUsersDrop*)

`POST /api/clusters/{clusterName}/users/drop`

Drops a user from the specified cluster.

3.4.2.2.169.1 Consumes

- application/json

3.4.2.2.169.2 Produces

- application/json

3.4.2.2.169.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
userform	`body`	ClusterUserForm	`models.ClusterUserForm`				User Form

3.4.2.2.169.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	User dropped successfully		schema
400	Bad Request	Error in request		schema
403	Forbidden	No Valid ACL		schema
500	Internal Server Error	Error dropping user" or "No valid cluster		schema

3.4.2.2.169.5 Responses

200 - User dropped successfully Status: OK

Schema

400 - Error in request Status: Bad Request

Schema

403 - No Valid ACL Status: Forbidden

Schema

500 - Error dropping user" or "No valid cluster Status: Internal Server Error

Schema

3.4.2.2.170 Send credentials to a specific user (*PostAPIClustersClusterNameUsersSendCredentials*)

`POST /api/clusters/{clusterName}/users/send-credentials`

This endpoint sends the credentials to the specified user via email.

3.4.2.2.170.1 Consumes

- application/json

3.4.2.2.170.2 Produces

- application/json

3.4.2.2.170.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
body	`body`	ServerCredentialMailForm	`models.ServerCredentialMailForm`				Credential Mail Form

3.4.2.2.170.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Credentials sent to user!		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Error sending email		schema

3.4.2.2.170.5 Responses

200 - Credentials sent to user! Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Error sending email Status: Internal Server Error

Schema

3.4.2.2.171 Update a cluster user (*PostAPIClustersClusterNameUsersUpdate*)

`POST /api/clusters/{clusterName}/users/update`

Updates the user information for a specified cluster.

3.4.2.2.171.1 Consumes

- application/json

3.4.2.2.171.2 Produces

- application/json

3.4.2.2.171.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
userform	`body`	ClusterUserForm	`models.ClusterUserForm`				User Form

3.4.2.2.171.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	User updated successfully		schema
400	Bad Request	Error in request		schema
403	Forbidden	No Valid ACL		schema
500	Internal Server Error	Error updating user" or "No valid cluster		schema

3.4.2.2.171.5 Responses

200 - User updated successfully Status: OK

Schema

400 - Error in request Status: Bad Request

Schema

403 - No Valid ACL Status: Forbidden

Schema

500 - Error updating user" or "No valid cluster Status: Internal Server Error

Schema

3.4.2.2.172 Reload cluster plans (*PostAPIClustersSettingsActionsReloadClustersPlans*)

`POST /api/clusters/settings/actions/reload-clusters-plans`

This endpoint reloads the cluster plans for all clusters.

3.4.2.2.172.1 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully reloaded plans		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.172.2 Responses

200 - Successfully reloaded plans Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.173 Set global settings for the server (*PostAPIClustersSettingsActionsSetSettingName-SettingValue*)

`POST /api/clusters/settings/actions/set/{settingName}/{settingValue}`

This endpoint sets the global settings for the server.

3.4.2.2.173.1 Consumes

- application/json

3.4.2.2.173.2 Produces

- application/json

3.4.2.2.173.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
settingName	`path`	string	`string`				Setting Name
settingValue	`path`	string	`string`				Setting Value

3.4.2.2.173.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully set setting		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	No cluster		schema

3.4.2.2.173.5 Responses

200 - Successfully set setting Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.174 Switch global settings for the server (*PostAPIClustersSettingsActionsSwitchSettingName*)

`POST /api/clusters/settings/actions/switch/{settingName}`

This endpoint switches the global settings for the server.

3.4.2.2.174.1 Consumes

- application/json

3.4.2.2.174.2 Produces

- application/json

3.4.2.2.174.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
settingName	`path`	string	`string`				Setting Name

3.4.2.2.174.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Successfully switched setting		schema
403	Forbidden	No valid ACL		schema

Code	Status	Description	Has headers	Schema
500	Internal Server Error	No cluster		schema

3.4.2.2.174.5 Responses

200 - Successfully switched setting Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - No cluster Status: Internal Server Error

Schema

3.4.2.2.175 User login (*PostAPILogin*)

`POST /api/login`

Authenticates a user and returns a JWT token upon successful login.

3.4.2.2.175.1 Consumes

- application/json

3.4.2.2.175.2 Produces

- application/json

3.4.2.2.175.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
userCredentials	body	ServerUserCredentials	models.ServerUserCredentials				User credentials

3.4.2.2.175.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	JWT token		schema
401	Unauthorized	Invalid credentials		schema
403	Forbidden	Error in request		schema
429	Too Many Requests	Too many requests		schema
500	Internal Server Error	Error signing token		schema

3.4.2.2.175.5 Responses

200 - JWT token Status: OK

Schema

ServerToken

401 - Invalid credentials Status: Unauthorized

Schema

403 - Error in request Status: Forbidden

Schema

429 - Too many requests Status: Too Many Requests

Schema

500 - Error signing token Status: Internal Server Error

Schema

3.4.2.2.176 Drop a server monitor from a cluster (*PostClusterClusterNameActionsDropserverHostPort*)

`POST /cluster/{clusterName}/actions/dropserver/{host}/{port}`

This endpoint allows dropping a server monitor or proxy monitor from a specified cluster.

3.4.2.2.176.1 Consumes

- application/json

3.4.2.2.176.2 Produces

- application/json

3.4.2.2.176.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
host	`path`	string	`string`				Host
port	`path`	string	`string`				Port

3.4.2.2.176.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Monitor dropped successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.176.5 Responses

200 - Monitor dropped successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.2.177 Drop a server monitor from a cluster (*PostClusterClusterNameActionsDropserverHostPortType*)

`POST /cluster/{clusterName}/actions/dropserver/{host}/{port}/{type}`

This endpoint allows dropping a server monitor or proxy monitor from a specified cluster.

3.4.2.2.177.1 Consumes

- application/json

3.4.2.2.177.2 Produces

- application/json

3.4.2.2.177.3 Parameters

Name	Source	Type	Go type	Separator	Required	Default	Description
clusterName	`path`	string	`string`				Cluster Name
host	`path`	string	`string`				Host
port	`path`	string	`string`				Port
type	`path`	string	`string`				Monitor Type (proxy or database)

3.4.2.2.177.4 All responses

Code	Status	Description	Has headers	Schema
200	OK	Monitor dropped successfully		schema
403	Forbidden	No valid ACL		schema
500	Internal Server Error	Cluster Not Found		schema

3.4.2.2.177.5 Responses

200 - Monitor dropped successfully Status: OK

Schema

403 - No valid ACL Status: Forbidden

Schema

500 - Cluster Not Found Status: Internal Server Error

Schema

3.4.2.3 Models

3.4.2.3.1 cluster.APIUser

Properties

Name	Type	Go type	Required	Default	Description	Example
grants	map of boolean	`map[string]bool`				
roles	map of boolean	`map[string]bool`				
user	string	`string`				

3.4.2.3.2 cluster.Agent

Properties

Name	Type	Go type	Required	Default	Description	Example
cpuCores	integer	`int64`				
cpuFreq	integer	`int64`				
hostName	string	`string`				
id	string	`string`				
memBytes	integer	`int64`				
memFreeBytes	integer	`int64`				
osKernel	string	`string`				
osName	string	`string`				
status	string	`string`				
version	string	`string`				

3.4.2.3.3 cluster.Alerts

Properties

Name	Type	Go type	Required	Default	Description	Example
errors	[]([StateStateHTTP](#state-state-http))	`[]*StateStateHTTP`				
warnings	[]([StateStateHTTP](#state-state-http))	`[]*StateStateHTTP`				

3.4.2.3.4 cluster.Cluster

Properties

Name	Type	Go type	Required	Default	Description	Example
activePassiveStatus	string	`string`				
agents	[]([ClusterAgent](#cluster-agent))	`[]*ClusterAgent`				
apiUsers	map of Cluster-APIUser	`map[string]ClusterAPIUser`				
backupList	ConfigBackupMetaMap	`ConfigBackupMetaMap`				
backupStat	Repmanv3BackupStat	`Repmanv3BackupStat`				
blacklist	[]([RegexpRegexp](#regexp-regexp-regexp))	`[]Regexp`				
canConnectVault	boolean	`bool`				
canInitNodes	boolean	`bool`				
cleanReplication	boolean	`bool`			used in testing	
config	GithubComSignal18ReplicationManagerConfigConfig	`GithubComSignal18ReplicationManagerConfigConfig`				
configurator	ConfigurationConfigurator	`ConfigurationConfigurator`				
dbServers	[]string	`[]string`				

Name	Type	Go type	Required	Default	Description	Example
dbServersCrashes	[][ClusterCrash](#clustercrash)	`[]*ClusterCrash`			This will be purged on all db node up	
diffVariables	[][ClusterVariableDiff](#clustervariable-diff)	`[]*ClusterVariableDiff`				
diskType	map of string	`map[string]string`				
failoverCounter	integer	`int64`				
failoverHistory	[][ClusterCrash](#clustercrash)	`[]*ClusterCrash`			This will be used for PITR	
failoverLastTime	integer	`int64`				
fsType	map of boolean	`map[string]bool`				
haveDBTLSCert	boolean	`bool`				
haveDBTLSOldCert	boolean	`bool`				
inBinlogBackup	boolean	`bool`				
inLogicalBackup	boolean	`bool`				
inPhysicalBackup	boolean	`bool`				
inResticBackup	boolean	`bool`				
inRollingRestart	boolean	`bool`				
isAlertDisable	boolean	`bool`				
isAllDbUp	boolean	`bool`				
isCapturing	boolean	`bool`				
isClusterDown	boolean	`bool`				
isDown	boolean	`bool`				
isExportPush	boolean	`bool`				
isFailable	boolean	`bool`				
isFailedArbitrator	boolean	`bool`				
isGettingSlowlog	boolean	`bool`				
isGitPull	boolean	`bool`				
isGitPush	boolean	`bool`				
isLostMajority	boolean	`bool`				
isMasterDown	boolean	`bool`				
isNeedDatabaseConfigChange	boolean	`bool`				
isNeedDatabaseReprov	boolean	`bool`				
isNeedDatabaseRestart	boolean	`bool`				
isNeedDatabaseRollingReprov	boolean	`bool`				
isNeedDatabaseRollingRestart	boolean	`bool`				
isNeedProxiesConfigChange	boolean	`bool`				
isNeedProxiesRestart	boolean	`bool`				
isNeedProxyRestart	boolean	`bool`				
isNotMonitoring	boolean	`bool`				
isPostgres	boolean	`bool`				
isProvision	boolean	`bool`				
isSplitBrain	boolean	`bool`				
isValidBackup	boolean	`bool`				
jobResults	ConfigTasksMap	`ConfigTasksMap`				
lastDelayStamp	string	`string`				
log	S18logHTTPLog	`S18logHTTPLog`				
logTask	S18logHTTPLog	`S18logHTTPLog`				
monitorSpins	string	`string`				
monitorTypes	map of string	`map[string]string`				

Name	Type	Go type	Required	Default	Description	Example
name	string	`string`				
proxyServers	[]string	`[]string`				
slaHistory	[][StateSLA](#stateSLA)	`[]*StateSLA`				
slavesConnected	integer	`int64`				
slavesOldestMasterFile	MasterFileOldestMasterFile	`MasterFileOldestMasterFile`				
sqlErrorLog	S18logHTTPLog	`S18logHTTPLog`				
sqlGeneralLog	S18logHTTPLog	`S18logHTTPLog`				
sstAvailablePorts	map of string	`map[string]string`				
stateMachine	ClusterCluster	`ClusterCluster`			dbUser string `json:"-"` oldDbUser string `json:"-"` dbPass string `json:"-"` oldDbPass string `json:"-"` rplUser string `json:"-"` rplPass string `json:"-"` proxysqlUser string `json:"-"` proxysqlPass string `json:"-"`	
tenant	string	`string`				
topology	string	`string`				
topologyTypes	map of string	`map[string]string`				
uptime	string	`string`				
uptimeFailable	string	`string`				
uptimeSemisync	string	`string`				
useBlacklist	boolean	`bool`				
useWhitelist	boolean	`bool`				
versionsMap	ConfigVersionsMap	`ConfigVersionsMap`				
vmType	map of boolean	`map[string]bool`				
waitingFailover	integer	`int64`				
waitingRejoin	integer	`int64`				
waitingSwitchover	integer	`int64`				
whitelist	[][RegexpRegexp](#regexp-regexp)	`[]Regexp`				
workLoad	ConfigWorkLoad	`ConfigWorkLoad`				
workingDir	string	`string`				

3.4.2.3.5 cluster.ClusterForm

Properties

Name	Type	Go type	Required	Default	Description	Example
clusterName	string	`string`				
orchestrator	string	`string`				

Name	Type	Go type	Required	Default	Description	Example
plan	string	`string`				

3.4.2.3.6 cluster.Crash

Properties

Name	Type	Go type	Required	Default	Description	Example
electedMasterURL	string	`string`				
failoverIOGtid	[]\[GtidGtid](#gtid-gtid)	`[]*GtidGtid`				
failoverMasterLogFile	string	`string`				
failoverMasterLogPos	string	`string`				
failoverSemiSyncSlaveStatus	boolean	`bool`				
newMasterLogFile	string	`string`				
newMasterLogPos	string	`string`				
switchover	boolean	`bool`				
unixTimestamp	integer	`int64`				
url	string	`string`				

3.4.2.3.7 cluster.DelayStat

Properties

Name	Type	Go type	Required	Default	Description	Example
counter	integer	`int64`			Increment	
delay	number	`float64`			Average Seconds of Delay	
delayCount	integer	`int64`			Number of Delay Occurred	
slaveErrCount	integer	`int64`			Number of Slave Err Occurred	

3.4.2.3.8 cluster.DelayStatHistory

Properties

Name	Type	Go type	Required	Default	Description	Example
delayDT	string	`string`				
delayStat	ClusterDelayStat	`ClusterDelayStat`				

3.4.2.3.9 cluster.Diff

Properties

Name	Type	Go type	Required	Default	Description	Example
serverName	string	`string`				
variableValue	string	`string`				

3.4.2.3.10 cluster.GraphiteFilterList

Properties

Name	Type	Go type	Required	Default	Description	Example
blacklist	string	`string`				
whitelist	string	`string`				

3.4.2.3.11 cluster.ServerBackupMeta

Properties

Name	Type	Go type	Required	Default	Description	Example
logical	ConfigBackupMetadata	`ConfigBackupMetadata`				
physical	ConfigBackupMetadata	`ConfigBackupMetadata`				

3.4.2.3.12 cluster.ServerDelayStat

Properties

Name	Type	Go type	Required	Default	Description	Example
current	ClusterDelayStat	`ClusterDelayStat`				
currentDT	string	`string`				
delayHistory	[][ClusterDelayStat](#ClusterDelayStat-delay-stat-history)	`[]*ClusterDelayStat`				
rotated	ClusterDelayStat	`ClusterDelayStat`				
total	ClusterServerDelayStat	`ClusterServerDelayStat`			Total Delay Average since SRM started	

3.4.2.3.13 cluster.ServerMonitor

Properties

Name	Type	Go type	Required	Default	Description	Example
HaveBinlogSlowUpdates	boolean	`bool`				
HaveHealthyReplica	boolean	`bool`				
HaveSshError	boolean	`bool`				
agent	string	`string`			used to provision service in orchestrator	
binaryLogDir	string	`string`				
binaryLogFile	string	`string`				
binaryLogFileOldest	string	`string`				
binaryLogFilePrevious	string	`string`				
binaryLogFiles	DbhelperBinaryLogMetaMap	`DbhelperBinaryLogMetaMap`				
binaryLogFilesCount	integer	`int64`				
binaryLogOldestTimestamp	integer	`int64`				
binaryLogPos	string	`string`				
binaryLogPurgeBefore	integer	`int64`				
binlogDumpThreads	integer	`int64`				
currentGtid	[][GtidGtid](#GtidGtid-gtid)	`[]*GtidGtid`				

Name	Type	Go type	Required	Default	Description	Example
datadir	string	`string`				
dbVersion	VersionVersion	`VersionVersion`				
dbdataDir	string	`string`				
delayStat	ClusterServerDelayStat	`ClusterServerDelayStat`				
domain	string	`string`			Use to store orchestrator CNI domain ..svc.	
domainId	integer	`int64`				
engineInnodbConfigStringsMap	ConfigStringsMap	`ConfigStringsMap`				
errorLog	S18logHTTPLog	`S18logHTTPLog`				
eventScheduler	boolean	`bool`				
eventStatus	[][DbhelperEvent](#dbhelperevent event)	`[]*DbhelperEvent`				
failCount	integer	`int64`				
failSuspectHeartbeat	integer	`int64`				
failoverIoGtid	[][GtidGtid](#gtidgtid gtid)	`[]*GtidGtid`				
failoverMasterLogFile	string	`string`				
failoverMasterLogPos	string	`string`				
failoverSemiSyncSlaveStatus	boolean	`bool`				
gtidBinlogPos	[][GtidGtid](#gtidgtid gtid)	`[]*GtidGtid`				
gtidExecuted	string	`string`				
hashUUID	integer	`int64`				
haveBinLogSync	boolean	`bool`				
haveBinlog	boolean	`bool`				
haveBinlogAnnotate	boolean	`bool`				
haveBinlogCompress	boolean	`bool`				
haveBinlogMixed	boolean	`bool`				
haveBinlogRow	boolean	`bool`				
haveBinlogSlowqueries	boolean	`bool`				
haveBinlogStatement	boolean	`bool`				
haveDiskMonitor	boolean	`bool`				
haveGtidStrictMode	boolean	`bool`				
haveInnodbChecksum	boolean	`bool`				
haveInnodbTrxCommit	boolean	`bool`				
haveLogGeneral	boolean	`bool`				
haveMariadbGtid	boolean	`bool`				
haveMetaDataLocksLog	boolean	`bool`				
haveMysqlGtid	boolean	`bool`				
haveNoMasterOnStart	boolean	`bool`				
havePFS	boolean	`bool`				
havePFSSlowQueryLog	boolean	`bool`				
haveQueryResponseTimeLog	boolean	`bool`				
haveReadOnly	boolean	`bool`				
haveSQLErrorLog	boolean	`bool`				
haveSemiSync	boolean	`bool`				
haveSlaveAggressive	boolean	`bool`				
haveSlaveConservative	boolean	`bool`				
haveSlaveIdempotent	boolean	`bool`				
haveSlaveMinimal	boolean	`bool`				
haveSlaveOptimistic	boolean	`bool`				

Name	Type	Go type	Required	Default	Description	Example
haveSlaveSerialized	boolean	`bool`				
haveSlowQueryLog	boolean	`bool`				
haveWsrep	boolean	`bool`				
host	string	`string`				
id	string	`string`			Unique name given by cluster & crc64(URL) used by test to provision	
ignored	boolean	`bool`				
ignoredRO	boolean	`bool`				
inCaptureMode	boolean	`bool`				
inPurgingBinaryLog	boolean	`bool`				
ioGtid	[][GtidGtid](#gtidgtid)	`[]*GtidGtid`				
ip	string	`string`				
isBackingUpBinaryLog	boolean	`bool`				
isCompute	boolean	`bool`			Used to idenfied spider compute nide	
isConfigGen	boolean	`bool`				
isDelayed	boolean	`bool`				
isFull	boolean	`bool`				
isGroupReplicationMaster	boolean	`bool`				
isGroupReplicationSlave	boolean	`bool`				
isInPFSQueryCapture	boolean	`bool`				
isInSlowQueryCapture	boolean	`bool`				
isLoadingJobsDone	boolean	`bool`				
isMaintenance	boolean	`bool`				
isMaxscale	boolean	`bool`				
isRefreshingBinlog	boolean	`bool`				
isRefreshingBinlogMeta	boolean	`bool`				
isRelay	boolean	`bool`				
isReseeding	string	`string`				
isSlave	boolean	`bool`				
isVirtualMaster	boolean	`bool`				
isWsrepDonor	boolean	`bool`				
isWsrepPrimary	boolean	`bool`				
isWsrepSynced	boolean	`bool`				
jobResults	ConfigTasksMap	`ConfigTasksMap`				
lastBackupMeta	ClusterServerBackupMeta	`ClusterServerBackupMeta`				
lastSeenReplication	[][DbhelperSlaveStatus](#dbhelperslavestatus-slave-status)	`[]*DbhelperSlaveStatus`				
logOutput	string	`string`				
longQueryTime	string	`string`				
longQueryTimeSaved	string	`string`				
masterStatus	DbhelperMasterStatus	`DbhelperMasterStatus`				
maxSlowQueryTime	timestamp	`int64`				
maxscaleHaveGtid	boolean	`bool`				
maxscaleServerName	string	`string`			Unique server Name in maxscale conf	

Name	Type	Go type	Required	Default	Description	Example
maxscaleServerStatus	string	`string`				
maxscaleVersion	integer	`int64`				
name	string	`string`				
needRefreshJobs	boolean	`bool`				
pointInTimeMeta	ConfigPointInTimeMeta	`ConfigPointInTimeMeta`				
port	string	`string`				
postgressDB	string	`string`				
prefered	boolean	`bool`				
preferedBackup	boolean	`bool`				
prevState	string	`string`				
process	OsProcess	`OsProcess`				
proxysqlHostgroup	string	`string`				
qps	integer	`int64`				
readOnly	string	`string`				
relayLogSize	integer	`int64`				
replicationHealth	string	`string`				
replicationSourceName	string	`string`				
replicationTags	string	`string`				
replications	[][DbhelperSlaveStatus](#slave-status)	`[]*DbhelperSlaveStatus`				
semiSyncMasterStatus	boolean	`bool`				
semiSyncSlaveStatus	boolean	`bool`				
serverId	integer	`int64`				
serviceName	string	`string`				
slaposDatadir	string	`string`				
slaveGtid	[]*[GtidGtid](#gtid)	`[]*GtidGtid`				
slaveVariables	ClusterSlaveVariables	`ClusterSlaveVariables`				
slowQueryCapture	boolean	`bool`				
slowQueryLog	string	`string`				
sourceClusterName	string	`string`			Used to idenfied server added from other clusters linked with multi source	
sstPort	string	`string`			used to send data to dbjobs	
state	string	`string`				
strict	string	`string`				
tlsConfigUsed	string	`string`			used to track TLS config during key rotation	
tunnelPort	string	`string`				
url	string	`string`				
user	string	`string`				
workLoad	ConfigWorkLoadsMap	`ConfigWorkLoadsMap`				

3.4.2.3.14 cluster.SlaveVariables

Properties

Name	Type	Go type	Required	Default	Description	Example
slaveParallelMaxQueued	integer	`int64`				
slaveParallelMode	string	`string`				
slaveParallelThreads	integer	`int64`				
slaveParallelWorkers	integer	`int64`				
slaveTypeConversions	string	`string`				

3.4.2.3.15 cluster.SlavesOldestMasterFile

Properties

Name	Type	Go type	Required	Default	Description	Example
oldestTimestamp	string	`string`				
prefix	string	`string`				
suffix	integer	`int64`				

3.4.2.3.16 cluster.Test

Properties

Name	Type	Go type	Required	Default	Description	Example
configfile	string	`string`				
configinit	GithubComSignal18ReplicationManagerConfigConfig	`GithubComSignal18ReplicationManagerConfigConfig`				
configtest	GithubComSignal18ReplicationManagerConfigConfig	`GithubComSignal18ReplicationManagerConfigConfig`				
name	string	`string`				
result	string	`string`				

3.4.2.3.17 cluster.UserForm

Properties

Name	Type	Go type	Required	Default	Description	Example
grants	string	`string`				
password	string	`string`				
roles	string	`string`				
username	string	`string`				

3.4.2.3.18 cluster.VariableDiff

Properties

Name	Type	Go type	Required	Default	Description	Example
diffValues	[]([ClusterDiff](#cluster-diff)	`[]*ClusterDiff`				
variableName	string	`string`				

3.4.2.3.19 config.BackupMetaMap

interface{}

3.4.2.3.20 config.BackupMetadata

Properties

Name	Type	Go type	Required	Default	Description	Example
backupMethod	integer	`int64`				
backupStrategy	integer	`int64`				
backupTool	string	`string`				
binLogFileName	string	`string`				
binLogFilePos	integer	`int64`				
binLogUuid	string	`string`				
checksum	string	`string`				
completed	boolean	`bool`				
compressed	boolean	`bool`				
dest	string	`string`				
encrypted	boolean	`bool`				
encryptionAlgo	string	`string`				
encryptionKey	string	`string`				
endTime	string	`string`				
id	integer	`int64`				
previous	integer	`int64`				
retentionDays	integer	`int64`				
size	integer	`int64`				
source	string	`string`				
startTime	string	`string`				

3.4.2.3.21 config.ConfigVariableType

Properties

Name	Type	Go type	Required	Default	Description	Example
available	boolean	`bool`				
id	integer	`int64`				
label	string	`string`				
name	string	`string`				

3.4.2.3.22 config.DockerRepo

Properties

Name	Type	Go type	Required	Default	Description	Example
image	string	`string`				
name	string	`string`				
tags	ConfigDockerTag	`ConfigDockerTag`				

3.4.2.3.23 config.DockerTag

Properties

Name	Type	Go type	Required	Default	Description	Example
results	[]([ConfigTagResult](#config-tag-result))	`[]*ConfigTagResult`				

3.4.2.3.24 config.Grant

Properties

Name	Type	Go type	Required	Default	Description	Example
enable	boolean	`bool`				
grant	string	`string`				

3.4.2.3.25 config.Partner

Properties

Name	Type	Go type	Required	Default	Description	Example
dbopsEmail	string	`string`				
domains	string	`string`				
id	integer	`int64`				
isDbops	integer	`int64`				
isSysops	integer	`int64`				
name	string	`string`				
stars	integer	`int64`				
sysopsEmail	string	`string`				

3.4.2.3.26 config.PeerCluster

Properties

Name	Type	Go type	Required	Default	Description	Example
api-credentials-acl-allow	string	`string`				
api-credentials-acl-allow-external	string	`string`				
api-public-url	string	`string`				
cloud18-cost-currency	string	`string`				
cloud18-database-read-srv-record	string	`string`				
cloud18-database-read-write-split-srv-record	string	`string`				
cloud18-database-read-write-srv-record	string	`string`				
cloud18-domain	string	`string`				
cloud18-external-dbops	string	`string`				
cloud18-external-sysops	string	`string`				
cloud18-infra-certifications	string	`string`				
cloud18-infra-cpu-freq	string	`string`				
cloud18-infra-cpu-model	string	`string`				
cloud18-infra-data-centers	string	`string`				
cloud18-infra-geo-localizations	string	`string`				
cloud18-infra-public-bandwidth	string	`string`				`0`
cloud18-monthly-dbops-cost	string	`string`				`0`
cloud18-monthly-infra-cost	string	`string`				`0`
cloud18-monthly-license-cost	string	`string`				`0`
cloud18-monthly-sysops-cost	string	`string`				`0`
cloud18-open-dbops	string	`string`				`false`

Name	Type	Go type	Required	Default	Description	Example
cloud18-open-sysops	string	`string`				`false`
cloud18-peer	string	`string`				`false`
cloud18-platform-description	string	`string`				
cloud18-promotion-pct	string	`string`				`0`
cloud18-shared	string	`string`				`false`
cloud18-sla-provision-time	string	`string`				`0`
cloud18-sla-repair-time	string	`string`				`0`
cloud18-sla-response-time	string	`string`				`0`
cloud18-sub-domain	string	`string`				
cloud18-sub-domain-zone	string	`string`				
cloud18-subscribed-dbops	string	`string`				`false`
cluster-name	string	`string`				
peer-users	[]string	`[]string`				
prov-db-cpu-cores	string	`string`				`0`
prov-db-disk-iops	string	`string`				`0`
prov-db-disk-size	string	`string`				`0`
prov-db-memory	string	`string`				`0`
prov-orchestrator	string	`string`				
prov-service-plan	string	`string`				

3.4.2.3.27 config.PointInTimeMeta

Properties

Name	Type	Go type	Required	Default	Description	Example
backup	integer	`int64`				
isInPITR	boolean	`bool`				
restoreTime	integer	`int64`				
useBinlog	boolean	`bool`				

3.4.2.3.28 config.Role

Properties

Name	Type	Go type	Required	Default	Description	Example
enable	boolean	`bool`				
role	string	`string`				

3.4.2.3.29 config.ServicePlan

Properties

Name	Type	Go type	Required	Default	Description	Example
bp	string	`string`				`0`
certs	string	`string`				
cpu	string	`string`				
dbacost	string	`string`				`0`
dbcores	string	`string`				`0`
dbcpufreq	string	`string`				
dbdatasize	string	`string`				`0`

Name	Type	Go type	Required	Default	Description	Example
dbiops	string	`string`				0
dbmemory	string	`string`				0
dbsystemsize	string	`string`				0
dc	string	`string`				
devise	string	`string`				
extdbops	string	`string`				
extsysops	string	`string`				
gti	string	`string`				0
gtr	string	`string`				0
id	string	`string`				0
infra	string	`string`				
infracost	string	`string`				0
licencecost	string	`string`				0
plan	string	`string`				
promo	string	`string`				0
provtime	string	`string`				0
prxcores	string	`string`				0
prxdatasize	string	`string`				0
syscost	string	`string`				0
zone	string	`string`				

3.4.2.3.30 config.StringsMap

interface{}

3.4.2.3.31 config.TagResult

Properties

Name	Type	Go type	Required	Default	Description	Example
name	string	`string`				

3.4.2.3.32 config.Tarball

Properties

Name	Type	Go type	Required	Default	Description	Example
OS	string	`string`				
checksum	string	`string`				
date_added	string	`string`				
flavor	string	`string`				
minimal	boolean	`bool`				
name	string	`string`				
notes	string	`string`				
short_version	string	`string`				
size	integer	`int64`				
updated_by	string	`string`				
url	string	`string`				
version	string	`string`				

3.4.2.3.33 config.TasksMap

interface{}

3.4.2.3.34 config.VersionsMap

interface{}

3.4.2.3.35 config.WorkLoad

Properties

Name	Type	Go type	Required	Default	Description	Example
busyTime	string	`string`				
connections	integer	`int64`				
cpuThreadPool	number	`float64`				
cpuUserStats	number	`float64`				
dbIndexSize	integer	`int64`				
dbTableSize	integer	`int64`				
qps	integer	`int64`				

3.4.2.3.36 config.WorkLoadsMap

interface{}

3.4.2.3.37 configurator.Configurator

Properties

Name	Type	Go type	Required	Default	Description	E
configPrxTags	[]\[Repmanv3Tag](#repmanv3-tag)	`[]*Repmanv3Tag`			from module	
configTags	[]\[Repmanv3Tag](#repmanv3-tag)	`[]*Repmanv3Tag`			from module	
dbServersTags	[]string	`[]string`			from conf	
dbServersTagsDiscover	[]string	`[]string`			from conf	
proxyServersTags	[]string	`[]string`				
proxyServersTagsDiscover	[]string	`[]string`				

3.4.2.3.38 dbhelper.BinaryLogMetaMap

interface{}

3.4.2.3.39 dbhelper.Event

Properties

Name	Type	Go type	Required	Default	Description	Example
db	string	`string`				
definer	string	`string`				
name	string	`string`				
status	integer	`int64`				

3.4.2.3.40 dbhelper.MasterStatus

Properties

Name	Type	Go type	Required	Default	Description	Example
binlogDoDB	string	`string`				
binlogIgnoreDB	string	`string`				
file	string	`string`				
position	integer	`int64`				

3.4.2.3.41 dbhelper.SlaveStatus

Properties

Name	Type	Go type	Required	Default	Description	Example
channelName	SQLNullString	`SQLNullString`				
connectionName	SQLNullString	`SQLNullString`				
eeplicateDoDomainIds	SQLNullString	`SQLNullString`				
execMasterLogPos	SQLNullString	`SQLNullString`				
executedGtidSet	SQLNullString	`SQLNullString`				
gtidIoPos	SQLNullString	`SQLNullString`				
gtidSlavePos	SQLNullString	`SQLNullString`				
lastIoErrno	SQLNullString	`SQLNullString`				
lastIoError	SQLNullString	`SQLNullString`				
lastSqlErrno	SQLNullString	`SQLNullString`				
lastSqlError	SQLNullString	`SQLNullString`				
masterHost	SQLNullString	`SQLNullString`				
masterLogFile	SQLNullString	`SQLNullString`				
masterPort	SQLNullString	`SQLNullString`				
masterServerId	integer	`int64`				
masterUser	SQLNullString	`SQLNullString`				
postgresExternalId	SQLNullString	`SQLNullString`				
readMasterLogPos	SQLNullString	`SQLNullString`				
relayMasterLogFile	SQLNullString	`SQLNullString`				
replicateDoDb	SQLNullString	`SQLNullString`				
replicateDoTable	SQLNullString	`SQLNullString`				
replicateIgnoreDb	SQLNullString	`SQLNullString`				
replicateIgnoreDomainIds	SQLNullString	`SQLNullString`				
replicateIgnoreServerIds	SQLNullString	`SQLNullString`				
replicateIgnoreTable	SQLNullString	`SQLNullString`				
replicateWildDoTable	SQLNullString	`SQLNullString`				
replicateWildIgnoreTable	SQLNullString	`SQLNullString`				
retrievedGtidSet	SQLNullString	`SQLNullString`				
secondsBehindMaster	SQLNullInt64	`SQLNullInt64`				
slaveHeartbeatPeriod	number	`float64`				
slaveIoRunning	SQLNullString	`SQLNullString`				
slaveSQLRunningState	SQLNullString	`SQLNullString`				
slaveSqlRunning	SQLNullString	`SQLNullString`				
usingGtid	SQLNullString	`SQLNullString`				

3.4.2.3.42 github_com_signal18_replication-manager_config.Config

Properties

Name	Type	Go type	Required	Default	Description
alertPushoverAppToken	string	`string`			
alertPushoverUserToken	string	`string`			
alertScript	string	`string`			
alertSlackChannel	string	`string`			
alertSlackUrl	string	`string`			
alertSlackUser	string	`string`			
alertTeamsProxyUrl	string	`string`			
alertTeamsState	string	`string`			
alertTeamsUrl	string	`string`			
analyzeUseSql	boolean	`bool`			
apiBind	string	`string`			
apiCredentials	string	`string`			
apiCredentialsACLAllow	string	`string`			
apiCredentialsACLAllowExternal	string	`string`			
apiCredentialsACLDiscard	string	`string`			
apiCredentialsACLDiscardExternal	string	`string`			
apiCredentialsExternal	string	`string`			
apiCredentialsSecureConfig	boolean	`bool`			
apiHttpsBind	boolean	`bool`			
apiOAuthClientID	string	`string`			
apiOAuthClientSecret	string	`string`			
apiOAuthProvider	string	`string`			
apiPort	string	`string`			
apiPublicUrl	string	`string`			
apiServer	boolean	`bool`			
apiSwaggerEnabled	boolean	`bool`			
apiTokenTimeout	integer	`int64`			
arbitrationExternal	boolean	`bool`			
arbitrationExternalHosts	string	`string`			
arbitrationExternalSecret	string	`string`			
arbitrationExternalUniqueId	integer	`int64`			
arbitrationFailedMasterScript	string	`string`			
arbitrationPeerHosts	string	`string`			
arbitrationReadTimout	integer	`int64`			
arbitratorBindAddress	string	`string`			
arbitratorDriver	string	`string`			
autorejoin	boolean	`bool`			
autorejoinBackupBinlog	boolean	`bool`			
autorejoinFlashback	boolean	`bool`			
autorejoinFlashbackOnSync	boolean	`bool`			
autorejoinFlashbackOnUnsync	boolean	`bool`			
autorejoinForceRestore	boolean	`bool`			
autorejoinLogicalBackup	boolean	`bool`			
autorejoinMysqldump	boolean	`bool`			
autorejoinPhysicalBackup	boolean	`bool`			
autorejoinScript	string	`string`			
autorejoinSlavePositionalHeartbeat	boolean	`bool`			
autorejoinZfsFlashback	boolean	`bool`			
autoseed	boolean	`bool`			
backup	boolean	`bool`			
backupBinlogs	boolean	`bool`			
backupBinlogsKeep	integer	`int64`			

Name	Type	Go type	Required	Default	Description
backupKeepDaily	integer	`int64`			
backupKeepHourly	integer	`int64`			
backupKeepMonthly	integer	`int64`			
backupKeepUntilValid	boolean	`bool`			
backupKeepWeekly	integer	`int64`			
backupKeepYearly	integer	`int64`			
backupLoadScript	string	`string`			
backupLockDDL	boolean	`bool`			
backupLogicalDumpSystemTables	boolean	`bool`			
backupLogicalDumpThreads	integer	`int64`			
backupLogicalLoadThreads	integer	`int64`			
backupLogicalType	string	`string`			
backupMyDumperOptions	string	`string`			
backupMyDumperRegex	string	`string`			
backupMyLoaderOptions	string	`string`			
backupMydumperPath	string	`string`			
backupMyloaderPath	string	`string`			
backupMysqlbinlogPath	string	`string`			
backupMysqlclientgPath	string	`string`			
backupMysqldumpOptions	string	`string`			
backupMysqldumpPath	string	`string`			
backupPhysicalType	string	`string`			
backupRestic	boolean	`bool`			
backupResticAws	boolean	`bool`			
backupResticAwsAccessKeyId	string	`string`			
backupResticBinaryPath	string	`string`			
backupResticRepository	string	`string`			
backupSaveScript	string	`string`			
backupStreaming	boolean	`bool`			
backupStreamingBucket	string	`string`			
backupStreamingDebug	boolean	`bool`			
backupStreamingEndpoint	string	`string`			
backupStreamingRegion	string	`string`			
binlogCopyMode	string	`string`			
binlogCopyScript	string	`string`			
binlogParseMode	string	`string`			
binlogRotationScript	string	`string`			
checkBinlogFilters	boolean	`bool`			
checkBinlogServerId	integer	`int64`			
checkGrants	boolean	`bool`			
checkReplicationErrantTrx	boolean	`bool`			
checkReplicationFilters	boolean	`bool`			
checkReplicationState	boolean	`bool`			
checkType	string	`string`			
cloud18	boolean	`bool`			
cloud18CostCurrency	string	`string`			
cloud18DatabaseReadSrvRecord	string	`string`			
cloud18DatabaseReadWriteSplitSrvRecord	string	`string`			
cloud18DatabaseReadWriteSrvRecord	string	`string`			
cloud18DbOps	string	`string`			
cloud18DbaUserCredential	string	`string`			
cloud18Domain	string	`string`			

Name	Type	Go type	Required	Default	Description
cloud18ExternalDbOps	string	`string`			
cloud18ExternalSysOps	string	`string`			
cloud18GitUser	string	`string`			
cloud18InfraCertifications	string	`string`			
cloud18InfraCpuFreq	string	`string`			
cloud18InfraCpuModel	string	`string`			
cloud18InfraDataCenters	string	`string`			
cloud18InfraDescription	string	`string`			
cloud18InfraGeoLocalizations	string	`string`			
cloud18InfraPublicBandwidth	number	`float64`			
cloud18MonthlyDbopsCost	number	`float64`			
cloud18MonthlyInfraCost	number	`float64`			
cloud18MonthlyLicenseCost	number	`float64`			
cloud18MonthlySysopsCost	number	`float64`			
cloud18OpenDbops	boolean	`bool`			
cloud18OpenSysops	boolean	`bool`			
cloud18PlatformDescription	string	`string`			
cloud18PromotionPct	number	`float64`			
cloud18SalesSubscriptionScript	string	`string`			
cloud18SalesSubscriptionValidateScript	string	`string`			
cloud18SalesUnsubscribeScript	string	`string`			
cloud18Shared	boolean	`bool`			
cloud18SlaProvisionTime	number	`float64`			
cloud18SlaRepairTime	number	`float64`			
cloud18SlaResponseTime	number	`float64`			
cloud18SponsorUserCredential	string	`string`			
cloud18SubDomain	string	`string`			
cloud18SubDomainZone	string	`string`			
cloud18SubscribedDbops	boolean	`bool`			
clusterHead	string	`string`			
compressBackups	boolean	`bool`			
dbServersBackupHosts	string	`string`			
dbServersConnectTimeout	integer	`int64`			
dbServersCredential	string	`string`			
dbServersHosts	string	`string`			
dbServersIgnoredHosts	string	`string`			
dbServersIgnoredReadonly	string	`string`			
dbServersLocality	string	`string`			
dbServersPreferedMaster	string	`string`			
dbServersReadTimeout	integer	`int64`			
dbServersStateChangeScript	string	`string`			
dbServersTlsCaCert	string	`string`			
dbServersTlsClientCert	string	`string`			
dbServersTlsClientKey	string	`string`			
dbServersTlsServerCert	string	`string`			
dbServersTlsServerKey	string	`string`			
dbServersUseGeneratedCert	boolean	`bool`			
delayStatCapture	boolean	`bool`			
delayStatRotate	integer	`int64`			
eeplicationRestartOnSqlLErrorMatch	string	`string`			
enterprise	boolean	`bool`			used to talk to opensvc collector
extproxy	boolean	`bool`			

Name	Type	Go type	Required	Default	Description
extproxyAddress	string	`string`			
failoverAtSync	boolean	`bool`			
failoverCheckDelayStat	boolean	`bool`			
failoverEventScheduler	boolean	`bool`			
failoverEventStatus	boolean	`bool`			
failoverFalsePositiveExternal	boolean	`bool`			
failoverFalsePositiveExternalPort	integer	`int64`			
failoverFalsePositiveHeartbeat	boolean	`bool`			
failoverFalsePositiveHeartbeatTimeout	integer	`int64`			
failoverFalsePositiveMaxscale	boolean	`bool`			
failoverFalsePositiveMaxscaleTimeout	integer	`int64`			
failoverFalsePositivePingCounter	integer	`int64`			
failoverLimit	integer	`int64`			
failoverLogFileKeep	integer	`int64`			
failoverMaxSlaveDelay	integer	`int64`			
failoverMdevCheck	boolean	`bool`			
failoverMdevLevel	string	`string`			
failoverMode	string	`string`			
failoverPostScript	string	`string`			
failoverPreScript	string	`string`			
failoverReadOnlyState	boolean	`bool`			
failoverResetTime	integer	`int64`			
failoverRestartUnsafe	boolean	`bool`			
failoverSemisyncState	boolean	`bool`			
failoverSuperReadOnlyState	boolean	`bool`			
failoverSwithToPrefered	boolean	`bool`			
failoverTimeLimit	integer	`int64`			
forceBinlogAnnotate	boolean	`bool`			
forceBinlogChecksum	boolean	`bool`			
forceBinlogCompress	boolean	`bool`			
forceBinlogPurge	boolean	`bool`			
forceBinlogPurgeMinReplica	integer	`int64`			
forceBinlogPurgeOnRestore	boolean	`bool`			
forceBinlogPurgeReplicas	boolean	`bool`			
forceBinlogPurgeTotalSize	integer	`int64`			
forceBinlogRow	boolean	`bool`			
forceBinlogSlowqueries	boolean	`bool`			
forceDiskRelaylogSizeLimit	boolean	`bool`			
forceDiskRelaylogSizeLimitSize	integer	`int64`			
forceInmemoryBinlogCacheSize	boolean	`bool`			
forceNoslaveBehind	boolean	`bool`			
forceSlaveGtidMode	boolean	`bool`			
forceSlaveGtidModeStrict	boolean	`bool`			
forceSlaveHeartbeat	boolean	`bool`			
forceSlaveHeartbeatRetry	integer	`int64`			
forceSlaveHeartbeatTime	integer	`int64`			
forceSlaveIdempotent	boolean	`bool`			
forceSlaveNoGtidMode	boolean	`bool`			
forceSlaveParallelMode	string	`string`			
forceSlaveReadonly	boolean	`bool`			
forceSlaveSemisync	boolean	`bool`			
forceSlaveStrict	boolean	`bool`			

Name	Type	Go type	Required	Default	Description
forceSyncBinlog	boolean	`bool`			
forceSyncInnodb	boolean	`bool`			
fullVersion	string	`string`			
gitMonitoringTicker	integer	`int64`			
gitUrl	string	`string`			
gitUrlPull	string	`string`			
gitUsername	string	`string`			
goArch	string	`string`			
goOS	string	`string`			
graphiteBlacklist	boolean	`bool`			
graphiteCarbonApiPort	integer	`int64`			
graphiteCarbonHost	string	`string`			
graphiteCarbonLinkPort	integer	`int64`			
graphiteCarbonPicklePort	integer	`int64`			
graphiteCarbonPort	integer	`int64`			
graphiteCarbonPprofPort	integer	`int64`			
graphiteCarbonServerPort	integer	`int64`			
graphiteEmbedded	boolean	`bool`			
graphiteMetrics	boolean	`bool`			
graphiteWhitelist	boolean	`bool`			
graphiteWhitelistTemplate	string	`string`			
haproxy	boolean	`bool`			
haproxyAPIPort	integer	`int64`			
haproxyAPIReadBackend	string	`string`			
haproxyAPIWriteBackend	string	`string`			
haproxyBinaryPath	string	`string`			
haproxyDebug	boolean	`bool`			
haproxyIpReadBind	string	`string`			
haproxyIpWriteBind	string	`string`			
haproxyJanitorWeights	string	`string`			
haproxyLogLevel	integer	`int64`			
haproxyMode	string	`string`			
haproxyPassword	string	`string`			
haproxyReadPort	integer	`int64`			
haproxyServers	string	`string`			
haproxyServers-ipv6	string	`string`			
haproxyStatPort	integer	`int64`			
haproxyWritePort	integer	`int64`			
haproxylUser	string	`string`			
heartbeatTable	boolean	`bool`			
http-use-react	boolean	`bool`			
httpAuth	boolean	`bool`			
httpBindAdress	string	`string`			
httpBootstrapButton	boolean	`bool`			
httpPort	string	`string`			
httpRefreshInterval	integer	`int64`			
httpRoot	string	`string`			
httpServer	boolean	`bool`			
httpSessionLifetime	integer	`int64`			
interactive	boolean	`bool`			
jobLogBatchSize	integer	`int64`			
kubeConfig	string	`string`			

Name	Type	Go type	Required	Default	Description
logBackupStream	boolean	`bool`			
logBackupStreamLevel	integer	`int64`			
logBinlogPurge	boolean	`bool`			
logBinlogPurgeLevel	integer	`int64`			
logConfigLoad	boolean	`bool`			
logConfigLoadLevel	integer	`int64`			
logFile	string	`string`			
logFileLevel	integer	`int64`			
logGit	boolean	`bool`			
logGitLevel	integer	`int64`			
logGraphite	boolean	`bool`			
logGraphiteLevel	integer	`int64`			
logHeartbeat	boolean	`bool`			
logHeartbeatLevel	integer	`int64`			
logLevel	integer	`int64`			
logOrchestrator	boolean	`bool`			
logOrchestratorLevel	integer	`int64`			
logProxy	boolean	`bool`			
logProxyLevel	integer	`int64`			
logRotateMaxAge	integer	`int64`			
logRotateMaxBackup	integer	`int64`			
logRotateMaxSize	integer	`int64`			
logSqlInMonitoring	boolean	`bool`			
logSst	boolean	`bool`			internal replication-manager sst
logSstLevel	integer	`int64`			internal replication-manager sst
logSyslog	boolean	`bool`			
logTask	boolean	`bool`			
logTaskLevel	integer	`int64`			
logTopology	boolean	`bool`			
logTopologyLevel	integer	`int64`			
logVault	boolean	`bool`			
logVaultLevel	integer	`int64`			
logWriterElection	boolean	`bool`			
logWriterElectionLevel	integer	`int64`			
mailFrom	string	`string`			
mailSmtpAddr	string	`string`			
mailSmtpPassword	string	`string`			
mailSmtpTlsSkipVerify	boolean	`bool`			
mailSmtpUser	string	`string`			
mailTo	string	`string`			
maxscale	boolean	`bool`			
maxscaleBinlog	boolean	`bool`			
maxscaleBinlogPort	integer	`int64`			
maxscaleDebug	boolean	`bool`			
maxscaleDisableMonitor	boolean	`bool`			
maxscaleGetInfoMethod	string	`string`			
maxscaleJanitorWeights	string	`string`			
maxscaleLogLevel	integer	`int64`			
maxscaleMaxinfoPort	integer	`int64`			
maxscalePass	string	`string`			
maxscalePort	string	`string`			
maxscaleReadPort	integer	`int64`			

Name	Type	Go type	Required	Default	Description
maxscaleReadWritePort	integer	`int64`			
maxscaleServerMatchPort	boolean	`bool`			
maxscaleServers	string	`string`			
maxscaleServers-ipv6	string	`string`			
maxscaleUser	string	`string`			
maxscaleWritePort	integer	`int64`			
maxscalemBinaryPath	string	`string`			
monitoringAddress	string	`string`			
monitoringAlertTrigger	string	`string`			
monitoringBasedir	string	`string`			
monitoringCapture	boolean	`bool`			
monitoringCaptureFileKeep	integer	`int64`			
monitoringCaptureTrigger	string	`string`			
monitoringCheckGrants	boolean	`bool`			
monitoringCloseStateScript	string	`string`			
monitoringConfdir	string	`string`			
monitoringConfdirBackup	string	`string`			
monitoringConfdirExtra	string	`string`			
monitoringDatadir	string	`string`			
monitoringDiskUsage	boolean	`bool`			
monitoringDiskUsagePct	integer	`int64`			
monitoringErreurLogLength	integer	`int64`			
monitoringIgnoreErrors	string	`string`			
monitoringInnoDBStatus	boolean	`bool`			
monitoringKeyPath	string	`string`			
monitoringKeyPathGitOverwrite	boolean	`bool`			
monitoringLongQueryLogLength	integer	`int64`			
monitoringLongQueryScript	string	`string`			
monitoringLongQueryTime	integer	`int64`			
monitoringLongQueryWithProcess	boolean	`bool`			
monitoringLongQueryWithTable	boolean	`bool`			
monitoringMergeConfigOnStart	boolean	`bool`			
monitoringOpenStateScript	string	`string`			
monitoringPause	boolean	`bool`			
monitoringPerformanceSchema	boolean	`bool`			
monitoringPlugins	boolean	`bool`			
monitoringProcesslist	boolean	`bool`			
monitoringQueries	boolean	`bool`			
monitoringQueryRules	boolean	`bool`			
monitoringQueryTimeout	integer	`int64`			
monitoringRestoreConfigOnStart	boolean	`bool`			
monitoringSSLCert	string	`string`			
monitoringSSLKey	string	`string`			
monitoringSaveConfig	boolean	`bool`			
monitoringScheduler	boolean	`bool`			
monitoringSchemaChange	boolean	`bool`			
monitoringSchemaChangeScript	string	`string`			
monitoringSharedir	string	`string`			
monitoringSocket	string	`string`			
monitoringTenant	string	`string`			
monitoringTicker	integer	`int64`			
monitoringTunnelCredential	string	`string`			

Name	Type	Go type	Required	Default	Description
monitoringTunnelHost	string	`string`			
monitoringTunnelKeyPath	string	`string`			
monitoringVariableDiff	boolean	`bool`			
monitoringWaitRetry	integer	`int64`			
monitoringWriteHeartbeat	boolean	`bool`			
monitoringWriteHeartbeatCredential	string	`string`			
myproxy	boolean	`bool`			
myproxyDebug	boolean	`bool`			
myproxyLogLevel	integer	`int64`			
myproxyPassword	string	`string`			
myproxyPort	integer	`int64`			
myproxyUser	string	`string`			
mysqlrouter	boolean	`bool`			
mysqlrouterDebug	boolean	`bool`			
mysqlrouterJanitorWeights	string	`string`			
mysqlrouterLogLevel	integer	`int64`			
mysqlrouterPass	string	`string`			
mysqlrouterPort	string	`string`			
mysqlrouterReadPort	integer	`int64`			
mysqlrouterReadWritePort	integer	`int64`			
mysqlrouterServers	string	`string`			
mysqlrouterUser	string	`string`			
mysqlrouterWritePort	integer	`int64`			
onpremiseSsh	boolean	`bool`			
onpremiseSshCredential	string	`string`			
onpremiseSshDbJobScript	string	`string`			
onpremiseSshPort	integer	`int64`			
onpremiseSshPrivateKey	string	`string`			
onpremiseSshStartDbScript	string	`string`			
onpremiseSshStartProxyScript	string	`string`			
onpremiseSshStopProxyScript	string	`string`			
opensvcAdminUser	string	`string`			
opensvcCodeapp	string	`string`			
opensvcCollectorAccount	string	`string`			
opensvcHost	string	`string`			
opensvcP12Certificate	string	`string`			
opensvcP12Secret	string	`string`			
opensvcRegister	boolean	`bool`			
opensvcUseCollectorApi	boolean	`bool`			
opensvcUser	string	`string`			
optimizeUseSql	boolean	`bool`			
printDelayStat	boolean	`bool`			
printDelayStatHistory	boolean	`bool`			
printDelayStatInterval	integer	`int64`			
provDBApplyDynamicConfig	boolean	`bool`			
provDBCompliance	string	`string`			
provDBForceWriteConfig	boolean	`bool`			
provDbAgents	string	`string`			
provDbBinaryBasedir	string	`string`			
provDbBinaryInTarball	boolean	`bool`			
provDbBinaryTarballName	string	`string`			
provDbBootstrapScript	string	`string`			

Name	Type	Go type	Required	Default	Description
provDbCleanupScript	string	`string`			
provDbClientBasedir	string	`string`			
provDbCpuCores	string	`string`			
provDbDatadirVersion	string	`string`			
provDbDiskDevice	string	`string`			
provDbDiskDockerSize	string	`string`			
provDbDiskFs	string	`string`			
provDbDiskFsCompress	string	`string`			
provDbDiskIops	string	`string`			
provDbDiskIopsLatency	string	`string`			
provDbDiskPool	string	`string`			
provDbDiskSize	string	`string`			
provDbDiskSnapshotKeep	integer	`int64`			
provDbDiskSnapshotPreferedMaster	boolean	`bool`			
provDbDiskSystemSize	string	`string`			
provDbDiskTempSize	string	`string`			
provDbDiskType	string	`string`			
provDbDockerImg	string	`string`			
provDbDomain	string	`string`			
provDbExpireLogDays	integer	`int64`			
provDbLoadCsv	string	`string`			
provDbLoadSql	string	`string`			
provDbMaxConnections	integer	`int64`			
provDbMemory	string	`string`			
provDbMemorySharedPct	string	`string`			
provDbMemoryThreadedPct	string	`string`			
provDbNetGateway	string	`string`			
provDbNetIface	string	`string`			
provDbNetMask	string	`string`			
provDbServiceType	string	`string`			
provDbStartScript	string	`string`			
provDbStopScript	string	`string`			
provDbTags	string	`string`			
provDbVolumeData	string	`string`			
provDbVolumeDocker	string	`string`			
provDockerDaemonPrivate	boolean	`bool`			
provNetCni	boolean	`bool`			
provNetCniCluster	string	`string`			
provOrchestrator	string	`string`			
provOrchestratorCluster	string	`string`			
provOrchestratorEnable	string	`string`			
provProxyAgents	string	`string`			
provProxyAgentsFailover	string	`string`			
provProxyBootstrapScript	string	`string`			
provProxyCleanupScript	string	`string`			
provProxyCompliance	string	`string`			
provProxyCpuCores	string	`string`			
provProxyDiskDevice	string	`string`			
provProxyDiskFs	string	`string`			
provProxyDiskPool	string	`string`			
provProxyDiskSize	string	`string`			
provProxyDiskType	string	`string`			

Name	Type	Go type	Required	Default	Description
provProxyDockerHaproxyImg	string	`string`			
provProxyDockerMaxscaleImg	string	`string`			
provProxyDockerMysqlrouterImg	string	`string`			
provProxyDockerProxysqlImg	string	`string`			
provProxyDockerShardproxyImg	string	`string`			
provProxyMemory	string	`string`			
provProxyNetGateway	string	`string`			
provProxyNetIface	string	`string`			
provProxyNetMask	string	`string`			
provProxyRouteAddr	string	`string`			
provProxyRouteMask	string	`string`			
provProxyRoutePolicy	string	`string`			
provProxyRoutePort	string	`string`			
provProxyServiceType	string	`string`			
provProxyStartScript	string	`string`			
provProxyStopScript	string	`string`			
provProxyTags	string	`string`			
provProxyVolumeData	string	`string`			
provSerialized	boolean	`bool`			
provServicePlan	string	`string`			
provServicePlanRegistry	string	`string`			
provSphinxAgents	string	`string`			
provSphinxCpuCores	string	`string`			
provSphinxDiskDevice	string	`string`			
provSphinxDiskFs	string	`string`			
provSphinxDiskPool	string	`string`			
provSphinxDiskSize	string	`string`			
provSphinxDiskType	string	`string`			
provSphinxDockerImg	string	`string`			
provSphinxMaxChildrens	string	`string`			
provSphinxMemory	string	`string`			
provSphinxReindexSchedule	string	`string`			
provSphinxServiceType	string	`string`			
provSphinxTags	string	`string`			
provTlsServerCa	string	`string`			
provTlsServerCert	string	`string`			
provTlsServerKey	string	`string`			
proxyServersBackendCompression	boolean	`bool`			
proxyServersBackendMaxConnections	integer	`int64`			
proxyServersBackendMaxReplicationLag	integer	`int64`			
proxyServersChangeStateScript	string	`string`			
proxyServersReadOnMaster	boolean	`bool`			
proxyServersReadOnMasterNoSlave	boolean	`bool`			
proxyjanitorAdminPort	string	`string`			
proxyjanitorBinaryPath	string	`string`			
proxyjanitorDebug	boolean	`bool`			
proxyjanitorLogLevel	integer	`int64`			
proxyjanitorPassword	string	`string`			
proxyjanitorPort	string	`string`			
proxyjanitorServers	string	`string`			
proxyjanitorServers-ipv6	string	`string`			
proxyjanitorUser	string	`string`			

Name	Type	Go type	Required	Default	Description
proxysql	boolean	`bool`			
proxysqlAdminPort	string	`string`			
proxysqlBinaryPath	string	`string`			
proxysqlBootstrap	boolean	`bool`			
proxysqlBootstrapHostgroups	boolean	`bool`			
proxysqlBootstrapQueryRules	boolean	`bool`			
proxysqlBootstrapVariables	boolean	`bool`			
proxysqlBootstrapyUsers	boolean	`bool`			
proxysqlDebug	boolean	`bool`			
proxysqlJanitorWeights	string	`string`			
proxysqlLogLevel	integer	`int64`			
proxysqlMultiplexing	boolean	`bool`			
proxysqlPassword	string	`string`			
proxysqlPort	string	`string`			
proxysqlReaderHostgroup	string	`string`			
proxysqlSaveToDisk	boolean	`bool`			
proxysqlServers	string	`string`			
proxysqlServersIpv6	string	`string`			
proxysqlUser	string	`string`			
proxysqlWriterHostgroup	string	`string`			
registryConsul	boolean	`bool`			
registryConsulCredential	string	`string`			
registryConsulDebug	boolean	`bool`			
registryConsulLogLevel	integer	`int64`			
registryConsulToken	string	`string`			
registryJanitorWeights	string	`string`			
registryServers	string	`string`			
replicationActivePassive	boolean	`bool`			
replicationCredential	string	`string`			
replicationDelayedHosts	string	`string`			
replicationDelayedTime	integer	`int64`			
replicationDynamicTopology	boolean	`bool`			
replicationErrorScript	string	`string`			
replicationMasterConnectRetry	integer	`int64`			
replicationMasterSlaveNeverRelay	boolean	`bool`			
replicationMasterSlavePgLogical	boolean	`bool`			
replicationMasterSlavePgStream	boolean	`bool`			
replicationMultiMaster	boolean	`bool`			
replicationMultiMasterConcurrentWrite	boolean	`bool`			
replicationMultiMasterGrouprep	boolean	`bool`			
replicationMultiMasterGrouprepPort	integer	`int64`			
replicationMultiMasterRing	boolean	`bool`			
replicationMultiMasterRingUnsafe	boolean	`bool`			
replicationMultiMasterWsrep	boolean	`bool`			
replicationMultiMasterWsrepPort	integer	`int64`			
replicationMultiMasterWsrepSSTMethod	string	`string`			
replicationMultiTierSlave	boolean	`bool`			
replicationMultisourceHeadClusters	string	`string`			
replicationSourceName	string	`string`			
replicationUseSsl	boolean	`bool`			
schedulerAlertDisable	boolean	`bool`			
schedulerAlertDisableCron	string	`string`			

Name	Type	Go type	Required	Default	Description
schedulerAlertDisableTime	integer	`int64`			
schedulerDatabaseLogsTableKeep	integer	`int64`			
schedulerDbServersAnalyze	boolean	`bool`			
schedulerDbServersAnalyzeCron	string	`string`			
schedulerDbServersLogicalBackup	boolean	`bool`			
schedulerDbServersLogicalBackupCron	string	`string`			
schedulerDbServersLogs	boolean	`bool`			
schedulerDbServersLogsCron	string	`string`			
schedulerDbServersLogsTableRotate	boolean	`bool`			
schedulerDbServersLogsTableRotateCron	string	`string`			
schedulerDbServersOptimize	boolean	`bool`			
schedulerDbServersOptimizeCron	string	`string`			
schedulerDbServersPhysicalBackup	boolean	`bool`			
schedulerDbServersPhysicalBackupCron	string	`string`			
schedulerDbServersReceiverPorts	string	`string`			
schedulerDbServersReceiverUseSSL	boolean	`bool`			
schedulerDbServersSenderPorts	string	`string`			
schedulerJobsSsh	boolean	`bool`			
schedulerJobsSshCron	string	`string`			
schedulerRollingReprov	boolean	`bool`			
schedulerRollingReprovCron	string	`string`			
schedulerRollingRestart	boolean	`bool`			
schedulerRollingRestartCron	string	`string`			
schedulerSlaRotateCron	string	`string`			
shardproxy	boolean	`bool`			
shardproxyCopyGrants	boolean	`bool`			
shardproxyCredential	string	`string`			
shardproxyDebug	boolean	`bool`			
shardproxyIgnoreTables	string	`string`			
shardproxyJanitorWeights	string	`string`			
shardproxyLoadSystem	boolean	`bool`			
shardproxyLogLevel	integer	`int64`			
shardproxyServers	string	`string`			
shardproxyServers-ipv6	string	`string`			
shardproxyUniversalTables	string	`string`			
slaposConfig	string	`string`			
slaposDbPartitions	string	`string`			
slaposHaproxyPartitions	string	`string`			
slaposMaxscalePartitions	string	`string`			
slaposProxysqlPartitions	string	`string`			
slaposShardproxyPartitions	string	`string`			
slaposSphinxPartitions	string	`string`			
sphinx	boolean	`bool`			
sphinxConfig	string	`string`			
sphinxDebug	boolean	`bool`			
sphinxJanitorWeights	string	`string`			
sphinxLogLevel	integer	`int64`			
sphinxPort	string	`string`			
sphinxServers	string	`string`			
sphinxServers-ipv6	string	`string`			
sphinxSqlPort	string	`string`			
sstSendBuffer	integer	`int64`			

Name	Type	Go type	Required	Default	Description
switchoverAtEqualGtid	boolean	`bool`			
switchoverAtSync	boolean	`bool`			
switchoverDecreaseMaxConn	boolean	`bool`			
switchoverDecreaseMaxConnValue	integer	`int64`			
switchoverLowerRelease	boolean	`bool`			
switchoverMaxSlaveDelay	integer	`int64`			
switchoverSlaveWaitCatch	boolean	`bool`			
switchoverWaitKill	integer	`int64`			
switchoverWaitRouteChange	integer	`int64`			
switchoverWaitTrx	integer	`int64`			
switchoverWaitWriteQuery	integer	`int64`			
sysbenchBinaryPath	string	`string`			
sysbenchBinaryTest	string	`string`			
sysbenchScale	integer	`int64`			
sysbenchTables	integer	`int64`			
sysbenchThreads	integer	`int64`			
sysbenchTime	integer	`int64`			
sysbenchV1	boolean	`bool`			
test	boolean	`bool`			
testInjectTraffic	boolean	`bool`			
topologyTarget	string	`string`			
vaultAuth	string	`string`			
vaultMode	string	`string`			
vaultMount	string	`string`			
vaultRoleId	string	`string`			
vaultSecretId	string	`string`			
vaultServerAddr	string	`string`			
vaultToken	string	`string`			
verbose	boolean	`bool`			
version	string	`string`			
withEmbed	string	`string`			
withTarball	string	`string`			

3.4.2.3.43 gtid.Gtid

Properties

Name	Type	Go type	Required	Default	Description	Example
domainId	integer	`int64`				
seqNo	integer	`int64`				
serverId	integer	`int64`				

3.4.2.3.44 opensvc.Addr

Properties

Name	Type	Go type	Required	Default	Description	Example
addr	string	`string`				
addr_type	string	`string`				
intf	string	`string`				
mask	string	`string`				

Name	Type	Go type	Required	Default	Description	Example
net_broadcast	string	`string`				
net_gateway	string	`string`				
net_name	string	`string`				
net_network	string	`string`				

3.4.2.3.45 opensvc.Host

Properties

Name	Type	Go type	Required	Default	Description	Example
cpu_cores	integer	`int64`				
cpu_freq	integer	`int64`				
id	integer	`int64`				
ips	[]([OpensvcAddr](#opensvc-addr))	`[]*OpensvcAddr`				
mem_bytes	integer	`int64`				
node_id	string	`string`				
nodename	string	`string`				
os_kernel	string	`string`				
os_name	string	`string`				
svc	[]([OpensvcService](#opensvc-service))	`[]*OpensvcService`				

3.4.2.3.46 opensvc.Service

Properties

Name	Type	Go type	Required	Default	Description	Example
id	integer	`int64`				
svc_id	string	`string`				
svc_status	string	`string`				
svcname	string	`string`				
updated	string	`string`				

3.4.2.3.47 os.Process

Properties

Name	Type	Go type	Required	Default	Description	Example
pid	integer	`int64`				

3.4.2.3.48 regexp.Regexp

interface{}

3.4.2.3.49 repmanv3.BackupStat

Properties

Name	Type	Go type	Required	Default	Description	Example
total_blob_count	integer	`int64`				
total_file_count	integer	`int64`				
total_size	integer	`int64`				

3.4.2.3.50 repmanv3.Tag

Properties

Name	Type	Go type	Required	Default	Description	Example
category	string	`string`				
id	integer	`int64`				
name	string	`string`				

3.4.2.3.51 s18log.HttpLog

Properties

Name	Type	Go type	Required	Default	Description	Ex
buffer	[]\[S18logHTTPMessage](#s18log-http-message)	`[]*S18logHTTPMessage`				
len	integer	`int64`				
line	integer	`int64`				

3.4.2.3.52 s18log.HttpMessage

Properties

Name	Type	Go type	Required	Default	Description	Example
group	string	`string`				
level	string	`string`				
text	string	`string`				
timestamp	string	`string`				

3.4.2.3.53 server.ApiResponse

Properties

Name	Type	Go type	Required	Default	Description	Example
data	string	`string`				
success	boolean	`bool`				

3.4.2.3.54 server.CredentialMailForm

Properties

Name	Type	Go type	Required	Default	Description	Example
type	string	`string`				
username	string	`string`				

3.4.2.3.55 server.DecodedData

Properties

Name	Type	Go type	Required	Default	Description	Example
data	string	`string`				

3.4.2.3.56 server.Heartbeat

Properties

Name	Type	Go type	Required	Default	Description	Example
cluster	string	`string`				
failed	integer	`int64`				
hosts	integer	`int64`				
id	integer	`int64`				
master	string	`string`				
secret	string	`string`				
status	string	`string`				
uuid	string	`string`				

3.4.2.3.57 server.ReplicationManager

Properties

Name	Type	Go type	Required	Default	Description	Example
agents	[][OpensvcHost](#) host)	`[]*OpensvcHost`				
arch	string	`string`				
backupBinlogs	map of boolean	`map[string]bool`				
backupLogicals	map of boolean	`map[string]bool`				
backupPhysicals	map of boolean	`map[string]bool`				
binlogParsers	map of boolean	`map[string]bool`				
canConnectVault	boolean	`bool`				
clusters	[]string	`[]string`				
config	GithubComSignal18ReplicationManagerConfigConfig	`GithubComSignal18ReplicationManagerConfigConfig`				
confs	map of GithubComSignal18ReplicationManagerConfigConfig	`map[string]GithubComSignal18ReplicationManagerConfigConfig`				
cpuprofile	string	`string`				
fullVersion	string	`string`				
graphiteTemplates	map of boolean	`map[string]bool`				
hasSavingCronjobQueue	boolean	`bool`				
hostname	string	`string`				
isGitPull	boolean	`bool`				
isGitPush	boolean	`bool`				

Name	Type	Go type	Required	Default	Description	Example
isSavingConfig	boolean	`bool`				
logs	S18logHTTPLog	`S18logHTTPLog`				
memprofile	string	`string`				
os	string	`string`				
osUser	UserUser	`UserUser`				
partner	ConfigPartner	`ConfigPartner`				
partners	[]([ConfigPartner partner)	`[]*ConfigPartner`				
serviceAcl	[]([ConfigGrant grant)	`[]*ConfigGrant`				
serviceDisk	map of string	`map[string]string`				
serviceFS	map of boolean	`map[string]bool`				
serviceOrchestrations	[]([ConfigConfigVariableType config-variable-type)	`[]*ConfigConfigVariableType`				
servicePlans	[]([ConfigServicePlan service-plan)	`[]*ConfigServicePlan`				
servicePool	map of boolean	`map[string]bool`				
serviceRepos	[]([ConfigDockerRepo docker-repo)	`[]*ConfigDockerRepo`				
serviceRoles	[]([ConfigRole role)	`[]*ConfigRole`				
serviceTarballs	[]([ConfigTarball tarball)	`[]*ConfigTarball`				
serviceVM	map of boolean	`map[string]bool`				
spitBrain	boolean	`bool`				
status	string	`string`				
termsDT	map of string	`map[string]string`				
tests	[]string	`[]string`				
uuid	string	`string`				
version	string	`string`				

3.4.2.3.58 server.token

Properties

Name	Type	Go type	Required	Default	Description	Example
token	string	`string`				

3.4.2.3.59 server.userCredentials

Properties

Name	Type	Go type	Required	Default	Description	Example
password	string	`string`				
username	string	`string`				

3.4.2.3.60 sql.NullInt64

Properties

Name	Type	Go type	Required	Default	Description	Example
int64	integer	`int64`				
valid	boolean	`bool`			Valid is true if Int64 is not NULL	

3.4.2.3.61 sql.NullString

Properties

Name	Type	Go type	Required	Default	Description	Example
string	string	`string`				
valid	boolean	`bool`			Valid is true if String is not NULL	

3.4.2.3.62 state.Sla

Properties

Name	Type	Go type	Required	Default	Description	Example
firsttime	integer	`int64`				
lasttime	integer	`int64`				
uptime	integer	`int64`				
uptimeFailable	integer	`int64`				
uptimeSemisync	integer	`int64`				

3.4.2.3.63 state.StateHttp

Properties

Name	Type	Go type	Required	Default	Description	Example
desc	string	`string`				
from	string	`string`				
number	string	`string`				

3.4.2.3.64 state.StateMachine

Properties

Name	Type	Go type	Required	Default	Description	Example
discovered	boolean	`bool`				
inFailover	boolean	`bool`				
inSchemaMonitor	boolean	`bool`				

3.4.2.3.65 user.User

Properties

Name	Type	Go type	Required	Default	Description	Example
gid	string	`string`			Gid is the primary group ID.On POSIX systems, this is a decimal number representing the gid.On Windows, this is a SID in a string format.On Plan 9, this is the contents of /dev/user.	
homeDir	string	`string`			HomeDir is the path to the user's home directory (if they have one).	
name	string	`string`			Name is the user's real or display name.It might be blank.On POSIX systems, this is the first (or only) entry in the GECOS fieldlist.On Windows, this is the user's display name.On Plan 9, this is the contents of /dev/user.	
uid	string	`string`			Uid is the user ID.On POSIX systems, this is a decimal number representing the uid.On Windows, this is a security identifier (SID) in a string format.On Plan 9, this is the contents of /dev/user.	
username	string	`string`			Username is the login name.	

3.4.2.3.66 version.Version

Properties

Name	Type	Go type	Required	Default	Description	Example
dist	VersionVersion	`VersionVersion`				
flavor	string	`string`				
major	integer	`int64`				
minor	integer	`int64`				
path	string	`string`				
release	integer	`int64`				
suffix	string	`string`				

3.4.3.0.1 Monitor Unprotected Endpoints

/api/login

INPUT: POST

```
{"username":"admin", "password":"mariadb"}
```

OUTPUT:

```
{"token":"hash"}
```

/api/status

OUPUT:

```
{"alive": true}
./replication-manager api  --url="https://127.0.0.1:10005/api/status"
```

/api/clusters/{clusterName}/status

Version 2.0 & 2.1

/clusters/{clusterName}/servers/{serverName}/master-status Return Code 200 if server id is a master

/clusters/{clusterName}/servers/{serverName}/slave-status Return Code 200 if server id is a slave

/clusters/{clusterName}/servers/{serverHost}/{serverPort}/master-status Return Code 200 if server id is a master

/clusters/{clusterName}/servers/{serverHost}/{serverPort}/slave-status Return Code 200 if server id is a slave
Version 2.1

/clusters/{clusterName}/servers/{serverName}/is-master Return Code 200 if server id is a master

/clusters/{clusterName}/servers/{serverName}/is-slave Return Code 200 if server id is a slave

/clusters/{clusterName}/servers/{serverHost}/{serverPort}/is-master Return Code 200 if server id is a master

/clusters/{clusterName}/servers/{serverHost}/{serverPort}/is-slave Return Code 200 if server id is a slave

3.4.3.0.2 API Protected Endpoints

/api/monitor

OUPUT:


```
{
    "version": "2.0.0-rc2",
    "full-version": "2.0.0-rc2-132-gae9091a",
    "os": "darwin",
    "arch": "amd64",
    "agents": [],
    "uuid": "7224a9d3",
    "hostname": "macbook-pro-de-apple-2.local",
    "status": "A",
    "spitbrain": false,
    "clusters": [
        "cluster_haproxy_masterslave"
    ],
    "tests": [
```

```
    "testSwitchoverAllSlavesDelayMultimasterNoRplChecksNoSemiSync",
    "testSwitchoverLongTransactionNoRplCheckNoSemiSync",
    "testSwitchoverLongQueryNoRplCheckNoSemiSync",
    "testSwitchoverLongTrxWithoutCommitNoRplCheckNoSemiSync",
    "testSwitchoverReadOnlyNoRplCheck",
    ........
    "testSlaReplAllSlavesStopNoSemiSync",
    "testSlaReplAllSlavesDelayNoSemiSync"
],
"config": {
    "monitoring-basedir": "system",
    "monitoring-datadir": "/var/lib/replication-manager",
    "monitoring-sharedir": "/opt/replication-manager/share",
    "monitoring-confdir": "/etc/replication-manager",
    "monitoring-ticker": 2,
    "monitoring-socket": "",
    "monitoring-tunnel-host": "",
    "monitoring-tunnel-credential": "",
    "monitoring-address": "",
    "monitoring-write-heartbeat": false,
    "monitoring-write-heartbeat-credential": "",
    "monitoring-schema-change": true,
    "monitoring-schema-change-script": "",
    "monitoring-queries": true,
    "monitoring-long-query-time": 10000,
    "monitoring-long-query-script": "",
    "monitoring-scheduler": true,
    "verbose": false,
    "log-file": "",
    "log-syslog": false,
    "log-level": 0,
    "log-sst": false,
    "db-servers-credential": "",
    "db-servers-hosts": "",
    "db-servers-tls-ca-cert": "",
    "db-servers-tls-client-key": "",
    "db-servers-tls-client-cert": "",
    "db-servers-prefered-master": "",
    "db-servers-ignored-hosts": "",
    "db-servers-connect-timeout": 5,
    "db-servers-read-timeout": 15,
    "db-servers-binary-path": "/usr/local/mysql/bin",
    "db-servers-locality": "127.0.0.1",
    "replication-master-connect-retry": 10,
    "replication-credential": "",
    "replication-source-name": "",
    "replication-use-ssl": false,
    "replication-multi-master-ring": false,
    "replication-multi-master-wsrep": false,
    "replication-multi-master": false,
    "replication-multi-tier-slave": false,
    "replication-master-slave-never-relay": true,
    "switchover-wait-kill": 5000,
    "switchover-wait-trx": 10,
```

"switchover-wait-write-query": 10,
"switchover-at-equal-gtid": false,
"switchover-at-sync": false,
"switchover-max-slave-delay": 0,
"switchover-slave-wait-catch": true,
"failover-limit": 5,
"failover-pre-script": "",
"failover-post-script": "",
"failover-readonly-state": true,
"failover-time-limit": 0,
"failover-at-sync": false,
"failover-event-scheduler": false,
"failover-event-status": false,
"failover-restart-unsafe": false,
"failover-falsepositive-ping-counte": 5,
"failover-reset-time": 0,
"failover-mode": "manual",
"failover-max-slave-delay": 30,
"failover-falsepositive-heartbeat": true,
"failover-falsepositive-maxscale": false,
"failover-falsepositive-heartbeat-timeout": 3,
"failover-falsepositive-maxscale-timeout": 14,
"failover-falsepositive-external": false,
"failover-falsepositive-external-port": 80,
"autorejoin": true,
"autorejoin-flashback": false,
"autorejoin-script": "",
"autorejoin-mysqldump": false,
"autorejoin-backup-binlog": true,
"autorejoin-flashback-on-sync": true,
"autorejoin-flashback-on-unsync": false,
"autorejoin-slave-positional-hearbeat": false,
"autorejoin-zfs-flashback": false,
"check-type": "tcp",
"check-replication-filters": true,
"check-binlog-filters": true,
"check-grants": true,
"check-replication-state": true,
"force-slave-heartbeat": false,
"force-slave-heartbeat-time": 3,
"force-slave-heartbeat-retry": 5,
"force-slave-gtid-mode": false,
"force-slave-no-gtid-mode": false,
"force-slave-semisync": false,
"force-slave-readonly": false,
"force-binlog-row": false,
"force-binlog-annotate": false,
"force-binlog-compress": false,
"force-binlog-slowqueries": false,
"force-binlog-checksum": false,
"force-inmemory-binlog-cache-size": false,
"force-disk-relaylog-size-limit": false,
"force-disk-relaylog-size-limit-size": 1000000000,
"force-sync-binlog": false,

"force-sync-innodb": false,
"orce-noslave-behind": false,
"http-bind-address": "localhost",
"http-port": "10001",
"http-server": true,
"http-root": "/opt/replication-manager/share/dashboard",
"http-auth": false,
"http-bootstrap-button": false,
"http-session-lifetime": 3600,
"mail-from": "mrm@localhost",
"mail-to": "",
"mail-smtp-addr": "localhost:25",
"heartbeat-table": false,
"extproxy": false,
"extproxy-address": "",
"shardproxy": false,
"shardproxy-servers": "127.0.0.1:3307",
"shardproxy-user": "root:mariadb",
"shardproxy-copy-grants": true,
"maxscale": false,
"maxscale-servers": "127.0.0.1",
"maxscale-port": "6603",
"maxscale-user": "admin",
"maxscale-pass": "mariadb",
"maxscale-write-port": 3306,
"maxscale-read-port": 3307,
"maxscale-read-write-port": 3308,
"maxscale-maxinfo-port": 3309,
"maxscale-binlog": false,
"MxsBinlogPort": 3309,
"maxscale-disable-monitor": false,
"maxscale-get-info-method": "maxadmin",
"maxscale-server-match-port": false,
"myproxy": false,
"myproxy-port": 4000,
"myproxy-user": "admin",
"myproxy-password": "repman",
"haproxy": true,
"haproxy-servers": "127.0.0.1",
"haproxy-write-port": 3306,
"haproxy-read-port": 3307,
"haproxy-stat-port": 1988,
"haproxy-ip-write-bind": "0.0.0.0",
"haproxy-ip-read-bind": "0.0.0.0",
"haproxy-binary-path": "/usr/local/bin/haproxy",
"proxysql": false,
"proxysql-servers": "127.0.0.1",
"proxysql-port": "6033",
"proxysql-admin-port": "6032",
"proxysql-user": "admin",
"proxysql-password": "admin",
"proxysql-writer-hostgroup": "0",
"proxysql-reader-hostgroup": "1",
"proxysql-copy-grants": true,

"proxysql-bootstrap": false,
"mysqlrouter": false,
"mysqlrouter-servers": "",
"mysqlrouter-port": "",
"mysqlrouter-use": "",
"mysqlrouter-pass": "",
"mysqlrouter-write-port": 0,
"mysqlrouter-read-port": 0,
"mysqlrouter-read-write-port": 0,
"sphinx": false,
"sphinx-servers": "127.0.0.1",
"sphinx-config": "/opt/replication-manager/share/sphinx/sphinx.conf",
"sphinx-sql-port": "9306",
"sphinx-port": "9312",
"registry-consul": false,
"registry-servers": "127.0.0.1",
"graphite-metrics": false,
"graphite-embedded": false,
"graphite-carbon-host": "127.0.0.1",
"graphite-carbon-port": 2003,
"graphite-carbon-api-port": 10002,
"graphite-carbon-server-port": 10003,
"graphite-carbon-link-port": 7002,
"graphite-carbon-pickle-port": 2004,
"graphite-carbon-pprof-port": 7007,
"sysbench-binary-path": "/usr/local/bin/sysbench",
"sysbench-time": 100,
"sysbench-threads": 4,
"arbitration-external": false,
"arbitration-external-secret": "",
"arbitration-external-hosts": "88.191.151.84:80",
"arbitration-external-unique-id": 0,
"arbitration-peer-hosts": "127.0.0.1:10001",
"arbitrator-bind-address": "",
"arbitrator-driver": "",
"FailForceGtid": false,
"test": true,
"test-inject-traffic": false,
"Enterprise": false,
"opensvc-host": "",
"opensvc-register": false,
"opensvc-admin-user": "",
"opensvc-user": "",
"opensvc-codeapp": "",
"prov-db-localhost-binary-path": "",
"prov-db-service-type": "",
"prov-db-agents": "",
"prov-db-memory": "",
"prov-db-disk-iops": "",
"prov-db-cpu-cores": "",
"prov-db-tags": "",
"prov-db-disk-size": "",
"prov-db-disk-fs": "",
"prov-db-disk-pool": "",

"prov-db-disk-device": "",
"prov-db-disk-type": "",
"prov-db-disk-snapshot-prefered-master": false,
"prov-db-disk-snapshot-keep": 0,
"prov-db-net-iface": "",
"prov-db-net-mask": "",
"prov-db-net-gateway": "",
"prov-db-docker-img": "",
"prov-db-datadir-version": "10.2",
"prov-db-load-sql": "",
"prov-db-load-csv": "",
"prov-proxy-service-type": "",
"prov-proxy-agents": "",
"prov-proxy-disk-size": "",
"prov-proxy-disk-fs": "",
"prov-proxy-disk-pool": "",
"prov-proxy-disk-device": "",
"prov-proxy-disk-type": "",
"prov-proxy-net-iface": "",
"prov-proxy-net-mask": "",
"prov-proxy-net-gateway": "",
"prov-proxy-route-addr": "",
"prov-proxy-route-port": "",
"rov-proxy-route-mask": "",
"prov-proxy-route-policy": "",
"prov-proxy-docker-shardproxy-img": "",
"prov-proxy-docker-maxscale-img": "",
"prov-proxy-docker-haproxy-img": "",
"prov-proxy-docker-proxysql-img": "",
"prov-proxy-docker-mysqlrouter-img": "",
"prov-proxy-tags": "",
"prov-sphinx-agents": "",
"prov-sphinx-docker-img": "",
"prov-sphinx-memory": "",
"prov-sphinx-disk-size": "",
"prov-sphinx-cpu-cores": "",
"prov-sphinx-max-childrens": "",
"prov-sphinx-disk-pool": "",
"prov-sphinx-disk-fs": "",
"prov-sphinx-disk-device": "",
"prov-sphinx-disk-type": "",
"prov-sphinx-tags": "",
"prov-sphinx-reindex-schedule": "",
"prov-sphinx-service-type": "",
"prov-tls-server-ca": "",
"prov-tls-server-cert": "",
"prov-tls-server-key": "",
"api-credentials": "admin:repman",
"api-port": "10005",
"api-bind": "0.0.0.0",
"alert-script": "",
"backup": false,
"scheduler-db-servers-logical-backup": true,
"scheduler-db-servers-physical-backup": true,

```
        "scheduler-db-servers-logs": true,
        "scheduler-db-servers-optimize": true,
        "scheduler-db-servers-logical-backup-cron": "0 0 1 * * 6",
        "scheduler-db-servers-physical-backup-cron": "0 0 0 * * *",
        "scheduler-db-servers-logs-cron": "@every 10m",
        "scheduler-db-servers-optimize-cron": "0 0 3 1 * 5",
        "backup-logical-type": "mysqldump",
        "backup-physical-type": "xtrabackup",
        "backup-keep-hourly": 1,
        "backup-keep-daily": 1,
        "backup-keep-weekly": 1,
        "backup-keep-monthly": 1,
        "backup-keep-yearly": 1,
        "backup-repo": "directory",
        "backup-repo-aws-key": "",
        "backup-repo-aws-secret": "",
        "backup-repo-aws-uri": ""
    },
    "logs": {
        "Buffer": [
            {
                "Group": "cluster_haproxy_masterslave",
                "Level": "STATE",
                "Timestamp": "2018/02/16 11:36:59",
                "Text": "OPENED WARN0073 : Xtrabackup task running on server 127.0.0.1:3313"
            },
            {
                "Group": "cluster_haproxy_masterslave",
                "Level": "STATE",
                "Timestamp": "2018/02/16 11:36:59",
                "Text": "OPENED ERR00060 : To many non closed task in scheduler, donor may not work on s
            },
      ........
```


3.4.3.1 Cluster Protected Endpoint

3.4.3.1.1 2.0

/api/clusters/{clusterName}


```
{
    "name": "cluster_haproxy_masterslave",
    "db-servers": [
        "8228534449735335205",
        "13916101692659507786",
        "9706829687233797502",
        "3862921442801615377"
    ],
    "db-servers-crashes": null,
    "proxies-servers": [
        "12217188530888569408"
    ],
    "failover-counter": 0,
    "failover-last-time": 0,
    "active-passive-status": "A",
```

```
    "config": {
        "monitoring-basedir": "system",
        "monitoring-datadir": "/var/lib/replication-manager",
        "monitoring-sharedir": "/opt/replication-manager/share",
        "monitoring-confdir": "/etc/replication-manager",
        "monitoring-ticker": 2,
        .....
        "backup-keep-daily": 1,
        "backup-keep-weekly": 1,
        "backup-keep-monthly": 1,
        "backup-keep-yearly": 1,
        "backup-repo": "directory",
        "backup-repo-aws-key": "",
        "backup-repo-aws-secret": "",
        "backup-repo-aws-uri": ""
    },
    "clean-replication": false,
    "is-down": false,
    "is-provisionned": false,
    "schedule": null
}
```


/api/clusters/{clusterName}/settings

Return cluster configuration file

/api/clusters/{clusterName}/actions/switchover

/api/clusters/{clusterName}/actions/failover

/api/clusters/{clusterName}/actions/replication/bootstrap/{topology}

/api/clusters/{clusterName}/actions/replication/cleanup

/api/clusters/{clusterName}/actions/services todo List agents services resources

/api/clusters/{clusterName}/actions/services/provision

/api/clusters/{clusterName}/actions/services/bootstrap

/api/clusters/{clusterName}/actions/start-traffic

/api/clusters/{clusterName}/actions/stop-traffic

/api/clusters/{clusterName}/actions/addserver/{host}/{port}

/api/clusters/{clusterName}/topology/servers

/api/clusters/{clusterName}/topology/master

/api/clusters/{clusterName}/topology/slaves

/api/clusters/{clusterName}/topology/proxies

/api/clusters/{clusterName}/topology/logs

/api/clusters/{clusterName}/topology/alerts

/api/clusters/{clusterName}/topology/crashes

/api/clusters/{clusterName}/tests

/api/clusters/{clusterName}/tests/actions/run/{testName}

/api/clusters/{clusterName}/settings

/api/clusters/{clusterName}/settings/reload

/api/clusters/{clusterName}/settings/switch/{parmaterName}

3.4.3.2 2.1

/api/clusters/{clusterName}/actions/master-physical-backup

3.4.3.3 Database Server Protected Endpoint

3.4.3.3.1 2.0

/api/clusters/{clusterName}/servers/{serverName}/actions/start

/api/clusters/{clusterName}/servers/{serverName}/actions/stop

/api/clusters/{clusterName}/servers/{serverName}/actions/backup

/api/clusters/{clusterName}/servers/{serverName}/actions/maintenance

/api/clusters/{clusterName}/servers/{serverName}/actions/unprovision

/api/clusters/{clusterName}/servers/{serverName}/actions/provision

3.4.3.3.2 2.1

/api/clusters/{clusterName}/servers/{serverName}/actions/optimize

/api/clusters/{clusterName}/servers/{serverName}/actions/backup-physical Post an event into replication_manager_schema.jobs for cron processing

/api/clusters/{clusterName}/servers/{serverName}/actions/backup-logical Post an event into replication_manager_schema.jobs for cron processing

/api/clusters/{clusterName}/servers/{serverName}/actions/backup-error-log Post an event into replication_manager_schema.jobs for cron processing

/api/clusters/{clusterName}/servers/{serverName}/actions/backup-slow-query-log Post an event into replication_manager_schema.jobs for cron processing

/api/clusters/{clusterName}/servers/{serverName}/actions/reseed/{backupMethod}

/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-innodb-monitor

/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-slow-query

/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-slow-query-capture

/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-slow-query

/api/clusters/{clusterName}/servers/{serverName}/actions/toogle-read-only

/api/clusters/{clusterName}/servers/{serverName}/actions/reset-master

/api/clusters/{clusterName}/servers/{serverName}/actions/start-slave

/api/clusters/{clusterName}/servers/{serverName}/actionsskip-replication-event

/api/clusters/{clusterName}/servers/{serverName}/processlist

/api/clusters/{clusterName}/servers/{serverName}/variables

/api/clusters/{clusterName}/servers/{serverName}/status

/api/clusters/{clusterName}/servers/{serverName}/status-delta

/api/clusters/{clusterName}/servers/{serverName}/status-innodb

/api/clusters/{clusterName}/servers/{serverName}/errorlog

/api/clusters/{clusterName}/servers/{serverName}/slowlog

/api/clusters/{clusterName}/servers/{serverName}/tables

/api/clusters/{clusterName}/servers/{serverName}/vtables

/api/clusters/{clusterName}/servers/{serverName}/schemas

/api/clusters/{clusterName}/servers/{serverName}/all-slaves-status

/api/clusters/{clusterName}/servers/{serverName}/master-status

/api/clusters/{clusterName}/servers/{serverName}/is-master

/api/clusters/{clusterName}/servers/{serverName}/is-slave

/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/is-master

/api/clusters/{clusterName}/servers/{serverName}/{serverPort}/is-slave

/api/clusters/{clusterName}/servers/{serverName}/service-opensvc

3.4.3.4 Proxy Protected Endpoint

/api/clusters/{clusterName}/proxies/{proxyName}/actions/unprovision

/api/clusters/{clusterName}/proxies/{proxyName}/actions/provision

3.6.1 Provisioning Agent

3.6.1.1 Get services manage by the agent

om mon

```
Threads                                                 s18-fr-4    s18-fr-5     s18-fr-6
 daemon                  running                      |
 dns                     running
 hb#1.rx                 running         [::]:10000 | X           X            /
 hb#1.tx                 running                    | O           O            /
 hb#2.rx                 running relay2.opensvc.com | O           O            /
 hb#2.tx                 running                    | O           O            /
 hb#3.rx                 running 37.187.220.6:10001 | O           O            /
 hb#3.tx                 running                    | O           O            /
 listener                running              :1214
 monitor                 running
 scheduler               running

Nodes                                                   s18-fr-4    s18-fr-5     s18-fr-6
 score                                                | 94          65           91
  load 15m                                            | 0.5         1.2          1.4
  mem                                                 | 30/98%:188g 65/98%:187g  24/98%:188g
  swap                                                | 3/90%:2.00g 81/90%:1023m 2/90%:1023m
 state                                                |

*/svc/*                                                 s18-fr-4    s18-fr-5     s18-fr-6
 arceau/svc/db1          up             ha    1/1   | O^
 arceau/svc/db2          up             ha    1/1   |             O^
 arceau/svc/db3          up             ha    1/1   |                          O^
 arceau/svc/prx1         up             ha    1/1   | O^
 arceau/svc/prx2         up             ha    1/1   |             O^
```

3.6.1.2 Check service status

Get to the node running the service
om arceau/svc/db3 print status

```
arceau/svc/db3                  up
`- instances
   `- s18-fr-6                  up         idle, started
      |- ip#01          ...../.. up         cni s18 10.60.48.92/20 eth12
      |- volume#01      ........ up         db3
      |- volume#02      ........ up         db3-sec
      |- container#01   ...../.. up         docker ghcr.io/opensvc/pause
      |- container#02   ...O./.. n/a        docker alpine
      |- container#db   ...../.. up         docker mariadb:11.4
      `- container#jobs ...../.. up         docker mariadb:11.4
```

3.6.1.3 Get service configuration

om arceau/svc/db3 print config

```
[DEFAULT]
app =
docker_daemon_private = false
nodes = s18-fr-6
```

```
orchestrate = ha
rollback = false
id = aa37e187-f2af-4383-a58c-9594e2bb2493

[container#01]
hostname = {svcname}.{namespace}.svc.{clustername}
image = ghcr.io/opensvc/pause
rm = true
run_args = --sysctl net.ipv4.tcp_tw_reuse=1 --sysctl net.core.somaxconn=1024  --sysctl net.ipv4.tcp_fin_
type = docker

[container#02]
command = -c 'wget --no-check-certificate -q -O- $REPLICATION_MANAGER_URL/static/configurator/opensvc/bo
configs_environment = env/REPLICATION_MANAGER_USER env/REPLICATION_MANAGER_URL
detach = false
entrypoint = /bin/sh
environment = REPLICATION_MANAGER_CLUSTER_NAME={namespace} REPLICATION_MANAGER_HOST_NAME={fqdn} REPLICAT
image = alpine
netns = container#01
optional = true
rm = true
secrets_environment = env/REPLICATION_MANAGER_PASSWORD
start_timeout = 30s
type = docker
volume_mounts = /etc/localtime:/etc/localtime:ro {name}:/bootstrap

[container#db]
##docker_image = quay.io/mariadb-foundation/mariadb-debug:10.11-mdev-33798-knielsen-pkgtest
#command = gdb -ex r -ex thread apply all bt -frame-arguments all full --args mariadbd
#run_args = --user mysql --cap-add SYS_PTRACE --ulimit nofile=262144:262144
environment = MYSQL_INITDB_SKIP_TZINFO=yes
image = {env.docker_image}
netns = container#01
rm = true
run_args = --tmpfs=/tmp:size=256m --ulimit nofile=262144:262144 --sysctl net.ipv4.tcp_tw_reuse=1 --sysct
secrets_environment = env/MYSQL_ROOT_PASSWORD
tags =
type = docker
volume_mounts = /etc/localtime:/etc/localtime:ro {name}/data:/var/lib/mysql:rw {name}/mysql-files:/var/l

[container#jobs]
command = /docker-entrypoint-initdb.d/dbjobs_launcher
entrypoint = /bin/bash
environment = MYSQL_INITDB_SKIP_TZINFO=yes
image = {env.docker_image}
netns = container#01
rm = true
run_args = --ulimit nofile=262144:262144
secrets_environment = env/MYSQL_ROOT_PASSWORD
tags =
type = docker
volume_mounts = /etc/localtime:/etc/localtime:ro {name}/jobs:/var/lib/replication-manager-jobs:rw {name}

[env]
```

```
docker_image = mariadb:11.4
nodes = s18-fr-6
size = 50g

[ip#01]
netns = container#01
network = s18
type = cni

[volume#01]
directories = run/mysqld
group = 999
name = {name}
pool = dbssd
size = {env.size}
user = 999

[volume#02]
dirperm = 700
name = {name}-sec
perm = 600
secrets = env/MYSQL_ROOT_PASSWORD:/
size = 1m
type = shm
user = 99
.....
```

Examine the **service** log you should found some error **om arceau/svc/db3 logs**

```
19:28:22,968 ip#01          INFO    skip allocate: an ip is already defined
19:28:23,027 ip#01          INFO    checking 192.168.100.50 availability
19:28:23,035 ip#01          ERROR   192.168.100.50 is already up on another host**
```

Working with services - **om arceau/svc/db3 provision** - **om arceau/svc/db3 stop** - **om arceau/svc/db3 unprovision** - **om arceau/svc/db3 start**

3.6.1.4 Get logs of a docker image

om arceau/svc/db3 docker logs {db}

```
2017-10-21 19:40:42 ProxySQL_Admin.cpp:2870:flush_mysql_variables___database_to_runtime(): [ERROR] Impos
2017-10-21 19:40:42 ProxySQL_Admin.cpp:2870:flush_mysql_variables___database_to_runtime(): [ERROR] Impos
Standard ProxySQL Admin rev. 0.2.0902 -- ProxySQL_Admin.cpp -- Sat Oct 29 13:47:26 2016
Standard MySQL Threads Handler rev. 0.2.0902 -- MySQL_Thread.cpp -- Sat Oct 29 13:47:26 2016
Standard MySQL Authentication rev. 0.2.0902 -- MySQL_Authentication.cpp -- Sat Oct 29 13:47:26 2016
2017-10-21 19:40:42 [INFO] New mysql_replication_hostgroups table
Standard Query Processor rev. 0.2.0902 -- Query_Processor.cpp -- Sat Oct 29 13:47:26 2016
In memory Standard Query Cache (SQC) rev. 1.2.0905 -- Query_Cache.cpp -- Sat Oct 29 13:47:26 2016
Standard MySQL Monitor (StdMyMon) rev. 1.2.0723 -- MySQL_Monitor.cpp -- Sat Oct 29 13:47:26 2016
```

3.6.1.5 Entering a docker image

om arceau/svc/db3 docker exec -ti {db} /bin/sh

3.6.1.6 Secret and config services

om arceau/*/env mon

```
arceau/cfg/e...                    s18-fr-4 s18-fr-5 s18-fr-6
arceau/cfg/env n/a              | /        /        /
arceau/sec/env n/a              | /        /        /
```

om arceau/*/env keys

```
REPLICATION_MANAGER_CLUSTER_NAME
REPLICATION_MANAGER_URL
REPLICATION_MANAGER_USER
MYSQL_ROOT_PASSWORD
REPLICATION_MANAGER_PASSWORD
SHARDPROXY_ROOT_PASSWORD
```

Decode key value **om arceau/cfg/env decode –key REPLICATION_MANAGER_URL**

List private network

om net ls

```
default
backendipv6
lo
s18
```

List disk pool

om pool ls

```
dbssd
default
drbd
drbd4k
drbdbigd
shm
tank
```

3.7.1 Logs

replication-manager uses a multi-tier logging system built on Logrus. Every message is written simultaneously to one or more destinations: a rotating log file, journald (via stdout), the web interface, and optionally syslog or alert channels such as Slack.

3.7.2 Log Files Written

3.7.2.1 Main Log

Set by `log-file`. All cluster monitoring, failover, switchover, and API activity is written here.

```
log-file = "/var/log/replication-manager.log"
```

3.7.2.2 Derived Logs

When `log-file` is set, three specialised logs are created automatically beside it by appending a suffix to the base name:

File	Suffix	Content
Main log	*(base)*	General monitoring, HA operations, API
Security log	`-security`	Authentication, authorisation, credential access
Workload log	`-workload`	Performance spikes, slow query events
Maintenance log	`-maintenance`	Backups, SST, tasks, purge, orchestrator provisioning

Example: if `log-file = /var/log/replication-manager.log` then:

```
/var/log/replication-manager.log
/var/log/replication-manager-security.log
/var/log/replication-manager-workload.log
/var/log/replication-manager-maintenance.log
```

3.7.2.3 Per-Cluster SQL Logs

Two SQL logs are written per cluster under the monitoring data directory:

```
{monitoring-datadir}/{cluster-name}/sql-error.log   - SQL errors returned by the database
{monitoring-datadir}/{cluster-name}/sql-general.log - All statements executed by replication-manager
```

The default `monitoring-datadir` is `/var/lib/replication-manager`.

3.7.2.4 Panic Log

If replication-manager panics, a JSON-formatted stacktrace is written to:

```
{monitoring-datadir}/panic.log
```

3.7.2.5 Carbon / Graphite API Log

When the embedded Graphite backend is enabled:

```
{monitoring-datadir}/carbonapi.log
```

3.7.3 Log Levels

Log verbosity is controlled with integers. A lower number means fewer, more severe messages; a higher number means more verbose output.

Level	Value	Meaning
ERROR	1	Errors only
WARN	2	Errors and warnings
INFO	3	Normal operational output (default)
DEBUG	4	Full diagnostic output

The global level applies to all output unless overridden per module:

```
log-level = 3
```

3.7.4 Per-Module Log Levels

Every subsystem has its own level key. Set any module to 4 (DEBUG) when diagnosing a specific area without flooding the rest of the log.

```
# Core operations
log-level-topology       = 2    # Topology discovery and changes
log-level-heartbeat      = 1    # Heartbeat monitoring
log-level-writer-election = 1   # Primary election process

# Config and Git
log-level-config-load    = 2    # Config file merging and reload
log-level-git            = 2    # Git config backup operations

# Backups
log-level-backup-stream  = 4    # Streaming backup verbosity
log-level-restic         = 3    # Restic backup operations
log-level-sst            = 1    # State Transfer (SST / IST)

# Proxies
# (set in the cluster section - no global key, use log-level)

# Orchestrator and provisioning
log-level-orchestrator   = 2    # Provisioning orchestrator calls

# Tasks and maintenance
log-level-task           = 3    # Scheduled task execution

# Database log forwarding
log-level-database-errors    = 1   # MariaDB/MySQL error log lines
log-level-database-sql-errors = 1  # SQL errors from general_log
log-level-database-slowquery  = 2  # Slow query log lines
log-level-database-audit      = 3  # Audit plugin events
log-level-database-optimize   = 4  # Optimizer trace
```

```
# Alerting
log-level-mailer         = 3    # Email alert delivery
log-level-support        = 2    # Support bundle creation

# Other
log-level-external-script = 3  # External script execution
log-level-stats          = 1    # Internal statistics collection
log-level-vault          = 1    # HashiCorp Vault operations
log-level-graphite       = 2    # Graphite/Carbon metrics
log-level-binlog-purge   = 1    # Binary log purge operations
log-level-sql            = 2    # SQL statements executed by repman

# Plugin log tailer
log-plugin               = false  # Enable log tailer plugins
log-level-plugin         = 2      # Plugin verbosity
```

3.7.5 Log Message Format

Each log line carries a timestamp, cluster name, module tag, level, and message:

```
2006/01/02 15:04:05 [mycluster] [heartbeat] INFO  - Primary heartbeat received
2006/01/02 15:04:06 [mycluster] [topology]  WARN  - Replica 10.0.1.11:3306 lag > threshold
2006/01/02 15:04:07 [mycluster] [general]   STATE - OPENED ERR00076: Lost primary
```

Level strings:

String	Meaning
`ERROR`	Hard error
`WARN`	Warning
`INFO`	Informational
`DEBUG`	Diagnostic detail
`STATE`	Cluster state change (`OPENED` / `RESOLV` prefix)
`ALERT`	Critical operational alert
`ALERTOK`	Alert cleared
`START`	Service start / bootstrap message
`TEST`	Regression test output
`BENCH`	Benchmark output

Module tags appearing in log lines:

Tag	Area
`general`	General monitoring
`topology`	Topology discovery
`heartbeat`	Heartbeat checks
`election`	Writer election
`sst`	State Transfer
`backup`	Backup streaming
`restic`	Restic operations
`task`	Task / job execution
`maintenance`	Maintenance operations
`proxy`	Generic proxy operations
`proxysql`	ProxySQL

Tag	Area
`haproxy`	HAProxy
`maxscale`	MaxScale
`orchestrator`	Provisioning orchestrator
`conf`	Config load and merge
`git`	Git backup
`vault`	HashiCorp Vault
`graphite`	Graphite metrics
`purge`	Binlog purge
`mailer`	Email alerting
`support`	Support bundles
`external-script`	External scripts
`stats`	Internal statistics
`sql`	SQL statements
`errorlog`	Database error log
`slowquery`	Database slow query log
`auditlog`	Database audit log
`sqlerrorlog`	Database SQL errors
`optimize`	Database optimizer trace
`plugin`	Log tailer plugins
`app`	Application-level

3.7.6 Log Rotation

Rotation is handled by lumberjack. All file-based logs (main, security, workload, maintenance, SQL) rotate automatically.

```
log-rotate-max-size   = 5    # Rotate when file reaches this size in MB
log-rotate-max-backup = 7    # Number of rotated files to keep
log-rotate-max-age    = 7    # Days before rotated files are deleted
```

Rotated files are gzip-compressed automatically. A daily forced rotation also runs via the internal scheduler.

3.7.7 Syslog

To forward all log entries to the local syslog daemon (rsyslog, syslog-ng, etc.):

```
log-syslog = true
```

replication-manager connects to `localhost:514` over UDP at `LOG_INFO` severity. The syslog hook is additive — file and syslog output both receive messages.

3.7.8 Viewing Logs on a Modern Linux System

3.7.8.1 journalctl — follow in real time

```
# Follow the main service journal
journalctl -u replication-manager -f

# Follow and show the last 100 lines
journalctl -u replication-manager -n 100 -f
```

3.7.8.2 journalctl — query by time

```
# Last hour
journalctl -u replication-manager --since "1 hour ago"

# Specific window
journalctl -u replication-manager --since "2024-06-01 00:00" --until "2024-06-01 06:00"

# Since last boot
journalctl -u replication-manager -b
```

3.7.8.3 journalctl — filter by priority

journald maps Logrus levels to syslog priorities automatically when replication-manager logs to stdout:

```
# Errors only
journalctl -u replication-manager -p err

# Warnings and above
journalctl -u replication-manager -p warning

# Full debug output
journalctl -u replication-manager -p debug
```

3.7.8.4 journalctl — output formats

```
# JSON output - pipe to jq
journalctl -u replication-manager -o json | jq '.MESSAGE'

# Short with microseconds
journalctl -u replication-manager -o short-precise

# Export to file
journalctl -u replication-manager --since today > /tmp/repman-today.log
```

3.7.8.5 journalctl — search inside messages

```
# Find all failover events
journalctl -u replication-manager -g "failover"

# Find specific cluster
journalctl -u replication-manager -g "\[mycluster\]"

# Find STATE changes
journalctl -u replication-manager -g "STATE"

# Find a specific error code
journalctl -u replication-manager -g "ERR00076"
```

3.7.8.6 Tailing the log file directly

If `log-file` is configured:

```
# Follow main log
tail -f /var/log/replication-manager.log

# Follow only security events
```

```
tail -f /var/log/replication-manager-security.log

# Follow maintenance/backup activity
tail -f /var/log/replication-manager-maintenance.log

# Filter a module in real time
tail -f /var/log/replication-manager.log | grep '\[topology\]'

# Filter an error code
tail -f /var/log/replication-manager.log | grep 'ERR0'

# Filter STATE changes only
tail -f /var/log/replication-manager.log | grep 'STATE'
```

3.7.8.7 Searching historical logs

```
# All ERR codes in the last 7 days of rotated logs
zgrep 'ERR0' /var/log/replication-manager.log* | sort

# All STATE OPENED events
zgrep 'OPENED' /var/log/replication-manager.log*

# Failover events across all rotated files
zgrep -h 'failover' /var/log/replication-manager.log* | sort -k1,2
```

3.7.9 Accessing Logs via the REST API

In-memory log buffers (last ~200 entries per cluster) are available through the API without reading the log files:

Endpoint	Content
`GET /api/clusters/{name}/log`	General cluster log
`GET /api/clusters/{name}/log-task`	Task and backup log
`GET /api/clusters/{name}/log-security`	Security events
`GET /api/clusters/{name}/log-workload`	Workload spikes

These endpoints require authentication. Use the admin or dba credential:

```
curl -sk -u admin:repman \
  https://localhost:10005/api/clusters/mycluster/log | jq .
```

3.7.10 Temporarily Increasing Verbosity

Bump a specific module to DEBUG without restarting — set the level via the API or edit the config and reload:

```
# Investigate a topology issue
log-level-topology = 4

# Trace a backup problem
log-level-backup-stream = 4
```

```
log-level-restic        = 4

# Debug SST / IST
log-level-sst = 4
```

After diagnosing, drop the levels back to their defaults to reduce log volume.

3.7.11 Diagnostic Quick Reference

Goal	Command
Follow all logs	`journalctl -u replication-manager -f`
Find switchovers	`journalctl -u replication-manager -g "switchover"`
Find failovers	`journalctl -u replication-manager -g "failover"`
Find all ERR codes	`journalctl -u replication-manager -g "ERR0"`
Find WARN codes	`journalctl -u replication-manager -g "WARN0"`
Find state changes	`journalctl -u replication-manager -g "STATE"`
Topology module only	`journalctl -u replication-manager -g "\[topology\]"`
Security events	`tail -f /var/log/replication-manager-security.log`
Backup / maintenance	`tail -f /var/log/replication-manager-maintenance.log`
Slow SQL	`tail -f /var/log/replication-manager-workload.log`
SQL executed by repman	`tail -f {datadir}/{cluster}/sql-general.log`
Panic stacktrace	`cat {datadir}/panic.log | jq .`

4.1.1 High Availability

replication-manager provides continuous, automated high availability for MariaDB and MySQL clusters. It monitors replication topology on every tick, enforces a target topology, and takes corrective action — switchover or failover — only when it is safe to do so. Every decision passes through the same monitoring loop that detected the problem, so no action is ever taken on incomplete or ambiguous information.

4.1.2 Topology Discovery

On every monitoring loop replication-manager connects to all configured servers and reconstructs the full replication graph: which server is the current primary, which are replicas, what GTID positions they hold, whether replication threads are running, and how far each replica lags behind.

The **default target topology is primary/replica** (also called master/slave). Other topologies — multi-primary, multi-tier replicas, binlog server, Galera, and more — can be declared as the target, and replication-manager will discover and validate the current state against that target on every loop.

replication-manager will not perform any automated action until the observed topology matches the declared target topology. This guarantee means that a partially-converged cluster, a cluster mid-rejoin, or a cluster where replica lag has not yet resolved will be left alone until the situation is fully understood. The same loop that detects a problem also validates that the cluster is in a known, stable state before acting.

4.1.3 False Positive Protection

Alongside topology discovery, every monitoring loop runs a series of false-positive checks before concluding that a primary is truly unreachable:

- Can the replication-manager host reach the primary directly?
- Can the replicas still reach the primary?
- Does an external arbitrator agree that the primary is down?
- Has the primary been unreachable for longer than the configured grace period?

Only when all applicable checks agree that the primary is genuinely lost will replication-manager consider an automated failover. This prevents split-brain and unnecessary failovers caused by transient network events, monitoring host isolation, or slow primaries under load.

4.1.4 Default Mode — Alert, Don't Act

By default replication-manager operates in **manual mode**: when a failure is detected and all false-positive checks confirm the primary is genuinely lost, an alert is sent to all configured channels and the cluster is held in a degraded-but-safe state. **No automatic failover is triggered.** This gives the supervision team time to assess the situation, check for external causes (network partition, storage failure, planned maintenance), and decide whether to promote a replica or wait for the primary to recover.

```
failover-mode = "manual"   # default - alert and wait for operator action
```

To enable fully automated failover without operator intervention:

```
failover-mode = "automatic"   # promote the best replica automatically
```

In automatic mode replication-manager still applies all false-positive and topology checks before acting — the difference is that the promotion step executes without waiting for a human decision.

Recommendation: run `manual` mode in production until the team is confident in the false-positive tuning, replication health scores, and alerting pipeline. Switch to `automatic` once those are validated.

4.1.5 Switchover — Planned Handoff

A **switchover** is a controlled primary promotion performed while the current primary is still reachable and healthy. It is the correct operation for planned maintenance, host migrations, software upgrades, and load rebalancing.

During a switchover replication-manager:

1. Makes the primary read-only and flushes all pending transactions
2. Waits for the best replica to catch up to the primary's GTID position
3. Promotes that replica to primary
4. Redirects all other replicas to replicate from the new primary
5. Optionally updates proxy routing (HAProxy, ProxySQL, MaxScale) to point to the new primary

Because the old primary is still alive, there is no data loss and the operation is fully reversible — another switchover brings the old primary back as the new leader.

4.1.6 Failover — Automatic Recovery

A **failover** is an emergency promotion triggered when the primary can no longer be reached and all false-positive checks have been exhausted. It is the operation that keeps the cluster writable when a primary host has crashed, lost power, or become permanently partitioned.

During a failover replication-manager:

1. Confirms primary loss through all configured arbitration checks
2. Elects the most advanced replica (highest GTID, lowest lag, best replication health score)
3. Promotes that replica to primary
4. Redirects surviving replicas to the new primary
5. Updates proxies

Because the old primary was not available to flush transactions, a small amount of data loss (transactions committed but not yet replicated) is possible depending on the replication mode configured (`semi-sync`, `binlog-commit-wait`, etc.). The number of failovers can be capped to prevent cascading failures.

4.1.7 Rejoin — Reintroducing an Old Primary

After a failover the old primary eventually comes back online. **Rejoin** is the process of safely reintroducing that server as a replica of the new primary.

Rejoin must handle the case where the old primary may have accepted writes that never replicated — those transactions must be rolled back or skipped before the server can safely join. replication-manager automates this using:

- **GTID-based flashback** — roll back diverging transactions on the old primary
- **Resync from backup** — restore the old primary from a recent backup and replay binlogs
- **Manual gate** — hold the server in a maintenance state until an operator approves the rejoin

The rejoin method is configurable per cluster based on acceptable data risk and recovery time objectives.

4.1.8 Reseeding — Adding a New Server

Reseeding is the process of provisioning a brand-new server and bringing it into the cluster as a replica. Unlike rejoin (which handles a server that was previously a member), reseeding starts from scratch:

1. A physical or logical backup is taken from the primary (or a designated donor replica)
2. The backup is restored on the new server
3. Replication is configured from the backup's GTID position
4. The server joins the cluster and begins catching up

replication-manager automates reseeding using the configured backup tools (mydumper, mariabackup, xtrabackup, or restic snapshots) and can target any server in the cluster as the donor to avoid load on the primary.

4.1.9 Summary

Operation	Trigger	Primary state	Data loss risk
Switchover	Planned / manual	Alive and reachable	None
Failover	Automatic / manual	Unreachable	Possible (depends on sync mode)
Rejoin	After failover	Returning online	Handled by rollback or restore
Reseeding	Adding new node	Running normally	None (backup-based)

4.2.0.1 Switchover Workflow

replication-manager is starting up with checks to disable potentially dangerous switchover cases. It looks for some excessively long WRITE queries, excessively long transactions and cancel the switchover in such case. Switchover also follows the state machine blocking states so that malfunctioning replication or wrong topology also will not enable switchover.

During switchover, **replication-manager** prevents additional writes via setting READ_ONLY flag and FTWRL on the old leader, if routers are still sending WRITE transactions, they can still pile up until timeout. To prevent this, **replication-manager** kills those blocked transactions.

Some additional caution to make sure that piled writes do not happen is that **replication-manager** decreases max_connections to the server to 1 and consumes the last possible connection by not killing himself. This works fine in practice, but to avoid a scenario where a node is left in such state where the database cannot be connected anymore (killing replication-manager in this critical section), we advise using extra port provided with MariaDB or MySQL pool of threads feature:

MariaDB Thread Pool

```
thread_handling = pool-of-threads
extra_port = 3307
extra_max_connections = 10
```

To better protect consistency it is strongly advised to disable **SUPER** privilege on users that perform writes, such as the MaxScale user used with Read-Write split module is instructed to monitor the replication lag via writing in the leader, privileges should be lower as described in Maxscale configuration settings.

4.2.0.2 Switchover Events Graph

[Figure: switchover]

4.3.0.1 Failover

replication-manager is checking all master failures up to N times, and when all false positive checks pass, the default behavior is to send alerts and put itself in waiting mode until a user forces the failover or that the master self-heals.

This default is know as the On-call mode and configured `failover-mode = "manual"`

Failover can be continue via user interaction with the console, the web server on default URL http://replication-manager-host:10001/, the API or the command line client.

Conditions for a possible failover are constantly monitored.

- A valid slave need to be available and up and running.
- The cluster configuration needs to be valid.

! Note: the definition of a valid slave is influenced by the prefered or ignored servers but since replication-manager 2.0 we first look at the most up to date position in all slaves. After founding this it can eventually say that failover is impossible because none of the slaves with highest position match the constraints , if multiple slaves have that same highest position, than the prefered master can be considered as a valid candidate.

Some users define failover replication constraints that can be added in the configuration file.

- Exceeding a given replication delay `failover-max-slave-delay`
- Time interval between failovers `failover-time-limit`

- Number of Failover can be limited `failover-limit`
- Synchronous state of the replication is lost `failover-at-sync`

We strongly advise to set the following best practice configuration in automatic failover:

```
failover-limit = 3
failover-time-limit = 10
failover-at-sync = false
failover-max-slave-delay = 30
```

For minimizing data lost in automatic failover:

```
failover-at-sync = true
failover-max-slave-delay = 0
```

A user can still force failover by ignoring replication checks via:

```
check-replication-state = false
```

This can be dynamically changed via the HTTP interface "Replication Checks Change" button.

Per default Semi-Sync replication status is not checked during failover, but this check can be enforced with semi-sync replication to enable to preserve OLD LEADER recovery at all costs, and not perform failover if none of the slaves are in SYNC status.

A user can change the sync check based on what is reported by the *SLA in-sync* metric, and decide that most of the time the replication is in sync, and when it's not, that the failover should be manual. Via http console, use the *Failover Sync* button.

All cluster down condition leads to a situation where it is possible to first restart a slave previously stopped before the entire cluster was shutdown, failover in such situation can promote a delayed slave by a big amount of time and lead to as much time data lost. By default **replication-manager** prevents such failover for the first slave unless you change `failover-restart-unsafe` to `true`. When using the default, it is advised to start the old master first, if not replication-manager will wait for the old master to show up again until it can failover again.

The previous scenario is not that frequent and one can favor availability in case the master never shows up again. The DC crash would have bring down all the nodes around the same time. So data lost can be

mitigated if you automate starting a slave node and failover on it via `failover-restart-unsafe=true` if the master cannot, or takes too long to recover from the crash.

[Figure: failover]

Positional is the old style of tracking replication positions. It is deprecated in all our supported databases major releases.

4.4.1.0.1 GTID

Support Status	Test Case
Production	170

replication-manager <=1.1 does not support MariaDB versions lower than 10.0, or MySQL versions lower than 5.6 and need MySQL GTID.

replication-manager 2.0 support positional replication

For MariaDB Server 10.0 and upper **replication-manager** always uses GTID positions for failover or switchover regardless of using GTID in the replication stream definition. In MariaDB, GTID are always present in the binary logs of MariaDB and can be used to point a slave to his master.

> For MariaDB Server 10.0 and upper **replication-manager** can force positional replication back after a failover or a switchover but this will disable crash safe slave and usage of concurrency in replication streams.

`force-slave-no-gtid-mode` (1.1)

Item	Value
Description	Automatically disable gtid mode on slave.
Type	Boolean
Default Value	false

4.4.1.0.2 Positional Replication

Support Status	Test Case
Experimental	1

Positional replication will be used for pointing replication on MySQL or Percona Servers when no GTID replication is set.

In this scenario **replication-manager** is working in best effort mode as it was designed to work agent less. **replication-manager** does not parse binary logs to try to match correct position based on the content of replication EVENTS. GTID are here for fixing this requirement and we state that using them is the correct way to go.

replication-manager 2.0 can correct the database lack of unique EVENT identifier via PSEUDO GTID it find positions by reversing binary log via SQL command.

`autorejoin-slave-positional-hearbeat` (2.0)

Item	Value
Description	Inject PseudoGTID and use them to rejoin extra slaves.
Type	Boolean
Default Value	false

replication-manager design choices have implications:

On MariaDB 10.0 and later, positional replication will be lost after first switchover in favor of using GTID POS.

For other non-GTID databases extra slaves are staying under the old master. They are later moved on elected master by fixing invalid relay architectures. That means that the old master needs to rejoin the cluster first and we can't give insurance that this will succeed, if does not, you may have to reseed all slaves.

One can be inventive like having tree topology with slaves not monitored by the same **replication-manager** cluster so that the new elected master already gets available extra slaves.

plugin:youtube

Support Status	Test Case
Production	175

replication-manager supports 2-node and x nodes setup, it is advised to use at least 3 nodes cluster to keep the cluster tolerant to failures when losing or stopping a slave.

replication-manager can ignore slaves or have a preferred node to failover or switchover.

By default **replication-manager** assumes a flat rejoin topology but can also promote multi-tier topology with some additional settings, the scenario being that you stop a slave and his master dies or change, when the old master rejoins the topology, it can keep his slave under himself instead of switching the slave to the new master.

`replication-multi-tier-slave` (2.0), `multi-tier-slave` (1.0)

Item	Value
Description	Enable relay slaves topology
Type	boolean
Default Value	false

Support Status	Test Case
Production	1

replication-manager supports 2-node multi-master topology detection. It is required to declare it explicitely in the configuration.

`replication-multi-master` (2.0), `multimaster` (0.7)

Item	Value
Description	Enable Master-Master topology
Type	boolean
Default Value	false

You just need to set one preferred master.

We advise to enable restart of database in read-only mode to cover the case where a failed node tries to rejoin but can't be contacted anymore from **replication-manager**, in such case no write traffic will be enabled on the rejoining node. In a Multi DC split brain with a proxy on each side, we can simply ensure the split brain will not make the database diverge on each side.

MariaDB configuration file:

```
read_only = 1
```

This flag ensures that in case of split brain + leader crash, when old leader is reintroduced it will not show up as a possible leader for WRITES.

MaxScale configuration file:

Maxscale needs to be instructed to monitor multi-master, the following settings tracks the read-only flag and routes queries to the writable node.

```
[Multi-Master Monitor]
type=monitor
module=mmmon
servers=server1,server2,server3
user=myuser
passwd=mypwd
detect_stale_master=true
```

Support Status	Test Case
Experimental	0

replication-manager supports replication tree or relay slaves architecture, in case of master death one of the slaves under the relay is promoted as a master.

replication-manager does not manage when the relay server crash to replace it with a slave.

But in **replication-manager 2.2** one can archive same goal using multi source, multi domains child clusters. All servers of the same tenant will be monitor and inter cluster replication will discovered, so creating replication from the leader of a tier cluster on the leader of a parent cluster, multi tiers topology can then be switchover and failover

```
Cluster-1-Leader
  -> Cluster-1-Replicate-1/Cluster2-Leader
     -> Cluster-2-Replicate-1
     -> Cluster-2-Replicate-2
  -> Cluster-1-Replicate-2/Cluster-3-Leader
     -> Cluster-3-Replicate-1
     -> Cluster-3-Replicate-2
```

`replication-multi-tier-slave` (2.0), `multi-tier-slave` (1.0)

Item	Value
Description	Enable relay slaves topology
Type	boolean
Default Value	false

Support Status	Test Case
Alpha	0

replication-manager can automate ring topology maintenance and failover.

A virtual master is elected on the ring to enable the proxies to reach a single node.

replication-manager failover close the ring around the available nodes and rejoin the node when maintenance is finished or if the node was failed and reintroduce the cluster.

replication-manager will put out a node of the ring if this node is set for maintenance via the the replication-manager clients. The node will still replicate as a slave of it's parent

A switchover is just changing the virtual master to the next node in the ring in descending order

`replication-multi-master-ring (2.0)`

Item	Value
Description	Enable Multi Master Ring topology
Type	boolean
Default Value	false

[Figure: Ring]

Support Status	Test Case
Experimental	0

`replication-multi-master-wsrep (2.0)`

Item	Value
Description	Enable Multi Master Galera
Type	boolean
Default Value	false

Galera cluster is not always adapted for a Write to all node architecture, for those reasons:

- It do not support serialized isolation level, READ LOCKS acquire no cluster locks and can only be validated for REPEATABLE READ or SERIALIZED on the local node the transaction is executed.
- It increase deadlock probability, while transactions are replicated in concurrency it's style take time to round trip the network, certification only apply on checkpointing, so the more nodes the more latency the more deadlock the workload may be exposed.

For those reasons your application as to be modify to write critical sections on a leader Galera node.

replication-manager can help you to get the best of Galera combine with layer7 proxy like MaxScale or ProxySQL.

replication-manager will elect a virtual master and failover or switchover it based on each Galera node status, if one node get excluded for any reason of that cluster.

replication-manager could help to get 2 node Galera Cluster with one **replication-manager** on each node and a **replication-manager-arb** to manage active-passive role on each. The active **replication-manager** can be use to start garbd and enable the cluster to survive.

Support Status	Test Case
Stable	0

`replication-multi-master-grouprep (2.3)`

Item	Value
Description	Enable MySQL Multi Master Group Replication Monitoring
Type	boolean
Default Value	false

Group replication is not always adapted for a Write to all node architecture, for following reasons:

- It increase deadlock probability, while transactions are replicated in concurrency it's style take time to round trip the network, certification only apply on checkpointing, so the more nodes the more latency the more deadlock the workload may be exposed.

For that reason your applications may have to always point to a single leader Group Replication node.

replication-manager can help you to get the best of Group Replication combine with layer7 proxy ProxySQL or layer 4 like HaProxy to point to a single node

replication-manager can monitor PRIMARY group status and can elect a virtual master on switchover, based on each Group Replication node status, if one node get excluded for any reason of that cluster it it removed from leader or reader group and move to status Unconnected being rejected from routers and proxies.

replication-manager Can bootstrap the group replication on already well configured node and can produce valid configuration using following config tags

prov-db-tags = "grouprep,nosplitpath"

replication-manager Does not yet check valid configuration for Group Replication but this can be added on demand, for now following the MySQL documentation is needed to setup correct deployment

Support Status	Test Case
Alpha	35

replication-manager supports MariaDB multi domain GTID, the product can switchover and failover on topology where each database node have a different domain id.

Writing on slaves is strongly not advice, it can be done but with caution to disable binary login on such WRITES.

Support Status	Test Case
Experiemental	0

replication-manager supports Maxscale binlog server architecture, in case of master death one of the slaves under the binlog server is promoted as a master.

replication-manager does not manage yet the binlog server crash to replace it with an other one.

See Maxscale

Support Status	Test Case
Experimental	0

replication-manager supports MariaDB multi source replication, one can setup a source to be monitored and create multiple clusters with the same servers to monitor multiple channels.

The code have been tested around MariaDB server but with MySQL and Percona in mind and can be adapted for both. We are waiting for more feadbacks and test cases to declare the feature mature.

`replication-source-name` (2.0), `master-connection` (1.1)

Item	Value
Description	Replication channel name to use for multisource
Type	string
Default Value	“”

Support Status	Test Case
Experiemental	0

[Figure: Ring]

replication-manager can create each federation table on multiple master shard group inside spider proxies

replication-manager on failover and switchover of one master shard reattach the table federation to the new master in the shard group.

replication-manager constantly rediscover tables and schemas on each master shard group to maintain table definitions.

replication-manager the advantage is that such topology maintain ACID capabilities between tables on different servers and can enable very efficient cross shard joins.

plugin:youtube

Support Status	Test Case
Production	1
Version	3.0.13

A staging cluster is a child cluster of a production cluster containing 3 nodes, one replica is detached from replication and proxies track the the status of the standalone node to enable dev team to test on production data

4.4.12.1 Staging config important setup :

```
## 4.4.12.2 To be change when proxy tracking state is enable
haproxy = false

## 4.4.12.3 Specify an other domain for staging
prov-db-domain = "3"

## 4.4.12.4  Specify an other source of replication for the staging cluster
replication-source-name = "staging"

## 4.4.12.5 Point the production cluster to enable mutti-tiers cluster master prod to master staging ext
replication-multisource-head-cluster="prod-cluster"

## 4.4.12.6 Proxy Upgrade script
db-servers-state-change-script = "/data/repman/script/database_state_change.sh"

## 4.4.12.7 To enable ssh to database and proxy hosts, use ssh-keygen ssh-copy-id for first deployment
onpremise-ssh = true
onpremise-ssh-credential = "root:"
onpremise-ssh-private-key = "/root/.ssh/id_rsa_preprod"
scheduler-jobs-ssh = true

## 4.4.12.8  To automate logical backup
monitoring-scheduler = true
```

4.4.12.9 Lesson learns:

- Production and staging monitoring and replication users should have same name same password so that 2 similar users does not get different password in staging after an initial restore from production
- Mydumper dist version test your restore

4.4.12.10 Todo :

- New cluster variable –topology-staging bool
- New cluster variable –topology-staging-refresh-script
- New cluster variable –topology-staging-post-detach-script (obfuscation data do whatever have to be done in db)
- Cluster menu api call to trigger: Staging refresh

Api call should trigger similar script that will comes embedded in share

```
#!/bin/bash
```

```
# Variables
REPLICATION_MANAGER_USER="admin"
REPLICATION_MANAGER_PASSWORD="xxx"
REPLICATION_MANAGER_URL="https://repman-01:10005"
REPLICATION_MANAGER_CLUSTER_NAME="staging"
REPLICATION_MANAGER_HOST_NAME=$1
REPLICATION_MANAGER_HOST_PORT="3306"
NB_SLAVES=0

##### This bloc is for getting the replication-manager token
GET="wget -q --no-check-certificate -O- --header Content-Type:application/json"
AUTH_DATA="{\"username\": \"$REPLICATION_MANAGER_USER\", \"password\": \"$REPLICATION_MANAGER_PASSWORD\"
TOKEN=$($GET --post-data "$AUTH_DATA" --header Accept:text/html $REPLICATION_MANAGER_URL/api/login)

function get {
    $GET --header Accept:application/json --header "Authorization: Bearer $TOKEN" "$@"
}

# Counting the slaves, depending, we will play one of the two following scenarios
NB_SLAVES=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/slaves
echo $NB_SLAVES

# Scenario 1 : 2 slaves, then we will stop the replication on one that will be the "staging"
if [ $NB_SLAVES -eq 2 ]; then
  echo "picking first slave \n"
  ID=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/slaves | jq
  PORT=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/slaves | j
  echo "$ID:$PORT"
  echo "Reseting first slave \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID/actions/reset-
  sleep 2
  echo "Stopping first server slave replication \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID/actions/stop-s
  sleep 20
  echo "Stopping first mariadb server \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID/actions/stop
  echo "Starting first mariadb server \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID/actions/start
  loop=true
  while $loop; do
    sleep 1
    IO_THREADS=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/sl
    SQL_THREADS=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/s
    echo $IO_THREADS
    echo $SQL_THREADS
    if [ "$IO_THREADS" == "\"No\"" ] || [ "$SQL_THREADS" == "\"No\"" ]; then
      loop=false
    fi
  done

  echo "Getting and saving slaves initial statuses \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/slaves | jq '.[1]
```

```
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID/actions/reset-
  sleep 2
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID/actions/stop
  echo "Waiting for database server $ID to status failed \n"
  loop=true
  while $loop; do
    ID_CHECK=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/serv
    if [ "$ID_CHECK" == "$ID" ]; then
      loop=false
    fi
  done
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID/actions/start
fi

if [ $NB_SLAVES -eq 1 ]; then
  echo "picking last slave and founding standalone \n"
  ID=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/servers | jq
  echo "found standalone server $ID \n"
  echo "reseting master position on standalone \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID/actions/reset-
  echo "setup replication manager for reseeding \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/settings/actions/switch/au
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/settings/actions/switch/au
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/settings/actions/switch/au
  sleep 2
  echo "Stopping database server $ID \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID/actions/stop
  echo "Waiting for database server $ID to status failed \n"
  loop=true
  while $loop; do
    ID_CHECK=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/serv
    if [ "$ID_CHECK" == "$ID" ]; then
      loop=false
    fi
  done
  echo "Start database server $ID \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID/actions/start
  sleep 20
  echo "Reseting replciation manager settings for reseed = false \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/settings/actions/switch/au
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/settings/actions/switch/au
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/settings/actions/switch/au

###### Now set last slave as standalone



# Get the last available slave
ID_SLAVE=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/servers
  echo "last slave found for staging $ID_SLAVE \n"
  echo "Stopping replication on last slave \n"
  get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID_SLAVE/actions/
  loop=true
  while $loop; do
```

```
    echo "Waiting replication to stop \n"
    sleep 5
    IO_THREADS=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/sl
    SQL_THREADS=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/s
#    IO_THREADS=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/s
#    SQL_THREADS=$(get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/
    echo $IO_THREADS
    echo $SQL_THREADS
    if [ "$IO_THREADS" == "\"No\"" ] || [ "$SQL_THREADS" == "\"No\"" ]; then
      loop=false
    fi
    done
    echo "Saving replication info in replication.save \n"
    get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/slaves | jq --a
#    get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/topology/slaves | jq '.
    echo "Reset all replication information \n"
    get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID_SLAVE/action
    sleep 2
    get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID_SLAVE/action
    sleep 2
    get $REPLICATION_MANAGER_URL/api/clusters/$REPLICATION_MANAGER_CLUSTER_NAME/servers/$ID_SLAVE/action
fi
```

- Adapt the script to not stop start the database nodes
- Adapt the script to use env variables deploy in the bash instead of hardcoded variables

Today we can customize status change script to trigger some dedicated proxy config reload bu need native proxy integration

4.4.12.11 To be removed after integration

database_state_change.sh

```
#!/bin/bash
# This script is given as sample and will be overwrite on upgrade
# db-servers-state-change-script
echo "Database state change script args"
echo "Script:$0, Cluster:$1, Host:$2, Port:$3, State:$4, OldState:$5"
if [ "$4" = "StandAlone" ]; then
        scp /data/repman/script/ha.cfg.$2 root@staging-proxy:/etc/haproxy/haproxy.cfg
        ssh root@staging-proxy "systemctl reload haproxy"
fi
```

4.4.12.12 Todo

- New cluster variables haproxy more variables haproxy-staging-port, haproxy-staging-bind, haproxy-staging-backend
- New cluster variable proxysql-write-track-state="master|standalone" , proxysql-read-track-state="slave|standalone"

Active-Passive topology is a special mode where **replication-manager** monitors a single active database server without managing replication or performing automatic failover. This topology is useful for scenarios requiring external orchestration or when automatic failover is not desired.

4.4.13.1 Configuration

`replication-active-passive` (3.1)

Item	Value
Description	Enable Active-Passive topology
Type	boolean
Default Value	false

4.4.13.2 Topology Characteristics

4.4.13.2.1 Behavior

In Active-Passive mode:

- **Single active server**: Only one server is designated as "master"
- **No replication management**: **replication-manager** does not configure or manage replication
- **No automatic failover**: Failover and switchover operations are disabled
- **No rejoin operations**: Server rejoin is skipped
- **No privilege checks**: REPLICATION and SUPER privilege checks are bypassed
- **Simplified monitoring**: Focuses on server availability and health metrics

4.4.13.2.2 Automatic Detection

Active-Passive topology is automatically enabled when: - Configuration explicitly sets `replication-active-passive = true` - Only one server is defined in the `db-servers-hosts` list

4.4.13.3 Use Cases

1. External Orchestration

Use Active-Passive mode when failover is managed by external systems: - Kubernetes operators - Cloud provider managed databases (RDS, Cloud SQL, Azure Database) - External clustering solutions (Pacemaker, Corosync) - Manual failover procedures

2. Development/Testing

Monitor a standalone database without replication complexity: - Local development environments - Integration testing - Database migration testing

3. Single-Server Production

Monitor production databases that don't require high availability: - Low-traffic applications - Non-critical workloads - Budget-constrained deployments

4. Monitoring-Only Deployments

Collect metrics and health information without infrastructure changes: - Performance monitoring - Backup scheduling - Schema change detection - Query analysis

4.4.13.4 Features Available in Active-Passive Mode

Active-Passive mode supports:

- **Monitoring**: Server health, status variables, processlist

- **Metrics collection**: Graphite, Prometheus endpoints
- **Backup management**: Logical and physical backups via restic
- **Schema monitoring**: Track schema changes and differences
- **Query monitoring**: Long queries, Performance Schema analysis
- **API access**: Full REST API for server information
- **Web UI**: Dashboard with server metrics and status

4.4.13.5 Features Disabled in Active-Passive Mode

Active-Passive mode disables:

- **Failover**: Manual and automatic failover operations blocked
- **Switchover**: Cannot perform switchover operations
- **Rejoin**: Automatic server rejoin disabled
- **Replication setup**: No CHANGE MASTER commands issued
- **Read-only enforcement**: Does not set `read_only` flag
- **Privilege checks**: Skips replication user privilege validation
- **Maintenance mode**: Maintenance flag not applied to master

4.4.13.6 Configuration Example

4.4.13.6.1 Minimal Configuration

```
[default]
db-servers-hosts = "192.168.1.100:3306"
db-servers-credential = "root:password"
replication-active-passive = true
monitoring-datadir = "/var/lib/replication-manager"
```

4.4.13.6.2 Full Monitoring Configuration

```
[myapp]
db-servers-hosts = "db.example.com:3306"
db-servers-credential = "monitor:password"
replication-credential = "replication:password"
replication-active-passive = true

# Monitoring
monitoring-datadir = "/var/lib/replication-manager/myapp"
monitoring-write-heartbeat = false
graphite-metrics = true
graphite-embedded = true

# Backups
backup-restic = true
backup-restic-repository = "s3:https://s3.amazonaws.com/my-backups"
backup-keep-daily = 7
backup-keep-weekly = 4

# Schema monitoring
monitoring-schema-change = true
monitoring-schema-change-script = "/usr/local/bin/schema-alert.sh"

# Query monitoring
monitoring-queries = true
monitoring-long-query-time = 5000
```

4.4.13.6.3 Multi-Server Active-Passive (No Replication)

```
[cluster1]
db-servers-hosts = "server1:3306,server2:3306,server3:3306"
db-servers-credential = "root:password"
replication-active-passive = true
```

When multiple servers are configured with Active-Passive mode: - First reachable server becomes "master" - Other servers are marked as "Unconnected" - No replication is expected or configured - Useful for monitoring multiple independent databases as a single logical cluster

4.4.13.7 Transitioning Between Modes

4.4.13.7.1 From Master-Slave to Active-Passive

To disable automatic failover on an existing replicated cluster:

```
replication-active-passive = true
```

replication-manager will: - Stop performing failover operations - Continue monitoring existing replication status - Maintain current topology unchanged - Allow manual intervention for topology changes

4.4.13.7.2 From Active-Passive to Master-Slave

To enable automatic failover:

1. Ensure replication is configured on database servers
2. Update configuration:

```
replication-active-passive = false
```

3. Restart **replication-manager** or reload configuration
4. **replication-manager** will detect topology and enable failover

4.4.13.8 API Behavior

In Active-Passive mode, API endpoints behave differently:

Blocked endpoints: - `POST /api/clusters/{cluster}/actions/switchover` - Returns error - `POST /api/clusters/{cluster}/actions/failover` - Returns error - `POST /api/clusters/{cluster}/servers/{server}/acti` - Returns error

Available endpoints: - All `GET` endpoints for monitoring - Backup management endpoints - Server maintenance endpoints (start, stop, provision) - Configuration management endpoints

4.4.13.9 Comparison with Other Topologies

Feature	Active-Passive	Master-Slave	Multi-Master
Replication	No	Yes	Yes
Auto Failover	No	Yes	Yes
Read-only enforcement	No	Yes	Conditional
Rejoin	No	Yes	Yes
Privilege checks	No	Yes	Yes
Server count	1+	2+	2+

4.4.13.10 Troubleshooting

Problem: Cannot perform switchover in Active-Passive mode

Solution: Active-Passive mode disables failover/switchover by design. Either: - Disable Active-Passive mode: `replication-active-passive = false` - Manually promote new master outside **replication-manager**

Problem: **replication-manager** not detecting replication

Solution: Active-Passive mode ignores replication status. If replication monitoring is needed: - Switch to appropriate topology (master-slave, multi-master, etc.) - Remove `replication-active-passive = true` from configuration

Problem: Multiple servers all showing as "Unconnected"

Solution: In Active-Passive with multiple servers, this is expected. Only the first server is marked as master. To change active server: - Use API: `PUT /api/clusters/{cluster}/servers/{server}/actions/set-master` - Or configure `db-servers-prefered-master` to specify desired active server

4.4.13.11 Best Practices

Monitoring without disruption: - Use Active-Passive mode when **replication-manager** should observe but not modify infrastructure - Ideal for migration periods or auditing existing setups

Backup automation: - Active-Passive mode fully supports backup scheduling and management - Use restic integration for point-in-time recovery

Schema tracking: - Enable `monitoring-schema-change` to detect unplanned schema modifications - Configure `monitoring-schema-change-script` for alerting

Metrics collection: - Active-Passive mode continues collecting all performance metrics - Use Graphite or Prometheus endpoints for visualization in Grafana

Security: - Use dedicated monitoring user with minimal privileges - No REPLICATION privilege required in Active-Passive mode - Recommended grants: `PROCESS`, `REPLICATION CLIENT`, `SELECT`

4.5.1.1 Database servers configuration (`db-servers`)

`db-servers-hosts` (2.0), `hosts` (1.1)

Item	Value
Description	List of database hosts to monitor, IP and port (optional), specified in the host:[port] format and separated by a comma
Type	list
Example	"127.0.0.1:5055,127.0.0.1:5056"

`db-servers-credential` (2.0), `user` (1.1)

Item	Value
Description	Database login with root privileges, specified in the [user]:[password] format
Type	string
Example	"root:password"

Database login credentials will use IP (not socket). If you are using local configuration with MySQL or MariaDB, please set up credentials and privileges for `127.0.0.1` instead of `localhost`.

```
If the host is not specified or is localhost, a connection to the local host occurs:

On Windows, the client connects using shared memory, if the server was started with the [shared_memory]

On Unix, MySQL programs treat the host name localhost specially, in a way that is likely different from
```

`db-servers-prefered-master` (2.0), `prefmaster` (1.1)

Item	Value
Description	Database preferred candidates for master election, in host:[port] format
Type	list
Example	"127.0.0.1:5055,127.0.0.1:5056"

If `db-servers-prefered-master` specified, it will failover to candidates mentioned list first. `db-servers-prefered-master` does not prevent candidates outside the list to be master. If no suitable candidates found within the list, then it will try to failover outside the list. If you want to exclude the database, you need to put the host address within `db-servers-ignored-hosts`.

`db-servers-ignored-hosts` (2.0), `ignore-servers` (1.1)

Item	Value
Description	Database list of hosts to ignore for master election
Type	list
Example	"127.0.0.1:5057,127.0.0.1:5058"

`db-servers-connect-timeout` (2.0), `connect-timeout` (1.1)

Item	Value
Description	Database connection timeout in seconds. The server will timeout if the connection cannot be established before that value.
Type	integer
Default Value	5

`db-servers-read-timeout` (2.0), `read-timeout` (1.1)

Item	Value
Description	Database I/O read timeout in seconds. The server will timeout if, on an already established connection, no data is received during a period equal to this option's value.
Type	integer
Default Value	15

`db-servers-tls-ca-cert` (2.0)

Item	Value
Description	Path to the database connection TLS authority certificate.
Type	string
Default Value	""

`db-servers-tls-client-cert` (2.0)

Item	Value
Description	Path to the database connection TLS client certificate.
Type	string
Default Value	""

`db-servers-tls-client-key` (2.0)

Item	Value
Description	Database TLS client key.
Type	string
Default Value	""

`db-servers-binary-path` (2.0), `mariadb-binary-path` (1.1)

Item	Value
Description	Path to mysqld binary for **replication-manager-tst** package to provisioning local clusters.
Type	string

Item	Value
Default Value	“/usr/local/mysql/bin”

`failover-mode` (2.0), `interactive` (0.6)

Item	Value
Description	Turn ON automatic failover.
Type	enum
Possible Value	“manual” , “automatic”
Default Value	“manual”

`failover-at-sync` (0.7)

Item	Value
Description	Failover only when state semisync is sync for last status.
Type	boolean
Default Value	false

`failover-max-slave-delay` (2.0), `maxdelay` (0.6)

Item	Value
Description	Election ignore slave with replication delay over this time in second.
Type	integer
Default Value	30

`failover-time-limit`(0.7)

Item	Value
Description	Failover is canceled if previous failover took place that number of second before the time now, 0 means unlimited, this is use to prevent hardware network issue this can limit flip flap failover issues.
Type	integer
Default Value	0

`failover-limit` (1.1), `failcount` (0.7)

Item	Value
Description	Failover is canceled if the number of failover reach that counter. The console, http or api can rest the internal counter, 0 means unlimited.
Type	integer
Default Value	5

`failover-readonly-state` (2.0), `readonly` (0.7)

Item	Value
Description	Failover and switchover set slaves to read-only
Type	boolean
Default Value	true

`failover-event-scheduler` (1.0)

Item	Value
Description	Failover activate and disable event scheduler.
Type	boolean
Default Value	false

`failover-event-status` (1.0)

Item	Value
Description	Failover change the event status ENABLE OR DISABLE ON SLAVE. This is used when all cluster have event scheduler enable.
Type	boolean
Default Value	false

`failover-post-script` (2.0), `post-failover-scripts` (0.7)

Item	Value
Description	Full path of a post failover script. This is call after the new leader has been setup.
Type	string
Default Value	“”

Used to re-enable traffic but before replication-manager take care of repointing other slaves.

Parameters passed to script :

oldMaster.Host, electedMaster.Host, oldMaster.Port, cluster.master.Port bash shell example : $1,$2,$3,$4

`failover-pre-script` (2.0), `pre-failover-scripts` (0.7)

Item	Value
Description	Full path of a pre failover script. This is call before the new leader has been setup.
Type	string
Default Value	“”

Used to disable traffic on old master before replication-manager continue with failover or switchover. At this point a flush table with read lock is already issue on the old master.

Parameters passed to script :

oldMaster.Host, electedMaster.Host, oldMaster.Port, cluster.master.Port bash shell example : $1,$2,$3,$4

`failover-restart-unsafe` (1.1)

Item	Value
Description	Failover to the first restarted node when full cluster down, can be a slave
Type	boolean
Default Value	false

4.5.2.0.1 Deprecated

`interactive` (2.0)

Item	Value
Description	Tell to monitor in post 2.0 release
Type	integer
Default Value	1

`failover-falsepositive-ping-counter` (2.0) `failcount` (0.7)

Item	Value
Description	Failover after this number of ping failures, interval is driven by `failover-falsepositive-heartbeat`.
Type	integer
Default Value	5

`failover-falsepositive-heartbeat` (1.1)

Item	Value
Description	Cancel failover if one slave can still fetch events from the master.
Type	boolean
Default Value	true

`failover-falsepositive-heartbeat-timeout` (1.1)

Item	Value
Description	Failover wait this timer in second that replication detect the failed master for `failover-falsepositive-heartbeat`.
Type	integer
Default Value	3

`failover-falsepositive-external` (1.1)

Item	Value
Description	Cancel failover when http call to host of the master port 80 return header response code 200 OK, some inetd scripts can be googled to make external monitoring.
Type	boolean
Default Value	false

`failover-falsepositive-external-port` (1.1)

Item	Value
Description	Custom TCP port for external check `failover-falsepositive-external`.
Type	integer
Default Value	80

`failover-falsepositive-maxscale` (1.1, 2.0-osc, 2.0-pro)

Item	Value
Description	Cancel failover when maxscale monitor detect alive master. See the maxscale configuration section for more details
Type	boolean
Default Value	"false"

`failover-falsepositive-maxscale-timeout` (1.1, 2.0-osc, 2.0-pro)

Item	Value
Description	Failover wait this timer in second that MaxScale monitor detect the failed master for `failover-falsepositive-maxscale`.
Type	integer
Default Value	14

4.5.2.2.1 Crash recovery

replication-manager <1.1 release only rejoin failed master that get equal GTID at election time in such case it can re-attach to the cluster

`autorejoin` (0.7)

Item	Value
Description	Used to turn out all scenarios of crash recovery
Type	boolean
Default Value	true

replication-manager 1.1, rejoining of dead master has been improved to cover more scenarios.

MariaDB clients binaries are used to remotely backup binlogs from rejoining node:

`autorejoin-backup-binlog` (0.7)

Item	Value
Description	Used to to fetch and save rejoining master binary logs, only delta binary logs with the last elected master are saved
Type	boolean
Default Value	true

mysqlbinlog –read-from-remote-server they are saved into the monitoring working directory in a crash sub directory for later reuse with flashback or for the DBA eyes in case of point in time recovery.

> Note that the server-id to backup binlog used by **replication-manager** is 1000, don't use it on your database cluster nodes

replication-manager track different crash state for rejoin:

GTID of the new leader at time of election is equal to GTID or lower of the joiner, we proceed with rejoin.

Joiner GTID is ahead of position saved of the last election.

We first backup extra events in **replication-manager** data directory.

If semisync replication was in sync status, **replication-manager** can do binary log flashback to come back to a physical state clients connections have never seen.

`autorejoin-flashback` (1.1)

Item	Value
Description	Automatically rejoin a failed server to the current master and flashback at the time of election if new master was elected.
Type	boolean
Default Value	false

`autorejoin-flashback-on-sync` (2.1)

Item	Value
Description	Use flashback only when replication state was sync on crash.
Type	boolean
Default Value	true

GTID is behind and semi-sync replication status at election was desynced, we can or flashback or restore from a full state transfert (SST)**replication-manager** since 1.1 force SST.

`autorejoin-flashback-on-unsync` (2.1)

Item	Value
Description	Use flashback also when replication state was unsync on crash.
Type	boolean
Default Value	false

GTID is ahead but semi-sync replication si not used, status at election is unknown, we restore the joiner via mysqldump from the new leader when replication-manager settings use the rejoin-mysqldump flag.

`autorejoin-mysqldump` (1.1)

Item	Value
Description	Automatically rejoin a failed server to the current master using mysqldump.
Type	boolean
Default Value	false

`autorejoin-zfs-flashback` (2.1)

Item	Value
Description	Automatically rejoin a failed server via last ZFS snapshot
Type	boolean
Default Value	false

The way this is done is via creating on the joiner a database table replication_manager_schema.snapback

If none of above method is enable **replication-manager** will call external scripts

`autorejoin-script` (1.1)

Item	Value
Description	Path to a rejoin script.
Type	boolean
Default Value	false

Script is passing the server to rejoin as first argument and the new master in current topology.

In some cascading failure scenarios **replication-manager** have not way to found replication position of an election, this will happen every time no slaves is available when the last master crashed. We advise 3 node cluster to limit painful reseeding of big databases.

`autorejoin-slave-positional-hearbeat` (2.0)

Item	Value
Description	Inject PseudoGTID and use them to rejoin extra slaves for positional replication.
Type	Boolean
Default Value	false

4.5.2.2.2 All cluster down

The default rejoining method is to not promote a slave as a master when the no crash information state can be found and to wait for the old master to recover.

`failover-restart-unsafe` (2.1)

Item	Value
Description	Failover when cluster down if a slave is start first.
Type	boolean
Default Value	false

High availability when false

Master/Slave/Kill	Read/Write/Err
MS-MK-MS	RW-RW-RW
MS-MK-KK-KS-MS	RW-RW-EE-RE-RW
MS-MK-KK-MK-MS	RW-RW-EE-RW-RW
MS-KM-SM	RW-RW-RW
MS-KM-KK-KM-SM	RW-RW-EE-RW-RW
MS-KM-KK-SK-SM	RW-RW-EE-RE-RW

Change the `failover-restart-unsafe` to flavor HA against protecting over data lost and do failover on first slave to ping: after a full DC crash or if the master never show up. It was reported to work with PeaceMaker when **replication-manager** is failover and the last node available is a slave. This mode rediscover the master from this slave from the replication information.

High availability when true

Master/Slave/Kill	Read/Write/Err	Lost
MS-MK-MS	RW-RW-RW	
MS-MK-KK-KM-SM	RW-RW-EE-RW-RW	L
MS-MK-KK-MK-MS	RW-RW-EE-RW-RW	
MS-KM-SM	RW-RW-RW	
MS-KM-KK-KM-SM	RW-RW-EE-RW-RW	
MS-KM-KK-MK-MS	RW-RW-EE-RW-RW	L

This setup can possibly elect a very late slave as first leader and when no crash information state is found for rejoining the old master than the replication-manager will provision it using full state transfer via mysqldump or external script

4.5.3.1 Switchover Configuration

`switchover-at-equal-gtid` (1.1), `gtidcheck` (0.7)

Item	Value
Description	Switchover only when slaves are fully in sync.
Type	boolean
Default Value	false

`switchover-at-sync` (1.1)

Item	Value
Description	Switchover Only when state semisync is sync for last status.
Type	boolean
Default Value	false

`switchover-wait-kill` (1.1), `wait-kill` (0.7)

Item	Value
Description	Switchover wait this many milliseconds before killing threads on demoted master.
Type	integer
Default Value	5000

`switchover-wait-write-query` (1.1), `wait-write-query` (0.7)

Item	Value
Description	Switchover is canceled if a write query is running for this time in second.
Type	integer
Default Value	10

`switchover-wait-trx` (2.0), `wait-trx` (1.0)

Item	Value
Description	Switchover is cancel after this timeout in second if can't return from FTWRL.
Type	integer
Default Value	10

`switchover-slave-wait-catch` (2.0)

Item	Value
Description	Switchover wait for replication to catch up before switching extra slaves, when using GTID don't wait can speed up switchover but may hide issues liek writing on the old master with super user
Type	boolean
Default Value	true

4.5.4.1 Replication Configuration

replication-manager to failover will be in charge of rebuilding the replication, following section explore how to proceed with it.

`replication-credential` (2.0), `rpluser` (0.6)

Item	Value
Description	Replication is created with such credential, in the [user]:[password] format.
Type	string
Default Value	repl:repman

`replication-use-ssl` (1.0)

Item	Value
Description	Replication is created using SSL encryption to replicate from master.
Type	boolean
Default Value	false

`replication-master-connect-retry` (2.0), `master-connect-retry` (1.0)

Item	Value
Description	Replication is created using this connection retry timeout in second.
Type	integer
Default Value	10

4.5.4.2 Topology Configuration

4.5.4.2.1 Master Slaves Configuration

replication-manager supports 2-node master slave setup, it is advice to use at least 3 nodes cluster to get the cluster tolerant to losing or stopping a slave.

By default **replication-manager** assume a flat topology but can enable multi-tier topology with some additional setting, scenario is you stop a slave and his master die, when the master rejoin the topology it can keep his slave under him instead of switching the slave to the new master

`replication-multi-tier-slave` (2.0), `multi-tier-slave` (1.0)

Item	Value
Description	Enable relay slaves topology
Type	boolean
Default Value	false

4.5.4.3 Multi Tier Slaves Configuration

`replication-multi-tier-slave` (2.0), `multi-tier-slave` (1.0)

Item	Value
Description	Enable relay slaves topology
Type	boolean
Default Value	false

replication-manager have support for replication tree or relay slaves architecture, in case of master death one of the slaves under the relay is promoted as a master.

replication-manager does not manage yet the relay crash to replace it with a slave

4.5.4.4 Multi Master Configuration

replication-manager supports 2-node multi-master topology detection. It is required to declare it the configuration configuration

`replication-multi-master (2.0), multimaster (0.7)`

Item	Value
Description	Enable Master-Master topology
Type	boolean
Default Value	false

You just need to set one preferred master

We advice to enable restart of database in read-only mode to cover the case where a failed node try to rejoin but can't be contacted anymore from **replication-manager**, in such case no write traffic will be enable on the rejoining node.In a Multi DC split brain with a proxy on each side we can simply ensure the split brain will not make the database diverge on each side

MariaDB configuration file:

```
read_only = 1
```

This flag ensures that in case of split brain + leader crash, when old leader is reintroduced it will not show up as a possible leader for WRITES.

MaxScale configuration file:

Maxscale need to be instructed to monitor multi master, following setting track the read-only flag and route queries to the writable node.

```
[Multi-Master Monitor]
type=monitor
module=mmmon
servers=server1,server2,server3
user=myuser
passwd=mypwd
detect_stale_master=true
```

4.5.4.5 Multi Source or Channel Configuration

replication-manager support monitoring of a specific replication channel, those feature are experimental and should be tested because they are not covered by QA

`replication-source-name` **(2.0)**, `master-connection` **(1.1)**

Item	Value
Description	Replication channel name to use for multisource
Type	string
Default Value	“”

4.5.4.6 Multi Master Ring Configuration

`replication-multi-master-ring (2.0)`

Item	Value
Description	Enable Multi Master Ring topology
Type	boolean
Default Value	false

Read more

4.5.4.7 Multi Master Galera Configuration

`replication-multi-master-wsrep (2.0)`

Item	Value
Description	Enable Multi Master Galera
Type	boolean
Default Value	false

Read more

4.5.5.1.0.1 HaProxy Configuration

We advice usage of HAProxy >=1.7 previously replication-manager can not get correct statistics

replication-manager can operate with HaProxy in different modes: local standby and remote via external check to replication-manger API since **replication-manager (2.0)**. Last mode is API based since **replication-manager (2.1)**.

`haproxy` (0.7)

Item	Value
Description	Enable Driving HaProxy.
Type	boolean
Default Value	false

`haproxy-servers` (0.7)

Item	Value
Description	Comma separated list of the HaProxy hosts.
Type	String
Default Value	"127.0.0.1"

`haproxy-mode` (0.7)

Item	Value
Description	Mode of driving haproxy standby will operate a local haproxy intsance and drive config+
Type	String
Default Value	runtimeapi

standby: Operate a local HAProxy instance via config generation and triggering start stop reload via socket

runtimeapi: use the tcp runtime api of HAProxy to modify some route to master and backend state according to replication status

externalcheck: do nothing but when provisioning, set external check script to replication-manager and HAProxy config file using external checks

dataplanapi: TODO: a sidecar process that runs next to HAProxy and provides API endpoints for managing HAProxy. It requires HAProxy version 1.9.0 or higher.

`haproxy-write-port` (0.7)

Item	Value
Description	HaProxy port to get database connection in WRITE.
Type	Integer
Default Value	3306

`haproxy-ip-write-bind` (0.7)

Item	Value
Description	If WRITE traffic bing to specific IP.
Type	String
Default Value	"0.0.0.0"

`haproxy-read-port` (0.7)

Item	Value
Description	HaProxy port to load balance read connection to all databases.
Type	Integer
Default Value	3306

`haproxy-ip-read-bind` (0.7)

Item	Value
Description	If READ traffic bing to specific IP.
Type	String
Default Value	"0.0.0.0"

`haproxy-stat-port` (0.7)

Item	Value
Description	HaProxy port to collect statistics.
Type	Integer
Default Value	1988

4.5.5.1.0.1.1 Local mode

`haproxy-binary-path` (0.7) In local mode **replication-manager**, need HaProxy to be install and on the same server **replication-manager** will than start one HaProxy Daemon per Cluster configured and maintain the configuration to route traffic to the topology.

Item	Value
Description	Full path to HaProxy binary.
Type	String
Default Value	"/usr/sbin/haproxy"

replication-manager generate some HaProxy configuration file. A template is located in the share directory used by replication-manager. For safety HaProxy is not stopped when replication-manager is stopped

replication-manager re-generate the HaProxy configuration file when the topology change and instruct HaProxy to reload this configuration during failover and switchover.

4.5.5.1.0.1.2 Remote mode using external checks HaProxy can call replication **replication-manage(2.0)** http handlers via the check-external feature to check that a backend is a valid Master or valid Slave

replication-manager (2.0) http server need to be enable and it's bind address need to be joinable from HaProxy for node health check to happen.

replication-manager-pro (2.0) can auto deploy HaProxy micro service, HaProxy config need the http-bind-address to be joinable from the proxy.

The haproxy conf that is deployed via micro services looks like this:

```
global

 daemon
 maxconn 4096
 stats socket /run/admin.sock level admin
 log 127.0.0.1 local0 debug
 external-check

defaults
   log global
   mode http
   option dontlognull
   option redispatch
   option clitcpka
   option srvtcpka
   retries 3
   maxconn 500000
   timeout http-request 5s
   timeout connect 5000ms
   timeout client 50000s
   timeout server 50000s

listen stats
   bind 0.0.0.0:1988
   mode http
   stats enable
   stats uri /
   stats refresh 2s
   stats realm Haproxy\ Stats
   stats show-legends

frontend my_write_frontend
    bind 0.0.0.0:3303
    option tcplog
    mode tcp
    default_backend service_write

frontend my_read_frontend
    bind 0.0.0.0:3302
    option tcplog
    mode tcp
    default_backend service_read

backend service_write
    mode tcp
    balance leastconn
    option external-check
    external-check path "/usr/bin:/bin"
```

```
    external-check command /usr/bin/checkmaster


server server1 192.168.100.70:3306  weight 100 maxconn 2000 check inter 1000
server server2 192.168.100.71:3306  weight 100 maxconn 2000 check inter 1000
server server3 192.168.100.72:3306  weight 100 maxconn 2000 check inter 1000

backend service_read
    mode tcp
    balance leastconn
    option external-check
    external-check path "/usr/bin:/bin"
    external-check command /usr/bin/checkslave


server server1 192.168.100.70:3306  weight 100 maxconn 2000 check inter 1000
server server2 192.168.100.71:3306  weight 100 maxconn 2000 check inter 1000
server server3 192.168.100.72:3306  weight 100 maxconn 2000 check inter 1000
```

The checkmaster script deployed looks like that: where http://192.168.100.20:10001 is the local dc **replication-manager (2.0)** web server

```
/usr/bin/checkmaster


#!/bin/sh
echo $3 $4
ret=`wget -O - -q http://192.168.100.20:10001/clusters/ux_dck_zpool_loop/servers/$3/$4/master-status`
n=`echo "$ret" | grep -c "200"`
if [ $n -eq 1 ]
then
 exit 0
fi
exit 1
```

cat /usr/bin/checkslave

```
#!/bin/sh
echo $3 $4
ret=`wget -O - -q http://192.168.100.20:10001/clusters/ux_dck_zpool_loop/servers/$3/$4/slave-status`
n=`echo "$ret" | grep -c "200"`
if [ $n -eq 1 ]
then
 exit 0
fi
exit 1
```

using **replication-manager** active/standby & arbitrator make sure each script point to it's local replication-manager. The all side loosing arbitration will report backend down.

4.5.5.1.0.1.3 Runtime API HAProxy Runtime API provides very powerful dynamic configuration capabilities with no service reloads or restarts https://www.haproxy.com/fr/blog/dynamic-configuration-haproxy-runtime-api/

replication-manager (2.1) is waiting for some specific naming convention for backend pool and server names in the proxy config

backend service_write

In this backend one need to define a single master node a named it leader

backend service_read

In this backend one need to define a all replica nodes and named them with the Id of replication manager aka db<hash(host:port)> One can get those ID via API (http interface / cluster menu / debug / servers)

The replication-manager will auto DRAIN the backend route if the replication is broken or is late or the server is in ignored list

4.5.5.2.0.1 ProxySQL Configuration

`proxysql` (2.0)

Item	Value
Description	Enable ProxySQL driver
Type	boolean
Default Value	false

`proxysql-servers` (2.0)

Item	Value
Description	Comma separated list of ProxySQL hosts
Type	String
Default Value	"127.0.0.1"

`proxysql-port` (2.0)

Item	Value
Description	ProxySQL port to get database connection
Type	String
Default Value	"6033"

`proxysql-admin-port` (2.0)

Item	Value
Description	ProxySQL port to get admin connection
Type	String
Default Value	"6032"

`proxysql-writer-hostgroup` (2.0)

Item	Value
Description	ProxySQL writer hostgroup ID
Type	String
Default Value	"0"

`proxysql-reader-hostgroup` (2.0)

Item	Value
Description	ProxySQL reader hostgroup ID
Type	String
Default Value	"1"

`proxysql-user` (2.0)

Item	Value
Description	ProxySQL admin user name
Type	String
Default Value	"admin"

`proxysql-password` (2.0)

Item	Value
Description	ProxySQL admin password
Type	String
Default Value	"admin"

`proxysql-bootstrap` (2.0)

Item	Value
Description	Auto create servers when start
Type	String
Default Value	true

`proxysql-bootstrap-users` (2.3)

Item	Value
Description	Auto push users when they are discovered
Type	String
Default Value	true

`proxysql-copy-grants` (2.0) `obsolete` use `proxysql-bootstrap-users`

Item	Value
Description	Auto push users when they are discovered
Type	String
Default Value	true

`proxysql-reader-hostgroup` (2.0)

Item	Value
Description	Define reader hostgroup
Type	Integer
Default Value	1

`proxysql-writer-hostgroup` (2.0)

Item	Value
Description	Define writer hostgroup
Type	Integer
Default Value	0

4.5.5.3.0.0.1 Maxscale Configuration **replication-manager** can operate with MaxScale with different modes.

Read the following sections to route database traffic using Maxscale.

Using **replication-manager** we can speed up the failover process of Maxscale, **replication-manager** will instruct MaxScale about the new master so that the Maxscale monitor delay is shortcut. This bring less risk against MaxScale sending traffic to the old master during the monitoring delay, **replication-manager** put extra barriers against writes during failover and switchover on the database level but not forever.

Maxscale Monitor Status Configuration With **replication-manager** one can get Maxscale Servers State in the HTTP server proxy tab.

replication-manager will monitor that the Maxscale Write Port point to the master in topology

`maxscale` (0.7)

Item	Value
Description	Enable MaxScale monitoring
Type	boolean
Default Value	false

`maxscale-servers` (2.0), `maxscale-host` (1.0)

Item	Value
Description	Comma separated list of the MaxScale hosts.
Type	String
Default Value	“”

`maxscale-write-port` (1.1)

Item	Value
Description	MaxScale port to get database connection in WRITE.
Type	Integer
Default Value	3306

`maxscale-read-port` (1.1)

Item	Value
Description	MaxScale port to load balance read connection to all databases.
Type	Integer
Default Value	3306

`maxscale-read-write-port` (1.1)

Item	Value
Description	MaxScale port to a read write spilling connection.

Item	Value
Type	Integer
Default Value	3306

replication-manager default monitoring method is to use MaxScale tcp row protocol. Configuration can be setup to use the MaxScale maxinfo plugin that provide a JSON REST service. In new Maxscale release a true REST API is now available but **replication-manager** do not use it yet.

`maxscale-get-info-method` (0.7)

Item	Value
Description	Enable MaxScale monitoring
Type	list
Possible Values	maxinfo,maxadmin,maxapi
Default Value	maxadmin

`maxscale-port` (0.7)

Item	Value
Description	MaxAdmin port to send command in tcp raw protocol.
Type	Integer
Default Value	6603

`maxscale-user` (0.7)

Item	Value
Description	MaxAdmin user in tcp raw protocol.
Type	String
Default Value	"admin"

`maxscale-password` (2.0) `maxscale-pass` (0.7)

Item	Value
Description	MaxAdmin password in tcp raw protocol.
Type	String
Default Value	"mariadb"

`maxscale-maxinfo-port` (0.7)

Item	Value
Description	MaxInfo port to for reading server monitor state
Type	Integer
Default Value	4003

Passive Mode Advised mode

MaxScale auto-discovers the new topology after failover or switchover. Replication Manager can reduce MaxScale monitor detection time of the master failure to reduce the time where it might block clients. This setup best works in 3 nodes in Master-Slaves cluster, because one slave is always available for re-discovering new topologies.

Example Maxscale settings:

```
[MySQL Monitor]
type=monitor
module=mysqlmon
servers=%%ENV:SERVERS_LIST%%
user=root
passwd=%%ENV:MYROOTPWD%%
monitor_interval=500
detect_stale_master=true

[Write Connection Router]
type=service
router=readconnroute
router_options=master
servers=%%ENV:SERVERS_LIST%%
user=root
passwd=%%ENV:MYROOTPWD%%
enable_root_user=true
```

> In case all slaves are down, MaxScale can still operate on the Master with the following maxscale monitoring setup : https://github.com/mariadb-corporation/MaxScale/blob/2.1/Documentation/Monitors/MySQL-Monitor.md#failover

`detect_stale_master`

In Maxscale 2.1 routing to last node have been introduce so that transparent support of 2 nodes cluster is possible [Doc]

`detect_stale_slave=true`

Use the following example grant for your MaxScale user:

```
CREATE USER 'maxadmin'@'%' IDENTIFIED BY 'maxpwd';
GRANT SELECT ON mysql.user TO 'maxadmin'@'%';
GRANT SELECT ON mysql.db TO 'maxadmin'@'%';
GRANT SELECT ON mysql.tables_priv TO 'maxadmin'@'%';
GRANT SHOW DATABASES, REPLICATION CLIENT ON *.* TO 'maxadmin'@'%';
GRANT ALL ON maxscale_schema.* TO 'maxadmin'@'%';
```

Also, to protect consistency it is strongly advised to disable *SUPER* privilege to users that perform writes, such as the MaxScale user when the Read-Write split module is instructed to check for replication lag:

```
[Splitter Service]
type=service
router=readwritesplit
max_slave_replication_lag=30
```

Driving Mode Operating MaxScale without monitor is the second Replication-Manager mode via:

`maxscale-disable-monitor` (1.1)

Item	Value
Description	MaxInfo port to for reading server monitor state
Type	Boolean
Default Value	false

replication-manager will assign each server status flags to the MaxScale backend servers via the MaxScale row protocol. This mode of operation is similar to the HAProxy but MaxScale server does not need to be collocated with **replication-manager**

If your are using old MaxScale release that does not support detect_stale_slave it can be used to support 2 nodes cluster

Slave Mode MaxScale drive **replication-manager** via calling some extranm scripts using **replication-manager-cli**.

Implications is that **replication-manager** need configuration for automatic failover. This is not always desirable, the false positive detection in **replication-manager** is much more advance o the forced failover may not succeed, i'm not aware of any retry solution in such case. This would force anyway usage of **replication-manager** command for more than a one shoot failure scenario.

MaxScale Binlog Server

`maxscale-binlog` (1.1)

Item	Value
Description	Enable support for MaxScale Binlog Server
Type	Boolean
Default Value	false

`maxscale-maxinfo-port` (1.1)

Item	Value
Description	MaxScale Binlog Server port
Type	Integer
Default Value	3306

`force-slave-gtid-mode` (1.1)

Item	Value
Description	Disable GTID on slave
Type	Boolean
Default Value	false

This parameter need to be enable for maxscale < 2.2

> From MaxScale 2.2 can support MariaDB GTID so force-gtid-mode=false is not needed anymore part of task https://github.com/mariadb-corporation/MaxScale/tree/MXS-1075 Maxscale Binlog router plugin configuration

`transaction_safety=On,mariadb10-compatibility=On,mariadb_gtid=On`

MaxScale Binlog Server need all databases node to have same binlog prefix

`bin_log='mariadb-bin'`

MaxScale Binlog Server Configuration

`router_options=mariadb10-compatibility=1,server-id=999,user=skysql,password=skyvodka,send_slave_heartbea`

4.5.5.4.0.0.1 MariaDBShardProxy Since version 1.1 **replication-manager** can manage Spider proxy for schema and table Sharding.

This type of linked table proxy preserve transaction consistency across shard group, transactions can be run against multiple shard clusters. Joins queries can be achieved inter shard.

This is done using Spider storage engine

replication-manager discovering the master tables on startup and maintain link to master during failover and switchover.

For every cluster definition you wan't to proxy add extra MariaDBShardProxy configuration:

`shardproxy` (2.0)

Item	Value
Description	Enable Driving mdbshardproxy.
Type	boolean
Default Value	false

`shardproxy-servers` (2.0)

Item	Value
Description	Comma separated list of the mdbshardproxy hosts.
Type	String
Default Value	"127.0.0.1:3306"

`shardproxy-credential` (1.0)

Item	Value
Description	Credential to connect to mdbshardproxy.
Type	String
Default Value	"root:mariadb"

4.5.5.4.1 Testing Configuration

Give a path to MariaDB 10.3 or Spiral Spider if you would like *replication-manager-tst* to bootstrap a local MariaDBShardProxy.

`mariadb-binary-path` (0.7)

Item	Value
Description	Full path to mdbshardproxy binary.
Type	String
Default Value	"/usr/local/mariadb"

This instance will use a default configuration file in

`/usr/share/tests/etc/mdbsproxy.cnf`

In local wrapper mode replication-manager never stop proxies to avoid disturbing the workload:)

plugin:youtube

The first supported micro service registry is Consul from *HashiCorp*, we are planning to add others like mdns, etcd, kubernetes, zookeeper.

4.5.5.5.0.0.1 Consul replication-manager (2.0) support consul registry, the registration service will be extended to other registries in future releases.

A Consul server and agent need to be install locally with **replication-manager** as advised in the HashiCorp documentation https://www.consul.io/docs/index.html

`registry-consul` (2.0)

Item	Value
Description	Registry service write_cluster and read_cluster to a local consul agent .
Type	Boolean
Values	true
Default Value	"consul"

`registry-servers` (2.0)

Item	Value
Description	Comma-separated list of registry addresses (default)
Type	String
Values	Unused today but can be use later to point to remote servers
Default Value	"127.0.0.1"

replication-manager (2.0) create a DNS entry **write_clustername** that follow the leader on failover and switchover.

```
dig @127.0.0.1 -p 8600 write_cluster_haproxy_multimasterring.service.consul SRV

; <<>> DiG 9.8.3-P1 <<>> @127.0.0.1 -p 8600 write_cluster_haproxy_multimasterring.service.consul SRV
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 56071
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;write_cluster_haproxy_multimasterring.service.consul. IN SRV

;; ANSWER SECTION:
write_cluster_haproxy_multimasterring.service.consul. 0 IN SRV 1 1 3315 macbook-pro-de-apple-2.local.noc
```

replication-manager (2.0) create a DNS entry **read_clustername** that round robin on all cluster nodes.

```
; <<>> DiG 9.8.3-P1 <<>> @127.0.0.1 -p 8600 read_cluster_haproxy_multimasterring.service.consul SRV
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 43134
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1
```

```
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;read_cluster_haproxy_multimasterring.service.consul. IN    SRV

;; ANSWER SECTION:
read_cluster_haproxy_multimasterring.service.consul. 0 IN SRV 1 1 3316 macbook-pro-de-apple-2.local.node
read_cluster_haproxy_multimasterring.service.consul. 0 IN SRV 1 1 3317 macbook-pro-de-apple-2.local.node
```

The services can be remove via the Consul agent entry are pending , **replication-manager** will not remove DNS entry on shutdown.

```
curl --request PUT http://localhost:8500/v1/agent/service/deregister/18259094474708091725
```

For querying catalogue service informations:

```
curl http://localhost:8500/v1/catalog/service/write_cluster_haproxy_multimasterring
[
    {
        "ID": "795b5921-0b0f-c914-6145-5570ba346b89",
        "Node": "macbook-pro-de-apple-2.local",
        "Address": "127.0.0.1",
        "Datacenter": "dc1",
        "TaggedAddresses": {
            "lan": "127.0.0.1",
            "wan": "127.0.0.1"
        },
        "NodeMeta": {
            "consul-network-segment": ""
        },
        "ServiceID": "write_cluster_haproxy_multimasterring",
        "ServiceName": "write_cluster_haproxy_multimasterring",
        "ServiceTags": [
            "v-789c32d033d03300040000ffff02c900ed"
        ],
        "ServiceAddress": "127.0.0.1",
        "ServicePort": 3317,
        "ServiceEnableTagOverride": false,
        "CreateIndex": 34410,
        "ModifyIndex": 34557
    }
]
```

4.5.5.6.0.1 External Scripts

replication-manager can automate route changes with popular proxies. however in some edge cases e.g. pacemaker or keepalived, logic like vip or hardware load balancer api calls can be managed through the external scripts option.

`failover-post-script` (2.0), `post-failover-script` (0.7)

Item	Value
Description	Full path of a post-failover script. This is called after the new leader has been setup. The previous master host and the newly elected master host are passed as sequential arguments.
Type	string
Default Value	“”
Example	“/usr/local/bin/vip-up.sh”

Arguments passed to the script: cluster.oldMaster.Host cluster.master.Host cluster.oldMaster.Port cluster.master.Port

`failover-pre-script` (2.0), `pre-failover-script` (0.7)

Item	Value
Description	Full path of a pre-failover script. This is called after the new leader has been elected. The previous master host and the newly elected master host are passed as sequential arguments.
Type	string
Default Value	“”
Example	“/usr/local/bin/vip-down.sh”

Arguments passed to the script: cluster.oldMaster.Host cluster.master.Host cluster.oldMaster.Port cluster.master.Port

4.5.5.6.0.2 External Proxy & Route

replication-manager can inject heartbeat into the route and check that the internal master is alway the one reach via the proxies or the route.

`extproxy` (2.0)

Item	Value
Description	External proxy can be used to specify a route manage with external scripts setup.
Type	Boolean
Default Value	false

`extproxy-address` (2.0)

Item	Value
Description	Network address when route is manage via external script, host:[port] format.
Type	String
Default Value	“”

replication-manage 2.0 can be used from proxies like HaProxy to check the status of the monitored servers.

We provide different checks for master and slave status.

```
https://{monitor-api-address}/api/clusters/{clusterName}/servers/{serverName}/master-status
https://{monitor-api-address}/api/clusters/{clusterName}/servers/{serverHost}/{serverPort}/master-status
http://{http-bind-address}:{http-port}/clusters/{clusterName}/servers/{serverHost}/{serverPort}/master-s
```

Return Code 200 if server id is a master

```
https://{monitor-api-address}/api/clusters/{clusterName}/servers/{serverName}/slave-status
https://{monitor-api-address}/api/clusters/{clusterName}/servers/{serverHost}/{serverPort}/slave-status
http://{http-bind-address}:{http-port}/clusters/{clusterName}/servers/{serverHost}/{serverPort}/slave-st
```

Return Code 200 if server id is a slave

> **{clusterName}** is the name of the section name define in the config file default should be it if no multiple clusters are defines
>
> **{serverName}** can be found via the client command **topology**

- ☒ Clusters in passive state return error code.
- ☒ Servers in maintenance, failed or suspect will return error code.
- ☒ Slaves with replication lags still return valid code
- ☒ Slaves with replication stopped return error code
- ☒ Master with READONLY return error code

replication-manager enforces best replication and database configuration practice using the following parameters. Every possible enforcement is producing warnings to let you know about what can be improved in your database setup.

4.5.6.1 Checks

`check-binlog-filters` (1.1)

Item	Value
Description	Check that all possible masters have equal binlog filters.
Type	Boolean
Default Value	true

`check-replication-filters` (1.1)

Item	Value
Description	Check that all possible slaves have equal replication filters.
Type	Boolean
Default Value	true

`check-replication-state` (1.1)

Item	Value
Description	Check that all replications are running fine.
Type	Boolean
Default Value	true

4.5.6.2 Enforce

`force-slave-readonly` (1.1)

Item	Value
Description	Automatically force READ ONLY on slaves.
Type	Boolean
Default Value	true

`force-sync-binlog` (1.1)

Item	Value
Description	Automatically force master crash safe.
Type	Boolean
Default Value	false

`force-sync-innodb` (1.1)

Item	Value
Description	Automatically force master innodb crash safe.
Type	Boolean
Default Value	false

`force-slave-semisync` (1.1)

Item	Value
Description	Automatically force semi-sync usage.
Type	Boolean
Default Value	false

`force-binlog-annotate-embedded` (1.1)

Item	Value
Description	Automatically activate annotate event allow to print SQL statement on slave using SHOW PROCESSLIST .
Type	Boolean
Default Value	false

`force-binlog-checksum` (1.1)

Item	Value
Description	Automatically force binlog checksum, miss functioning network or inter continental networking can corrupt replication protocol. Checksum detect and repair the protocol by re-transmitting.
Type	Boolean
Default Value	false

`force-binlog-compress` (1.1)

Item	Value
Description	Automatically force on disk binlog compression on MariaDB 10.2.
Type	Boolean
Default Value	false

`force-binlog-row` (1.1)

Item	Value
Description	Automatically activate binlog row format on master.
Type	Boolean
Default Value	false

`force-binlog-slowqueries` (1.1)

Item	Value
Description	Automatically activate logging of slow query run from replication in slow log.
Type	Boolean
Default Value	false

`force-slave-gtid-mode` (1.1)

Item	Value
Description	Automatically activate gtid mode on slave.
Type	Boolean
Default Value	false

`force-slave-gtid-mode-strict` (2.1)

Item	Value
Description	Enforce activate gtid strict mode on slave.
Type	Boolean
Default Value	false

It was found that it can be used keeping default non strict mode when slave restart, when using Memory table on slave it would break strict mode replication such parameter enable replication manager to set back strict mode after the restart

`force-slave-no-gtid-mode` (1.1)

Item	Value
Description	Automatically disable gtid mode on slave.
Type	Boolean
Default Value	false

`force-slave-heartbeat` (1.1)

Item	Value
Description	Automatically activate replication heartbeat on slave.
Type	Boolean
Default Value	true

`force-slave-heartbeat-retry` (1.1)

Item	Value
Description	Replication heartbeat retry on slave.
Type	Integer

Item	Value
Default Value	5

`force-slave-heartbeat-time` (1.1)

Item	Value
Description	Replication heartbeat time.
Type	Integer
Default Value	3

`force-noslave-behind` (1.1)

Item	Value
Description	Automatically force no slave behing.
Type	Boolean
Default Value	true

4.6.1 Troubleshooting High Availability

This page lists every file and API endpoint replication-manager writes or exposes during normal operation, failovers, and crashes — and explains how to read them when something goes wrong.

4.6.2 Data Directory Layout

All runtime state is stored under `monitoring-datadir` (default `/var/lib/replication-manager`). Each cluster gets its own sub-directory named after the cluster section in the config file.

```
/var/lib/replication-manager/
  panic.log                         # stacktrace on daemon panic (JSON)
  carbonapi.log                     # embedded Graphite backend log
  <cluster-name>/
      clusterstate.json             # full cluster state snapshot (persisted every tick)
      sla.json                      # SLA counters (uptime, failover counts)
      queryrules.json               # active ProxySQL query rules
      agents.json                   # OpenSVC agent list
      <cluster-name>.toml           # active runtime config (written by API/GUI changes)
      cache.toml                    # cached config values
      immutable.toml                # config values locked from API changes
      overwrite.toml                # config values that override the main config
      failover.YYYYMMDDHHMMSS.json  # one file per failover/switchover event
      sql_error.log                 # SQL errors from all DB connections
      sql_general.log               # all SQL statements executed by replication-manager
      ca-cert.pem                   # generated CA certificate
      ca-key.pem                    # generated CA private key
      server-cert.pem               # server TLS certificate
      server-key.pem                # server TLS private key
```

4.6.3 Key Files Explained

4.6.3.1 `clusterstate.json`

Written on every monitoring tick. Contains the complete cluster snapshot including: - Current primary and all replicas with their status, GTID positions, and lag - Active error and warning state codes (e.g. `ERR00076`, `WARN0108`) - Crash history (`dbServersCrashes`) — cleared when all servers are back up - Failover history (`failoverHistory`) — kept across restarts for PITR reference

```
cat /var/lib/replication-manager/mycluster/clusterstate.json | jq .
# Show current state codes
cat /var/lib/replication-manager/mycluster/clusterstate.json | jq '.crashes'
# Show SLA
cat /var/lib/replication-manager/mycluster/sla.json | jq .
```

4.6.3.2 `failover.YYYYMMDDHHMMSS.json`

One file is written per failover or switchover event, named with a timestamp. Contains: - Old primary URL and elected new primary URL - Binlog file and position at the moment of the event - GTID positions (IO GTID, executed GTID set) - Semi-sync slave status at failover time - Whether it was a switchover (`"switchover": true`) or failover

```
# List all failover events, most recent first
ls -lt /var/lib/replication-manager/mycluster/failover.*.json
```

```
# Inspect the most recent event
ls -t /var/lib/replication-manager/mycluster/failover.*.json | head -1 | xargs cat | jq .
```

The number of retained failover files is controlled by `failover-log-file-keep` (default: 5).

4.6.3.3 sql_error.log and sql_general.log

- `sql_error.log` — every SQL error returned when replication-manager queries the database (connection failures, permission errors, invalid queries)
- `sql_general.log` — every SQL statement replication-manager executes against the cluster (useful to audit what the daemon does at each step of a failover or rejoin)

```
# Watch SQL errors in real time
tail -f /var/lib/replication-manager/mycluster/sql_error.log

# Find statements executed during a failover window
grep "2024-06-01 03:" /var/lib/replication-manager/mycluster/sql_general.log
```

4.6.3.4 panic.log

If replication-manager itself crashes, a JSON-formatted stacktrace is written here. Send this file to Signal18 support when reporting a daemon crash.

```
cat /var/lib/replication-manager/panic.log | jq .
```

4.6.4 Log Files

See the Logs reference for the full list of log files and journalctl commands. For HA troubleshooting the most relevant are:

File	What to look for
Main log (`log-file`)	`STATE`, `ERRO`, `WARNO` lines around the incident time
Maintenance log (`*-maintenance.log`)	Rejoin and reseed progress
`sql_error.log`	Permission or connectivity errors that blocked a failover step
`sql_general.log`	Exact SQL sequence replication-manager ran during failover/rejoin

```
# Find all state changes in the main log
grep 'STATE' /var/log/replication-manager.log

# Find failover-related lines around a specific time
grep '2024-06-01 03:' /var/log/replication-manager.log | grep -E 'failover|Failover|ERRO|STATE'

# Follow the maintenance log during a rejoin
tail -f /var/log/replication-manager-maintenance.log
```

4.6.5 REST API Endpoints

All in-memory state is also accessible over the API without reading files:

Endpoint	Content
`GET /api/clusters/{name}/topology/servers`	All servers with status, GTID, lag, replication health
`GET /api/clusters/{name}/topology/master`	Current primary details
`GET /api/clusters/{name}/topology/slaves`	All replicas with replication thread status
`GET /api/clusters/{name}/topology/alerts`	Active alert and warning state codes
`GET /api/clusters/{name}/topology/crashes`	Crash and failover history (in-memory)
`GET /api/clusters/{name}/topology/logs`	General cluster log buffer
`GET /api/clusters/{name}/topology/http-logs`	HTTP/API log buffer

```
# Quick cluster health check
curl -sk -u admin:repman https://localhost:10005/api/clusters/mycluster/topology/servers | jq '.[] | {ho

# Active alerts
curl -sk -u admin:repman https://localhost:10005/api/clusters/mycluster/topology/alerts | jq .

# Crash history
curl -sk -u admin:repman https://localhost:10005/api/clusters/mycluster/topology/crashes | jq .
```

4.6.6 Increasing Verbosity for an Investigation

To get more detail without restarting the daemon, bump the relevant module log levels via config reload or API:

```
# In cluster config - reload with SIGHUP or via API
log-level           = 4   # full DEBUG for all modules
log-level-topology  = 4   # topology discovery detail
log-level-sql       = 4   # every SQL statement with timing
log-level-sst       = 4   # state transfer / rejoin steps

# Follow debug output in real time
journalctl -u replication-manager -f -p debug

# Or if log-file is configured
tail -f /var/log/replication-manager.log | grep -E 'DEBUG|topology|rejoin|failover'
```

4.6.7 Common First Steps

Symptom	Where to look
Unexpected failover	`failover.*.json` + main log `STATE` lines around the event time
Failover did not trigger	Main log — check false-positive check results (`ERR00023`, grace period messages)
Replica not rejoining	Maintenance log + `sql_error.log` for permission or GTID errors
Wrong server promoted	`clusterstate.json` — check replication health scores at the time
Daemon crashed	`panic.log` — JSON stacktrace
Proxy not updated after failover	Main log — search `[proxy]`, `[haproxy]`, `[proxysql]` module tags

Symptom	Where to look
Config not taking effect	`<cluster>.toml`, `cache.toml`, `immutable.toml` in the datadir — check which layer wins

5.1.1 Monitoring Overview

replication-manager runs a continuous monitoring loop that tracks every server in every managed cluster. On each tick (default every 2 seconds, controlled by `monitoring-ticker`) it connects to each database and proxy server, reads state, and updates an in-memory topology model that drives the GUI, API, alerting, and failover decisions.

5.1.2 What the Monitor Tracks

5.1.2.1 Replication topology

- Server role (primary / replica / candidate / down)
- GTID positions and replication lag
- Semi-sync acknowledgement state
- Slave SQL/IO thread status
- Relay log position and relay log size
- Replication domain IDs (multi-source)

5.1.2.2 Server health

- Global status variables (connections, InnoDB metrics, query cache, etc.)
- Global variables (for config drift detection)
- InnoDB engine status
- Error log events
- Schema changes (DDL detection — requires `monitoring-schema-change`)

5.1.2.3 Workload

- Active processlist (up to 50 longest-running queries — `monitoring-processlist`)
- Performance Schema digest statistics (`monitoring-performance-schema`)
- Query routing rules consolidated from all proxies (`monitoring-query-rules`)

5.1.2.4 Proxies

- ProxySQL, HAProxy, and MaxScale connection counts and routing state
- Backend server weights and online/offline transitions

5.1.2.5 Metrics

- All collected counters are pushed to the embedded Graphite database and exposed as Prometheus metrics
- See Metrics for Graphite and Prometheus configuration

5.1.3 Monitoring Loop

```
Every monitoring-ticker seconds:
  for each cluster:
    for each server (parallel goroutines):
      connect → read status → update topology model
    evaluate topology → compute failover eligibility
    fire alerts if state changed (see Alerting chapter)
    push metrics to Graphite
    write API state
```

When `monitoring-capture` is enabled, replication-manager automatically saves a diagnostic snapshot (SHOW SLAVE STATUS, SHOW PROCESSLIST, SHOW GLOBAL STATUS, SHOW ENGINE INNODB STATUS) for a configurable number of ticks whenever a monitored error code appears. See Logs and Troubleshooting for details.

5.1.4 Accessing Monitoring Data

5.1.4.1 GUI

The web dashboard shows the live topology, server state, processlist, and alerts. The React dashboard is served at the replication-manager API address (default port 10001).

5.1.4.2 REST API

```
GET /api/clusters/{clusterName}/topology/servers
GET /api/clusters/{clusterName}/topology/master
GET /api/clusters/{clusterName}/topology/slaves
GET /api/clusters/{clusterName}/topology/alerts
GET /api/clusters/{clusterName}/topology/logs
```

5.1.4.3 Prometheus

```
GET /api/prometheus
```

Returns all collected metrics in Prometheus text exposition format. Point your Prometheus scraper at this endpoint.

5.1.4.4 CLI

```
replication-manager-cli status --cluster=my-cluster
replication-manager-cli status --cluster=my-cluster --with-errors
```

5.1.5 Configuration

Area	Where documented
Monitoring intervals, directories, capture	Daemon Monitoring
Graphite and Prometheus metrics	Metrics
Processlist capture	Processlist Monitoring
Performance Schema	Performance Schema
Multi-cluster topology	Clustering
Log files and debug tools	Logs and Troubleshooting
Alert channels and triggers	Alerting

5.2.1.1 Server daemon configuration (`monitoring,log`)

`log-file`

Item	Value
Description	Write messages to log file per default messages are output to stdout.
Type	string
Example	"/var/log/replication-manager.log"

`log-file` (2.0), `logfile` (0.6)

Item	Value
Description	Write messages to log file per default messages are output to stdout
Type	string
Example	`/var/log/replication-manager.log`

`log-level`

Item	Value
Description	Log verbosity level, og level >3 are very verbose and print the server monitoring workflow only to use in debugging
Type	int
Example	1 to 7

`log-syslog` (2.0)

Item	Value
Description	Mirror all log output to the local UDP syslog port in addition to the normal log file. Allows log aggregation systems (Graylog, Splunk, ELK) to receive replication-manager events via syslog.
Type	Boolean
Default	`false`

`log-rotate-max-age` (2.1)

Item	Value
Description	Rotate after this number of days
Type	integer
Default	7

`log-rotate-max-backup` (2.1)

Item	Value
Description	Keep this number of log files
Type	integer
Default	7

`log-rotate-max-size` (2.1)

Item	Value
Description	Rotate after this size in Mb
Type	integer
Default	5

`log-sql-in-monitoring` (2.1)

Item	Value
Description	Log all SQL queries send to backend for monitoring servers
Type	boolean
Default	false

The sql_general.log can be found under each cluster directory and can be use to track what was trigger to backend during failover, rejoin step

`verbose` (2.1)

Item	Value
Description	Add more debugging information
Type	int
Example	1

`memprofile`

Item	Value
Description	Write a memory profile to a file readable by pprof.
Type	string
Default	`/tmp/repmgr.mprof`

`monitoring-datadir` (2.0), `working-directory` (1.1)

Item	Value
Description	Path to write temporary and persistent files
Type	string
Example	`/var/lib/replication-manager`

`monitoring-sharedir` (2.0), `share-directory` (1.1)

Item	Value
Description	Path to the share files provided with packaging with no write access
Type	string
Default Value	`/usr/share/replication-manager`

`monitoring-basedir` (2.0)

Item	Value
Description	Path to a basedir where a data and share directory can be found, used mostly for developer or collocation of the product with a tar.gz deployment.
Type	string
Default Value	`/usr/local/replication-manger`

`monitoring-save-config` (2.1)

Item	Value
Description	Save configuration changes to /clusterd
Type	bool
Default Value	false

`monitoring-ticker` (2.0), `read-timeout` (1.1)

Item	Value
Description	Monitoring interval in seconds.
Type	integer
Default Value	2

`monitoring-write-heartbeat` (2.0)

Item	Value
Description	Inject heartbeat into proxy or via external VIP. Without this option long inactive write period and database expire log days can make switchover to failed without founding last binlog position
Type	bool
Default Value	2

`monitoring-write-heartbeat-credential` (2.0)

Item	Value
Description	Credential DB user to inject heartbeat into proxy or via external VIP.
Type	bool

Item	Value
Default Value	2

`title`

Item	Value
Description	An explicit description of the managed cluster
Type	string
Default Value	`CRM production MariaDB cluster`

`monitoring-schema-change` (2.0)

Item	Value
Description	Monitor schema change
Type	bool
Default Value	true

This parameter is needed for alerting on schema change and for sharding proxy to push DDL change

`monitoring-schema-change-script` (2.1)

Item	Value
Description	Monitor schema change external script
Type	string
Default Value	“”

`monitoring-performance-schema` (2.1)

Item	Value
Description	Monitor performance schema
Type	bool
Default Value	true

This enable to get slow queries templating from performance schema

`monitoring-processlist` (2.1)

Item	Value
Description	Enable capture 50 longest process via processlist
Type	bool
Default Value	true

`monitoring-queries` (2.1)

Item	Value
Description	Monitor long queries
Type	bool
Default Value	true

Currently unused

`monitoring-query-rules` (2.1)

Item	Value
Description	Monitor query routing from proxies
Type	bool
Default Value	true

Enable to build a consolidated view of multiple cluster proxies query rules

`monitoring-capture` (2.1)

Item	Value
Description	Enable capture on internal error for 5 monitor loops
Type	bool
Default Value	true

The capture files are found in the cluster datadir

Capture are show slave status, show processlist, show global status, show innodb status

`monitoring-capture-trigger` (2.1)

Item	Value
Description	Enable capture on internal error for 5 monitor loops
Type	string
Default Value	"ERR00076,ERR00041"

Trigger to capture on max connections threshold and slave delay

`monitoring-capture-file-keep` (2.1)

Item	Value
Description	Purge capture file keep that number of them
Type	integer
Default Value	5

5.2.1.1.1 Deprecated

`daemon` (2.0)

Item	Value
Description	Tell to monitor in post 2.0 release
Type	integer
Default Value	1

replication-manager embeds a graphite database to store metrics about databases and proxies status.

replication-manager 2.1 expose Prometheus metrics for all servers under clusters

`graphite-embedded` (1.1)

Item	Value
Description	Enable Graphite Server.
Type	Boolean
Default Value	false

`graphite-metrics` (1.1)

Item	Value
Description	Collect and send metrics to a Graphite Server.
Type	Boolean
Default Value	false

`graphite-carbon-host` (1.1)

Item	Value
Description	Host of the Graphite server to send metrics.
Type	String
Default Value	"127.0.0.1"

`graphite-carbon-port` (1.1)

Item	Value
Description	TCP & UDP port of the Graphite server to send metrics.
Type	Integer
Default Value	2003

`graphite-carbon-api-port` (1.1)

Item	Value
Description	Graphite Carbon API port.
Type	Integer
Default Value	10002

`graphite-carbon-link-port` (1.1)

Item	Value
Description	Graphite Carbon Link port.
Type	Integer
Default Value	7002

`graphite-carbon-pickle-port` (1.1)

Item	Value
Description	Graphite Carbon Pickle port.
Type	Integer
Default Value	2004

`graphite-carbon-pprof-port` (1.1)

Item	Value
Description	Graphite Carbon pprof port.
Type	Integer
Default Value	7007

`graphite-carbon-server-port` (1.1)

Item	Value
Description	Graphite Carbon HTTP port.
Type	Integer
Default Value	10003

HTTP Graph are display via the giraffe JS library.

Customizing the trends can be done modifying.

`/usr/share/replication-manager/dashboard/static/graph.js`

Set the replication-manager host or IP address to tell where to found the graphite server

Replication and some databases status metrics are pushed inside carbon, the metrics are pushed with the server-id prefix name. to get unicity against nodes

[Figure: graphs]

We can advise usage of Statd and Collectd on each database node and send metrics to replication-manager whisper port to add extra system metrics to the cluster.

One can create custom dashboard via Grafana. The port to use as direct data source in grafana is drive by `graphite-carbon-api-port` default 10002.

`http://127.0.0.1:10002/`

[Figure: source]

In **replication-manager 2.0** more metrics are pushed to graphite using prefix

- ☒ Each database global status mysql.host.mysql_global_status_(metric)
- ☒ Each database global variables mysql.host.mysql_global_variables_(metric)
- ☒ Each database show slave status mysql.host.mysql_slave_status_(metric)
- ☒ Each database show engine InnoDB mysql.host.engine_innodb_(metric)
- ☒ Top 20 database performance schema mysql.host.pfs.digest_(metric)

Some Grafana dashboard using such metric names can be found in the share directory of **replication-manager**.

[Figure: source]

[Figure: source]

Prometheus URI available 2.1 http://demo.signal18.io/api/prometheus

replication-manager can monitor long-running queries and sleeping transactions using the processlist. This feature captures the longest queries or transactions across all monitored database servers and makes them available for analysis.

5.2.3.1 Basic Processlist Monitoring

`monitoring-processlist` (2.0)

Item	Value
Description	Enable capture of longest queries or transactions via processlist
Type	boolean
Default Value	true

When enabled, **replication-manager** captures processlist information during each monitoring loop. The number of processes captured is controlled by `monitoring-processlist-limit`.

`monitoring-processlist-limit` (2.0)

Item	Value
Description	Maximum number of processes to capture per server
Type	string
Default Value	"50"

Limiting the processlist size reduces monitoring bandwidth and overhead while still capturing the most significant queries and transactions.

`monitoring-processlist-information-schema` (3.0)

Item	Value
Description	Use INFORMATION_SCHEMA.PROCESSLIST instead of SHOW FULL PROCESSLIST
Type	boolean
Default Value	true

Using INFORMATION_SCHEMA allows **replication-manager** to join with INNODB_TRX tables to retrieve transaction information. This is required for transaction monitoring features.

5.2.3.2 Transaction Monitoring

`monitoring-processlist-transactions` (3.1)

Item	Value
Description	Report transactions in process, including sleeping transactions
Type	boolean
Default Value	false

When enabled, **replication-manager** monitors both active queries and long-running transactions. Sleeping transactions are particularly important to identify as they can hold locks and block other operations.

The processlist will be sorted by transaction duration rather than query duration, making it easy to identify transactions that have been open for a long time.

Key features: - Captures sleeping transactions that are holding InnoDB locks - Shows transaction isolation level, tables locked, rows modified - Displays transaction start time and duration - Identifies read-only vs read-write transactions

Information captured for each transaction: - `trx_time`: Transaction duration in seconds - `trx_isolation_level`: Transaction isolation level (READ-COMMITTED, REPEATABLE-READ, etc.) - `trx_tables_in_use`: Number of tables currently in use - `trx_tables_locked`: Number of tables locked - `trx_lock_structs`: Number of lock structures - `trx_lock_memory_bytes`: Memory used for locks - `trx_rows_locked`: Number of rows locked - `trx_rows_modified`: Number of rows modified - `trx_is_read_only`: Whether transaction is read-only (1) or read-write (0)

`monitoring-processlist-inactive` (2.0)

Item	Value
Description	Show inactive connections (sleeping queries) in processlist
Type	boolean
Default Value	false

When enabled, all connections are shown including those with Command='Sleep'. When combined with `monitoring-processlist-transactions`, this shows sleeping transactions that may be holding locks.

5.2.3.3 Query Monitoring

For monitoring slow queries specifically, see the long query monitoring parameters:

- `monitoring-queries`: Enable long query monitoring
- `monitoring-long-query-time`: Threshold in milliseconds
- `monitoring-long-query-with-process`: Use processlist to capture slow queries
- `monitoring-long-query-with-table`: Use slow_log table to capture slow queries

5.2.3.4 Usage Example

To monitor sleeping transactions that may be causing blocking:

```
monitoring-processlist = true
monitoring-processlist-transactions = true
monitoring-processlist-information-schema = true
monitoring-processlist-limit = "50"
```

This configuration will: - Capture the 50 longest-running transactions per server - Include sleeping transactions holding locks - Join with INNODB_TRX to get detailed transaction metrics - Sort by transaction duration to highlight long-running transactions

5.2.3.5 API Access

Transaction processlist data is available via the REST API at:

```
GET /api/clusters/{cluster}/servers/{server}/processlist
```

The web UI displays this information in the server detail view under "Processlist" tab.

5.2.3.6 Performance Considerations

Querying INFORMATION_SCHEMA.PROCESSLIST and INNODB_TRX tables has minimal overhead but can be noticeable on servers with thousands of connections. Adjust `monitoring-processlist-limit` to control the impact.

On MariaDB, the query uses TIME_MS for millisecond precision. On MySQL/Percona 8.0+, TIME_MS is also available. On older versions, TIME (seconds) is used.

replication-manager can collect advanced metrics from MySQL/MariaDB Performance Schema, including mutex waits, latch (rwlock) waits, and memory usage. These metrics are collected during each monitoring loop and exported to Graphite for trending analysis.

5.2.4.1 Basic Performance Schema Monitoring

`monitoring-performance-schema` (2.0)

Item	Value
Description	Enable Performance Schema monitoring
Type	boolean
Default Value	true

When enabled, **replication-manager** queries Performance Schema tables to collect query statistics and performance metrics.

`monitoring-performance-schema-instruments` (2.0)

Item	Value
Description	Monitor Performance Schema instruments configuration
Type	boolean
Default Value	true

Monitors the enabled/disabled state of Performance Schema instruments from `setup_instruments` table.

5.2.4.2 Mutex Wait Monitoring

`monitoring-performance-schema-mutex` (3.1)

Item	Value
Description	Monitor InnoDB mutex wait events
Type	boolean
Default Value	true

When enabled, **replication-manager** collects mutex wait statistics from Performance Schema and includes them in global status metrics sent to Graphite.

Data collected: - Queries `performance_schema.events_waits_summary_global_by_event_name` - Filters for `EVENT_NAME LIKE 'wait/synch/mutex/innodb%'` - Collects `COUNT_STAR` for each mutex event - Metric names formatted as `WAIT_SYNCH_MUTEX_INNODB_*`

Use cases: - Identify InnoDB mutex contention - Diagnose performance bottlenecks caused by mutex waits - Track mutex wait trends over time - Correlate mutex waits with application workload patterns

Requirements: - Performance Schema must be enabled (`performance_schema = ON`) - Requires MariaDB 10.5+ or MySQL 5.7+

5.2.4.3 Latch (RWLock) Wait Monitoring

`monitoring-performance-schema-latch` (3.1)

Item	Value
Description	Monitor InnoDB latch (rwlock) wait events
Type	boolean
Default Value	true

When enabled, **replication-manager** collects read-write lock (rwlock) wait statistics from Performance Schema and includes them in global status metrics sent to Graphite.

Data collected: - Queries `performance_schema.events_waits_summary_global_by_event_name` - Filters for `EVENT_NAME LIKE 'wait/synch/rwlock/innodb%'` - Collects `COUNT_STAR` for each rwlock event - Metric names formatted as `WAIT_SYNCH_RWLOCK_INNODB_*`

Use cases: - Identify InnoDB rwlock contention - Diagnose performance issues with concurrent reads/writes - Track latch wait trends over time - Analyze impact of read-heavy vs write-heavy workloads

Requirements: - Performance Schema must be enabled (`performance_schema = ON`) - Requires MariaDB 10.5+ or MySQL 5.7+

5.2.4.4 Performance Schema Memory Monitoring

`monitoring-performance-schema-memory` (3.1)

Item	Value
Description	Monitor Performance Schema memory usage
Type	boolean
Default Value	true

When enabled, **replication-manager** collects memory usage statistics from Performance Schema memory instrumentation and includes them in global status metrics sent to Graphite.

Data collected: - Queries `performance_schema.memory_summary_global_by_event_name` - Collects memory allocation statistics by event type - Tracks `CURRENT_COUNT_USED`, `CURRENT_NUMBER_OF_BYTES_USED`, `HIGH_COUNT_USED`, `HIGH_NUMBER_OF_BYTES_USED`

Use cases: - Monitor memory consumption by InnoDB subsystems - Identify memory leaks or excessive allocations - Track memory usage trends over time - Diagnose out-of-memory conditions

Requirements: - Performance Schema must be enabled (`performance_schema = ON`) - Memory instrumentation must be enabled in Performance Schema consumers - Requires MariaDB 10.5+ or MySQL 5.7+

5.2.4.5 Configuration Example

To enable all Performance Schema advanced monitoring features:

```
monitoring-performance-schema = true
monitoring-performance-schema-instruments = true
monitoring-performance-schema-mutex = true
monitoring-performance-schema-latch = true
monitoring-performance-schema-memory = true
```

To disable mutex/latch monitoring but keep memory monitoring:

```
monitoring-performance-schema = true
monitoring-performance-schema-mutex = false
monitoring-performance-schema-latch = false
monitoring-performance-schema-memory = true
```

5.2.4.6 Metrics Export

When these features are enabled, **replication-manager** adds the collected metrics to the global status query result. Metrics are then:

1. Stored in the embedded Graphite database (if enabled)
2. Exported to external Graphite server (if configured)
3. Available via Prometheus metrics endpoint
4. Visible in Grafana dashboards

Metric naming convention: - Mutex waits: `mysql.{server_id}.mysql_global_status_wait_synch_mutex_innodb_{ev` - Latch waits: `mysql.{server_id}.mysql_global_status_wait_synch_rwlock_innodb_{event}` - Memory: `mysql.{server_id}.mysql_global_status_memory_{event}`

5.2.4.7 Performance Considerations

Impact: - Mutex/latch monitoring adds one UNION clause to the global status query per feature - Performance Schema queries are fast (indexed) but can add milliseconds to monitoring loop - Memory monitoring queries are more expensive than mutex/latch queries

Recommendations: - Enable these features in production to identify contention issues - Monitor the monitoring loop duration to ensure it stays within acceptable limits - Disable specific features if monitoring overhead becomes a concern - Use Graphite/Grafana to visualize trends rather than real-time analysis

5.2.4.8 Database Configuration

Ensure Performance Schema is properly configured in your database server:

```
# Enable Performance Schema
performance_schema = ON

# Enable wait event instruments (for mutex/latch monitoring)
performance-schema-instrument = 'wait/synch/%=ON'

# Enable memory instruments (for memory monitoring)
performance-schema-instrument = 'memory/%=ON'

# Enable consumers
performance-schema-consumer-events-waits-current = ON
performance-schema-consumer-events-waits-history = ON
performance-schema-consumer-events-waits-history-long = ON
```

5.2.4.9 Troubleshooting

Metrics not appearing: - Verify Performance Schema is enabled: `SHOW VARIABLES LIKE 'performance_schema'` - Check instruments are enabled: `SELECT * FROM performance_schema.setup_instruments WHERE NAME LIKE 'wait/synch/%'` - Ensure `graphite-metrics = true` or Prometheus endpoint is configured - Check **replication-manager** logs for SQL errors

High monitoring overhead: - Reduce `monitoring-wait-retry` interval - Disable mutex/latch monitoring if not needed - Consider enabling only on specific clusters, not all

No mutex/latch events: - If `COUNT_STAR = 0` for all events, there is genuinely no wait contention - This is expected on idle or low-traffic databases - Metrics only appear when there is actual mutex/latch activity

5.2.5.0.1 Active standby with external arbitrator

When inside a single DC, we can use a single replication-manager that performs failover using keepalived, corosync, openha or etcd, but if you run on 2 DC it is possible to run two replication-manager instances in the same infrastructure. Both instances will use a heartbeat mechanism via the http protocol.

Make sure you activate the web server of **replication-manager**.

To configure active standby and arbitration, use the following settings on each **replication-manager**:

`arbitration-external` (1.0)

Item	Value
Description	Enable external arbitration on split brain.
Type	Boolean
Default Value	false

`arbitration-external-hosts` (1.0)

Item	Value
Description	Hosts list of external arbitrator service.
Type	List
Default Value	"arbitrator-fr.signal18.io,arbitrator-us.signal18.io"

Define one secret arbitration-external-secret it should be unique across all users of replication-manager, it is use by the arbitrator to identify your cluster, organization name and random alpha-numeric are very welcome.

`arbitration-external-secret` (1.0)

Item	Value
Description	Full path to an alerting script.
Type	String
Default Value	"cluster01.signal18.io"

Give each **replication-manager** server a different value for arbitration-external-unique-id and instruct it for the other **replication-manager** in the cluster.

`arbitration-external-id` (1.0)

Item	Value
Description	Unique value on each replication-manager.
Type	Integer
Default Value	0

`arbitration-peer-hosts` (1.0)

Item	Value
Description	Peer replication-manager node.

Item	Value
Type	String
Default Value	"replication-manager01:10002"

`arbitration-failed-master-script` (2.1)

Item	Value
Description	External script when a master lost arbitration during split brain
Type	String
Default Value	""

5.2.5.0.2 Internals

On each instance instruct it's peer replication-manager node

On instance "127.0.0.1:10001" arbitration-peer-hosts ="127.0.0.1:10002"

On instance "127.0.0.1:10002" arbitration-peer-hosts ="127.0.0.1:10001"

Once done start the fist replication-manager.

```
INFO[2017-03-20T09:48:38+01:00] [cluster_test_2_nodes] ERROR :Get http://127.0.0.1:10001/heartbeat: dial
INFO[2017-03-20T09:48:38+01:00] [cluster_test_2_nodes] INFO : Splitbrain
INFO[2017-03-20T09:48:38+01:00] [cluster_test_3_nodes] CHECK: External Abitration
INFO[2017-03-20T09:48:38+01:00] [cluster_test_3_nodes] INFO :Arbitrator say winner
INFO[2017-03-20T09:48:40+01:00] [cluster_test_2_nodes] ERROR :Get http://127.0.0.1:10001/heartbeat: dial
INFO[2017-03-20T09:48:40+01:00] [cluster_test_2_nodes] INFO : Splitbrain
INFO[2017-03-20T09:48:40+01:00] [cluster_test_3_nodes] CHECK: External Abitration
INFO[2017-03-20T09:48:40+01:00] [cluster_test_3_nodes] INFO Arbitrator say :winner
```

The split brain detection is trigger because your are the first instance to start, the peer **replication-manager** is not joinable so it ask for an arbitration to arbitration-external-hosts where the arbitrator daemon is running.

The arbitrator will enable that node to enter Active Mode

When you start the peer replication-manager server, the split brain is resolve and replication-manager will detect an other active instance is running so it will get the Standby status.

> failover in a **replication-manager** cluster is requesting an arbitration. If arbitrator can't be contacted, you can come back to normal command line mode to failover but make sure you stopped all other running replication-manager.

5.2.5.0.3 Run a private arbitrator

`arbitrator-bind-address` (2.0 arb)

Item	Value
Description	Arbitrator bind address
Type	String
Default Value	"0.0.0.0:10001"

`arbitrator-driver` (2.0 arb)

Item	Value
Description	Arbitrator backend storage type
Type	Enum
List Values	sqlite,mysql
Default Value	“sqlite”

Configuration for MySQL backend should be enable via this configuration.

```
[arbitrator]
title = "arbitrator"
db-servers-hosts = "192.168.0.201:3306"
db-servers-credential = "user:password"
[default]
```

Start it via

```
/usr/bin/replication-manager-arb arbitrator --arbitrator-port=80
```

5.3.1 Logs and Troubleshooting

replication-manager writes several log files and diagnostic files. Knowing where each one lives and what it contains is the first step when investigating unexpected behaviour.

5.3.2 replication-manager Daemon Log

The main process log captures everything replication-manager does: topology discovery, failover decisions, API calls, job execution, and internal errors.

Setting	Effect
`log-file`	Path to write the daemon log. Defaults to stdout if not set. Example: `/var/log/replication-manager.log`
`log-level`	Verbosity from 1 (errors only) to 7 (full monitoring loop trace). Levels above 3 print every monitoring tick and are intended for debugging only.
`log-syslog`	Mirror all log output to the local UDP syslog port in addition to the log file. Useful for feeding Graylog, Splunk, or ELK without a file agent.
`log-rotate-max-age`	Rotate logs older than this many days. Default: `7`.
`log-rotate-max-backup`	Keep at most this many rotated log files. Default: `7`.
`log-rotate-max-size`	Rotate when the log file exceeds this size in MB. Default: `5`.
`verbose`	Add extra debugging output on top of `log-level`.

All of these are set in the global (non-cluster) section of the configuration file. See Daemon Monitoring for the full parameter reference.

5.3.2.1 Typical log-level guide

Level	When to use
`1`	Production — errors and critical events only
`2`	Default — normal operations including state changes
`3`	Failover debugging — shows decision logic
`5-7`	Full trace — every SQL sent to monitored servers, every monitoring tick

---

5.3.3 Per-Cluster Diagnostic Files

These files are written inside the replication-manager data directory, one subdirectory per cluster:

```
{monitoring-datadir}/{cluster-name}/
   sql_general.log      All SQL sent to backend servers during monitoring
   capture_*.log        Snapshot dumps triggered by error codes
   {host}_{port}/
       ...              Per-server config and state files
```

5.3.3.1 sql_general.log

Enabled by `log-sql-in-monitoring = true`. Records every SQL query that replication-manager sends to monitored servers (SHOW SLAVE STATUS, SHOW GLOBAL STATUS, etc.).

This log is valuable during failover and rejoin analysis: you can replay exactly what replication-manager queried and in what order to reconstruct the decision path.

```
log-sql-in-monitoring = true
```

5.3.3.2 Capture files

When `monitoring-capture` is enabled, replication-manager automatically dumps a diagnostic snapshot whenever a monitored error code fires. Each capture saves:

- `SHOW SLAVE STATUS`
- `SHOW PROCESSLIST`
- `SHOW GLOBAL STATUS`
- `SHOW ENGINE INNODB STATUS`

Capture is triggered by the error codes listed in `monitoring-capture-trigger` (default: `ERR00076,ERR00041` — max connections threshold and slave delay). The dump runs for 5 monitoring ticks and the oldest files are purged once the count exceeds `monitoring-capture-file-keep`.

Setting	Default	Effect
`monitoring-capture`	`true`	Enable automatic capture on error
`monitoring-capture-trigger`	`"ERR00076,ERR00041"`	Comma-separated error codes that start a capture
`monitoring-capture-file-keep`	`5`	Maximum number of capture files to retain

5.3.4 Memory Profiling

```
memprofile = "/tmp/repmgr.mprof"
```

When set, replication-manager writes a Go memory profile to this path. Load it with `go tool pprof` to investigate memory growth:

```
go tool pprof /tmp/repmgr.mprof
```

5.3.5 Common Troubleshooting Steps

5.3.5.1 Replication-manager won't connect to a server

1. Set `log-level = 5` temporarily and restart
2. Watch the daemon log for the connection attempt and error
3. Verify `db-servers-hosts`, `db-servers-credential`, and network access
4. Check that `log-sql-in-monitoring = true` to see the exact queries being sent

5.3.5.2 Failover triggered unexpectedly

1. Check the daemon log around the time of the event — look for the `ERR` code that fired
2. Check the capture files in `{monitoring-datadir}/{cluster-name}/` — they contain the exact server state at the moment of the error

3. Look at `monitoring-capture-trigger` to see which codes arm a capture

5.3.5.3 Alert firing but no notification received

1. Verify the error code is in `monitoring-alert-trigger` (see Alerting Configuration Guide)
2. Check if the code is in `monitoring-ignore-errors` (suppression list)
3. Check if `scheduler-alert-disable` is active (scheduled blackout)
4. Set `log-level = 3` to see alert dispatch in the daemon log

5.3.5.4 High log volume

- Lower `log-level` back to `1` or `2` for production
- Enable `log-rotate-max-size` to cap file growth
- Use `log-syslog` with a log aggregator that handles volume and retention externally

6.1.1 Maintenance

replication-manager embeds a cluster scheduler that automates database maintenance tasks: backups, log collection, table optimization, rolling restarts, and more. The scheduler is disabled by default — each task is individually enabled and given its own cron expression.

6.1.2 How the Job System Works

Maintenance tasks are coordinated through a message queue table that replication-manager creates and manages on every monitored database server. The flow is:

[Figure: dbjobnodetorepman]

1. **Scheduler fires** — the Robfig cron scheduler triggers a task at the configured interval
2. **Job is enqueued** — replication-manager opens a TCP connection to the database server and inserts a row into `replication_manager_schema.jobs`. All writes to this table use `sql_log_bin=0` so they do not produce binlog events
3. **A port is reserved** — replication-manager picks a free port from `scheduler-db-servers-sender-ports` to receive the streamed result. It passes this address and port in the job row
4. **The dbjobs script runs** — a shell script on the database host reads the job table, picks up the pending task, marks it `processing`, executes the work, and streams the output back to replication-manager via `socat`
5. **Result is received** — replication-manager accepts the stream on the reserved TCP port and writes it to the appropriate destination (log file, backup directory, etc.)
6. **Job is closed** — the script writes the result into the `result` field and sets `done=1`; replication-manager confirms completion on the next check

6.1.2.1 Jobs Table

```
CREATE TABLE replication_manager_schema.jobs (
  id      INT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
  task    VARCHAR(20),
  port    INT,
  server  VARCHAR(255),
  done    TINYINT      NOT NULL DEFAULT 0,
  state   TINYINT      NOT NULL DEFAULT 0,
  result  MEDIUMTEXT,
  payload MEDIUMTEXT,
  start   DATETIME,
  end     DATETIME,
  KEY idx1 (task, done),
  KEY idx2 (result(1), task),
  KEY idx3 (task, state),
  UNIQUE  (task)
) ENGINE=InnoDB;
```

The `state` column tracks the task lifecycle: - `0` — available (pending) - `1` — processing - `3` — finished recently (post-job check) - `5` — halted (replication-manager restarted mid-task; aborted for safety)

6.1.3 Scheduled Tasks

Task	Config key	Description
Logical backup	`scheduler-db-servers-logical-backup`	mydumper or mysqldump full backup
Physical backup	`scheduler-db-servers-physical-backup`	mariabackup or xtrabackup xbstream
Log collection	`scheduler-db-servers-logs`	Fetch error log and slow query log from each server
Log table rotate	`scheduler-db-servers-logs-table-rotate`	Rotate `mysql.general_log` / `slow_log` tables
Optimize	`scheduler-db-servers-optimize`	`OPTIMIZE TABLE` on all schemas
Analyze	`scheduler-db-servers-analyze`	`ANALYZE TABLE` on all schemas
Rolling restart	`scheduler-rolling-restart`	Restart replicas then primary one at a time
Rolling reprov	`scheduler-rolling-reprov`	Re-provision services on a rolling basis (pro)
SSH job runner	`scheduler-jobs-ssh`	Trigger the dbjobs script via SSH (osc)

Each task has a matching `-cron` key for its schedule, e.g.:

```
monitoring-scheduler                      = true
scheduler-db-servers-logical-backup       = true
scheduler-db-servers-logical-backup-cron  = "0 0 1 * * *"   # 01:00 every day

scheduler-db-servers-logs                 = true
scheduler-db-servers-logs-cron            = "0 0 */6 * * *"  # every 6 hours

scheduler-db-servers-optimize             = true
scheduler-db-servers-optimize-cron        = "0 0 3 * * 0"    # Sundays at 03:00
```

6.1.4 How the dbjobs Script Is Delivered

The script that runs on each database node is a bash script named `dbjobs_new`. How it gets there depends on the orchestration mode:

6.1.4.1 replication-manager-pro (container mode)

In pro mode with OpenSVC, Kubernetes, or local orchestration, the `dbjobs_new` script is packaged into a gzip'd config tarball. An **init container** that shares the same network namespace as the database container downloads this tarball from replication-manager at startup and unpacks it. The script runs inside the container on the same host as the database process, connecting to the database over `127.0.0.1`.

6.1.4.2 replication-manager-osc (SSH mode)

In osc mode, or when using the `onpremise` orchestrator in pro, replication-manager delivers and runs the script over SSH:

1. The script is unpacked into the server's data directory at `{datadir}/init/init/dbjobs_new`
2. On each scheduler tick (controlled by `scheduler-jobs-ssh-cron`), replication-manager SSHs into the database host, prepends the environment variable block, and pipes the script through the remote shell

Enable SSH job execution:

```
scheduler-jobs-ssh       = true
scheduler-jobs-ssh-cron  = "*/1 * * * * *"   # run every minute

onpremise-ssh            = true
onpremise-ssh-port       = 22
onpremise-ssh-credential = "deploy"           # user; key auth used if no password
onpremise-ssh-private-key = "/home/repman/.ssh/id_rsa"
```

To use a custom script instead of the generated one:

```
onpremise-ssh-db-job-script = "/opt/custom/dbjobs.sh"
```

6.1.5 Environment Variables Injected by replication-manager

When the script runs via SSH, replication-manager prepends a block of `export` statements before the script body. The script can use these variables without hardcoding any credentials or addresses:

Variable	Content
`REPLICATION_MANAGER_URL`	Full HTTPS URL of the replication-manager API (`https://host:port`)
`REPLICATION_MANAGER_URL_HOST`	API hostname / IP (`monitoring-address`)
`REPLICATION_MANAGER_URL_PORT`	API port (`api-port`)
`REPLICATION_MANAGER_USER`	API admin username
`REPLICATION_MANAGER_PASSWORD`	API admin password
`REPLICATION_MANAGER_HOST_NAME`	Database server hostname
`REPLICATION_MANAGER_HOST_PORT`	Database server port
`REPLICATION_MANAGER_HOST_USER`	Database user replication-manager connects with
`REPLICATION_MANAGER_HOST_PASSWORD`	Database password (also exported as `MYSQL_ROOT_PASSWORD`)
`MYSQL_ROOT_PASSWORD`	Alias for `REPLICATION_MANAGER_HOST_PASSWORD`
`REPLICATION_MANAGER_CLUSTER_NAME`	Cluster section name from the config file

These variables allow the script to call back into the replication-manager API, connect to the local database, and stream results without any static configuration.

6.1.6 How Results Are Streamed Back

Tasks that produce output (backups, log fetches) stream their data back to replication-manager using `socat`. replication-manager opens a TCP listener on a port from `scheduler-db-servers-sender-ports` and records that address in the job row. The script reads the address from the row and pipes the output directly:

```
# Example: stream a physical backup back to replication-manager
mariabackup --stream=xbstream ... | socat -u stdio TCP:$ADDRESS
```

replication-manager writes the received stream to: - **Logs** (error log, slow query log): `{datadir}/{cluster}/{host_port}/` - **Backups**: `{datadir}/Backups/{cluster}/{host_port}/` - **Restic archive**: streamed into Restic after local write (if `backup-restic = true`)

Configure the TCP address replication-manager advertises to the database nodes:

```
monitoring-address              = "10.0.1.5"          # must be reachable from DB hosts
scheduler-db-servers-sender-ports = "4000,4001,4002"  # port pool; one per concurrent task
```

If `scheduler-db-servers-sender-ports` is not set, replication-manager picks any available port on the host. In firewalled environments, setting an explicit port pool allows precise firewall rules.

6.2.1.1 Scheduler

`monitoring-scheduler` (2.1)

Item	Value
Description	"Enable task scheduler"
Type	boolean
Default Value	true

6.2.1.2 CRON Expression Format

A cron expression represents a set of times, using 6 space-separated fields.

Field name	Mandatory?	Allowed values	Allowed special characters
Seconds	Yes	0-59	* / , -
Minutes	Yes	0-59	* / , -
Hours	Yes	0-23	* / , -
Day of month	Yes	1-31	* / , - ?
Month	Yes	1-12 or JAN-DEC	* / , -
Day of week	Yes	0-6 or SUN-SAT	* / , - ?

Note: Month and Day-of-week field values are case insensitive. "SUN", "Sun", and "sun" are equally accepted.

Special Characters Asterisk (*)

The asterisk indicates that the cron expression will match for all values of the field; e.g., using an asterisk in the 5th field (month) would indicate every month.

Slash (/)

Slashes are used to describe increments of ranges. For example 3-59/15 in the 1st field (minutes) would indicate the 3rd minute of the hour and every 15 minutes thereafter. The form "*/…" is equivalent to the form "first-last/…", that is, an increment over the largest possible range of the field. The form "N/…" is accepted as meaning "N-MAX/…", that is, starting at N, use the increment until the end of that specific range. It does not wrap around.

Comma (,)

Commas are used to separate items of a list. For example, using "MON,WED,FRI" in the 5th field (day of week) would mean Mondays, Wednesdays and Fridays.

Hyphen (-)

Hyphens are used to define ranges. For example, 9-17 would indicate every hour between 9am and 5pm inclusive.

Question mark (?)

Question mark may be used instead of '*' for leaving either day-of-month or day-of-week blank.

6.2.1.3 CRON jobs configuration

`scheduler-db-servers-logical-backup` (2.1)

Item	Value
Description	Schedule logical backup

Item	Value
Type	boolean
Default Value	true

Logical backup in **Replication-manager (2.1)** are done via mysqldump client from the replication-manager host to the master database with non blocking and replication position options.

It's alway safer to backup the master despite it was advice to backup slaves, Signal18 don't think this a best practice to backup slave.

Yes Logical backup are slow, but for good reason it's supposed not to get all your databases CPU and IO so that you may take it without trouble to your workload!

Logical backup are needed for long time archiving of your data, it's the only way to prevent physical data corruption that can happen with hardware issues, databases bugs, Kernel and FS bugs.

`scheduler-db-servers-logical-backup-cron` (2.1)

Item	Value
Description	Logical backup cron expression represents a set of times, using 6 space-separated fields.
Type	string
Default Value	"0 1 * * 0" Once every week

`scheduler-db-servers-physical-backup` (2.1)

Item	Value
Description	Schedule physical backup
Type	boolean
Default Value	true

`scheduler-db-servers-physical-backup-cron` (2.1)

Item	Value
Description	Logical backup cron expression represents a set of times, using 6 space-separated fields or expression.
Type	string
Default Value	"0 0 0 * * *" Once per day at midnight

Physical backups are day to day backups in MySQL & MariaDB and works like FS snapshots, tracking database changes while copying physical pages. It is non blocking when InnoDB storage engine is mostly used for most tables , big non transactional table will be lock for WRITES a (READ LOCK) will still enable to READ non transactional tables.

Physical backups like innodbhotbackup, xtrabackup and mariabackup need to take place on the database host itself but can be stream to **replication-manager** for archiving or reused for re-seeding new slaves or old died masters.

`scheduler-db-servers-logs` (2.1)

Item	Value
Description	Schedule database logs fetching
Type	boolean
Default Value	true

`scheduler-db-servers-logs-cron` (2.1)

Item	Value
Description	Logical backup cron expression represents a set of times, using 6 space-separated fields or expression.
Type	string
Default Value	"@every 10m" Every 10 minutes

replication-manager take care of monitoring replication but can be extend to a central point for database error logs and query log analyze, using this option the logs of the database are requested on this schedule to replication-manager for a central point of analyze and alerting. We will provide the tools to classify the queries and understand databases miss configuration and workload

Our donor script will purge the log on the database server after sent out so that growing logs don't consume the disk space and memory of the db host itself.

`scheduler-db-servers-optimize` (2.1)

Item	Value
Description	"Schedule database optimize"
Type	boolean
Default Value	true

`scheduler-db-servers-optimize-cron` (2.1)

Item	Value
Description	Logical backup cron expression represents a set of times, using 6 space-separated fields or expression.
Type	string
Default Value	""0 0 3 1 * 5"" Every month the first at 3AM on Saturday

scheduler-db-servers-optimize-cron",

6.2.2.1 Backups

Supported backups methods are mysqldump, mydumper, xtrabackup, mariabackup

A physical and a logical backup can be schedule via configuration variables.

replication-manager store the last backup into a data directory of replication-manager under: /backups/// From **replication-manager 2.1** it is possible to store backups using s3 protocol and backups are available via fuse mounting directory /s3///

In **replication-manager 2.1** >, backups can be archive and purge via interaction with the Restic binaries (need pre install) that interconnect to an s3 protocol bucket, Restic enable block level encryption & deduplication and will store the entire backup directory for mixing multiple files backups like mydumper.

When backup are executed they are stored compressed under the replication-manager datadir

`<data_directory>/backups/<cluster_name>/<server_name>_<server_port>.`

The can be used to provision new nodes or restore broken slaves,

But long archiving may be required. A full restic integration is used for this.

`backup-logical-type` (2.1)

Item	Value
Description	river,mysqldump,mydumper
Type	string
Default Value	"mysqldump"

mysqldump only so far

`backup-physical-type` (2.1)

Item	Value
Description	xtrabackup,mariabackup
Type	string
Default Value	"xtrabackup"

`backup-mysqldump-options` (2.1)

Item	Value
Description	MySQLDump Extra Options
Type	string
Default Value	"–hex-blob –single-transaction –verbose –all-databases –add-drop-database –system=all"

When using MySQL Flavor or old MariaDB version it may needed to remove system=al option that is printing system tables as SQL like for USERS, GRANTS , ROLES, SERVERS

`backup-lockddl` (2.2)

Item	Value
Description	Use MariaDB backup stage
Type	boolean
Default Value	true

`backup-logical-load-threads` (2.1)

Item	Value
Description	MyLoader can benefite multi threads for restore
Type	Interger
Default Value	2

`backup-logical-dump-threads` (2.1)

Item	Value
Description	MyDumper and MySQLDump can benefite multi threads for dumping
Type	Interger
Default Value	2

This is not advice to dump with to many threads as it can consume a lot of ressource when backup happen on leader, the default and advice way of backing up a db

`backup-mydumper-path` (2.1)

Item	Value
Description	Path to mydumper binary
Type	string
Default Value	"/usr/bin/mydumper"

`backup-myloader-path` (2.1)

Item	Value
Description	Path to myloader binary
Type	string
Default Value	"/usr/bin/myloader"

`backup-mysqldump-path` (2.1)

Item	Value
Description	Path to mysqldump binary
Type	string
Default Value	""

When empty a default binary is used provided in share directory.

It can be old and it's advice to install a database package similar or most up to date version to your database version.

`backup-mysqlclient-path` (2.1)

Item	Value
Description	Path to mysql client binary
Type	string
Default Value	“”

To restore mysqldump “replicaction-manager” pipe the SQL to the mysql client

When empty a default binary is used provided in share directory

`db-servers-backup-hosts` (2.1)

Item	Value
Description	Database list of hosts to backup when set can backup a slave
Type	list
Default Value	“”

Use the same string as in db-servers-hosts

6.2.2.1.0.1 Binlogs Backups

replication-manager (2.1) can backup binlogs it is monitoring binlogs and on rotation will copy the previous binlog

`backup-binlogs` (2.1)

Item	Value
Description	Archive binlogs
Type	Boolean
Default Value	false

`backup-mysqlbinlog-path` (2.1)

Item	Value
Description	Path to mysqlbinlog binary
Type	string
Default Value	“”

When empty a default binary is used provided in share directory.

It can be old and it’s advice to install a database package similar or most up to date version to your database version.

`backup-binlogs-keep` (2.1)

Item	Value
Description	
Type	Integer
Default Value	10

6.2.2.2.0.1 Archiving Backups

`backup-restic` (2.1)

Item	Value
Description	Turn on Backup store and restore with restic
Type	boolean
Default Value	false

`backup-restic-repository` (2.1)

Item	Value
Description	Restic AWS or Minio backend repository
Type	string
Default Value	"s3:https://s3.signal18.io/backups"

`backup-restic-aws-access-key-id` (2.1)

Item	Value
Description	AWS or Minio key id
Type	string
Default Value	"admin"

`backup-restic-aws-access-secret` (2.1)

Item	Value
Description	AWS or Minio key secret
Type	string
Default Value	"secret"

`backup-restic-binary-path` (2.1)

Item	Value
Description	Path to restic binary
Type	string
Default Value	"/usr/bin/restic"

`backup-restic-password` (2.1)

Item	Value
Description	Restic backend password
Type	string
Default Value	"secret"

`backup-keep-hourly` (2.1)

Item	Value
Description	Keep this number of hourly backup
Type	integer
Default Value	1

`backup-keep-daily` (2.1)

Item	Value
Description	Keep this number of daily backup
Type	integer
Default Value	1

`backup-keep-weekly` (2.1)

Item	Value
Description	Keep this number of weekly backup
Type	integer
Default Value	4

`backup-keep-monthly` (2.1)

Item	Value
Description	Keep this number of monthly backup
Type	integer
Default Value	12

`backup-keep-yearly` (2.1)

Item	Value
Description	Keep this number of yearly backups
Type	integer
Default Value	2

`backup-restic-local-repository` (3.0)

Item	Value
Description	Restic local repository path. When empty, uses per-cluster repository in datadir/backups/archive/{clustername}
Type	string
Default Value	“”

Set this to specify a custom local repository path. Useful for multi-cluster environments where you want centralized backup storage.

`backup-restic-aws` (3.0)

Item	Value
Description	Enable S3 archiving. When false, backups are stored in local repository
Type	boolean
Default Value	false

When enabled, restic archives to S3/Minio using the configured `backup-restic-repository`. When disabled, backups are stored locally in `datadir/backups/archive/`.

`backup-restic-timeout` (3.1)

Item	Value
Description	Timeout for restic operations in seconds
Type	integer
Default Value	3600

Controls how long **replication-manager** will wait for restic backup, restore, and maintenance operations to complete before timing out.

6.2.2.2.1 Disk Space Management

`backup-restic-purge-oldest-on-disk-space` (3.1)

Item	Value
Description	Automatically purge oldest backups when disk space is critically low
Type	boolean
Default Value	true

When enabled, **replication-manager** monitors disk space usage and automatically purges the oldest backup snapshots when disk usage exceeds the critical threshold. This prevents backups from filling the disk and causing system-wide issues.

`backup-restic-purge-oldest-on-disk-threshold` (3.1)

Item	Value
Description	Disk usage percentage threshold for automatic purge. 0 uses backup-disk-threshold-crit value
Type	integer
Default Value	0

When set to 0, the threshold from `backup-disk-threshold-crit` is used. Otherwise, this specific value overrides the general threshold for restic purge operations.

`backup-check-size` (2.1)

Item	Value
Description	Check free space before processing backup
Type	boolean
Default Value	true

When enabled, **replication-manager** verifies sufficient disk space is available before starting a backup operation.

`backup-disk-threshold-warn` (3.0)

Item	Value
Description	Warning threshold for disk usage in percentage
Type	integer
Default Value	85

When disk usage exceeds this percentage, **replication-manager** issues warnings but continues backup operations.

`backup-disk-threshold-crit` (3.0)

Item	Value
Description	Critical threshold for disk usage in percentage. Backups are skipped above this value
Type	integer
Default Value	95

When disk usage exceeds this percentage, **replication-manager** will: - Skip new backup operations to prevent disk exhaustion - Trigger automatic purge if `backup-restic-purge-oldest-on-disk-space` is enabled - Generate critical alerts

6.2.2.2.2 Configuration Examples

6.2.2.2.2.1 Local Repository Configuration

```
backup-restic = true
backup-restic-binary-path = "/usr/bin/restic"
backup-restic-local-repository = ""
backup-restic-aws = false
backup-restic-password = "changeme"
backup-restic-purge-oldest-on-disk-space = true
backup-disk-threshold-crit = 90
```

This configuration stores backups locally in per-cluster directories and automatically purges old backups when disk usage exceeds 90%.

6.2.2.2.2.2 S3/Minio Configuration

```
backup-restic = true
backup-restic-aws = true
backup-restic-repository = "s3:https://s3.amazonaws.com/my-backups"
```

```
backup-restic-aws-access-key-id = "AKIAIOSFODNN7EXAMPLE"
backup-restic-aws-access-secret = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
backup-restic-password = "changeme"
backup-restic-timeout = 7200
```

This configuration archives backups to S3 with a 2-hour timeout for operations.

6.2.2.2.2.3 Retention Policy

```
backup-keep-hourly = 24
backup-keep-daily = 7
backup-keep-weekly = 4
backup-keep-monthly = 12
backup-keep-yearly = 3
```

This retention policy keeps: - Last 24 hourly backups - Last 7 daily backups - Last 4 weekly backups - Last 12 monthly backups - Last 3 yearly backups

6.2.2.2.3 Restic Task Management

replication-manager 3.1 includes task queue management for restic operations. Long-running backup operations are tracked and can be:

- Paused via API: `PUT /api/clusters/{cluster}/servers/{server}/actions/task-pause`
- Resumed via API: `PUT /api/clusters/{cluster}/servers/{server}/actions/task-resume`
- Cancelled via API: `PUT /api/clusters/{cluster}/servers/{server}/actions/task-cancel`

Task status is visible in the web UI under server details.

6.2.2.2.4 Automatic Cleanup Workflow

When `backup-restic-purge-oldest-on-disk-space` is enabled:

1. **replication-manager** monitors disk usage before each backup
2. If usage exceeds `backup-disk-threshold-crit`:
   - New backups are skipped
   - Oldest snapshot is identified via `restic snapshots`
   - Snapshot is removed via `restic forget`
   - Repository is pruned via `restic prune`
   - Disk space is rechecked
3. Process repeats until disk usage drops below critical threshold
4. Normal backup operations resume

6.2.2.2.5 Performance Considerations

- Restic operations are I/O intensive, especially `prune` and `check`
- Set `backup-restic-timeout` appropriately for repository size
- Network timeouts may occur with S3 for large repositories
- Purge operations can take significant time on full disks
- Monitor restic task status via API to track progress

6.2.2.3.0.1 Streaming Backups

replication-manager 2.1 can fuse mount the entire backup directory to an S3 bucket

`backup-streaming` (2.1)

Item	Value
Description	Backup streaming to cloud
Type	Boolean
Default Value	false

`backup-streaming-aws-access-key-id` (2.1)

Item	Value
Description	Backup AWS key identity
Type	string
Default Value	"admin"

`backup-streaming-aws-access-secret` (2.1)

Item	Value
Description	Backup AWS key secret
Type	string
Default Value	"secret"

`backup-streaming-bucket` (2.1)

Item	Value
Description	Backup S3 bucket
Type	string
Default Value	"repman"

`backup-streaming-endpoint` (2.1)

Item	Value
Description	Backup AWS endpoint
Type	string
Default Value	"https://s3.signal18.io/"

`backup-streaming-region` (2.1)

Item	Value
Description	Backup AWS region
Type	string
Default Value	"fr-1"

`backup-streaming-debug` (2.1)

Item	Value
Description	Debug mode for streaming to cloud
Type	Boolean
Default Value	false

Schema and Data Monitoring

Replication Manager provides two complementary monitoring systems for database integrity: **schema monitoring** tracks structural changes across your cluster, and **data checksumming** detects and repairs row-level divergence between the primary and its replicas.

6.2.3.1 Schema Monitoring

6.2.3.1.1 How Collection Works

Schema monitoring operates in two phases. First, Replication Manager builds a list of all `schema.table` pairs. Then, in a background thread, it collects full metadata for each table — column definitions, index structures, collations, and data types — computing a CRC64 fingerprint over the result.

Several mechanisms prevent concurrent collections from running and enforce a minimum interval between runs, controlled by:

Parameter	Default	Description
`monitoring-schema-scan-timeout`	`30`	Maximum seconds for a single schema metadata scan

6.2.3.1.2 Enabling the Scheduler

All maintenance schedulers are **disabled by default** and must be opted in explicitly.

Parameter	Default	Description
`monitoring-scheduler`	`false`	Master switch — enables the internal scheduler engine
`monitoring-schema-scheduler`	`true`	Enables the schema monitoring schedule
`monitoring-schema-scheduler-cron`	`0 0 1 1 * *`	Cron schedule for schema collection (6-field format)

The default cron runs once a day at 1 AM. This conservative default is intentional — clusters with large table dictionaries can have expensive metadata scans, and once-a-day collection is sufficient for most environments.

6.2.3.1.3 Collection Scope

Fine-grained control over what gets collected:

Parameter	Default	Description
`monitoring-schema-columns`	`true`	Collect column definitions
`monitoring-schema-indexes`	`true`	Collect index definitions
`monitoring-schema-on-replicas`	`true`	Mirror schema collection on all replicas
`monitoring-schema-ignore-tables`	`""`	Comma-separated `schema.table` patterns to exclude

6.2.3.1.4 Change Detection

Once metadata has been collected on the primary and all replicas, Replication Manager continuously compares CRC64 fingerprints across all cluster nodes to detect structural drift — schema divergence between replicas and their primary, missing or added tables, column type changes, collation changes.

Parameter	Default	Description
`monitoring-schema-change`	`true`	Enable schema change detection and alerting
`monitoring-schema-change-script`	`""`	Optional external script invoked on schema change event

6.2.3.1.5 Special Case: Shard Proxy

When a MariaDB Spider shard proxy is in use (`shardproxy = true`), schema changes must be detected and pushed to the Spider proxy node much faster than the default daily schedule allows. In this configuration, schema monitoring runs continuously in the main cluster loop, throttled only by `monitoring-schema-scan-timeout`. If you use sharding, verify this parameter is tuned appropriately for your table count.

6.2.3.1.6 Persisting the Table Dictionary

To avoid expensive cold-start scans after a process restart, the full per-server table dictionary — including schema metadata and checksum state — is persisted to disk after every collection. It is automatically reloaded on startup before the first monitoring cycle runs.

The file is stored per server inside the working directory:

`{working-dir}/{cluster-name}/{host}_{port}/serverstate.json`

For example:

`/var/lib/replication-manager/belair/db1.belair.svc.cloud18_3306/serverstate.json`

6.2.3.2 Data Checksumming

Schema monitoring tells you *structure* is consistent. Data checksumming tells you *rows* are consistent.

6.2.3.2.1 Triggering a Checksum

A checksum run can be initiated in three ways:

- **Scheduler** — runs automatically on the configured cron schedule
- **GUI** — from the Shards tab, at the cluster, schema, or individual table level
- **API** — `POST /api/clusters/{cluster}/settings/actions/switch/monitoring-checksum-scheduler`

Parameter	Default	Description
`monitoring-checksum-scheduler`	`false`	Enable scheduled automatic checksumming
`monitoring-checksum-scheduler-cron`	`0 0 2 * * 5`	Cron schedule — default is every Friday at 2 AM

6.2.3.2.2 How Checksumming Works

Tables are processed **serially**. For each table, the algorithm:

1. **Splits the table into chunks** of 2,000 rows using range predicates on the primary key, stored in a working table `replication_manager_schema.table_chunk`.
2. **Computes a CRC32 checksum** for each chunk on the primary using `REPEATABLE READ` isolation and `binlog_format = STATEMENT` to ensure the checksum write itself replicates to all replicas identically.
3. **Waits for replica convergence** using GTID sequence numbers — each replica must have applied the primary's transactions up to the point the chunk was written before its result is read.
4. **Compares chunk checksums** between primary and each replica. Any divergent chunk is recorded with its full range predicate so it can be repaired later.

Tables without a `PRIMARY KEY` or `UNIQUE KEY` cannot be chunked and are marked **N/A**.

6.2.3.2.3 Table Sync States

Each table carries one of five sync states visible in the GUI:

State	Meaning
(blank)	Not yet checksummed
`WA`	Waiting — queued for a checksum run that is already in progress
`PR`	In Progress — currently being checksummed, shows a progress bar
`OK`	All chunks match across all replicas
`ER`	One or more chunks diverge on at least one replica
`NA`	Cannot checksum — no primary/unique key, or a process error occurred

6.2.3.2.4 Excluding Expected Divergence

Some tables diverge by design. Replication Manager uses the `replication_manager_schema.jobs` table to schedule maintenance tasks on individual nodes, intentionally writing those rows without binary logging so they stay node-local. The `replication_manager_schema.table_checksum` working table is also ephemeral. Both are excluded by default:

Parameter	Default
`monitoring-checksum-ignore-tables`	`replication_manager_schema.jobs,replication_manager_schema`

Add any additional `schema.table` pairs to this comma-separated list to suppress known-good divergences.

6.2.3.2.5 Supported Primary Key Types

The chunking engine supports the full range of primary key data types and compositions. The regression test suite at `share/sql/checksum-repair-test.sql` covers:

Single-column keys: `TINYINT`, `SMALLINT`, `INT`, `BIGINT`, `UNSIGNED BIGINT`, `VARCHAR`, `CHAR`, `UUID CHAR(36)`, `UUID BINARY(16)`, `DATE`, `DATETIME`, `TIMESTAMP`, `DECIMAL`, `ENUM`

Composite keys: `INT + VARCHAR`, `BIGINT + DATE`, `UUID CHAR(36) + TINYINT`, `ENUM + DATE`, `ENUM + DATETIME(6)`, `ENUM + TIMESTAMP(3)`, `ENUM + YEAR`

6.2.3.3 Repairing Divergent Tables

When a table is in `ER` state, Replication Manager stores the exact range predicate for every divergent chunk on every affected replica. Repair can be triggered from the GUI or API at the table, schema, or cluster level.

The repair process per chunk:

```
START TRANSACTION;
CREATE OR REPLACE TEMPORARY TABLE tmp_repair
    AS SELECT * FROM schema.table WHERE {chunk_range} FOR UPDATE;
DELETE FROM schema.table WHERE {chunk_range};
INSERT INTO schema.table SELECT * FROM tmp_repair;
COMMIT;
```

This transaction runs on the **primary** with `binlog_format = ROW`, so replicas receive an exact row-level copy of the repaired chunk via standard replication — no direct writes to replicas are required.

After all chunks are repaired, the table is automatically re-checksummed to confirm convergence.

6.2.3.4 GUI: Shards and Schema Graph

The **Shards tab** provides a table-level view of the entire schema dictionary with:

- **Sync status badges** for each table (`OK`, `ER`, `NA`, `PR` with live progress bar)
- **In-progress tables sorted to the top** automatically during a checksum run
- **Filter pills** to quickly isolate tables by sync state
- **Per-table and per-schema actions** — Checksum, Repair, Analyze

The **Graph view** within the Shards tab visualises relationships between tables using three edge types:

Edge type	Style	Description
Foreign key	Solid	Declared `FOREIGN KEY` constraints
Name match	Dashed	Columns sharing the same name across tables (implicit join candidates)
Workload query	Dotted	Tables co-appearing in captured query plans

Name-match edges are **off by default** as they can produce a large number of implicit edges in schemas with common column naming conventions such as `id`, `created_at`, or `status`. Toggle each edge type independently using the filter controls in the graph toolbar.

6.2.3.5 Failover Policy and Data Divergence

6.2.3.5.1 How Divergence Affects Replica Election

When a checksum run completes, each replica that has one or more divergent chunks is marked with `isDataDiverge = true` in the cluster state. This flag is evaluated every time Replication Manager selects a candidate for failover or switchover.

During candidate election, any replica flagged as divergent is excluded and logged with:

```
ERR00103: Skip slave in election {server} data diverge in checksum
```

This exclusion is tracked in the election audit trail under `IgnoredDataDivergence` so operators can see exactly why a replica was bypassed.

If divergence has been detected on **all** replicas, no candidate can be found and the operation is blocked with:

```
ERR00032: No candidates found in slaves list
```

This escalation from a per-server warning to a cluster-level error is intentional: a single divergent replica is a warning that should be investigated and repaired, but a fully divergent replica set is a signal that automatic promotion would risk serving stale or inconsistent data.

6.2.3.5.2 Controlling Failover Behaviour

Replication Manager's core design principle is **availability first**. Blocking all failovers because of data divergence would be the wrong default for most production environments where a brief inconsistency is preferable to an extended outage.

Parameter	Default	Description
`failover-divergent-data`	`true`	Allow failover and switchover to replicas with divergent checksum data

With the default of `true`, divergent replicas participate in candidate election normally. The `ERR00103` warning is still raised and recorded — giving operators full visibility — but it does not block promotion.

Setting this to `false` enforces a **strict consistency policy**: no replica with a known data divergence will ever be promoted. This is appropriate for environments where data correctness takes absolute precedence over availability, such as financial ledgers, compliance-critical datasets, or systems where serving stale data has regulatory consequences.

6.2.3.5.3 Recommended Workflow for Strict Mode

If you run with `failover-divergent-data = false`, make sure your operational process accounts for the fact that an unrepaired divergence will block all automated failover. The recommended cycle is:

1. Run checksums on schedule (`monitoring-checksum-scheduler = true`)
2. Monitor for `ERR00103` alerts — each one identifies a specific replica and set of divergent chunks
3. Trigger repair from the GUI or API to re-sync the affected chunks via the primary
4. After repair the table is automatically re-checksummed; on success `isDataDiverge` is cleared
5. The replica re-enters the candidate pool for the next failover or switchover

6.2.3.5.4 GUI Setting

The `failover-divergent-data` toggle is available in **Settings → Replication → Failover** under the label *"Failover enable on divergent data"*, allowing it to be changed at runtime without a configuration file edit or restart.

replication-manager (2.1) can stream backups out and in to a new or de-sync database node.

Streaming is organized around a jobs queue table on each database node that is dequeue with a cron script. **replication-manager** is auto creating this job table on cluster nodes.

For donor node to stream jobs **replication-manager (2.1)** open a TCP connection with a timeout of 120s and populate the jobs (replication_manager_scehma.jobs) table with the action and the receiver port.

For Joiner replication manager use port 4444.

To organize streaming you need to implement backup policy via replication-manager scheduler or do an on demand backup via the GUI or API.

Scheduler default collect daily physical backup. Extra package xtrabackup or mariadbbackup need to be install on the remote host.

Reseeding endpoint API get 3 methods:

Method	Descrition
logicalmaster	Direct mysqldump of master via replication-manager piping host
logicalbackup	Last mysqldump backup
physicalbackup	Last xtrabackup or mariadb-backup backup

6.2.4.1 Donor and joiner cron script

In **replication-manager-pro** we push the donor script in services compliance but if you are using **replication-manager-osc** you can customize similar script in crontab schedule every minutes.

Job script Bootstrap script is os dependent

7.1.1 Security

replication-manager enforces security at every layer: the REST API is TLS-only, all database connections support mutual TLS with CA validation, sensitive configuration values are AES-encrypted at rest, and secrets can be sourced from HashiCorp Vault at runtime. API access is controlled through a credential-based ACL system.

7.1.2 Feature Summary

7.1.2.1 Process Privilege Drop (`--user`)

replication-manager can start as `root` to bind a low-numbered HTTPS port and then immediately drop to a less-privileged OS user via `syscall.Setuid` / `syscall.Setgid` once the listeners are ready. The drop is irreversible — the process cannot regain root after this point. See Configuration Guide for the full drop sequence, directory ownership requirements, and a systemd unit example.

7.1.2.2 TLS — API and Web Interface

The monitoring API and web GUI are served over HTTPS. Server certificate and key are configured per instance. Clients that cannot present a valid certificate can be denied.

7.1.2.3 TLS — Database Connections

All connections replication-manager opens to MariaDB/MySQL nodes (monitoring, replication setup, failover) can be secured with TLS. Supported options include server certificates, client certificates, CA chain validation, and per-cluster SSL mode overrides (`db-servers-tls-*`).

7.1.2.4 Encrypted Configuration

Passwords, API tokens, and Vault credentials stored in TOML files are AES-encrypted. replication-manager decrypts them at startup using a per-cluster key. Plain-text values are never written back after first encryption.

7.1.2.5 HashiCorp Vault Integration

Credentials can be retrieved from Vault at runtime using AppRole authentication. Vault paths, role ID, secret ID, and auth mode are configurable. The integration supports both KV v1 and v2 secret engines.

Config key	Purpose
`vault-server-addr`	Vault server URL
`vault-role-id`	AppRole role ID
`vault-secret-id`	AppRole secret ID
`vault-mode`	Secret engine mode
`vault-auth`	Authentication method

7.1.2.6 API Credentials and ACLs

The REST API authenticates all requests against a configurable credential list (`api-credentials`). ACL rules allow fine-grained whitelisting and blacklisting of credentials for both internal and external API access:

- `api-credentials-acl-allow` / `api-credentials-acl-allow-external`
- `api-credentials-acl-discard` / `api-credentials-acl-discard-external`

7.1.2.7 Provisioning TLS

TLS certificates for the provisioning orchestration layer (OpenSVC, Kubernetes) are configured separately from the monitoring API (`prov-tls-server-ca`, `prov-tls-server-cert`, `prov-tls-server-key`).

7.1.3 HTTPS Bastion — Browser Terminal Access

replication-manager doubles as a **secure bastion host**: operators can open `bash`, `mysql`, and `mytop` terminals to any managed database node or proxy directly in the browser, with all traffic carried over the existing HTTPS/WebSocket connection. No SSH port needs to be exposed to the operator's workstation.

Authentication uses the same RSA JWT as the REST API. Role-based credential selection ensures users connect to the database with the appropriate privilege level — the credential is injected server-side and never sent to the browser. Sessions can be made resumable via tmux or screen.

See HTTPS Bastion and Terminal for configuration, role mapping, OpenSVC integration, and ACL grants.

7.1.4 Supply Chain and Regulatory Compliance

A **CycloneDX SBOM** (Software Bill of Materials) is published with every release, listing all 279 Go module dependencies with versions, PURLs, and hashes. Operators can feed the SBOM into standard vulnerability scanners (Grype, Trivy, OSV-Scanner) or SCA platforms (Dependency-Track, Snyk).

Signal18 publishes the SBOM and maintains a vulnerability disclosure process as part of compliance with the **EU Cyber Resilience Act (CRA)**. See SBOM and CRA Compliance for the full details including where to download the SBOM, how to regenerate it, and how to report a vulnerability.

7.1.5 Configuration Guide

See Configuration Guide for all TLS, Vault, encryption, and ACL configuration keys.

7.2.1 Process User and Privilege Drop

7.2.1.1 `--user`

Description	OS username to drop privileges to after the HTTPS and HTTP listeners are ready. Must be started as root (UID 0) for this option to take effect. If omitted and started as root, replication-manager continues running as root with a log warning.
Type	String
Default	`""`
Example	`"repman"`

replication-manager supports a **privilege-drop model**: it starts as `root` to bind the HTTPS API port (which may be 443 or another low-numbered port), then immediately drops to a less-privileged OS user once the listeners are accepting connections.

7.2.1.1.1 Drop sequence

```
1. Start as root
2. Bind HTTPS API port and HTTP dashboard port
3. Wait until both listeners are ready
4. chown -R <user>:<group> {monitoring-datadir}     ← ownership hand-off
5. syscall.Setgid(<gid>)                             ← drop group
6. syscall.Setuid(<uid>)                             ← drop user (irreversible)
7. Adjust any config dirs that are no longer writable under the new user
8. Start cluster monitoring goroutines
```

Steps 5–6 use `syscall.Setuid` / `syscall.Setgid`, which are **irreversible** on Linux — the process cannot regain root privileges after this point.

7.2.1.1.2 File ownership requirements Before starting, the target user must own (or have write access to) the key directories:

```
# Create a dedicated system user
useradd -r -s /sbin/nologin repman

# Give it ownership of config and data directories
chown -R repman:repman /etc/replication-manager
chown -R repman:repman /var/lib/replication-manager

# The encryption key (generated as root) should remain root-readable only
chmod 600 /etc/replication-manager/.replication-manager.key
```

replication-manager will automatically `chown` the working directory (`monitoring-datadir`) to the target user before dropping, so files written during the root phase (encryption keys, initial config) are accessible afterwards. If any writable config sub-directories are not accessible under the new user, they are transparently remapped to paths under the target user's home directory (`~/.config/replication-manager/`).

7.2.1.1.3 systemd unit example

```
[Unit]
Description=replication-manager
After=network.target

[Service]
```

```
Type=simple
# Start as root so the process can bind privileged ports, then drop to repman
ExecStart=/usr/bin/replication-manager monitor --user repman --config /etc/replication-manager/config.to
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

If replication-manager does not need to bind a port below 1024, you can skip the privilege drop entirely and start directly as the target user:

```
[Service]
User=repman
Group=repman
ExecStart=/usr/bin/replication-manager monitor --config /etc/replication-manager/config.toml
```

In this case the `--user` flag is ignored (only root can call `setuid`).

7.2.1.1.4 Docker rootless alternative The official Docker images provide rootless variants that run as `repman` (UID/GID 10001) from the start, removing the need for any privilege drop:

```
docker run -u repman \
  -v /etc/replication-manager:/etc/replication-manager \
  -v /var/lib/replication-manager:/var/lib/replication-manager \
  signal18/replication-manager:latest-rootless
```

7.2.2 Configuration File Security

replication-manager provides password obfuscating security by implementing AES encryption.

An encryption key must be generated by running `replication-manager keygen` as root. This ensures that no unprivileged user can read the contents of the encryption key.

With the key now generated, you can create encrypted passwords using `replication-manager password`. Example:

```
# replication-manager password secretpass
Encrypted password hash: 50711adb2ef2a959577edbda5cbe3d2ace844e750b20629a9bcb
```

You can now replace your password in the configuration file using this encrypted hash:

```
user = "root:50711adb2ef2a959577edbda5cbe3d2ace844e750b20629a9bcb"
```

When an encryption key is detected at `replication-manager monitor` start, the encrypted passwords will be automatically decrypted by the application. There is no further configuration change required.

7.2.3 API Security Configuration

replications-manager-cli clients and API use JWT TLS protocol over https.

The REST API is secured using encrypted token that is used to validate user:password credential of the API, the client will use same default password and so enter the API without asking a password but if the configuration `api-credentials` is changed, all client will prompt for the password unless given the correct parameter `user` and `password` flags.

`api-credentials` (2.0), `user` (1.1)

Item	Value
Description	Rest API credential in [user]:[password] format
Type	string
Default Value	“admin:repman”

At startup of the monitor some x509 certificates are loaded from the *replication-manager* share directory to ensure TLS https secure communication.

Replace the files with your own certificate to make sure your deployment is truly secured.

‘monitoring-ssl-cert (2.1)

Item	Value
Description	HTTPS & API TLS certificate
Type	string
Default Value	“”

‘monitoring-ssl-key (2.1)

Item	Value
Description	HTTPS & API TLS key
Type	string
Default Value	“”

```
# Key considerations for algorithm "RSA"  2048-bit
openssl genrsa -out server.key 2048

# Key considerations for algorithm "ECDSA"  secp384r1
# List ECDSA the supported curves (openssl ecparam -list_curves)
openssl ecparam -genkey -name secp384r1 -out server.key
openssl req -new -x509 -sha256 -key server.key -out server.crt -days 3650
```

In addition at startup replication-manager monitor will generate in memory extra self signed RSA certificate to ensure token encryption exchange for JWT

7.2.4 Database Security Configuration

`db-servers-tls-ca-cert` (2.0)

Item	Value
Description	Path to the database connection TLS authority certificate.
Type	string
Default Value	“”

`db-servers-tls-client-cert` (2.0)

Item	Value
Description	Path to the database connection TLS client certificate.
Type	string
Default Value	“”

`db-servers-tls-client-key` (2.0)

Item	Value
Description	Database TLS client key.
Type	string
Default Value	“”

`replication-use-ssl` (1.0)

Item	Value
Description	Replication is created using SSL encryption to replicate from master.
Type	boolean
Default Value	false

Replication-Manager does not set MASTER_SSL_CA , MASTER_SSL_CERT , MASTER_SSL_KEY in CHANGE MASTER command, instead it relies on MySQL MariaDB to get setup for the replication to be using SSL. with this flag replication just add MASTER_SSL=1 to the replication command.

```
[client]
ssl-ca=cacert.pem
ssl-cert=client-cert.pem
ssl-key=client-key.pem

[mysqld]
ssl-ca=cacert.pem
ssl-cert=server-cert.pem
ssl-key=server-key.pem
```

7.2.5 Vault Security Configuration

replication-manager provides password obfuscating security by using Vault services.

To access your vault server, you must specify its address in the parameter `vault-server-addr` flag. Here, the documentation to setup a Vault server.

`vault-server-addr` (2.3)

Item	Value
Description	Vault server address
Type	string
Default Value	“”

To authenticate and access your data stored in your Vault server, you need to use an authentication method. For now, it is only possible to connect using the Approle method. Here, the documentation for creating an Approle role. Be carefull to create the necessary policies for your auth role, you can find documentation here

`vault-auth` (2.3)

Item	Value
Description	Vault auth method
Type	string
Default Value	"approle"

Specify your role and secret id obtained by creating your approle role with the parameters `vault-role-id` and `vault-secret-id` flags.

`vault-role-id` (2.3)

Item	Value
Description	Vault role id
Type	string
Default Value	""

`vault-secret-id` (2.3)

Item	Value
Description	Vault secret id
Type	string
Default Value	""

```
[mycluster]
vault-server-addr="http://vault.infra.svc.cloud18:8200"
vault-auth="approle"
vault-role-id="bgsg2f1af-7ce4-e938-46d0-4fbuzibfe4r0d"
vault-secret-id="91b1a039-1597-54b7-f304-c703gyct9d4e"
```

Vault allow multiple services to store secret data. **Replication-manager** provides two modes to use Vault services. Vault can be used to store keys in a secret on a Vault server, the config_store_v2 mode. Vault also provides a feature for automatic password rotation, implemented in the database_engine mode.

`vault-mode` (2.3)

Item	Value
Description	Vault mode
Type	string
Default Value	"config_store_v2"

7.2.5.1 Config_store_v2

In this case, the secret path to the remote secret must be specified in the parameters `db-server-credential` flag for the database credential and `replication-credential` flag for the replication-manager credential as

`user:password`. The key of your secret, which is store at your secret path, has to be `db-server-credential` for your database credentials and `replication-credential` for the replication credential.

Exemple :

```
secret path : kv/applications/repman
key : db-server-credential
secret key : user:password
```

That means you can store different keys at the same secret path!

We also need to specify the mount directory of the secret in the `vault-mount` variable.

`vault-mount` (2.3)

Item	Value
Description	Vault mount for the secret
Type	string
Default Value	"secret"

Exemple of configuration for a secret store at /kv/applications/repman secret Vault path.

```
[mycluster]
vault-mode="config_store_v2"
db-servers-credential = "applications/repman"
replication-credential = "applications/repman"
vault-mount="kv"
```

In this case, we store database server credential and replication credential at the same secret path, in two different key named by their respective parameters flags.

Be carefull to create the necessary policies to acces for your secret Vault path, you can find documentation here

7.2.5.2 Database_engine

Vault also allow to perform automatic password rotations with a management service of database credential. It allow to create a vault user that will performs password rotations on the specified database(s) at the desired regular time interval. First, you have to create a database role in which you will specify the connection url of your database, the username and password which allows Vault to access the database, the allowed role for the database and a SQL statement that will perform the rotation password. Here, the documentation to create a database role. Then, you have to create a static-role, with this documentation. A static-role is define by a username, a password and a rotation period.

Exemple of a database and static-role configuration :

```
#Database configuration
vault write database/config/my-mysql-database \
    plugin_name=mysql-database-plugin \
    connection_url="{{username}}:{{password}}@tcp(proxysql1.emma.svc.cloud18:3306)/" \
    allowed_roles="repman-monitor, repman-replication" \
    username="vaultuser" \
    password="vaultpass"

#SQL statement to perform password rotation
tee rotation.sql <<EOF
ALTER USER '{{name}}'@'%' IDENTIFIED BY '{{password}}';
```

```
EOF

#Static-role configuration
vault write database/static-roles/repman-monitor \
    db_name=my-mysql-database \
    rotation_statements=@rotation.sql \
    username="repman" \
    rotation_period=600

vault write database/static-roles/repman-replication \
    db_name=my-mysql-database \
    rotation_statements=@rotation.sql \
    username="repman-rep" \
    rotation_period=600
```

Once these roles are set up, Vault will perform automatic password rotations for all roles defined in the database role configuration. To allow the repman to access the created roles, you have to specify the secret path to access the roles must be specified in the parameters `db-server-credential` flag for the database credential role and `replication-credential` flag for the replication-manager credential role.

Exemple of configuration for two static-role store at "database/static-creds/repman-monitor" and "database/static-creds/repman-replication" secret Vault path.

```
[mycluster]
vault-mode="database_engine"
db-servers-credential = "database/static-creds/repman-monitor"
replication-credential = "database/static-creds/repman-replication"
```

7.2.5.3 Rotation of Database Credentials

So far, replication-manager is taking care of the monitoring and the replication credentials. When an authentication error state is trigger, replication-manager will presume that the password has been rotated and will fetch Vault secret path again. Also, we provide a password rotation service which can be accessible from the replication-manager interface or with an api call. This features is available in both Vault mode. In the config_store_v2 mode, replication-manager will generate a new password and update it in the vault secret and the databases. In the database_engine mode, replication-manager will send a resquest of rotation password to vault services, which will generate all the rotation password in the databases.

7.3.1 SBOM and EU Cyber Resilience Act

The **EU Cyber Resilience Act (CRA)** establishes mandatory cybersecurity requirements for products with digital elements placed on the European market. One of its core obligations is the publication of a **Software Bill of Materials (SBOM)** — a machine-readable inventory of every software component, library, and dependency included in a product.

Signal18 publishes a CycloneDX SBOM for every release of replication-manager to help operators, integrators, and auditors understand the full composition of the software they deploy.

7.3.2 The Cyber Resilience Act

The CRA entered into force in November 2024. Its obligations phase in over a transition period:

Date	Obligation
September 2026	Vulnerability and incident reporting obligations apply
December 2027	All other CRA requirements fully apply

7.3.2.1 What the CRA requires

For a product like replication-manager the main obligations are:

- **SBOM** — a structured, machine-readable list of all software components and their versions, published with each release
- **Vulnerability handling** — a documented process to receive, triage, and remediate reported vulnerabilities, and to notify affected users when fixes are available
- **Incident reporting** — actively exploited vulnerabilities must be reported to ENISA or the relevant national authority within 24 hours of discovery; significant security incidents within 72 hours
- **Security updates** — security fixes must be provided for the expected supported lifetime of the product and made available separately from feature updates
- **Secure development** — evidence of a security-aware software development lifecycle (dependency scanning, code review, signed releases)

7.3.2.2 Open-source software and the CRA

The CRA contains an exemption (Recital 18) for purely community-developed open-source software that is not commercialized. replication-manager is developed by Signal18 as part of a commercial offering. Signal18 therefore treats its CRA obligations as applying in full and has taken proactive steps to comply ahead of the mandatory deadlines.

7.3.3 Software Bill of Materials

7.3.3.1 Format

replication-manager's SBOM uses the **CycloneDX 1.6** format (JSON). CycloneDX is a widely supported open standard for SBOM, maintained by OWASP, and natively understood by most vulnerability scanners and supply-chain security tools.

The SBOM is generated automatically using Anchore's SBOM Action (backed by Syft) on every published GitHub release.

7.3.3.2 What is included

Each SBOM lists the **279 direct and transitive Go module dependencies** of replication-manager, including:

- Module name and version (semver)
- Package URL (PURL) in `pkg:golang/` format
- License identifiers where detectable
- Hashes (SHA-256) for integrity verification

The root component entry identifies the exact build:

```
{
  "type": "library",
  "name": "github.com/signal18/replication-manager",
  "version": "v3.0.x",
  "purl": "pkg:golang/github.com/signal18/replication-manager@v3.0.x?type=module&goos=linux&goarch=amd64
}
```

7.3.3.3 How to get the SBOM

The SBOM is attached as a release asset to every GitHub release:

```
https://github.com/signal18/replication-manager/releases/latest
  → sbom-cyclonedx.json
```

A development snapshot (`sbom.json`, generated with `cyclonedx-gomod`) is also committed to the repository root and updated with significant dependency changes.

7.3.3.4 Generating the SBOM locally

To regenerate the SBOM from source (requires cyclonedx-gomod):

```
cyclonedx-gomod mod -json -licenses -output sbom.json .
```

Or using Syft (produces the same CycloneDX 1.6 output):

```
syft packages . --output cyclonedx-json=sbom-cyclonedx.json
```

7.3.4 Using the SBOM

7.3.4.1 Vulnerability scanning

Feed the SBOM into any CycloneDX-compatible scanner to identify known CVEs in replication-manager's dependencies:

```
# Grype (Anchore)
grype sbom:sbom-cyclonedx.json

# OSV-Scanner (Google)
osv-scanner --sbom sbom-cyclonedx.json

# Trivy (Aqua)
trivy sbom sbom-cyclonedx.json
```

7.3.4.2 Software composition analysis

Import the SBOM into your organization's SCA or ASPM platform (Dependency-Track, FOSSA, Snyk, etc.) to track replication-manager as a dependency in your own software inventory. Most platforms accept CycloneDX JSON natively.

7.3.4.3 License compliance

The SBOM includes SPDX license expressions for each dependency. Use this to verify that the license obligations of all included components are compatible with your organization's policies before deployment.

7.3.5 Vulnerability Disclosure

7.3.5.1 Reporting a vulnerability

Security vulnerabilities in replication-manager should be reported privately to Signal18 before public disclosure. Contact:

security@signal18.io

Please include: - A description of the vulnerability and its potential impact - Steps to reproduce or a proof-of-concept - The version(s) of replication-manager affected - Your preferred contact method for follow-up

7.3.5.2 Response commitment

Milestone	Target
Acknowledgement of report	48 hours
Triage and severity assessment	7 days
Fix available for critical/high	30 days
Public disclosure	Coordinated with reporter

Signal18 follows a coordinated disclosure model: fixes are prepared and made available before public disclosure. Reporters are credited unless they request anonymity.

7.3.5.3 Security advisories

Confirmed vulnerabilities are published as GitHub Security Advisories on the replication-manager repository. Subscribe to repository notifications to receive alerts.

7.3.6 Signed Releases

Release artifacts are signed to allow integrity verification. The public key and signature files are published alongside each release on the GitHub releases page.

7.3.7 Regulatory References

Document	Summary
EU CRA (Regulation 2024/2847)	Full text of the Cyber Resilience Act
ENISA CRA guidance	ENISA implementation guidance
CycloneDX specification	SBOM format specification

Document	Summary
OWASP CycloneDX	OWASP project page

7.4.1 HTTPS Bastion and Terminal

replication-manager provides browser-based terminal access to every database node and proxy it manages — without opening SSH ports to browser clients. All terminal traffic is carried over a **WebSocket connection tunnelled through the existing HTTPS API**, so the replication-manager TLS endpoint is the only port that needs to be reachable from the operator's browser.

This makes replication-manager act as a **secure HTTPS bastion**: it authenticates the user, enforces ACL and role checks, and proxies the terminal I/O to the target host over a back-end SSH connection or, for OpenSVC-provisioned services, via the OpenSVC agent's terminal server.

7.4.2 Terminal Types

Four command types are available for each database server and proxy:

Command	Description
`bash`	SSH shell to the target host (or OpenSVC container shell via gotty/tty-share)
`mysql`	Interactive `mysql` client connected to the target database
`mytop`	`mytop` live process monitor connected to the target database
(global)	Bash shell on the replication-manager host itself (admin/cloud18 user only)

7.4.3 WebSocket API Endpoints

All terminal connections use the WebSocket protocol (upgraded from HTTPS GET requests). The browser or API client connects over WSS and receives a prompt for a JWT token as the first message.

```
# Global terminal (replication-manager host shell)
GET /api/terminal/connect

# Database server terminal
GET /api/terminal/connect/clusters/{clusterName}/servers/{serverName}
GET /api/terminal/connect/clusters/{clusterName}/servers/{serverName}/{command}

# Proxy terminal
GET /api/terminal/connect/clusters/{clusterName}/proxies/{serverName}
GET /api/terminal/connect/clusters/{clusterName}/proxies/{serverName}/{command}
```

Where `{command}` is one of `bash`, `mysql`, or `mytop`. Omitting `{command}` defaults to `bash`.

7.4.3.1 OpenSVC container selector (bash only)

For bash terminals on OpenSVC-provisioned clusters, the `rid` query parameter selects which container to open the shell into:

`rid` value	Target
`container#db`	Database container (default)
`container#jobs`	Jobs/maintenance container

```
GET /api/terminal/connect/clusters/mycluster/servers/db1?rid=container#jobs
```

7.4.3.2 List active sessions

```
GET /api/terminals
GET /api/clusters/{clusterName}/terminals
```

Returns the list of terminal sessions currently open for the authenticated user.

7.4.4 Authentication Flow

1. Browser upgrades the HTTPS connection to a WebSocket
2. replication-manager sends: `Connected. Waiting for token…`
3. Client sends the JWT Bearer token obtained from the normal API login (`/api/login`)
4. replication-manager validates the JWT signature (RSA) and extracts the user identity
5. ACL check: the user's roles are validated against the cluster's `api-credentials-acl-allow`
6. On success, the terminal session starts; on failure the connection is closed with an error message

Because the JWT is sent as the first WebSocket message (not as an HTTP header), the authentication hand-off is fully contained within the encrypted WebSocket channel.

7.4.5 Role-Based Database Credential Selection

For `mysql` and `mytop` terminals, replication-manager selects which database user to connect with based on the authenticated API user's role:

Role / Grant	DB user used
`sysops`	Cluster root user (`db-servers-credential`) — full access
`sponsor`	Sponsor user (`cloud18-sponsor-user-credentials`) — read-only/limited
`dbops`, `extsysops`, `extdbops`, or `terminal-db` grant	DBA user (`cloud18-dba-user-credentials`)

If the sponsor or DBA user does not exist yet on the target server, replication-manager creates it automatically before opening the session.

For `bash` terminals, the connection uses the on-premise SSH credential configured for the cluster (`onpremise-ssh-credential`).

7.4.6 Back-End Transport

7.4.6.1 Non-OpenSVC clusters (on-premise / SSH)

replication-manager opens a direct SSH connection from the replication-manager host to the database or proxy host. The operator's browser never talks to the SSH port — only replication-manager does.

```
Browser  WSS  replication-manager  SSH  database host
```

Required configuration:

```
onpremise-ssh            = true
onpremise-ssh-port       = 22
```

```
onpremise-ssh-credential = "root:"          # user:password or user: (key auth)
onpremise-ssh-private-key = "/etc/replication-manager/id_rsa"  # optional
```

7.4.6.2 OpenSVC clusters

For services provisioned by OpenSVC, replication-manager queries the OpenSVC API for the terminal URL served by the OpenSVC agent running alongside the database container.

Depending on the OpenSVC agent version:

Agent version	Protocol	Binary
v2	gotty	`gotty-client`
v1	tty-share	`tty-share`

```
Browser  WSS  replication-manager  gotty/tty-share  OpenSVC agent   container
```

Configure the client binary paths if not in the system PATH:

```
backup-gotty-client-path = "/usr/local/bin/gotty-client"
tty-share-binary-path    = "/usr/local/bin/tty-share"
```

7.4.7 Session Resume

When `terminal-session-resume` is enabled, disconnecting from a terminal does not kill the shell process — it is kept alive in a `tmux` or `screen` session on the replication-manager host (for global terminals) or on the remote host. Reconnecting to the same endpoint re-attaches to the existing session.

Session state is persisted across replication-manager restarts in `{monitoring-datadir}/tty.state.json`.

Setting	Default	Effect
`terminal-session-resume`	`false`	Keep shell alive after disconnect and allow re-attach
`terminal-session-manager`	`"tmux"`	Multiplexer to use: `tmux` or `screen`

7.4.8 Configuration Reference

7.4.8.1 Enabling the feature

Terminal access is **disabled by default** and must be explicitly enabled. In non-OpenSVC environments it also requires setting the flag at the server (non-cluster) level:

```
terminal-session-enabled = true
```

For OpenSVC environments the flag can be set per cluster. For other orchestrators it is forced to `false` unless set at the server level.

7.4.8.2 All terminal settings

`terminal-session-enabled`

Description	Enable the browser terminal bastion. Must be set to `true` to allow WebSocket terminal connections. Defaults to `false` on non-OpenSVC clusters unless set at the global server level.
Type	Boolean
Default	`false`

`terminal-session-resume`

Description	Keep the shell process alive after the WebSocket disconnects and allow re-attach on reconnect. Requires `tmux` or `screen` on the replication-manager host for global terminals.
Type	Boolean
Default	`false`

`terminal-session-manager`

Description	Multiplexer used to host resumable sessions.
Type	String (`tmux` | `screen`)
Default	`"tmux"`

`onpremise-ssh`

Description	Connect to database and proxy hosts over SSH for `bash` terminal sessions (and for other on-premise operations).
Type	Boolean
Default	`false`

`onpremise-ssh-port`

Description	SSH port on target hosts.
Type	Integer
Default	22

`onpremise-ssh-credential`

Description	`user:password` for SSH authentication. If password is empty, key-based authentication is used.
Type	String
Default	`"root:"`

`onpremise-ssh-private-key`

Description	Path to the SSH private key. If empty, replication-manager looks for keys in the running user's `~/.ssh/` directory.
Type	String
Default	`""`

`backup-gotty-client-path`

Description	Path to the `gotty-client` binary used for OpenSVC v2 agent terminal sessions. If empty, the binary is located via `PATH`.
Type	String
Default	`""`

`tty-share-binary-path`

Description	Path to the `tty-share` binary used for OpenSVC v1 agent terminal sessions. If empty, `tty-share` is located via `PATH`.
Type	String
Default	`""`

7.4.9 ACL and Grants

Terminal grants can be added to any API user independently of their base role. Three granular grants are available:

Grant	Allows
`terminal-db`	Open `bash`, `mysql`, and `mytop` terminals to database servers
`terminal-proxy`	Open `bash` and `mysql` terminals to proxy servers
`terminal-global`	Open a shell on the replication-manager host itself

Add grants in `api-credentials-acl-allow`:

```
api-credentials-acl-allow = "admin:cluster db proxy prov global grant show sale extrole terminal,dba:clu
```

The `terminal` keyword in the ACL grants all three terminal grants at once. Individual grants (`terminal-db`, `terminal-proxy`, `terminal-global`) allow finer control.

7.4.10 Security Notes

- **No SSH port exposure** — the browser only connects to the HTTPS API port. The SSH connection is made server-side by replication-manager.
- **JWT authentication on every connection** — the WebSocket channel is authenticated using the same RSA-signed JWT as the REST API; there is no separate credential.
- **Role-enforced DB credentials** — users never supply a database password in the terminal; replication-manager injects the appropriate credential based on the user's role.

- **Disabled by default** — `terminal-session-enabled = false` until explicitly turned on.
- **Audit trail** — all terminal session starts and ACL rejections are written to the replication-manager daemon log.

8.1.1 Alerting

replication-manager continuously monitors cluster state and emits alerts the moment a condition is detected — and resolves them automatically once it clears. Every alert is sent through all configured channels simultaneously, and a matching resolve notification follows when the state returns to normal.

8.1.2 Alert Channels

Channel	Description
Email	SMTP with optional TLS, configurable sender and recipients
Slack	Incoming webhook, custom channel and username
Microsoft Teams	Webhook with proxy support
Mattermost	Webhook with message attachments
Pushover	Mobile push notifications (app token + user token)
Custom script	Shell script or binary called on each alert/resolve event

---

8.1.3 What Triggers an Alert

Alerts fire on any cluster state transition — new error or warning states detected on the monitoring tick that were absent on the previous tick. Common triggers:

- Primary down or unreachable
- Replica lag exceeding the failover threshold
- SQL or IO replication thread stopped on a replica
- Split-brain or arbitration failure
- Binlog inconsistency between nodes
- Backup or disk space warnings
- Configuration or grant mismatches

8.1.4 Alert Lifecycle

1. **Trigger** — `SetState()` adds an error or warning code to the current cluster state
2. **Detect** — next monitoring tick compares current state to the previous tick's state
3. **Send** — codes that are new (present now, absent before) dispatch an alert to all channels
4. **Resolve** — codes that cleared (absent now, present before) dispatch a resolve notification
5. **Suppress** — the scheduler can silence alerts during planned maintenance windows using a cron expression (`alert-disable`)

8.1.5 Configuration Guide

See Configuration Guide for all SMTP, Slack, Teams, Mattermost, Pushover, and script settings.

8.2.1 Alerting Configuration Guide

replication-manager can deliver alerts through multiple channels when cluster state changes. Alerts fire when a monitored error or warning code first appears in the cluster state and again when it clears.

8.2.2 Alert Triggering

`monitoring-alert-trigger`

Description	Comma-separated list of error/warning codes that trigger an alert notification. Only codes in this list cause notifications to be sent — all other state changes are logged but do not fire the alert channels.
Type	String
Default	`"ERR00027,ERR00042,ERR00087,ERR00002,WARN0023,WARN0100,WAR`
Example	`"ERR00027,ERR00042,WARN0100"`

`monitoring-ignore-errors`

Description	Comma-separated list of error or warning codes to suppress entirely. Matching codes are never logged and never trigger alerts. Use this to silence known false positives.
Type	String
Default	`""`
Example	`"WARN0067,WARN0066"`

8.2.3 Alert Blackout Window

Alerts can be suppressed on a schedule — useful for planned maintenance windows or known noisy periods (e.g. nightly batch jobs).

`scheduler-alert-disable`

Description	Enable the scheduled alert blackout. When `true`, replication-manager silences all alert channels during the window defined by `scheduler-alert-disable-cron` and `scheduler-alert-disable-time`. Can also be toggled at runtime from the GUI.
Type	Boolean
Default	`false`

`scheduler-alert-disable-cron`

Description	Cron expression (6 fields: second minute hour day month weekday) that defines when the alert blackout window starts.
Type	String

Default	`"0 0 0 * * 0-4"` (midnight, Monday–Friday)
Example	`"0 30 22 * * *"` (22:30 every day)

`scheduler-alert-disable-time`

Description	Duration of the alert blackout window in seconds.
Type	Integer
Default	`3600` (1 hour)

8.2.4 External Script

An external script is called on every state change for every monitored server, regardless of `monitoring-alert-trigger`. It receives the affected server information as arguments.

`alert-script`

Description	Full path to a script to execute when a server state changes. The script is called with three positional arguments: server URL, previous state, current state.
Type	String
Default	`""` (disabled)
Example	`"/opt/scripts/repman-alert.sh"`

```
#!/bin/bash
# Arguments: $1=server_url  $2=previous_state  $3=current_state
echo "$(date) $1 changed from $2 to $3" >> /var/log/repman-alerts.log
```

8.2.5 Email

Email alerts require a reachable SMTP server. The mail alert fires for all state transitions.

`mail-from`

Description	Sender address for alert emails. Must be a valid user on the configured SMTP server.
Type	String
Default	`"mrm@localhost"`

`mail-to`

Description	Comma-separated list of recipient addresses. Leave empty to disable email alerting.
Type	String
Default	`""`
Example	`"ops@example.com,dba@example.com"`

mail-smtp-addr

Description	SMTP server address in `host:port` format.
Type	String
Default	`"localhost:25"`
Example	`"smtp.example.com:587"`

mail-smtp-user

Description	SMTP authentication username. Leave empty for unauthenticated relay.
Type	String
Default	`""`

mail-smtp-password

Description	SMTP authentication password.
Type	String
Default	`""`

mail-smtp-tls-skip-verify

Description	Connect to SMTP with TLS but skip certificate verification.
Type	Boolean
Default	`false`

mail-max-pool

Description	Maximum number of persistent SMTP connections to keep open. `0` disables connection pooling (a new connection is opened for each alert).
Type	Integer
Default	`0`

mail-timeout

Description	Timeout in seconds when waiting for an SMTP connection from the pool. `0` means no timeout.
Type	Integer
Default	`5`

8.2.6 Slack

Slack alerts use an incoming webhook URL. Slack alerting is global to the replication-manager instance — all clusters share the same webhook.

alert-slack-url

Description	Slack incoming webhook URL. Leave empty to disable Slack alerting.
Type	String
Default	""
Example	See Slack → App Management → Incoming Webhooks for your workspace URL

alert-slack-channel

Description	Slack channel to post alerts to.
Type	String
Default	"#support"

alert-slack-user

Description	Display name for the Slack bot posting the alert.
Type	String
Default	""

8.2.7 Pushover

Pushover delivers mobile push notifications to iOS and Android via the Pushover service.

alert-pushover-app-token

Description	Pushover application token (created at pushover.net). Leave empty to disable Pushover alerting.
Type	String
Default	""

alert-pushover-user-token

Description	Pushover user or group key — identifies who receives the notification.
Type	String
Default	""

8.2.8 Microsoft Teams

Teams alerts are sent via an incoming webhook. Alerting can be filtered to specific severity levels.

alert-teams-url

Description	Microsoft Teams incoming webhook URL. Leave empty to disable Teams alerting.
Type	String
Default	`""`

`alert-teams-proxy-url`

Description	HTTP proxy URL to use when calling the Teams webhook (for environments where outbound HTTP requires a proxy).
Type	String
Default	`""`

`alert-teams-state`

Description	Comma-separated list of severity prefixes to forward to Teams. Only state changes whose code starts with one of these prefixes are sent. Leave empty to send all triggered alerts.
Type	String
Default	`""`
Example	`"ERR"` (errors only) · `"ERR,WARN"` (errors and warnings) · `"ERR,WARN,INFO"` (all)

8.2.9 Syslog

`log-syslog`

Description	Mirror all log output to the local UDP syslog port in addition to the normal log file. Allows log aggregation systems (Graylog, Splunk, ELK) to receive replication-manager events via syslog.
Type	Boolean
Default	`false`

8.2.10 Cloud18 Integration

When the instance is registered with the Signal18 SSO, alert notifications can also be forwarded to the Signal18 Mattermost instance. These settings are global (not per-cluster).

`cloud18-alert`

Description	Enable forwarding of alerts to the Cloud18 Slack/Mattermost channel.
Type	Boolean
Default	`true`

cloud18-alert-slack-channel

Description	Mattermost channel name to post Cloud18 alerts to.
Type	String
Default	`"signal18_alert"`

cloud18-alert-slack-url

Description	Webhook URL for the Cloud18 Mattermost instance.
Type	String
Default	`"https://meet.signal18.io/hooks/…"`

cloud18-alert-slack-user

Description	Display name for the Cloud18 alert bot.
Type	String
Default	`"repman"`

8.2.11 Checking Alert Status via API

The following endpoints allow external monitoring systems to poll cluster health:

```
GET /api/status
# {"alive": "running"} or {"alive": "starting"}

GET /api/clusters/{clusterName}/status
# {"alive": "running"} or {"alive": "errors"}

GET /api/clusters/{clusterName}/topology/alerts
# Returns the current active alert list
```

Via CLI:

```
replication-manager-cli status
# running

replication-manager-cli status --cluster=my-cluster
# running  or  errors

replication-manager-cli status --cluster=my-cluster --with-errors
# JSON object listing all active errors and warnings
```

9.1.1 Provisioning

replication-manager-pro can provision and manage the complete lifecycle of database and proxy infrastructure — creating service definitions, deploying containerised or bare-metal instances, bootstrapping replication topology, and tearing everything down — across five orchestration backends.

> **Requires:** `replication-manager-pro` flavor. The `osc` release supports `onpremise` and `local` orchestrators only.

9.1.2 Orchestration Backends

Orchestrator	`prov-orchestrator`	Description
OpenSVC	`opensvc`	Service cluster with Docker/Podman micro-services across a pool of agents. Primary supported backend.
Kubernetes	`kube`	Native K8s deployment using a kubeconfig file.
SlapOS	`slapos`	Distributed service mesh from Nexedi.
On-Premise SSH	`onpremise`	SSH-based bootstrap onto existing bare-metal or VM hosts. No agent required.
Local	`local`	Processes started directly on the replication-manager host. For testing only.

9.1.3 What Can Be Provisioned

9.1.3.1 Database Services

- **Containers**: MariaDB, MySQL, Percona — any Docker/Podman image
- **Resources**: CPU cores, memory, shared memory, tmpfs
- **Disk**: size, filesystem (ext4, ZFS, Btrfs), pool, compression, device type
- **Snapshots**: daily snapshots on preferred primary, configurable retention
- **Network**: interface, netmask, gateway, CNI virtual networks
- **Bootstrap**: replication topology, initial data load (SQL/CSV), benchmark table setup
- **Config**: dynamic config application, binary log settings, max connections, expire-logs-days

9.1.3.2 Proxy Services

All proxy types share resource (memory, CPU, disk) and network configuration:

Proxy	Notes
HAProxy	TCP load balancer, read/write split
ProxySQL	Query routing, connection pooling
MaxScale	MariaDB-native proxy with binlog routing
ShardProxy	Spider-based sharding proxy
MySQL Router	Lightweight routing for InnoDB Cluster
Sphinx	Full-text search engine as a service

9.1.3.3 Service Plans

Pre-defined plans set the number of database nodes and resource tiers in a single setting (`prov-service-plan`). Plans are registered in a central registry and can be switched dynamically, triggering automatic cluster resize.

9.1.4 Bootstrap Workflow

When you provision a cluster, replication-manager executes in order:

1. Provision all database service containers/processes
2. Provision all proxy service containers/processes
3. Bootstrap the replication topology
4. Create the sponsor/monitoring user if configured
5. Wait for proxies to synchronise with the new topology
6. Optionally initialise a benchmark (Sysbench) table

Serialised provisioning mode (`prov-db-service-plan-batch`) controls whether services are started one-by-one (dependency ordering) or in parallel.

9.1.5 Configuration Guide

See the sub-sections for detailed configuration of the Software Configurator, Orchestrators, and Service Plans.

The collector API is the oldest way to deployed services in OpenSVC , a more new way is via using the cluster API delivering http2 API from any agent node.

replication-manager is using a secure client API to the OpenSVC collector. This collector is used for posting actions to a cluster of agents, fetch cluster nodes information and uploading his own set of playbooks for provisioning.

The Signal18.io SAS collector can be used for faster testing or not to have to maintain an extra piece of infrastructure, if not possible, an evaluation version of the collector needs to be installed, or it can be installed co-located to **replication-manager** or on a separate machine.

replication-manager talks to the default SAS collector or it can be setup to talk to on premise collector via the following parameters:

`opensvc-host` (1.1)

Item	Value
Description	Address of the OpenSVC collector
Type	String
Default	"ci.signal18.io:9443"
Example	"127.0.0.1:443"

`opensvc-admin-user` (1.1)

Item	Value
Description	Admin credential of the OpenSVC collector
Type	String
Default	"root@localhost.localdomain:opensvc"
Example	"root@signa18.io:secret"

To use the Signal18 SAS collector you need to login to signal18.io and request for your evaluation or licence account.yaml file. Copy it into the share/opensvc directory of **replication-manager-pro**

9.2.1.3.1 Disks

`prov-db-disk-fs` (1.1)

Item	Value
Description	Database type of FS deployment
Type	Enum
Values	xfs,ext4,zfs,ufs
Example	"zfs"

File system many drivers are available we do test xfs ext4 zfs . Many other drivers like ceph or drbd would need additional testing to be used as extra options may need to be added.

OpenSVC Agent drivers can provision disk resources on many SAN arrays and Cloud API, contact support if you need custom type of disk provisioning for your architecture.

`prov-db-disk-pool` (1.1)

Item	Value
Description	Database disk pool type for Micro Services deployment
Type	Enum
Values	none,zpool,lvm
Example	"zpool"

`prov-db-disk-type` (1.1)

Item	Value
Description	Database disk pool type for Micro Services deployment
Type	Enum
Values	loopback,physical,pool,directory
Example	"loopback"

When loopback instead of a real device the FS path is needed instead of device path

`prov-db-disk-device` (1.1)

Item	Value
Description	Database disk device path for Micro Services deployment
Type	String
Example	"/srv"

Depends on `prov-db-disk-type`

physical: define the device /dev/XXXXXXXX pool: define the pool name loopback: define the path to create the loopback file directory: define the path to create the service_name directory

9.2.1.3.2 network

Network please check availability of the ip before using them , also some opensvc deployemetn can manage range of dhcp ip and DNS entries

`prov-db-net-iface` (1.1)

Item	Value
Description	Database ethernet device.
Type	String
Example	"br0"

`prov0-db-net-gateway` (1.1)

Item	Value
Description	Database network gateway.
Type	String
Example	"192.168.1.254"

`prov-db-net-mask` (1.1)

Item	Value
Description	Database eth device
Type	String
Example	"255.255.255.0"

SlapOS use replication API call to request actions to be performed and can bootstrap software configuration via giving the following parameters

`slapos-db-partitions` (2.1)

Item	Value
Description	List databases slapos partitions path
Type	String
Default	“”
Example	“/srv/lib/slapos/partitions/0/data”

`slapos-proxysql-partitions` (2.1)

Item	Value
Description	List proxysql slapos partitions
Type	String
Default	“”
Example	“/srv/lib/slapos/partitions/0/data”

`slapos-haproxy-partitions` (2.1)

Item	Value
Description	List haproxy slapos partitions
Type	String
Default	“”
Example	“/srv/lib/slapos/partitions/0/data”

`slapos-maxscale-partitions` (2.1)

Item	Value
Description	List maxscale slapos partitions path
Type	String
Default	“”
Example	“/srv/lib/slapos/partitions/0/data”

`slapos-shardproxy-partitions` (2.1)

Item	Value
Description	List spider proxy slapos partitions path
Type	String
Default	“”
Example	“/srv/lib/slapos/partitions/0/data”

`slapos-sphinx-partitions` (2.1)

Item	Value
Description	List sphinx proxy slapos partitions path
Type	String
Default	“”
Example	“/srv/lib/slapos/partitions/0/data”

`kube-config` (2.1)

Item	Value
Description	path to K8S cluster config file
Type	String
Default	“”
Example	“/Users/apple/.kube/config”

prov-db-binary-basedir (2.1)

Item	Value
Description	Path to mysqld mariadbd or mysql_instal_db binary
Type	String
Default	"/usr/local/mysql/bin"
Example	"/usr/local/mysql/bin"

db-servers-binary-path (1.1)

Item	Value
Description	Deprecated for prov-db-binary-basedir
Type	String
Default	"/usr/local/mysql/bin"
Example	"/usr/local/mysql/bin"

haproxy-binary-path (2.0)

Item	Value
Description	Path to HaProxy
Type	String
Default	"/usr/sbin/haproxy"
Example	"/usr/local/bin/haproxy"

proxysql-binary-path (2.1)

Item	Value
Description	Path to ProxySQL
Type	String
Default	"/usr/sbin/proxysql"
Example	"/usr/sbin/proxysql"

maxscale-binary-path (2.1)

Item	Value
Description	Path to ProxySQL
Type	String
Default	"/usr/sbin/maxscale"
Example	"/usr/sbin/maxscale"

backup-mysqlclient-path (2.1)

Item	Value
Description	Path to ProxySQL
Type	String
Default	"/usr/sbin/maxscale"

Item	Value
Example	“/usr/sbin/maxscale”

Some actions during bootstrap are run via command line mysql client

9.2.4.1 Sample MariaDB from source deployed in /usr/local

prov-orchestrator = “local” prov-db-binary-basedir= “/usr/local/bin” prov-db-client-basedir= “/usr/local/bin” backup-mysqlclient-path =“/usr/local/bin/mysql” backup-mysqlbinlog-path = “/usr/local/bin/mysqlbinlog”

In **replication-manager 2.2** provision can call some extra local scripts

`prov-db-bootstrap-script` (2.2)

Item	Value
Description	Path to database bootstrap script
Type	String
Default	“”
Example	“/home/deploy/install_database_ubuntu_2004.sh”

`prov-proxy-bootstrap-script` (2.2)

Item	Value
Description	Path to proxy bootstrap script
Type	String
Default	“”
Example	“/home/deploy/install_proxy_ubuntu_2004.sh”

`prov-db-cleanup-script` (2.2)

Item	Value
Description	Path to database cleanup script
Type	String
Default	“”
Example	“/home/deploy/cleanup_database_ubuntu_2004.sh”

`prov-proxy-cleanup-script` (2.2)

Item	Value
Description	Path to proxy cleanup script
Type	String
Default	“”
Example	“/home/deploy/cleanup_proxy_ubuntu_2004.sh”

`prov-db-start-script` (2.2)

Item	Value
Description	Path to database start script
Type	String
Default	“”
Example	“/home/deploy/start_database_ubuntu_2004.sh”

`prov-db-stop-script` (2.2)

Item	Value
Description	Path to database stop script
Type	String
Default	“”
Example	“/home/deploy/stop_database_ubuntu_2004.sh”

`prov-proxy-start-script` (2.2)

Item	Value
Description	Path to proxy start script
Type	String
Default	“”
Example	“/home/deploy/start_proxy_ubuntu_2004.sh”

`prov-proxy-stop-script` (2.2)

Item	Value
Description	Path to proxy stop script
Type	String
Default	“”
Example	“/home/deploy/stop_proxy_ubuntu_2004.sh”

replication-manager (2.2) can connect via SSH using it's own user SSH private key and connect to the db or proxy host on a given sudoer root user to bootstrap the configuration files and start stop each service

We plan to extend this to deploy tar.gz binary distribution of each software in a way it does not conflict with repository distribution this would enable to upgrade and downgrade versions out of regular OS packaging

`onpremise-ssh` (2.2)

Item	Value
Description	Connect to host via SSH using user private key
Type	Boolean
Default	false
Example	true

`OnPremiseSSHPort` (2.2)

Item	Value
Description	Connect to host via SSH using ssh port
Type	Integer
Default	22
Example	2222

`onpremise-ssh-credential` (2.2)

Item	Value
Description	Connect to host via SSH using user private key
Type	String
Default	"root"
Example	"admin"

9.3.1.1 Software Configurator

The **Software Configurator** is replication-manager's built-in rules engine for generating, delivering, and tracking database and proxy configuration files. It translates a set of cluster **tags** and **hardware resource settings** into ready-to-use `my.cnf` files, directory structures, and bootstrap scripts — packaged as a `config.tar.gz` archive that an init container or SSH script can unpack directly into the service file system.

The configurator runs on every cluster. It is the sole source of truth for the configuration files deployed to each monitored database server.

9.3.1.2 How It Works

```
prov-db-tags  +  prov-db-memory / disk / iops / cores


Compliance module  (embedded opensvc/moduleset_mariadb.svc.mrm.db.json)

          evaluates tag filters → selects matching rulesets
          substitutes %%ENV:…%% template variables


  <datadir>/<cluster>/<host_port>/init/
          etc/mysql/
                  conf.d/        tag-generated .cnf fragments (symlinked)
                  rc.d/          ordered symlinks to active fragments
                  custom.d/      user overlay (01_preserved, 02_delta, 03_agreed)
          init/
                  dbjobs_new     maintenance job script
          data/
                   .system/       InnoDB undo/redo/tmp directory skeleton

config.tar.gz   ←  packaged from init/ tree, served via HTTP API
```

Each time tags or resource settings change, replication-manager regenerates all config archives. The init container (container mode) or SSH provisioner (osc mode) unpacks the archive into the live service volume on the next apply or rolling restart.

9.3.1.3 Config Discovery

When a cluster is first connected — or when you request it explicitly — replication-manager reads the **live database variables and installed plugins** and automatically derives the matching tags. This means you can point replication-manager at an existing, hand-tuned MariaDB server and have it reconstruct the tag set that describes that configuration.

Discovery maps variables to tags in the following ways:

Variable	Tag derived
`INNODB_DOUBLEWRITE=OFF`	`nodoublewrite`
`INNODB_FLUSH_LOG_AT_TRX_COMMIT 1`	`nodurable`
`INNODB_FLUSH_METHOD O_DIRECT`	`noodirect`
`LOG_BIN_COMPRESS=ON`	`compressbinlog`
`INNODB_DEFRAGMENT=ON`	`autodefrag`
`INNODB_COMPRESSION_DEFAULT=ON`	`compresstable`

Variable	Tag derived
QUERY_CACHE_SIZE=0	noquerycache
SLOW_QUERY_LOG=ON	slow
GENERAL_LOG=ON	general
PERFORMANCE_SCHEMA=ON	pfs
LOG_OUTPUT=TABLE	logtotable
HAVE_SSL=YES	ssl
READ_ONLY=ON	readonly
SKIP_NAME_RESOLVE=OFF	resolvdns
LOCAL_INFILE=ON	localinfile
LOG_BIN=OFF	nobinlog
LOG_SLAVE_UPDATES=OFF	nologslaveupdates
RPL_SEMI_SYNC_MASTER_ENABLED=ON	semisync
GTID_STRICT_MODE=ON	gtidstrict
TX_ISOLATION=READ-COMMITTED	readcommitted
LOWER_CASE_TABLE_NAMES=1	lowercasetable
USER_STAT_TABLES=PREFERABLY_FOR_QUERIES	eits
BINLOG_FORMAT=STATEMENT	statement
BINLOG_FORMAT=ROW	row
JOIN_CACHE_LEVEL=8	hashjoin
JOIN_CACHE_LEVEL=6	mrrjoin
JOIN_CACHE_LEVEL=2	nestedjoin
SQL_MODE=ORACLE	sqlmodeoracle
SQL_MODE=""	sqlmodeunstrict
Plugin BLACKHOLE installed	blackhole
Plugin QUERY_RESPONSE_TIME	userstats
Plugin SQL_ERROR_LOG	sqlerror
Plugin METADATA_LOCK_INFO	metadatalocks
Plugin SERVER_AUDIT	audit
Plugin CONNECT	connect
Plugin SPIDER	spider
Plugin MROONGA	mroonga
Plugin TOKUDB_CACHE_SIZE variable present	tokudb
Plugin ROCKSDB_BLOCK_CACHE_SIZE variable present	myrocks
Plugin S3_PAGECACHE_BUFFER_SIZE variable present	s3
Plugin CRACKLIB_PASSWORD_CHECK	pwdcheckcracklib
Plugin SIMPLE_PASSWORD_CHECK	pwdchecksimple
wsrep plugin active	wsrep

Discovery also reads memory values directly from the running server (INNODB_BUFFER_POOL_SIZE, KEY_BUFFER_SIZE, ARIA_PAGECACHE_BUFFER_SIZE, etc.) and sets prov-db-memory to the detected total — so the generated config targets the same memory footprint as the existing server.

9.3.1.4 Config Delivery

9.3.1.4.1 Container mode (pro / OpenSVC / Kubernetes)

The generated archive is served at:

GET /api/clusters/{clusterName}/servers/{host}/{port}/config

An **init container** that shares the network namespace with the database container fetches this URL at service startup and unpacks it:

```
# OpenSVC container spec
[container#0002]
type    = docker
image   = busybox
netns   = container#0001
command = sh -c 'wget -qO- http://{env.mrm_api_addr}/api/clusters/{env.mrm_cluster_name}/servers/{env.ip
```

```
# Kubernetes init container
initContainers:
- name: install
  image: busybox
  command:
  - sh
  - -c
  - 'wget -qO- http://replication-manager:10001/api/clusters/my-cluster/servers/db1/3306/config | tar xz
```

9.3.1.4.2 SSH mode (osc / onpremise)

replication-manager regenerates the archive locally and can push it to database hosts over SSH. The config is unpacked into the server's data directory, and the `dbjobs_new` script is placed in `{datadir}/init/init/dbjobs_new`.

9.3.1.4.3 Secure download

By default the config endpoint requires no credentials so init containers can bootstrap without pre-provisioned tokens. To require JWT authentication (protects embedded passwords):

```
api-credentials-secure-config = true
```

When enabled, the bootstrap script fetches a session token first using environment variables (`REPLICATION_MANAGER_USER`, `REPLICATION_MANAGER_PASSWORD`, `REPLICATION_MANAGER_URL`) and then requests the config archive with Bearer auth.

A pre-built bootstrap script that handles this flow is served at:

```
/static/configurator/opensvc/bootstrap
/static/configurator/onpremise/repository/debian/mariadb/bootstrap
/static/configurator/onpremise/repository/redhat/mariadb/bootstrap
```

9.3.1.5 Starting the Database

The configurator provides two distinct scripts for on-premise (SSH) operation — **bootstrap** and **start** — with different behavior around config fetching and data directory initialization.

9.3.1.5.1 Bootstrap vs Start

Script	When to use	Config fetch	Data directory
`bootstrap`	First-time provisioning of a new node	Always downloads fresh config	**Wipes** `/var/lib/mysql`, copies `.system` skeleton
`start`	All subsequent restarts	Conditional (see below)	Copies `.system` with `cp -rpn` — **never overwrites existing files**

Scripts are served by replication-manager at:

```
/static/configurator/onpremise/repository/debian/mariadb/bootstrap
/static/configurator/onpremise/repository/debian/mariadb/start
/static/configurator/onpremise/repository/redhat/mariadb/start
/static/configurator/onpremise/package/linux/mariadb/start
```

Both scripts receive all credentials and addressing via injected environment variables (`REPLICATION_MANAGER_URL`, `REPLICATION_MANAGER_USER`, `REPLICATION_MANAGER_PASSWORD`, `REPLICATION_MANAGER_CLUSTER_NAME`, `REPLICATION_MANAGER_HOST_NAME`, `REPLICATION_MANAGER_HOST_PORT`). See Environment Variables in the Maintenance chapter.

9.3.1.5.2 Conditional Config Fetch on Start

The `start` script does **not** unconditionally re-fetch the config archive. Before downloading anything it checks:

```
GET /api/clusters/{clusterName}/servers/{host}/{port}/need-config-fetch
```

- **200** → a new config is needed; the script downloads `config.tar.gz`, unpacks it, applies the `.cnf` files, and then starts the database
- **500** → no fetch needed; the script skips the download entirely and starts the database with the **existing config files already on disk**

This matters in normal restarts: if nothing changed on the replication-manager side since the last start, the server just starts immediately using its local config — no network round-trip to fetch the archive.

9.3.1.5.3 Controlling Config Fetch

The `need-config-fetch` response is controlled by a per-server cookie that replication-manager manages. The cookie state is driven by:

`prov-db-start-fetch-config`

Description	When `true` (default), replication-manager clears the "no-fetch" cookie on each monitoring tick, so the next start will re-fetch config. Set to `false` to permanently suppress config fetching on start — the database will always start with whatever config is already on disk.
Type	Boolean
Default	`true`

```
# Never re-fetch config on start; trust the existing files on disk
prov-db-start-fetch-config = false
```

Use `false` when you are managing config files externally (e.g., configuration management tools, manual tuning) and do not want replication-manager to overwrite them on restart.

9.3.1.5.4 Preserving an Existing my.cnf

When the `start` or `bootstrap` script copies new `.cnf` files from the archive, it applies a **non-destructive rule for `my.cnf`**:

- If `my.cnf` already exists and does **not** start with `# Generated by Signal18 replication-manager`, it is assumed to be a hand-written file. The script renames the new Signal18-generated file to `my.cnf.new` and keeps the existing `my.cnf` untouched.
- If `my.cnf` starts with `# Generated by Signal18 replication-manager`, it was placed by a previous run and is replaced normally.

To override this and force the Signal18 config to win regardless:

```
export REPLICATION_MANAGER_FORCE_CONFIG=true
```

9.3.1.5.5 The `.system` Directory

The archive always contains a `data/.system/` skeleton. This directory holds all files that must live inside the MySQL datadir but are **not** user data:

```
/var/lib/mysql/.system/
  innodb/
     undo/          InnoDB undo tablespaces (innodb_undo_directory)
     redo/          InnoDB redo log files (innodb_log_group_home_dir)
  aria/              Aria engine transaction log (aria_log_dir_path)
  tmp/               Temporary files (tmpdir)
  repl/              Replication relay logs and position files
  logs/
     error.log      MariaDB error log
     slow-query.log Slow query log
     server_audit.log  Audit plugin log
     sql_errors.log SQL error log plugin
  tokudb/            TokuDB data files
```

The generated `.cnf` fragments point InnoDB, Aria, and log variables at these paths using relative syntax (`./.system/...`). The configurator rewrites these to absolute paths depending on context:

- **Docker / container**: `./.system` → `/var/lib/mysql/.system`
- **SlapOS**: `./.system` → `{basedir}/var/lib/mysql/.system`
- **Galera (wsrep)**: system file paths are excluded from the archive entirely to avoid overwriting Galera's own state files

On **bootstrap**, the entire `/var/lib/mysql` is removed and the `.system` tree is copied in fresh — this is intentional for first-time provisioning.

On **start**, the `.system` tree is copied with `cp -rpn` which means **no file is ever overwritten**. InnoDB undo and redo logs, replication relay logs, and log files already present on disk are left completely intact. Only missing directories and files from the archive skeleton are created.

9.3.1.5.6 nosplitpath — Flat Datadir Layout

By default all `.system` sub-paths are used. If your environment cannot accommodate a `.system` hidden directory inside the datadir — for example because your backup tooling, snapshot strategy, or storage layout requires a flat datadir — add the `nosplitpath` tag:

```
prov-db-tags = "...,nosplitpath"
```

With `nosplitpath` set, all paths that normally point into `.system/` are moved to the root datadir or a dedicated location:

Path	Default (without `nosplitpath`)	With `nosplitpath`
Error log	`{datadir}/.system/logs/error.log`	`{datadir}/error.log`
Slow query log	`{datadir}/.system/logs/slow-query.log`	`{datadir}/slow-query.log`
Audit log	`{datadir}/.system/logs/server_audit.log`	`{datadir}/server_audit.log`
SQL error log	`{datadir}/.system/logs/sql_errors.log`	`{datadir}/sql_errors.log`
Replication relay logs	`{datadir}/.system/repl/`	`{datadir}/`
Maintenance job dir	`{datadir}/.system/jobs/`	`/var/lib/replication-manager-jobs/`

InnoDB undo and redo log directories follow the same pattern — the generated `.cnf` points them into

`{datadir}` rather than `{datadir}/.system/innodb/undo` and `{datadir}/.system/innodb/redo`.

When the `nosplitpath` tag is added or removed at runtime, replication-manager detects that the live `innodb_undo_directory` value no longer matches the expected configuration and sets a **config path cookie** to trigger config regeneration and a scheduled restart — the same mechanism used for any other path variable change. No manual restart scheduling is needed.

9.3.1.6 Regenerating Config

To force immediate regeneration of all config archives for a cluster (without waiting for the next monitoring tick):

```
POST /api/clusters/{clusterName}/actions/regenerate-configs
```

replication-manager re-evaluates tags and resource settings, writes new `init/` trees, and rebuilds all `config.tar.gz` archives. Servers pick up the updated config on their next init-container launch or SSH apply cycle.

9.3.1.7 Filesystem Layout

Inside the replication-manager working directory:

```
{working-dir}/{cluster-name}/{host}_{port}/
   config.tar.gz         ← packed archive served to init containers
   init/
      etc/mysql/
         conf.d/       tag-generated .cnf fragments
         rc.d/         ordered symlinks (loaded by main my.cnf)
         custom.d/     user overlays (preserved, delta, agreed)
         ssl/          TLS certificates
      init/
          dbjobs_new    maintenance job script
   01_preserved.cnf      server-specific locked variables (see Config Tracking)
   02_delta.cnf          calculated drift between expected and deployed
   03_agreed.cnf         manually accepted deviations
   preserved_variables.cnf  cluster-wide preserved variables
```

`etc/mysql/rc.d` contains numbered symlinks that control load order. `etc/mysql/custom.d` is read last, so user overlays always win over tag-generated fragments. This is where the three-layer preserved/delta/agreed files land inside the container.

See Config Tracking for a full explanation of `01_preserved.cnf`, `02_delta.cnf`, and `03_agreed.cnf`.

See Tags for the complete tag reference.

See Configuration Guide for all `prov-db-*` settings.

9.3.2.1 Database Tags

Tags are set via `prov-db-tags` (comma-separated). Each tag activates a matching ruleset in the compliance module, which contributes one or more `.cnf` fragments to the generated configuration. Tags can also be added at runtime through the GUI or API without a cluster restart.

```
prov-db-tags = "innodb,noquerycache,threadpool,slow,pfs,semisync,row,ssl,docker,linux"
```

9.3.2.1.1 Storage Engines

Tag	Effect
`innodb`	Enable InnoDB as default engine; tune buffer pool, log files, IOPS capacity
`myrocks`	Enable MyRocks (RocksDB) engine; tune block cache
`tokudb`	Enable TokuDB engine; tune TokuDB cache
`s3`	Enable S3 engine (column-store cold storage); tune S3 page cache
`spider`	Install Spider storage engine plugin
`sphinx`	Install Sphinx search engine plugin
`blackhole`	Install BLACKHOLE engine (replication topology use)
`connect`	Install CONNECT engine plugin
`mroonga`	Install Mroonga full-text search engine
`oqgraph`	Install OQGRAPH graph engine
`archive`	Install ARCHIVE engine plugin
`wsrep`	Configure Galera Cluster (wsrep) settings

9.3.2.1.2 Disk and I/O

Tag	Effect
`ssd`	Tune I/O settings for SSD (higher concurrency, `innodb_flush_neighbors=0`)
`zfs`	Enable ZFS-specific tuning (disable `O_DIRECT`, tune page size)
`nodoublewrite`	Disable InnoDB doublewrite buffer (`innodb_doublewrite=OFF`)
`noodirect`	Disable `O_DIRECT` flush method
`noaio`	Disable asynchronous I/O (`innodb_use_native_aio=0`)
`smallredolog`	Reduce InnoDB redo log size to 128 MB (dev/test)
`nodurable`	Relax durability settings (`innodb_flush_log_at_trx_commit=2`, `sync_binlog=0`)
`autodefrag`	Enable online InnoDB defragmentation (`innodb_defragment=ON`)
`compresstable`	Enable InnoDB page compression by default
`compressbinlog`	Enable binary log compression (`log_bin_compress=ON`)

9.3.2.1.3 Logging

Tag	Effect
`slow`	Enable slow query log
`general`	Enable general query log
`sqlerror`	Enable SQL error log plugin
`pfs`	Enable Performance Schema
`userstats`	Enable user statistics (QUERY_RESPONSE_TIME plugin, `userstat=ON`)
`metadatalocks`	Enable metadata lock instrumentation plugin
`audit`	Enable MariaDB Audit Plugin (SERVER_AUDIT)
`logtotable`	Route log output to `mysql.general_log` / `mysql.slow_log` tables
`diskmonitor`	Enable disk usage monitoring

9.3.2.1.4 Network and Threads

Tag	Effect
`threadpool`	Enable MariaDB thread pool
`noquerycache`	Disable query cache (`query_cache_size=0`, `query_cache_type=0`)
`resolvdns`	Enable DNS hostname resolution (`skip_name_resolve=OFF`)
`proxyprotocol`	Enable PROXY protocol support

9.3.2.1.5 Replication

Tag	Effect
`row`	Set binary log format to ROW
`statement`	Set binary log format to STATEMENT
`nobinlog`	Disable binary logging
`semisync`	Enable semi-synchronous replication
`nologslaveupdates`	Disable `log_slave_updates` on replicas
`multidomains`	Enable multi-domain replication (unique domain IDs per cluster)
`gtidstrict`	Enable GTID strict mode
`mysqlgtid`	Use MySQL-compatible GTID format
`idempotent`	Set `slave_exec_mode=IDEMPOTENT`
`lossyconv`	Allow lossy type conversions on replicas
`readonly`	Set server to read-only mode

9.3.2.1.6 Security

Tag	Effect
`ssl`	Generate and deploy TLS certificates; configure `ssl-ca`, `ssl-cert`, `ssl-key`
`pwdchecksimple`	Install `simple_password_check` plugin
`pwdcheckcracklib`	Install `cracklib_password_check` plugin (requires OS cracklib library)
`encryptfile`	Enable file-at-rest encryption
`localinfile`	Enable `LOCAL INFILE` (disabled by default for security)

9.3.2.1.7 Optimizer

Tag	Effect
`compresstables`	Enable compressed row format for InnoDB tables
`lowercasetable`	Set `lower_case_table_names=1`
`sqlmodeunstrict`	Clear SQL mode (permissive mode, no `STRICT_TRANS_TABLES`)
`sqlmodeoracle`	Set `sql_mode=ORACLE`
`noautocommit`	Disable autocommit (`autocommit=0`)
`eits`	Enable extended index and table statistics (`userstat=ON`, `use_stat_tables=PREFERABLY_FOR_QUERIES`)
`readcommitted`	Set transaction isolation to READ COMMITTED
`readuncommitted`	Set transaction isolation to READ UNCOMMITTED
`reapeatableread`	Set transaction isolation to REPEATABLE READ
`serialized`	Set transaction isolation to SERIALIZABLE
`hashjoin`	Set `join_cache_level=8` (hash join)
`mrrjoin`	Set `join_cache_level=6` (MRR join)
`nestedjoin`	Set `join_cache_level=2` (nested loop)
`subquerycache`	Enable `SUBQUERY_CACHE` optimizer switch
`semijoincache`	Enable `SEMIJOIN_WITH_CACHE` optimizer switch
`firstmatch`	Enable `FIRSTMATCH` optimizer switch
`extendedkeys`	Enable `EXTENDED_KEYS` optimizer switch
`loosescan`	Enable `LOOSESCAN` optimizer switch
`noicp`	Disable `INDEX_CONDITION_PUSHDOWN`
`nointoexists`	Disable `IN_TO_EXISTS` rewrite
`noderivedmerge`	Disable `DERIVED_MERGE`
`noderivedwithkeys`	Disable `DERIVED_WITH_KEYS`
`nomrr`	Disable `MRR` optimizer switch
`noouterjoincache`	Disable `OUTER_JOIN_WITH_CACHE`
`nosemijoincache`	Disable `SEMI_JOIN_WITH_CACHE`
`notableelimination`	Disable `TABLE_ELIMINATION`

9.3.2.1.8 Character Set

Tag	Effect
`bm4ci`	Set `character_set_server=utf8mb4`, case-insensitive collation
`bm4cs`	Set `character_set_server=utf8mb4`, case-sensitive collation
`utf8ci`	Set `character_set_server=utf8`, case-insensitive collation
`utf8cs`	Set `character_set_server=utf8`, case-sensitive collation

9.3.2.1.9 Platform / Orchestrator

Tag	Effect
`docker`	Adjust paths for container filesystems (`.system` → `/var/lib/mysql/.system`)
`linux`	Use Linux-specific startup and socket paths
`rpm`	Use RPM-style paths (`/etc/my.cnf.d/`, Red Hat layout)
`package`	Use Debian/Ubuntu package layout

Tag	Effect
`nosplitpath`	Flatten all `.system` sub-paths back into the root datadir. Logs, replication relay files, and InnoDB undo/redo directories are placed directly in `{datadir}` instead of `{datadir}/.system/…`. The maintenance job directory moves to `/var/lib/replication-manager-jobs`. Use this when backup tooling, snapshots, or storage layout cannot accommodate a hidden `.system` subdirectory inside the datadir. Adding or removing this tag at runtime triggers automatic config regeneration and a scheduled restart.

9.3.2.2 Proxy Tags

Proxy tags are set via `prov-proxy-tags`. Available tags for ProxySQL, HAProxy, and MaxScale:

Tag	Effect
`readonmaster`	Allow read queries on the primary server (set automatically from `db-servers-read-on-master`)
`ssl`	Enable TLS on proxy connections
`docker`	Container filesystem paths
`linux`	Linux socket and init paths

9.3.2.3 Compliance Module

Tags are matched against the embedded compliance module (`share/opensvc/moduleset_mariadb.svc.mrm.db.json`). Each tag corresponds to a **filterset** (fset) in the module. A filterset groups one or more rulesets that each contribute:

- A `.cnf` file fragment placed under `etc/mysql/conf.d/`
- A symlink in `etc/mysql/rc.d/` that controls load order
- Optional `# mariadb_command:` or `# mariadb_default:` SQL lines for runtime application

You can supply a custom compliance module file instead of the embedded one:

```
prov-db-compliance  = "/etc/replication-manager/custom_db_module.json"
prov-proxy-compliance = "/etc/replication-manager/custom_proxy_module.json"
```

9.3.3.1 Config Tracking

The configurator does not just generate a one-time config archive — it continuously monitors the live database configuration and tracks **drift** between what replication-manager expects to be deployed and what is actually running. This lets you detect unplanned changes, preserve intentional deviations, and keep config in a known state across rolling restarts and node replacements.

9.3.3.2 Three-Layer Config System

Each server's data directory contains three override files that sit in `etc/mysql/custom.d/` inside the config archive (read after all tag-generated fragments):

```
{datadir}/{cluster}/{host_port}/
  01_preserved.cnf    server-specific locked values
  02_delta.cnf        calculated drift (expected vs deployed)
  03_agreed.cnf       manually accepted deviations
```

These files are not generated from tags — they represent the **state of the deployed server** relative to the configurator's expectations, and are managed by replication-manager automatically (with overrides you can set via API or GUI).

9.3.3.2.1 01_preserved.cnf — Server-specific locked values

Variables listed here are **frozen** for this server. The configurator will not attempt to change them during regeneration. Use this for server-specific paths, hardware-specific tuning, or any setting that legitimately differs from the rest of the cluster.

Example: a server with a different disk layout needs a custom `innodb_data_home_dir`:

```
[mysqld]
# Preserved - non-standard disk layout on this host
innodb_data_home_dir = /data2/mysql
```

9.3.3.2.2 02_delta.cnf — Calculated drift

Written automatically by replication-manager on every monitoring tick. It contains variables where the **deployed (on-disk) value differs from the expected (tag-generated) value**. This file tells you exactly what diverged and by how much.

The delta file has a 4-layer safety framework for writing runtime values: 1. Empty values are never written 2. Read-only variables (e.g. `version`, `have_ssl`) are excluded 3. Only whitelisted runtime-fallback variables are included 4. Failed servers are excluded to avoid partial state

If the delta is empty after a config regeneration and restart, the server is fully converged.

9.3.3.2.3 03_agreed.cnf — Manually accepted deviations

When a delta value is reviewed and accepted — either by an operator using the GUI, or via the API — the variable moves from `02_delta.cnf` to `03_agreed.cnf`. This signals that the deviation is **known and intentional**, not drift to be remediated.

Accepting a variable via API:

```
POST /api/clusters/{clusterName}/servers/{serverName}/actions/set-variable-accepted
Body: {"variable": "MAX_CONNECTIONS"}
```

9.3.3.3 Cluster-Wide Preserved Variables

In addition to per-server preserved files, you can define **cluster-wide** variables that must be preserved across all servers. These are stored in:

`{working-dir}/{cluster-name}/preserved_variables.cnf`

This file is readable and writable via the API:

```
GET  /api/clusters/{clusterName}/preserved-vars-cnf
POST /api/clusters/{clusterName}/preserved-vars-cnf
```

The file format is a my.cnf-style `[mysqld]` section, with optional per-server exclusions:

```
[mysqld]
# Variables preserved cluster-wide during config regeneration

# This variable will be preserved on all servers
MAX_CONNECTIONS = 500

# This variable will be preserved on all servers EXCEPT the listed ones
# INNODB_BUFFER_POOL_SIZE = 4096 # exclude: server1,server2
```

Changes written via the POST endpoint are applied immediately without a restart — replication-manager reloads preserved variables automatically.

9.3.3.4 Configuration Keys

Two config flags control preserved variable behavior during config tar.gz generation:

`prov-db-config-preserve`

Description	Copy the preserved variables into the generated `config.tar.gz`. When `true` (default), `01_preserved.cnf`, `02_delta.cnf`, and `03_agreed.cnf` are included in `custom.d/` inside the archive so the next init-container launch picks them up. Set to `false` to get a clean config from tags only.
Type	Boolean
Default	`true`

`prov-db-config-preserve-vars`

Description	Semicolon-separated list of variable names (or `name=value` pairs) to add to `01_preserved.cnf` at every regeneration. Use this to hard-code values that must survive any config update. Example: `innodb_data_home_dir=/var/lib/mysql;max_connections=1000`
Type	String
Default	`""`

9.3.3.5 Monitoring Config Changes

replication-manager detects when the deployed config changes on disk (checksum comparison). When a change is detected:

1. Preserved variables are reloaded from `preserved_variables.cnf`
2. The delta file (`02_delta.cnf`) is recalculated for the affected server
3. A log entry is written at INFO level

This happens within the normal monitoring loop interval (`monitoring-ticker`), so config drift is detected quickly without polling overhead.

9.3.3.6 Reading Variables from Config Files

Before the database is running (e.g., during provisioning or after a crash), replication-manager can read configuration values directly from the deployed `.cnf` files rather than from `SHOW VARIABLES`. This allows it to know the intended configuration even for offline servers.

The read order mirrors MySQL's include chain: 1. Main `my.cnf` with `includedir` directives 2. `conf.d/` fragments (tag-generated) 3. `custom.d/` overrides (preserved, delta, agreed)

Variables read this way are used for topology decisions, failover calculations, and provisioning validation.

9.3.4.1 Configuration Guide

All configurator settings are per-cluster (set under the `[cluster-name]` section or in `[DEFAULT]`).

9.3.4.2 Database Resource Settings

These settings describe the hardware allocated to each database service. The configurator uses them to size buffer pools, log files, thread counts, and IOPS capacity automatically.

`prov-db-memory`

Description	Total memory in MiB allocated to the database service. The configurator distributes this across engines according to `prov-db-memory-shared-pct`.
Type	String
Default	`"256"`
Example	`"4096"`

`prov-db-memory-shared-pct`

Description	How `prov-db-memory` is split across shared global buffers. Comma-separated `engine:percent` pairs. The `threads` key reserves a share for per-thread buffers (sized by `prov-db-memory-threaded-pct`).
Type	String
Default	`"threads:16,innodb:60,myisam:10,aria:10,rocksdb:1,tokudb:`

`prov-db-memory-threaded-pct`

Description	How the `threads` share from `prov-db-memory-shared-pct` is split across per-connection thread buffers.
Type	String
Default	`"tmp:70,join:20,sort:10"`

`prov-db-disk-size`

Description	Disk space in GiB for the database data volume.
Type	String
Default	`"20"`

`prov-db-disk-iops`

Description	Provisioned random IOPS for the database disk. Used to size `innodb_io_capacity` (set to IOPS/3) and `innodb_io_capacity_max` (set to IOPS).
Type	String
Default	`"300"`

prov-db-disk-iops-latency

Description	Average disk I/O latency in seconds. Used with `prov-db-disk-iops` to calculate the optimal number of InnoDB write I/O threads (`innodb_write_io_threads = latency × iops`).
Type	String
Default	`"0.002"`

prov-db-cpu-cores

Description	Number of CPU cores available to the database service. Sets `thread_pool_size` and `innodb_read_io_threads`.
Type	String
Default	`"1"`

9.3.4.3 Tag Settings

prov-db-tags

Description	Comma-separated list of compliance tags applied to all database servers in this cluster. Each tag activates one or more ruleset fragments in the compliance module.
Type	String
Default	`"semisync,row,innodb,noquerycache,threadpool,slow,pfs,doc`
Example	`"innodb,semisync,row,slow,pfs,ssl,threadpool,noquerycache`

prov-proxy-tags

Description	Comma-separated list of compliance tags applied to proxy servers (ProxySQL, HAProxy, MaxScale).
Type	String
Default	`""`

9.3.4.4 Config Preservation

prov-db-config-preserve

Description	Include the three-layer override files (`01_preserved.cnf`, `02_delta.cnf`, `03_agreed.cnf`) in the generated `config.tar.gz`. Set to `false` to generate a clean config from tags only (useful for resetting a server to a known-good state).
Type	Boolean

Default	`true`

`prov-db-config-preserve-vars`

Description	Semicolon-separated list of variable names or `name=value` pairs to hard-code into `01_preserved.cnf` on every regeneration. Values listed here survive all tag changes and config updates.
Type	String
Default	`""`
Example	`"innodb_data_home_dir=/var/lib/mysql;max_connections=1000"`

9.3.4.5 Compliance Module Override

`prov-db-compliance`

Description	Path to a custom compliance module JSON file for database configuration. If unset, replication-manager uses the embedded `moduleset_mariadb.svc.mrm.db.json`.
Type	String
Default	`""` (use embedded module)
Example	`"/etc/replication-manager/custom_db_module.json"`

`prov-proxy-compliance`

Description	Path to a custom compliance module JSON file for proxy configuration.
Type	String
Default	`""` (use embedded module)

9.3.4.6 Replication and Domain

`prov-db-expire-log-days`

Description	Value for `expire_logs_days` in the generated configuration.
Type	Integer
Default	`5`

`prov-db-max-connections`

Description	Value for `max_connections` in the generated configuration.
Type	Integer
Default	`1000`

prov-domain

Description	MariaDB replication domain ID. Used to avoid conflicts in multi-source setups. If set to `"0"` and the cluster has a `master-conn` (multi-source), the domain is calculated automatically from a CRC32 of the cluster name.
Type	String
Default	`"0"`

9.3.4.7 Binary Distribution

prov-db-binary-in-tarball

Description	Bundle the MariaDB/MySQL binary tarball inside `config.tar.gz` for air-gapped environments. Requires `prov-db-binary-tarball-name` to be set.
Type	Boolean
Default	`false`

prov-db-binary-tarball-name

Description	Filename of the binary tarball to bundle. The tarball is fetched from the URL returned by the `GetTarballUrl` lookup before being included.
Type	String
Default	`"mysql-8.0.17-macos10.14-x86_64.tar.gz"`

9.3.4.8 Start Behavior

prov-db-start-fetch-config

Description	When `true`, replication-manager signals each server to re-fetch the config archive on the next start (the `start` script checks `/need-config-fetch` and downloads a fresh `config.tar.gz`). Set to `false` to suppress config fetching — the database starts with whatever config files are already on disk.
Type	Boolean
Default	`true`

9.3.4.9 Bootstrap Scripts

prov-db-bootstrap-script

Description	Path to a custom shell script run on the database host at first bootstrap (after the config archive is unpacked). If unset, the built-in bootstrap script for the target OS/package layout is used.
Type	String
Default	`""`

`prov-proxy-bootstrap-script`

Description	Path to a custom shell script run on proxy hosts at first bootstrap.
Type	String
Default	`""`

9.3.4.10 Security

`api-credentials-secure-config`

Description	Require a valid JWT Bearer token to download `config.tar.gz` from the `/api/clusters/{name}/servers/{host}/{port}/config` endpoint. When `false` (default), the endpoint is open so init containers can bootstrap without pre-provisioned credentials.
Type	Boolean
Default	`false`

9.3.4.11 Proxy Resources

`prov-proxy-memory`

Description	Memory in MiB allocated to proxy service containers.
Type	String
Default	`"256"`

`prov-proxy-cpu-cores`

Description	CPU cores allocated to proxy service containers.
Type	String
Default	`"1"`

`prov-proxy-disk-size`

Description	Disk in GiB for proxy service containers.
Type	String

Default	"2"

From **replication-manager 2.1** a service plan can be used to enable pre cap cluster resource deployment. In this scenario external API can be used to select a plan and a placement to external available orchestrator clusters

A list of plans can be exposed inside some external API:

http://gsx2json.com/api?id=130326CF_SPaz-flQzCRPE-w7FjzqU1NqbsM7MpIQ_oU&sheet=1&columns=false

If a plan is set for a cluster, **replication-manager** auto set provisioning variables for the resources describe by the plan
Using ACL **replication-manager** can restrict later user change of such resources.

Clusters and nodes selection will be enable via an external API that will send such request

in: {"region": "eu", "n_instances": 2, "req_cpu": 2, "req_mem": "100g", "req_swap": 0}

And is waiting for similar answers

resp: [{"cluster_id": , "cluster_name": "cl1", "nodes": [{"nodename": "node1", "az": "fr1"}, {"nodename": "node2", "az": "de1"}, …]}, {"cluster_id": , "cluster_name": "cl2", "nodes": [{"nodename": "node1", "az": "it1"}, {"nodename": "node2", "az": "cz1"}, …]}, …]

ACL describe how **replication-manager** interact with user for placement prov-placement-manual prov-placement-first prov-placement-interactive

10.1.1 Building from source

- Download a Go release (Go 1.8 minimum): https://golang.org/dl/ or use your system's release if >= 1.8
- Create a build directory: `mkdir -p ~/go`
- Point GOPATH environment variable to this directory: `export GOPATH=~/go`
- Clone the source: `go get github.com/signal18/replication-manager`
- Compile and install: `go install github.com/signal18/replication-manager`
- Add the go binaries directory to your path: `export PATH=$PATH:~/go/bin`
- Compile all binaries: `make bin`

10.1.2 Compilation flags

It is possible to disable features via the following tags

Flags	Value
WithProvisioning	ON/OFF
WithArbitration	ON/OFF
WithArbitrationClient	ON/OFF
WithProxysql	ON/OFF
WithHaproxy	ON/OFF
WithMaxscale	ON/OFF
WithMariadbshardproxy	ON/OFF
WithMonitoring	ON/OFF
WithMail	ON/OFF
WithHttp	ON/OFF
WithSpider	ON/OFF
WithEnforce	ON/OFF
WithDeprecate	ON/OFF
WithTargz	ON/OFF

10.2.0.1 Non-regression tests

replication-manager can play some predefined scenarios in a non-regression testing framework The tests can be called via the command line client.

replication-manager is allowed to play tests only when setting the `test` variable in the predefined testing cluster in the configuration file or via using *replication-manager-tst** monitor binaries. where this flag is always on.

You probably don't want to play tests on an existing cluster, so instead of that **replication-manager-tst** or **replication-manager-pro** can provision a testing cluster when the test is starting and unprovision it when the test finishes. Some client parameters can disable the auto provisioning. If you want to get access to the running servers such as examine the database logs and more.

replication-manager-tst provisions a local host setup using different ports. Because haproxy is supported on multiple OSes, it's the preferred way to perform local testing.

```
[Cluster_Test_2_Nodes]
title = "Cluster_Test_2_Nodes"
db-servers-hosts = "127.0.0.1:3310,127.0.0.1:3311"
db-servers-preferedd-master = "127.0.0.1:3310"
db-servers-credential = "root:xxxx"
db-servers-connect-timeout = 1
replication-credential = "root:xxxx"
haproxy = true
haproxy-binary-path = "/usr/sbin/haproxy"
haproxy-write-port=3303
haproxy-read-port=3304
test=true
mariadb-binary-path = "/usr/local/mysql/bin"
sysbench-binary-path = "/usr/sbin/sysbench"
sysbench-threads = 4
sysbench-time = 60
```

Run the TST release with extra parameters not defined in configuration files as on OSX:

```
replication-manager-tst --config=etc/config.toml.sample.tst.masterslave-haproxy  monitor   --haproxy-bin
```

Some tests are requiring sysbench and haproxy, please refer to the Configuration Dependencies section to proceed further.

10.2.0.1.1 Command line print all tests

```
./replication-manager-cli --config=/etc/replication-manager/mrm.cnf --config-group=cluster_test_2_nodes
INFO[2017-02-22T21:40:02+01:00] [testSwitchOverLongTransactionNoRplCheckNoSemiSync testSwitchOverLongQue
```

Command-line running some tests via passing a list of tests in run-tests ALL is a special test to run all available tests.

```
./replication-manager-cli --config=/etc/replication-manager/cluster_test_2_nodes.cnf --cluster=cluster_t
```

Test returns some JSON structure that get useful informations for debugging

PASS FAIL ERR

10.3.0.1 Report Bugs

Bugs can be reported in github https://github.com/signal18/replication-manager/issues for a list of issues.

Bugs will be proceed in priority when report under a support subscription via http:/support.signal18.io

10.4.0.1 Profiling CPU usage

Take a 30s CPU profile go tool pprof http://localhost:10001/debug/pprof/profile top5

See proof documentation

11.1.1 Plugin Architecture

Available since: replication-manager **v3.1.24**

replication-manager extends its monitoring capabilities through a plugin system. On every monitoring tick each enabled plugin is invoked, receives a full snapshot of the monitored server state as JSON on its standard input, and writes findings and/or score checks as JSON on its standard output.

Plugins are organised in three tiers:

Tier	Implementation	Availability	Examples
Static	In-process Go functions, no subprocess	All instances, no registration required	`errorlog`, `sqlerrorlog`, `slowlog`, `auditlog`
Community	External compiled binaries	Instances registered at gitlab.signal18.io (free)	All workload, security and score plugins
Enterprise	External compiled binaries	Signal18 Support Contract customers	`plugin-critical-alerts`

Community plugins are the external plugin library shipped with replication-manager. Registering your instance at gitlab.signal18.io unlocks the full community plugin set. Registration is free — it allows Signal18 to understand how the product is used in production so the roadmap can be focused where it creates the most value.

Enterprise plugins are developed from Signal18's field experience supporting production deployments and are available to customers with an active Support Contract.

11.1.2 Lifecycle — External Plugins

```
Monitor tick (every monitoring-ticker seconds)

  for each server in cluster

        for each enabled external plugin

              Build wire.Request JSON (server snapshot)
              exec plugin binary  ←  stdin: JSON Request
                                     stdout: JSON Response
              Decode wire.Response
              Append findings → SecurityStateMachine / LogStateMachine
              Append score_checks → SecurityScore

  SecurityStateMachine.GetOpenStates()
         feed remediation engine, alerts, dashboard
```

Plugins are **stateless** and **ephemeral**: a new process is spawned for every evaluation and exits after writing one JSON response. State across ticks must be kept by replication-manager itself (state machines, Graphite metrics).

11.1.3 Wire Protocol — JSON stdin/stdout

All plugins share the types defined in `cluster/logplugin/plugins/wire/wire.go`.

11.1.3.1 Request

One JSON object written to the plugin's stdin before it starts reading:

Field	Go type	Description
`server_url`	`string`	`host:port` of the monitored server
`server_variables`	`map[string]string`	Full `SHOW GLOBAL VARIABLES` snapshot (lowercase keys)
`database_users`	`[]DBUser`	`mysql.user` rows — no credential hashes
`cluster_context`	`ClusterContext`	Cluster-level facts (backup encryption, proxy presence, …)
`error_log`	`[]Msg`	Recent error log lines
`slow_log`	`[]SlowMsg`	Recent slow query log entries
`process_list`	`[]Process`	`SHOW PROCESSLIST` snapshot
`binlog_events`	`[]BinlogEvent`	Recent decoded binlog QUERY events
`config`	`map[string]string`	Per-plugin configuration key-value pairs

DBUser fields:

Field	Type	Description
`user`	string	Account username (`""` = anonymous)
`host`	string	Account host (`%` = wildcard)
`plugin`	string	Authentication plugin name
`password_empty`	bool	`true` when `authentication_string` is empty
`account_locked`	bool	`true` when `ACCOUNT LOCKED`

11.1.3.2 Response

One JSON object written to stdout before the process exits:

Field	Go type	Description
`findings`	`[]Finding`	Security or operational findings
`score_checks`	`[]ScoreCheck`	Named pass/fail compliance checks

Finding fields:

Field	Type	Description
`err_key`	string	Unique code, e.g. `SEC0103`, `WARN0301`
`severity`	string	`"WARNING"`, `"ERROR"`, or `"SECURITY"`
`description`	string	Human-readable explanation including server URL

ScoreCheck fields:

Field	Type	Description
`name`	string	Check identifier, e.g. `HasTableEncryption`
`pass`	bool	`true` = check passes
`description`	string	Explanation

11.1.4 Plugin Configuration

Each plugin receives its configuration through two mechanisms, in priority order:

1. **`config` map in the JSON Request** — set via the replication-manager GUI or TOML file under `[cluster.plugin-config.plugin-name]`
2. **Scoped environment variables** — fallback when the config map key is absent

Environment variables follow the naming pattern `REPMAN_<PLUGINNAME>_<KEY>` where `<PLUGINNAME>` is the plugin name without the `plugin-` prefix, uppercased with hyphens converted to underscores.

Example — `plugin-connection-storm`:

```
[mycluster.plugin-config.plugin-connection-storm]
sleep-ratio-threshold = 0.75
min-connections = 20
```

Equivalent environment variables (fallback):

```
REPMAN_CONNECTION_STORM_SLEEP_RATIO_THRESHOLD=0.75
REPMAN_CONNECTION_STORM_MIN_CONNECTIONS=20
```

Helper functions in `wire.go` read from both sources transparently:

```
threshold := wire.CfgFloat(req.Config, "sleep-ratio-threshold",
    wire.EnvFloat("REPMAN_CONNECTION_STORM_SLEEP_RATIO_THRESHOLD", 0.60))
```

11.1.5 Plugin Discovery and Loading

External plugins are placed in the cluster's plugins directory. At startup and on reload replication-manager scans the directory and registers every executable file whose name matches a known plugin prefix via `LoadPluginsFromDir`.

Built-in plugins are registered in `logplugin.GlobalRegistry`. External plugins are registered in the per-cluster `cluster.pluginRegistry`. The `/api/clusters/{name}/plugins` endpoint returns the combined list from the per-cluster registry.

11.1.5.1 Plugin Signing

When `plugin-signing-public-key` is configured, replication-manager verifies every external plugin binary against an Ed25519 signature stored in a `.sig` sidecar file under `<share>/plugins/` before executing it. Plugins that fail verification are rejected and logged as errors — they are never executed.

```
plugin-signing-public-key = "/etc/replication-manager/plugin-signing.pub"
```

The public key ships with the package at `<ShareDir>/plugins/plugin-signing.pub`. To sign a plugin binary:

```
# Generate key pair (once)
replication-manager plugin-key-gen
```

```
# Sign a plugin
replication-manager plugin-sign --binary share/plugins/plugin-connection-storm \
    --key etc/plugin-signing.key \
    --out share/plugins/plugin-connection-storm.sig
```

11.1.6 Security Considerations

- **Process isolation:** each plugin runs as a child process of replication-manager with the same OS user. No elevated privileges are granted.
- **No network access required:** plugins operate entirely on the JSON snapshot passed via stdin; they do not need database credentials.
- **No persistent state:** plugins must not write files or modify system state. All output goes to stdout.
- **Signature verification:** in production environments always enable `plugin-signing-public-key` to prevent execution of tampered binaries.
- **Timeout:** plugin processes are killed if they do not produce a response within the monitoring tick timeout. A timed-out plugin emits an error log line and contributes no findings for that tick.

11.1.7 Global Plugin Settings

Config key	Default	Description
`log-plugin`	`false`	Master switch — enable the external plugin evaluation loop
`log-level-plugin`	`0`	Verbosity for the plugin subsystem (0=off … 4=debug)
`plugin-signing-public-key`	`""`	Path to Ed25519 public key for binary verification

11.2.1 Built-in Log Plugins

The four built-in plugins run **in-process** — they are Go functions registered in `logplugin.GlobalRegistry`, not external binaries. They share the same evaluation loop as external plugins but execute without spawning a subprocess.

Built-in plugins use **Graphite-backed spike detection**: a rolling baseline is maintained over a configurable window and a finding is raised when the current value exceeds `baseline + spike-sigma ×` . This makes them adaptive to the normal workload of each individual server.

11.2.2 errorlog

Source: `cluster/logplugin/errorlog.go`

Analyses the MariaDB/MySQL error log for spikes in warning and error messages. Useful for catching sudden increases in replication errors, InnoDB warnings, authentication failures, and other server health indicators.

Configuration:

Key	Default	Description
`timeframe-hours`	`1`	Rolling window of error log lines to inspect
`min-log-level`	`Warning`	Minimum severity — only lines at or above this level are counted
`spike-sigma`	`2`	Standard deviations above rolling mean to trigger a finding

Min log level values:

Value	Lines included
`System`	Startup and shutdown messages only
`Note`	Informational and above
`Warning`	Warnings and errors *(default)*
`ERROR`	Errors only

Configuration example:

```
[mycluster.plugin-config.errorlog]
enabled = true
timeframe-hours = 2
min-log-level = "Warning"
spike-sigma = 3
```

11.2.3 sqlerrorlog

Source: `cluster/logplugin/sqlerrorlog.go`

Analyses the SQL error log (the log of SQL statements that returned errors) for spikes. High error rates in the SQL log can indicate application bugs, connection pool exhaustion, or privilege misconfigurations.

Configuration:

Key	Default	Description
`timeframe-hours`	1	Rolling window of SQL error log lines
`spike-sigma`	2	Standard deviations above rolling mean

11.2.4 slowlog

Source: `cluster/logplugin/slowlog.go`

Analyses the slow query log for spikes in slow query count and latency. A sudden spike indicates a query regression, a missing index being dropped, or an unexpected table scan caused by data growth.

Configuration:

Key	Default	Description
`timeframe-hours`	1	Rolling window of slow log entries
`spike-sigma`	2	Standard deviations above rolling mean

Note: the slow query log must be enabled on the server (`slow_query_log = ON`, `long_query_time` set appropriately) for this plugin to receive data.

11.2.5 auditlog

Source: `cluster/logplugin/auditlog.go`

Analyses the MariaDB audit log for activity drift. Instead of simple spike detection, `auditlog` compares a **current window** against a longer **baseline window** to detect unusual shifts in the volume or pattern of audited operations. This catches slow-building anomalies that a pure spike detector would miss.

Configuration:

Key	Default	Description
`current-window-hours`	1	Recent audit log window for current activity baseline
`baseline-window-hours`	24	Historical window for expected activity comparison
`spike-sigma`	2	Standard deviations of drift to trigger a finding

Note: requires the `server_audit` plugin to be loaded on the server (`server_audit_logging = ON`).

11.2.6 Spike Detection Algorithm

All four built-in plugins share the same underlying algorithm implemented in `cluster/logplugin/spike.go`:

1. Collect the metric (e.g. error count) from all log lines within `timeframe-hours`
2. Load the rolling history of that metric from Graphite
3. Compute `mean` and (standard deviation) over the history window
4. If `current_value > mean + spike_sigma ×` : raise a finding

5. Store the current value back into Graphite for future baseline calculation

The Graphite storage is local to the replication-manager instance. No external Graphite server is required — replication-manager ships its own embedded metric store.

Tuning `spike-sigma`:

Value	Sensitivity
`1.0`	Very sensitive — many false positives on bursty workloads
`2.0`	Balanced *(default)* — good for most production systems
`3.0`	Conservative — only fires on clear anomalies
`4.0+`	Very conservative — use on highly variable workloads

11.3.1 Workload Plugins

Tier: Community — requires a free registered instance at gitlab.signal18.io

Workload plugins detect **operational anomalies** — performance problems, resource saturation, and regressions — by analysing the real-time server state passed through the wire protocol. They emit findings with `WARN` error keys that are routed to the main HA log and can trigger alerts.

All workload plugins are **external binaries** located in the cluster plugins directory.

11.3.2 plugin-innodb-corruption

Binary: `plugin-innodb-corruption`
Finding: `WARN0300`
Source: `cluster/logplugin/plugins/plugin-innodb-corruption/main.go`

Scans the error log for InnoDB corruption indicators within the last 24 hours. Keywords include `corruption`, `corrupted`, `Database page corruption`, `InnoDB: Error: page`, `space id and page`, `is in the future`, and `checksum mismatch`.

Prerequisites: none — the error log is always available.

11.3.3 plugin-connection-storm

Binary: `plugin-connection-storm`
Finding: `WARN0307`
Source: `cluster/logplugin/plugins/plugin-connection-storm/main.go`

Detects connection pool saturation through two complementary signals:

1. **Sleep ratio** — when `sleeping_connections / total_connections` `sleep-ratio-threshold` a connection leak is indicated: clients are opening connections but not closing them, and the server is spending capacity maintaining idle sessions.
2. **Lock waits** — when the number of threads stuck waiting on any lock (metadata lock, table lock, row lock) reaches `lock-wait-count`, a lock storm is indicated.

Evaluation is skipped entirely when total connections are below `min-connections` to avoid false positives on idle servers.

Prerequisites: none — uses `SHOW PROCESSLIST`, available on all servers.

Configuration:

Key	Default	Description
`sleep-ratio-threshold`	`0.60`	Fraction of sleeping / total connections to trigger
`lock-wait-count`	`3`	Concurrent threads in lock-wait state to trigger
`min-connections`	`10`	Minimum total connections before evaluating

11.3.4 plugin-error-storm

Binary: `plugin-error-storm`
Finding: `WARN0302`
Source: `cluster/logplugin/plugins/plugin-error-storm/main.go`

Groups error log entries by a **template fingerprint** (numbers and quoted string literals are stripped so that variant messages with different values are counted together as one template) and fires when any template appears `storm-threshold` or more times within `storm-window-mins` minutes.

Both the MariaDB/MySQL error log and the SQL error log are scanned.

Prerequisites: none — the error log is always available.

Configuration:

Key	Default	Description
`storm-threshold`	`10`	Occurrences of the same error template to trigger
`storm-window-mins`	`5`	Rolling time window in minutes

11.3.5 plugin-full-table-scan-spike

Binary: `plugin-full-table-scan-spike`
Finding: `WARN0304`
Source: `cluster/logplugin/plugins/plugin-full-table-scan-spike/main.go`

Reads `performance_schema.events_statements_summary_by_digest` and fires when **both** conditions hold simultaneously:

- `full_scan_executions / total_executions  scan-ratio-threshold`
- `full_scan_executions  min-full-scan-count`

Digests with fewer than `min-exec-count` total executions are excluded to filter out one-off or infrequent queries. The finding reports the top three offending query digests.

Prerequisites:

Variable	Required value	Notes
`performance_schema`	`ON`	Must be enabled at server startup
`performance_schema_consumer_events_statements_summary_by_digest`	`ON`	Usually ON by default when performance_schema is enabled

Configuration:

Key	Default	Description
`scan-ratio-threshold`	`0.30`	Fraction of full-scan executions / total to trigger
`min-full-scan-count`	`10`	Absolute minimum full-scan count required
`min-exec-count`	`5`	Minimum digest execution count to include

11.3.6 plugin-metadata-lock-contention

Binary: `plugin-metadata-lock-contention`
Finding: `WARN0305`
Source: `cluster/logplugin/plugins/plugin-metadata-lock-contention/main.go`

Reads `information_schema.METADATA_LOCK_INFO` (requires the MariaDB `METADATA_LOCK_INFO` plugin) and fires when either:

- Any single MDL wait duration `lock-wait-ms-threshold` milliseconds, **or**
- The total number of concurrent MDL waits `lock-count-threshold`

Metadata lock waits occur when a DDL statement (`ALTER TABLE`, `DROP TABLE`) holds an exclusive metadata lock, blocking all concurrent DML on the affected table until the DDL completes.

Prerequisites:

Requirement	How to enable
MariaDB only	Not available on MySQL (no `METADATA_LOCK_INFO` plugin)
`metadata_lock_info` plugin	`INSTALL SONAME 'metadata_lock_info'` on each monitored server

Configuration:

Key	Default	Description
`lock-wait-ms-threshold`	`5000`	Single MDL wait duration in ms to trigger
`lock-count-threshold`	`3`	Concurrent MDL waits count to trigger

11.3.7 plugin-replication-lag-predictor

Binary: `plugin-replication-lag-predictor`
Finding: `WARN0303`
Source: `cluster/logplugin/plugins/plugin-replication-lag-predictor/main.go`

Detects DML write bursts in the slow log *before* lag appears in `seconds_behind_master`. High DML rates in the slow log predict that the binary log will grow faster than the replica can consume it.

Fires when: `DML_count_in_window / window_mins` `write-rate-threshold queries/min`

DML verbs counted: `INSERT`, `UPDATE`, `DELETE`, `REPLACE`, `LOAD DATA`.

Prerequisites:

Variable	Required value	Notes
`slow_query_log`	`ON`	Enable the slow query log
`long_query_time`	e.g. `0` or `1`	Set low enough to capture DML-heavy queries

Configuration:

Key	Default	Description
`window-mins`	`5`	Observation window in minutes
`write-rate-threshold`	`50`	DML queries per minute to trigger

11.3.8 plugin-slow-query-regression

Binary: `plugin-slow-query-regression`
Finding: `WARN0301`
Source: `cluster/logplugin/plugins/plugin-slow-query-regression/main.go`

Compares the current slow-log average latency against the PFS historical baseline for each query digest. A query is flagged as regressed when:

`current_avg_ms / pfs_avg_ms  regression-factor`

The current window is the last `timeframe-hours` hours of slow log. Only digests with at least `min-executions` PFS executions are included to ensure a meaningful baseline. Up to five regressions are reported per tick.

Prerequisites:

Variable	Required value	Notes
`slow_query_log`	`ON`	Current latency baseline source
`long_query_time`	e.g. `0` or `1`	Set low enough to capture the queries you want to track
`performance_schema`	`ON`	Historical baseline source (`events_statements_summary_by_digest`)

Configuration:

Key	Default	Description
`timeframe-hours`	`1`	Slow log window to use as current baseline
`regression-factor`	`3.0`	Multiplier over PFS average to flag as regression
`min-executions`	`5`	Minimum PFS execution count for a valid baseline

11.3.9 plugin-tmp-table-storm

Binary: `plugin-tmp-table-storm`
Finding: `WARN0306`
Source: `cluster/logplugin/plugins/plugin-tmp-table-storm/main.go`

Reads PFS to detect queries creating on-disk temporary tables. Fires when **either**:

- On-disk tmp table count `disk-tmp-threshold`, **or**
- `disk_tmp / (disk_tmp + mem_tmp)  ratio-threshold`

Only digests with at least `min-exec-count` executions are included. Common causes: missing indexes on `GROUP BY` / `ORDER BY` columns, or `tmp_table_size` / `max_heap_table_size` set too small.

Prerequisites:

Variable	Required value	Notes
performance_schema	ON	Must be enabled at server startup

Configuration:

Key	Default	Description
disk-tmp-threshold	20	Absolute on-disk tmp table count to trigger
ratio-threshold	0.20	Disk/total tmp ratio to trigger
min-exec-count	3	Minimum digest execution count to include

11.3.10 plugin-off-hours-access

Binary: plugin-off-hours-access
Finding: WARN0309
Source: cluster/logplugin/plugins/plugin-off-hours-access/main.go

Scans the audit log for connections or DML from non-exempt accounts outside configured business hours. Useful for PCI-DSS and HIPAA compliance auditing.

A finding is raised when all of the following hold:

- Access time is outside allowed-hours-start–allowed-hours-end (local time)
- Account is not listed in always-allowed-users
- Operation type is in allowed-operations
- Event falls within the last timeframe-hours

Prerequisites:

Requirement	How to enable
server_audit plugin	INSTALL SONAME 'server_audit'
server_audit_logging	SET GLOBAL server_audit_logging = ON
server_audit_events	Include CONNECT, QUERY, QUERY_DML, QUERY_DDL as needed

Configuration:

Key	Default	Description
allowed-hours-start	8	Business hours start, hour of day 0–23
allowed-hours-end	20	Business hours end, hour of day 1–24
always-allowed-users	root,replication_manager	Accounts that are always permitted
allowed-operations	QUERY,QUERY_DML,QUERY_DDL,CONNECT	Audit operation types to inspect
timeframe-hours	1	Audit log window to scan

11.3.11 plugin-privilege-escalation

Binary: `plugin-privilege-escalation`
Finding: `WARN0308`
Source: `cluster/logplugin/plugins/plugin-privilege-escalation/main.go`

Watches the audit log for DDL statements that modify user privileges performed by any account not in `allowed-admin-users`.

Watched operations: `GRANT`, `REVOKE`, `CREATE USER`, `ALTER USER`, `DROP USER`, `RENAME USER`, `SET PASSWORD`.

Prerequisites:

Requirement	How to enable
`server_audit` plugin	`INSTALL SONAME 'server_audit'`
`server_audit_logging`	`SET GLOBAL server_audit_logging = ON`
`server_audit_events`	Must include `QUERY_DDL`

Configuration:

Key	Default	Description
`allowed-admin-users`	`root,replication_manager`	Accounts permitted to perform privilege DDL
`timeframe-hours`	`24`	Audit log window to scan

11.3.12 plugin-binlog-cleartext-password

Binary: `plugin-binlog-cleartext-password`
Finding: `WARN0310`
Source: `cluster/logplugin/plugins/plugin-binlog-cleartext-password/main.go`

Scans binlog QUERY events for SQL statements that contain a cleartext password literal:

- `CREATE USER … IDENTIFIED BY 'pwd'`
- `ALTER USER … IDENTIFIED BY 'pwd'`
- `GRANT … IDENTIFIED BY 'pwd'`
- `SET PASSWORD … = 'pwd'`

Password values are partially redacted in findings (first and last character shown). Findings are capped at `max-findings` per tick to avoid log flooding.

Prerequisites:

Requirement	How to enable
Binary logging	`log_bin` enabled (set in `my.cnf`, requires restart)
`binlog_format`	`MIXED` or `ROW` — STATEMENT format is also captured via QUERY events

Configuration:

Key	Default	Description
`timeframe-hours`	`1`	Binlog event window to scan
`max-findings`	`10`	Maximum findings per evaluation tick

11.3.13 plugin-binlog-creditcard-leak

Binary: `plugin-binlog-creditcard-leak`
Finding: `WARN0311`
Source: `cluster/logplugin/plugins/plugin-binlog-creditcard-leak/main.go`

Scans binlog QUERY events for potential credit card Primary Account Numbers (PANs) using two validation layers:

1. **Regex** — matches 13–19 digit sequences with optional space or dash separators (covers Visa, Mastercard, Amex, Discover, UnionPay formats)
2. **Luhn algorithm** — validates candidates to filter phone numbers, order IDs, and timestamps

PANs are masked in findings (last four digits shown). Findings are capped at `max-findings` per tick.

Prerequisites:

Requirement	How to enable
Binary logging	`log_bin` enabled (set in `my.cnf`, requires restart)

Configuration:

Key	Default	Description
`timeframe-hours`	`1`	Binlog event window to scan
`max-findings`	`10`	Maximum findings per evaluation tick

11.3.14 WARN Code Reference

Code	Plugin	Condition	Notes
WARN0300	plugin-innodb-corruption	InnoDB corruption keyword in error log	24h window
WARN0301	plugin-slow-query-regression	Query latency regressed vs PFS baseline	Up to 5 per tick
WARN0302	plugin-error-storm	Error template count exceeded threshold	Fingerprint-based dedup
WARN0303	plugin-replication-lag-predictor	DML write rate exceeds threshold	Leading indicator for lag
WARN0304	plugin-full-table-scan-spike	Full-scan ratio exceeded threshold	Top 3 digests reported
WARN0305	plugin-metadata-lock-contention	MDL wait duration or count exceeded	Requires MariaDB MDL plugin
WARN0306	plugin-tmp-table-storm	On-disk tmp table count or ratio exceeded	Check tmp_table_size

Code	Plugin	Condition	Notes
WARN0307	plugin-connection-storm	Sleep ratio or lock-wait count exceeded	Connection leak indicator
WARN0308	plugin-privilege-escalation	Privilege DDL by non-admin account	Audit log required
WARN0309	plugin-off-hours-access	DB access outside business hours	Audit log required
WARN0310	plugin-binlog-cleartext-password	Cleartext password in binlog	PAN masked in finding
WARN0311	plugin-binlog-creditcard-leak	Credit card PAN detected in binlog	Luhn validated, masked

11.4.1 Security Plugins

Tier: Community — requires a free registered instance at gitlab.signal18.io

Security plugins audit the database server's configuration, user accounts, and activity logs for security weaknesses. They emit findings with `SEC` error keys. When a finding is open (not resolved), the **remediation engine** proposes and, for supported codes, automatically applies a fix.

Security findings are routed to `security.log`, separate from the main HA log.

11.4.2 plugin-security-no-password-user

Binary: `plugin-security-no-password-user`
Finding: `SEC0100`
Source: `cluster/logplugin/plugins/plugin-security-no-password-user/main.go`

Flags every database account that has an empty `authentication_string` and is not protected by a socket-based authentication plugin.

An account with no password and no socket auth can be logged into from any host that can reach the database port without any credentials.

Logic: - Skip if `password_empty = false` - Skip if `account_locked = true` (locked accounts cannot be used to log in) - Skip if plugin is socket-based: `unix_socket`, `auth_socket`, `gssapi`, `authentication_pam`, `auth_pam` - Skip if account is in `ignored-users`

Configuration:

Key	Default	Description
`ignored-users`	`""`	Comma-separated `'user'@'host'` pairs or bare usernames to suppress

Automated fix: `ALTER USER … ACCOUNT LOCK` — reversible, safe.

11.4.3 plugin-security-weak-auth

Binary: `plugin-security-weak-auth`
Finding: `SEC0101`
Source: `cluster/logplugin/plugins/plugin-security-weak-auth/main.go`

Flags accounts using weak or deprecated authentication plugins.

Strong plugins (no finding): `ed25519`, `parsec`, `auth_gssapi`, `gssapi`, `unix_socket`, `auth_socket`, `authentication_pam`, `auth_pam`, `caching_sha2_password`, `sha256_password`

Weak plugins (trigger SEC0101): - `mysql_native_password` — SHA-1 based, deprecated in MySQL 8.0, removed in MySQL 8.4 - `mysql_old_password` — DES-based, dangerously weak

Accounts with an empty plugin string fall back to the server default (often `mysql_native_password`) and are optionally flagged via `include-empty`.

Configuration:

Key	Default	Description
`ignored-users`	`""`	Comma-separated accounts to suppress
`include-empty`	`true`	Also flag accounts with no explicit plugin set

Remediation: advisory only. Migration from `mysql_native_password` to `ed25519` (MariaDB) or `caching_sha2_password` (MySQL 8+) requires updating application connection strings — no automated fix is provided.

11.4.4 plugin-security-local-infile

Binary: `plugin-security-local-infile`
Finding: `SEC0102`
Source: `cluster/logplugin/plugins/plugin-security-local-infile/main.go`

Raises SEC0102 when `local_infile = ON`.

With `local_infile` enabled a malicious or compromised server can instruct a client to read arbitrary local files and send their contents upstream via `LOAD DATA LOCAL INFILE` attack (CVE-2016-3440, CIS MySQL Benchmark control 4.5).

No configuration keys — the check is unconditional.

Automated fix: `DropDBTag("with_sec_localinfile")` → executes `SET GLOBAL local_infile = 0` on all servers immediately. Risk: **safe**.

11.4.5 plugin-security-hardening

Binary: `plugin-security-hardening`
Source: `cluster/logplugin/plugins/plugin-security-hardening/main.go`

Evaluates a set of CIS MySQL/MariaDB Benchmark hardening controls. Raises findings for:

11.4.5.1 SEC0103 — require_secure_transport=OFF

Plaintext (non-TLS) client connections are permitted.

Trigger: `require_secure_transport = OFF` in SHOW GLOBAL VARIABLES.

Remediation: `cnf_template` advisory — operator must deploy the `with_sec_securetransport` tag (see Compliance Tags below). Risk: **moderate** (existing non-TLS sessions will be dropped).

11.4.5.2 SEC0104 — general_log=ON

All SQL statements, including those containing plaintext passwords, are written to the general query log.

Trigger: `general_log = ON`.

Automated fix: `DropDBTag("with_log_general")` → executes `SET GLOBAL general_log = 0` immediately. Risk: **safe**.

11.4.5.3 SEC0105 — secure_file_priv=''

The server can read or write any filesystem path via `LOAD DATA` / `SELECT INTO OUTFILE`.

Trigger: `secure_file_priv` is empty string.

Remediation: `cnf_template` advisory — `secure_file_priv` is a read-only variable, requires restart. Risk: **disruptive**.

11.4.5.4 SEC0106 — skip_name_resolve=OFF

DNS lookups are enabled for connecting clients; hostname spoofing is possible.

Trigger: `skip_name_resolve = OFF`.

Remediation: `cnf_template` advisory — `skip_name_resolve` is a read-only variable, requires restart. All GRANT statements using hostnames must be converted to IP addresses first. Risk: **disruptive**.

11.4.5.5 SEC0107 — Anonymous user account

An anonymous (`user=''`) account exists; anyone can connect without a username.

Trigger: a row with `User = ''` exists in `mysql.user`.

Automated fix: `DROP USER IF EXISTS ''@host` for every anonymous account across all servers. Risk: **safe**.

11.4.5.6 SEC0108 — Wildcard-host privileged user

An account uses host `%` and holds elevated privilege (SUPER, ADMIN, ALL, or similar).

Trigger: `Host = '%'` combined with a sensitive privilege.

Accounts listed in `wildcard-priv-ignored-users` and accounts using socket-based auth plugins are excluded.

Remediation: informational only — restricting wildcard-host accounts requires reviewing which applications depend on them.

11.4.5.7 SEC0109 / SEC0110 / SEC0111 — At-rest encryption (via plugin-score-encryption)

Code	Variable	Condition
SEC0109	`innodb_encrypt_tables` / `aria_encrypt_tables`	Both OFF
SEC0110	`encrypt_binlog` / `binlog_encryption`	Both OFF
SEC0111	`encrypt_tmp_files` / `encrypt_tmp_disk_tables`	Both OFF

Automated fix: `AddDBTag("with_sec_keyfileencrypt")` — deploys the file-key-management plugin configuration and triggers a **rolling restart**. Risk: **disruptive**.

Pre-requisite: the key file must exist on every server before the tag is applied.

11.4.5.8 SEC0112 — Audit logging not active

The `server_audit` plugin is not loaded or `server_audit_logging = OFF`.

Automated fix: `AddDBTag("audit")` → `INSTALL SONAME 'server_audit'` and enables logging at runtime. Risk: **safe**.

11.4.5.9 SEC0113 / SEC0114 / SEC0115 — Password validation plugins *(MariaDB only)*

Code	Plugin	Variable
SEC0113	`simple_password_check`	`simple_password_check_minimal_length`
SEC0114	`cracklib_password_check`	`cracklib_password_check_dictionary`
SEC0115	`password_reuse_check`	`password_reuse_check_interval`

Each code fires when the corresponding plugin is not loaded **or** when `strict_password_validation = OFF` (making validation advisory only).

Automated fix: `INSTALL SONAME '<plugin>'` at runtime via the compliance tag, plus `plugin_load_add` deployed to `my.cnf` for persistence. Risk: **safe**.

SEC0115 requires MariaDB **10.7+** — servers below this version are skipped automatically.

11.4.5.10 SEC0118 — Hostname grants with skip_name_resolve=ON

When `skip_name_resolve = ON`, MySQL/MariaDB never resolves DNS hostnames. Any account whose `Host` column contains a hostname (not an IP, not `%`, not `localhost`) will silently fail every connection attempt — the grant is effectively dead.

Trigger: `skip_name_resolve = ON` AND at least one account in `mysql.user` has a hostname in `Host`.

Remediation: informational — operator must `RENAME USER 'user'@'hostname' TO 'user'@'<ip>'` for each affected account.

Configuration for plugin-security-hardening:

Key	Default	Description
`wildcard-priv-ignored-users`	`""`	Accounts to exclude from the SEC0108 wildcard-host check

---

11.4.6 Remediation Engine

11.4.6.1 Remediation Plan

`GET /api/clusters/{clusterName}/security/remediation-plan`

Returns a JSON document listing every open security finding with its available fixes:

```
{
  "cluster": "prod-db",
  "generated_at": "2026-04-10T08:00:00Z",
```

```
  "open_findings": 3,
  "remediations": [
    {
      "err_key": "SEC0102",
      "server": "db1:3306",
      "description": "Server db1:3306: local_infile=ON ...",
      "auto_fixable": true,
      "fixes": [
        {
          "type": "drop_tag",
          "tag": "with_sec_localinfile",
          "description": "Drop 'with_sec_localinfile' tag ...",
          "risk": "safe"
        }
      ]
    }
  ]
}
```


Fix types:

Type	Description
`drop_tag`	Remove a compliance tag; executes `mariadb_default:` SQL and removes the `.cnf`
`add_tag`	Add a compliance tag; deploys `.cnf` and executes `mariadb_command:` SQL
`cnf_template`	Informational — suggested `.cnf` content to create manually
`drop_anon_users`	Drop all anonymous accounts
`lock_no_password_users`	`ACCOUNT LOCK` on all no-password, non-socket accounts

Risk levels:

Level	Meaning
`safe`	Applied at runtime, no service interruption, fully reversible
`moderate`	Applied at runtime but may interrupt active connections
`disruptive`	Requires server restart — triggers rolling restart

11.4.6.2 Apply a Fix

`POST /api/clusters/{clusterName}/security/fix-state/{errKey}`

No request body required. Returns `{"status":"ok"}` on success.

What each fix does:

Code	Action
`SEC0100`	`ACCOUNT LOCK` on every no-password, non-socket account across all servers
`SEC0102`	`DropDBTag("with_sec_localinfile")` → `SET GLOBAL local_infile = 0`

Code	Action
SEC0104	DropDBTag("with_log_general") → SET GLOBAL general_log = 0
SEC0107	DROP USER IF EXISTS ''@host for every anonymous account
SEC0109-SEC0111	AddDBTag("with_sec_keyfileencrypt") → deploy encryption .cnf + rolling restart
SEC0112	AddDBTag("audit") → INSTALL SONAME 'server_audit'
SEC0113	AddDBTag("with_sec_pwdchecksimple") → INSTALL SONAME 'simple_password_check'
SEC0114	AddDBTag("with_sec_pwdcheckcracklib") → INSTALL SONAME 'cracklib_password_check'
SEC0115	AddDBTag("with_sec_pwdcheckreuse") → INSTALL SONAME 'password_reuse_check'

11.4.6.3 Compliance Tags

Replication-manager's remediation engine works through the **compliance module tag mechanism**: safe and moderate fixes deploy or remove a .cnf fragment on all servers rather than executing raw SQL directly. This ensures the fix survives server restarts.

Special comment lines inside each .cnf file control runtime execution:

Prefix	When executed
# mariadb_command:	Immediately when the tag is **added**
# mariadb_default:	Immediately when the tag is **removed**
# mariadb_alert:	Advisory note — logged, not executed

If no runtime SQL line is present, the variables are read-only and a **restart cookie** is set — the settings take effect on the next server start.

Existing security tags:

Tag	SEC Code	Dynamic	Description
with_log_general	SEC0104	Yes	Enables/disables general_log
with_sec_localinfile	SEC0102	Yes	Enables/disables local_infile
with_sec_keyfileencrypt	SEC0109/110/111	No (restart)	File-key-management + InnoDB/binlog/tmp encryption
with_sec_ssl	(SSL prereq)	No (FLUSH SSL)	SSL certificate configuration
with_sec_ed25519	SEC0101	No	Loads auth_ed25519 plugin
with_sec_pwdchecksimple	SEC0113	No	Loads simple_password_check
with_sec_pwdcheckcracklib	SEC0114	No	Loads cracklib_password_check

Tag	SEC Code	Dynamic	Description
`with_sec_pwdcheckreuse`	SEC0115	No	Loads `password_reuse_check`

Proposed tags (cnf_template advisories until added to the module):

Tag	SEC Code	Dynamic	Description
`with_sec_securetransport`	SEC0103	Yes (MariaDB 10.3.5+)	`require_secure_transport = ON`
`with_sec_securefilepriv`	SEC0105	No (restart)	`secure_file_priv = /var/lib/mysql-files`
`with_sec_skipnameresolve`	SEC0106	No (restart)	`skip_name_resolve = ON`

11.4.7 SEC Code Reference

Code	Plugin	Condition	Auto-fixable	Risk
SEC0100	plugin-security-no-password-user	Empty password, no socket auth	Yes	Safe
SEC0101	plugin-security-weak-auth	Weak/deprecated auth plugin	No	—
SEC0102	plugin-security-local-infile	`local_infile = ON`	Yes	Safe
SEC0103	plugin-security-hardening	`require_secure_transport = OFF`	Partial (cnf_template)	Moderate
SEC0104	plugin-security-hardening	`general_log = ON`	Yes	Safe
SEC0105	plugin-security-hardening	`secure_file_priv = ''`	Partial (cnf_template)	Disruptive
SEC0106	plugin-security-hardening	`skip_name_resolve = OFF`	Partial (cnf_template)	Disruptive
SEC0107	plugin-security-hardening	Anonymous account exists	Yes	Safe
SEC0108	plugin-security-hardening	Wildcard-host privileged account	No	—
SEC0109	plugin-score-encryption	Table encryption disabled	Yes	Disruptive
SEC0110	plugin-score-encryption	Binlog encryption disabled	Yes	Disruptive
SEC0111	plugin-score-encryption	Tmp file encryption disabled	Yes	Disruptive
SEC0112	plugin-security-hardening	Audit plugin not loaded	Yes	Safe
SEC0113	plugin-security-hardening	simple_password_check missing	Yes	Safe
SEC0114	plugin-security-hardening	cracklib_password_check missing	Yes	Safe

Code	Plugin	Condition	Auto-fixable	Risk
SEC0115	plugin-security-hardening	password_reuse_check missing	Yes	Safe (MariaDB 10.7+)
SEC0118	plugin-security-hardening	Hostname grants + skip_name_resolve ON	No	—

11.4.8 Security Considerations

- **Account operations (SEC0100, SEC0107)** use `ACCOUNT LOCK` and `DROP USER IF EXISTS` — executed directly via the server connection pool, not through the compliance module. `ACCOUNT LOCK` is reversible.
- **Encryption fixes (SEC0109/0110/0111)** require a pre-configured key file at the path specified in `with_sec_keyfileencrypt`. Applying the tag without a valid key file will cause the server to fail to start after the rolling restart.
- **Rolling restart** requires a functioning replication topology. Review cluster health before applying disruptive fixes.
- **SEC0108 (wildcard host)** is informational only — restricting wildcard-host accounts requires reviewing application dependencies.
- **SEC0101 (weak auth plugin)** migration requires updating application connection strings; no automated fix is provided.

11.5.1 Score Plugins

Score plugins compute **binary pass/fail security checks** from the server snapshot. They emit `ScoreCheck` entries (not findings) that feed the **SecurityScore** gauge visible in the cluster dashboard. A passing check contributes positively to the score; a failing check deducts points.

Score plugins can also emit **SEC findings** alongside score checks when a condition is both measurable as a score and significant enough to warrant an alert entry.

Score plugins are external binaries with no configurable parameters — their calculation is derived entirely from the current server state (SHOW GLOBAL VARIABLES, mysql.user, cluster context).

11.5.2 plugin-score-ssl

Binary: `plugin-score-ssl`
Source: `cluster/logplugin/plugins/plugin-score-ssl/main.go`

Checks whether TLS/SSL is enabled and properly configured on the server.

Score checks:

Check name	Pass condition
`HasSSL`	`have_ssl = YES`
`HasSSLCA`	`ssl_ca` is set

11.5.3 plugin-score-encryption

Binary: `plugin-score-encryption`
Source: `cluster/logplugin/plugins/plugin-score-encryption/main.go`

Evaluates at-rest encryption status for tables, binary logs, and temporary files. Also emits SEC findings when encryption is disabled.

Score checks:

Check name	Pass condition
`HasTableEncryption`	`innodb_encrypt_tables = ON` OR `aria_encrypt_tables = ON`
`HasBinlogEncryption`	`encrypt_binlog = ON` OR `binlog_encryption = ON`
`HasTmpEncryption`	`encrypt_tmp_files = ON` OR `encrypt_tmp_disk_tables = ON`
`HasBackupEncryption`	Cluster has Restic backup encryption configured

Also emits: SEC0109 (table), SEC0110 (binlog), SEC0111 (tmp) when the corresponding check fails.

11.5.4 plugin-score-auth

Binary: `plugin-score-auth`
Source: `cluster/logplugin/plugins/plugin-score-auth/main.go`

Checks whether strong password validation is enforced on the server.

Score checks:

Check name	Pass condition
HasStrongPwd	MariaDB: strict_password_validation = ON AND (simple_password_check OR cracklib_password_check) loaded. MySQL: validate_password = ON or validate_password_policy set
HasPwdReuse	password_reuse_check_interval variable present (MariaDB 10.7+)

11.5.5 plugin-score-network

Binary: plugin-score-network
Source: cluster/logplugin/plugins/plugin-score-network/main.go

Checks network security configuration.

Score checks:

Check name	Pass condition
HasSecureTransport	require_secure_transport = ON
HasLocalInfileOff	local_infile = OFF

11.5.6 plugin-score-proxy

Binary: plugin-score-proxy
Source: cluster/logplugin/plugins/plugin-score-proxy/main.go

Checks whether the cluster is using a traffic routing proxy for HA and read/write splitting.

Score checks:

Check name	Pass condition
HasProxy	Cluster has at least one proxy configured (MaxScale, ProxySQL, HAProxy, etc.)

11.5.7 plugin-score-passwords

Binary: plugin-score-passwords
Source: cluster/logplugin/plugins/plugin-score-passwords/main.go

Checks for the presence of no-password and weak-auth accounts.

Score checks:

Check name	Pass condition
HasNoEmptyPwd	No accounts with empty authentication_string (excluding socket-auth accounts)

Check name	Pass condition
`HasNoWeakAuth`	No accounts using `mysql_native_password` or `mysql_old_password`

11.5.8 plugin-score-lts

Binary: `plugin-score-lts`
Source: `cluster/logplugin/plugins/plugin-score-lts/main.go`

Checks whether the server is running a Long Term Support (LTS) release of MariaDB.

Score checks:

Check name	Pass condition
`HasLTS`	Server version matches a known MariaDB LTS release series (10.6, 10.11, 11.4, …)

11.5.9 plugin-score-audit

Binary: `plugin-score-audit`
Source: `cluster/logplugin/plugins/plugin-score-audit/main.go`

Checks whether the MariaDB audit plugin is active.

Score checks:

Check name	Pass condition
`HasAuditPlugin`	`server_audit_logging = ON`

11.5.10 SecurityScore Dashboard

The SecurityScore is a percentage computed from all `ScoreCheck` results across all enabled score plugins:

```
SecurityScore = (passing_checks / total_checks) × 100
```

The score is displayed per-server in the cluster dashboard and can be used to: - Track hardening progress over time - Gate automated operations (e.g. block provisioning if score is below threshold) - Generate compliance reports

Score plugins have no configurable parameters. To change what is scored, enable or disable individual score plugins via the GUI or TOML configuration.

11.6.1 Developing Plugins

External plugins are standalone Go binaries that follow a simple JSON stdin/stdout contract. No special framework is required — only the `wire` package from the replication-manager module.

11.6.2 Quick Start

11.6.2.1 Create the plugin directory

```
cluster/logplugin/plugins/plugin-<category>-<name>/
  main.go
```

Category conventions: `workload`, `security`, `score`, `binlog`.

11.6.2.2 Write the plugin

```go
package main

import (
    "encoding/json"
    "fmt"
    "os"

    "github.com/signal18/replication-manager/cluster/logplugin/plugins/wire"
)

func main() {
    // 1. Decode the request from stdin
    var req wire.Request
    if err := json.NewDecoder(os.Stdin).Decode(&req); err != nil {
        fmt.Fprintf(os.Stderr, "decode error: %v\n", err)
        os.Exit(1)
    }

    // 2. Read configuration (config map first, env var as fallback)
    threshold := wire.CfgFloat(req.Config, "my-threshold",
        wire.EnvFloat("REPMAN_<PLUGINNAME>_MY_THRESHOLD", 0.50))

    // 3. Inspect the server snapshot
    var findings []wire.Finding

    if val := req.ServerVariables["some_variable"]; val == "UNSAFE" {
        findings = append(findings, wire.Finding{
            ErrKey:      "WARN0399",
            Severity:    "WARNING",
            Description: fmt.Sprintf("Server %s: some_variable is unsafe", req.ServerURL),
        })
    }

    // 4. Write the response to stdout
    json.NewEncoder(os.Stdout).Encode(wire.Response{Findings: findings})
}
```

11.6.2.3 Build the plugin

```
cd cluster/logplugin/plugins/plugin-<category>-<name>
go build -o ../../../../share/plugins/plugin-<category>-<name> .
```

11.6.2.4 Sign the plugin (if signature verification is enabled)

```
replication-manager plugin-sign \
    --binary share/plugins/plugin-<category>-<name> \
    --key etc/plugin-signing.key \
    --out share/plugins/plugin-<category>-<name>.sig
```

11.6.3 Wire Protocol Reference

11.6.3.1 wire.Request

```go
type Request struct {
    ServerURL       string            `json:"server_url"`
    ServerVariables map[string]string `json:"server_variables"`
    DatabaseUsers   []DBUser          `json:"database_users"`
    ClusterContext  ClusterContext    `json:"cluster_context"`
    ErrorLog        []Msg             `json:"error_log"`
    SlowLog         []SlowMsg         `json:"slow_log"`
    ProcessList     []Process         `json:"process_list"`
    BinlogEvents    []BinlogEvent     `json:"binlog_events"`
    Config          map[string]string `json:"config,omitempty"`
}
```

`ServerVariables` keys are always **lowercase** (e.g. `require_secure_transport`, not `REQUIRE_SECURE_TRANSPORT`). Boolean values may be `"ON"` / `"OFF"` or `"1"` / `"0"` depending on the MariaDB/MySQL version — always test both.

11.6.3.2 wire.DBUser

```go
type DBUser struct {
    User          string `json:"user"`
    Host          string `json:"host"`
    Plugin        string `json:"plugin"`
    PasswordEmpty bool   `json:"password_empty"`
    AccountLocked bool   `json:"account_locked"`
}
```

11.6.3.3 wire.Finding

```go
type Finding struct {
    ErrKey      string `json:"err_key"`
    Severity    string `json:"severity"`
    Description string `json:"description"`
}
```

Severity values:

Value	Routing
`"WARNING"`	Main HA log
`"ERROR"`	Main HA log

Value	Routing
"SECURITY"	Dedicated security.log + SecurityStateMachine

11.6.3.4 wire.ScoreCheck

```go
type ScoreCheck struct {
    Name        string `json:"name"`
    Pass        bool   `json:"pass"`
    Description string `json:"description"`
}
```

11.6.3.5 wire.Response

```go
type Response struct {
    Findings    []Finding    `json:"findings"`
    ScoreChecks []ScoreCheck `json:"score_checks"`
}
```

11.6.4 Configuration Helpers

The `wire` package provides helpers to read configuration from the request `Config` map with typed conversion and an environment variable fallback.

```go
// Integer with env fallback
hours := wire.CfgInt(req.Config, "timeframe-hours",
    wire.EnvInt("REPMAN_MYNAME_TIMEFRAME_HOURS", 1))

// Float with env fallback
ratio := wire.CfgFloat(req.Config, "ratio-threshold",
    wire.EnvFloat("REPMAN_MYNAME_RATIO_THRESHOLD", 0.30))

// String with env fallback
users := wire.CfgStr(req.Config, "ignored-users",
    wire.EnvStr("REPMAN_MYNAME_IGNORED_USERS", ""))

// Bool with env fallback
flag := wire.CfgBool(req.Config, "include-empty",
    wire.EnvStr("REPMAN_MYNAME_INCLUDE_EMPTY", "true") != "false")
```

Environment variable naming convention:

```
REPMAN_<PLUGIN_NAME_UPPER>_<KEY_UPPER>
```

Where `<PLUGIN_NAME_UPPER>` is the plugin name without the `plugin-` prefix, with hyphens replaced by underscores and uppercased.

Plugin name	Env prefix
plugin-connection-storm	REPMAN_CONNECTION_STORM_
plugin-security-hardening	REPMAN_SECURITY_HARDENING_
plugin-binlog-creditcard-leak	REPMAN_BINLOG_CREDITCARD_LEAK_

11.6.5 Error Code Assignment

Assign error codes from the appropriate range:

Prefix	Range	Category
`WARN`	0300–0399	Workload and operational anomalies
`SEC`	0100–0199	Security findings
`INFO`	0300–0399	Informational nudges

Check existing codes in the current plugins to avoid collisions.

11.6.6 Registering GUI Configuration

To expose plugin configuration in the replication-manager GUI, add the plugin to `pluginKnownKeys()` in `share/dashboard_react/src/Pages/Settings/PluginsSettings.jsx`:

```
case 'plugin-<category>-<name>':
    return ['my-threshold', 'timeframe-hours']
```

Also add entries in `pluginKeyType()`, `pluginKeyRange()`, `pluginKeyLabel()`, `pluginKeyDefault()`, and `pluginKeyHelp()` for each key. Each key gets: - A type: `'int'`, `'float'`, `'text'`, or `'bool'` - A numeric range for number inputs (min, max, step) - A human-readable label - A default value matching the plugin source code - A markdown help string explaining the calculation

---

11.6.7 Adding an Automated Remediation

If the finding has a fix via a compliance module tag, add an entry to `secTagMap` in `cluster/cluster_sec_fix.go`:

```
"WARN0399": {action: "drop_tag", tag: "with_my_tag", risk: "safe"},
```

Then add a `case "WARN0399":` to `FixSecState()` in the same file.

If the fix is a direct database operation (e.g. locking accounts), implement it as a helper function and call it from `FixSecState()`.

If the fix requires a server restart, add `go cluster.RollingRestart()` after the tag is applied and set the risk to `"disruptive"`.

11.6.8 Testing a Plugin Locally

The simplest way to test a plugin is to pipe a sample JSON request into it:

```
echo '{
  "server_url": "127.0.0.1:3306",
  "server_variables": {"some_variable": "UNSAFE"},
  "database_users": [],
  "config": {"my-threshold": "0.75"}
}' | ./plugin-<category>-<name>
```

The output should be a valid `wire.Response` JSON object:

```
{"findings":[{"err_key":"WARN0399","severity":"WARNING","description":"Server 127.0.0.1:3306: some_varia
```

For integration testing, enable the plugin in a development cluster, set `log-level-plugin = 4`, and watch the plugin log in the monitoring output.

11.6.9 Checklist

- ☐ Plugin binary placed in `share/plugins/`
- ☐ Signature file created if signing is enabled
- ☐ Env var names follow `REPMAN_<PLUGINNAME>_<KEY>` convention
- ☐ All config keys read via `wire.CfgXxx` with env fallback
- ☐ Error code checked against existing codes — no collision
- ☐ `Description` always includes `req.ServerURL` so findings are traceable to a server
- ☐ Response always written even when `findings` is empty (write `wire.Response{}`)
- ☐ Plugin exits 0 on success; non-zero exit is logged as a plugin error
- ☐ GUI keys registered in `PluginsSettings.jsx` with correct type, range, and help text
- ☐ Remediation entry added to `secTagMap` / `FixSecState` if automated fix exists

12.1.1 Replication best practices

12.1.1.1 Parallel replication

The history of MariaDB replication has reached a point where replication can almost in any case catch up with the master. It can be ensured using new features like Group Commit improvement, optimistic in-order parallel replication and semi-synchronous replication.

MariaDB 10.1 settings for in-order optimistic parallel replication:

```
slave_parallel_mode = optimistic
slave_domain_parallel_threads = %%ENV:CORES%%
slave_parallel_threads = %%ENV:CORES%%
expire_logs_days = 5
sync_binlog = 1
log_slave_updates = ON
```

12.1.1.2 Semi-synchronous replication

Semi-synchronous replication enables to delay transaction commit until the transactional event reaches at least one replica. The "In Sync" status will be lost only when a tunable replication delay is attained. This Sync status is checked by **replication-manager** to compute the last SLA metrics, the time we may auto-failover without losing data and when we can reintroduce the dead leader without re-provisioning it.

The MariaDB recommended settings for semi-sync:

```
plugin_load = "semisync_master.so;semisync_slave.so"
rpl_semi_sync_master = ON
rpl_semi_sync_slave = ON
loose_rpl_semi_sync_master_enabled = ON
loose_rpl_semi_sync_slave_enabled = ON
rpl_semi_sync_master_timeout = 10
```

Such parameters will print an expected warning in error.log on slaves about SemiSyncMaster Status switched OFF.

> **Important Note**: semisync SYNC status does not guarantee that the old leader is replication consistent with the cluster in case of crash MDEV-11855 or shutdown MDEV-11853 of the master,the failure can leave more data in the binary log but it guarantees that no client applications have seen those pending transactions if they have not reach a replica.
>
> This leads to a situation where semisync is used to slowdown the workload to the speed of the network until it reaches a timeout where it is not possible to catch up anymore. A crash or shutdown will lead to the requirement of re-provisioning the old leader from another node in most heavy write scenarios.
>
> Setting rpl_semi_sync_master_wait_point to AFTER_SYNC may limit the number of extra transactions inside the binlog after a crash but those transactions would have been made visible to the clients and may have been lost during failover to an other node. It is highly recommended to keep AFTER_COMMIT to make sure the workload is safer.

12.2.0.1 Enforcing best practices

replication-manager (1.1) can dynamically enforce best database practices around the replication usage.

It can dynamically configure the server it monitors via SET GLOBAL VARIABLES when it is possible to do so without database restart.

> Enforcement can be lost if replication-manager monitoring is shutdown and the database is restarted.

replication-manager (2.0) produces warnings if one of the possible practices is not found in the database nodes and not dynamically enforced. Some non-tunable warnings are produced for:

- missing log-slave-update directive
- non-strict GTID mode

replication-manager ignores enforcement and warnings on the nodes excluded from election using the ignore list.

- force-slave-heartbeat
- force-slave-gtid-mode
- force-slave-semisync
- force-slave-readonly
- force-binlog-row
- force-binlog-annotate
- force-binlog-slowqueries
- force-inmemory-binlog-cache-size
- force-disk-relaylog-size-limit
- force-sync-binlog
- force-sync-innodb
- force-binlog-checksum

replication-manager default enforcements are `force-slave-readonly` and `force-slave-heartbeat`

We advise to permanently set the variables inside your database node configuration, to disable most dynamic enforcements on the long run.

12.3.0.1 Troubleshooting failover switchover

The log verbosity can be increase to get better understanding for the route cause of some issue

```
replication-manager-cli api  url=https://127.0.0.1:3000/api/clusters/cluster_name/settings/switch/verbos
```

12.3.0.2 Troubleshooting internal status

replication-manager track internal status inside following class settings, clusters, servers, master, slaves, crashes, alerts

You can explore those status via the show client per default JSON file have all classes but it can be filtered with the –get option

```
replication-manager-cli show
```

You can attach the command result with the log file, the increase verbosity log file to a signal18.io support ticket.

12.3.0.3 Troubleshoot Crashes

replication-manager (2.O) record a crash information in the replication-manager datadir for each failover or switchover. It helps the DBA to refer to it in case of later reporting. The binary log state of the elected master or his own replication state when he was still a master get lost when the traffic take over, such a backup is always good to have and save DBA time to create it manually.

replication-manager (1.1) Save some binlog informations on crash when old master restart. Before this happen one can query the current recorded elections via the API, the HTTP URL `/crashes` or form the client.


```
[
    {
        "URL": "127.0.0.1:3310",
        "FailoverMasterLogFile": "8228534449735335205-bin.000001",
        "FailoverMasterLogPos": "459",
        "FailoverSemiSyncSlaveStatus": true,
        "FailoverIOGtid": [
            {
                "DomainID": 0,
                "ServerID": 3310,
                "SeqNo": 1
            }
        ],
        "ElectedMasterURL": "127.0.0.1:3311"
    }
]
```


replication-manager track crashes under the data directory under the json file `cluster_name.json`


```
{
    "servers": "127.0.0.1:3310,127.0.0.1:3311",
    "crashes": [
        {
            "URL": "127.0.0.1:3310",
            "FailoverMasterLogFile": "8228534449735335205-bin.000001",
            "FailoverMasterLogPos": "459",
            "FailoverSemiSyncSlaveStatus": true,
            "FailoverIOGtid": [
                {
```

```
                    "DomainID": 0,
                    "ServerID": 3310,
                    "SeqNo": 1
                }
            ],
            "ElectedMasterURL": "127.0.0.1:3311"
        }
    ],
    "sla": {
        "firsttime": 1506375653,
        "uptime": 170132,
        "uptimeFailable": 170119,
        "uptimeSemisync": 401
    }
}
```


Those internal states are saved every 60 seconds to disk and reloaded on startup.

replication-manager will clear the crashes informations when the all cluster topology is back to no ERROR, some stopped slave can still need those informations to rejoin correctly the cluster.

replication-manager when old master rejoin, binlogs used for rejoin are saved with extra election informations in a directory `/datadir/crash*Unixtime*`

13.1.0.1 Features

- CORE: Master slave
- CORE: Multi Master 2 nodes
- CORE: Semi-sync replication
- CORE: Email alerts
- CORE: Configuration file
- CORE: AES Password encryption
- CORE: On-leave mode
- CORE: Log facilities and verbosity
- API: Bootstrap
- HTTP: Daemon mode
- HTTP: Failover SLA tracking
- TESTS: Non regression tests via http
- PROXY: HaProxy wrapper
- Docker images

13.2.0.1 Features

- CORE: Multi cluster
- CORE: Multi proxies
- CORE: Rejoin failed nodes
- CORE: Rejoin flashback

- CORE: Rejoin mysqldump

- CORE: Backup lost events

- CORE: Trends storage in whisper

- CORE: Active Standby mode
- CORE: External arbitrator
- CORE: Enforce replication practice
- CORE: False positive via replication heartbeat
- CORE: False positive via external http call
- CORE: False positive via maxscale monitor

- CORE: MultiTier replication topology
- CORE: Alert on node state change
- PROXY: MariaDB sharding proxy
- PROXY: MaxScale 2 nodes master-slave driving
- PROXY: MaxScale integration to disable traffic on READ_ONLY flag
- PROXY: Maxscale binlog server support
- HTTP: MaxScale monitor state server display
- HTTP: Trends display
- TESTS: Non regression tests, via command line
- TESTS: Non regression tests, more tests
- TESTS: Non regression tests, sysbench wrapper in tests
- TESTS: Non regression tests, all topology bootstrap

13.3.0.1 Features

- CORE: Client/Server architecture with JWT API
- CORE: Docker provision via OpenSVC

- CORE: Support database secure TLS connection
- CORE: Support Multi domain GTID
- CORE: Support Positional Replication
- CORE: Support Positional Replication via PseudoGTID
- CORE: Support Callable external alert scripts
- CORE: Support save and reload SLA
- CORE: Support adding node to monitor
- CORE: Support basedir installation
- CORE: Support Galera Replication

- CORE: Support MySQL7 super read only mode

- CORE: Support Ring Replication

- PROXY: Support ProxySQL
- PROXY: Support Database Maintenance
- PROXY: Support Consul DNS
- CLIENT: Support cluster status
- CLIENT: Support monitor status
- CLIENT: Support show objects

13.4.0.1 Features

- CORE: Dynamic config cluster store default config in $datadir/cluster.d
- CORE: IPV6 support [IPV6host]:port
- CORE: Schema change monitoring
- CORE: Use of MariaDB SHUTDOWN [WAIT FOR ALL SLAVES]
- CORE: Use of MariaDB mysqldump system=ALL
- CORE MONITORING: Multi Tenant Cluster Auth
- CORE MONITORING: SMTP can use Auth credentials
- CORE MONITORING: Slack alerting
- CORE MONITORING: Alerts preserved when flapping connections state on unstable network, reducing false positive alert
- CORE MONITORING: Support for MySQL 8.0 and MariaDB 10.5
- CORE MAINTENANCE: Job scheduler rolling restart,
- CORE MAINTENANCE: Job scheduler rolling reprovision
- CORE MAINTENANCE: Job scheduler rolling optimize
- CORE MAINTENANCE: Job scheduler logical backup
- CORE MAINTENANCE: Job scheduler physical backup
- CORE MAINTENANCE: Job scheduler remote ssh job execution
- CORE MAINTENANCE: Tracking config change for rolling restart and rolling upgrade
- CORE MAINTENANCE: Streaming db error log on schedule SST external script
- CORE MAINTENANCE: Capture slow queries via db table, output in local log, log rotation
- CORE MAINTENANCE: slow queries digest
- CORE MAINTENANCE: Capture loop for database infos to log (processlist,slave status, status, innodb status) trigger by alert
- CORE MAINTENANCE: Failover Switchover store state
- CORE MAINTENANCE: Failover Switchover capture SQL Logs produce by replication-manager
- CORE MAINTENANCE: Cluster configurator generation of tar.gz
- CORE BACKUP: Streaming xtrabackup mariabackup xbstream to replication-manager
- CORE BACKUP: Streaming db slow query on schedule SST external script
- CORE BACKUP: Streaming backups for reseeding new node on schedule SST external script
- CORE BACKUP: Streaming backups to restic for achiving
- CORE BACKUP: Capture master binary logs and archive
- CORE BACKUP: mydumper integration
- CORE BACKUP: expose mysqldump parameter in configuration
- CORE SHARDING: Backend schema discover, pushdown spider table to proxy assuming universal table for duplicate names

- CORE SHARDING: Backend schema discover, pushdown spider table to proxy assuming hash of PK
- CORE SHARDING: Resharding table via API call
- CORE SHARDING: Table move cluster via API call
- CORE REPLICATION: Delayed replication
- CORE REPLICATION: MYSQL GTID errant transactions detection on non ignored slave
- CORE REPLICATION: Checksum tables
- CORE REPLICATION: Multi source aggregation slaves
- CORE ROUTING: MyProxy internal proxying in go based on the Vitess parser and Siddon proxy
- CORE ROUTING: HaProxy Runtime API
- HTTP/API: Database Process
- HTTP/API: Database slow Queries
- HTTP/API: Database Query Digest PFS/SLOW
- HTTP/API: Database Query Explain
- HTTP/API: Database Metadata Locks
- HTTP/API: Database Variables
- HTTP/API: Database Status Delta
- HTTP/API: Database Queries Response Time
- HTTP/API: Prometheus URI (done)
- HTTP/API: Cancel Rolling Restart
- HTTP/API: Need rolling restart or reprov on config change
- HTTP/API: Download Configurations
- PRO CORE REPLICATION: Rejoin via ZFS snapback to last snapshot for prefered master when binlog ahead (done)
- PRO CORE: OpenSVC agent 1.9 using overlay network (Weave, ipip tunneling)
- PRO CORE: OpenSVC agent 1.9 direct cluster API
- PRO CORE: Kubernetes (In progress)
- PRO CORE: Orchestrator onpremise start server via ssh systemctl start
- PRO CORE: Orchestrator onpremise provision server via ssh bootstrap configuration
- PRO CORE: Track cluster state for reprovisioning, start, stop based on configuration change and rolling actions

13.5.0.1 Features

- CORE: Multi Domain Parent Cluster
- CORE: Proxy to use interface
- CORE: CDC to Kafka
- CORE: CDC to ColumnStore
- CORE: GRPC API to save restore configs in store plugin
- CORE: GRPC API to save restore plugin sqllite
- CORE: Scripting for on demand provisioning
- CORE: MariaDB backup stage support
- CORE: MySQL GTID multi source channel full support
- CORE MONITORING: Alerting via pushover cloud

13.6.0.1 Features

- CORE: MySQL Group Replication
- HTTP: V2 API V2 GRPC swagger /help
- CORE: MariaDB Galera
- CORE: Password rotation in Vault KV or via Vault database plugins
- CORE: MariaDB backup support via SST job
- CORE: Dynamic config tracking of changed system variables
- CORE: Merge dynamic config into system configs
- CONFIGURATOR : MariaDB EITS add sampling for statististic to avoid disk over usage

13.7.0.1 Features

- CORE: React GUI migration
- CORE: Staging with multi source cluster
- CORE: Monitoring tooling versions
- CORE: MySQL 8.X support
- SECURITY: sysops dbops sponsor ROLES
- SECURITY: Run server on unpriviledge user –user=repman
- SECURITY: Better support for transparent SSL tooling MariaDB 11.x MySQL 8.x
- PEERING: SSO via gitlab.signal18.io
- PEERING: Sharing Cluster with api grants to gitlab.signal18.io users
- PEERING: Selling Cluster workflow to gitlab.signal18.io users
- PEERING: Chat with support when signed with gitlab.signal18.io users
- PEERING: Custom alerting pull when when signed with gitlab.signal18.io users
- PEERING: Web TTY to docker container, client tools or ssh sessions for dedicated ROLES
- MAINTENANCE: Per cluster restic backup snapshot

- MAINTENANCE: Binary log backups
- MAINTENANCE: Job Execution log streeming to API
- MAINTENANCE: Job maintenance dedicated logs
- MAINTENANCE: Encrypting at rest when logs sending
- MAINTENANCE: Heavy testing on backup restore and expose more parameters for backups
- MAINTENANCE: Preserving database logs when sending

13.8.0.1 Features

13.8.0.1.1 Monitoring Enhancements

- **Sleeping Transactions Monitoring**: Monitor long-running transactions including sleeping transactions that hold InnoDB locks. Tracks transaction duration, isolation level, tables locked, rows modified, and memory usage. See documentation
- **Performance Schema Mutex Monitoring**: Collect InnoDB mutex wait events from Performance Schema. Identifies mutex contention and performance bottlenecks. See documentation
- **Performance Schema Latch Monitoring**: Monitor InnoDB read-write lock (rwlock) wait events. Diagnoses concurrency issues with read/write operations. See documentation
- **Performance Schema Memory Monitoring**: Track memory consumption by InnoDB subsystems via Performance Schema memory instrumentation. See documentation
- **Per-Module Log Levels**: Configure log verbosity independently for each subsystem (backup, proxy, heartbeat, scheduler, etc.). 20+ new `log-level-*` parameters for fine-grained logging control

13.8.0.1.2 Topology

- **Active-Passive Topology**: New topology mode for monitoring single servers or multiple independent servers without replication management. Disables automatic failover while maintaining full monitoring capabilities. See documentation

13.8.0.1.3 Backup Enhancements

- **Restic Task Management**: Background task queue for backup operations with pause/resume/cancel support via API
- **Automatic Disk Space Management**: Restic automatically purges oldest backups when disk space is critically low (`backup-restic-purge-oldest-on-disk-space`)
- **Disk Threshold Configuration**: Separate thresholds for warning and critical disk usage (`backup-disk-threshold-warn`, `backup-disk-threshold-crit`)
- **Restic Timeout Control**: Configure operation timeouts for long-running backup/restore operations (`backup-restic-timeout`)
- **Local Repository Support**: Use local filesystem repositories with per-cluster isolation (`backup-restic-local-repository`)
- **MariaDB Backup Tools**: Native support for `mariadb-backup` and `mariadb-import` (formerly `maria-backup`)

13.8.0.1.4 Configuration Management

- **Configurator Diff Tracking**: Track differences between proposed and current configuration
- **Preserve Remote Configuration**: Configurator maintains remote settings during updates
- **Deprecated Configuration Detection**: Automatically identify outdated configuration parameters

13.8.0.1.5 Backup & Restore

- **Cross-Cluster User Restore**: Restore backups between clusters while merging database USERS tables

13.8.0.1.6 Provisioning

- **OpenSVC Go Agent**: Migration to new OpenSVC go-based agent
- **OpenSVC Service Logging**: Enhanced logging for OpenSVC agent operations
- **OpenSVC Failover Monitoring**: Split-brain detection and monitoring for OpenSVC clusters

13.8.0.1.7 API & Integration

- **Application Deployments**: Support for managing application deployments via peering
- **Compact JSON Responses**: Reduced API response sizes for improved performance
- **OAuth Authentication**: OAuth2 provider integration for API authentication (parameters: `api-oauth-client-id`, `api-oauth-client-secret`, `api-oauth-provider-url`, `api-oauth-redirect-url`)

13.8.0.1.8 Scheduler

- **Enhanced Scheduler System**: Comprehensive task scheduling with cron support for database maintenance, backups, and administrative operations
- **Rolling Restart/Reprovision**: Schedule automatic rolling restarts and reprovisioning operations
- **SLA Rotation**: Automated SLA metric rotation via scheduler

13.8.0.1.9 Database Compatibility

- **MySQL 8.4 Support**: Version parsing and SSL mode compatibility fixes for MySQL 8.4
- **PostgreSQL Enhancements**: Improved PostgreSQL logical and streaming replication support

13.8.0.1.10 Performance & Reliability

- **Job Script Versioning**: Automatic upgrades and version tracking for database job scripts
- **Database Jobs Schema Upgrades**: API endpoint for upgrading jobs schema
- **SQL Error Tailers**: Monitor and tail SQL error logs for analysis
- **Audit Log Tailers**: Capture and monitor audit log entries

13.8.0.1.11 Development & Build

- **SBOM Generation**: Software Bill of Materials generation for security compliance

- CORE: Windows compatibility
- CORE: Consul API integration

14.1 Installation & Setup

14.1.1 Why does connection fail when using "localhost" as the database host?

Problem: Connection failures or socket errors when using `localhost` in `db-servers-hosts`.

Cause: On Unix systems, MySQL treats "localhost" specially and attempts a Unix socket connection rather than a TCP connection. This differs from network-based connections.

Solution: Use the IP address `127.0.0.1` instead of `localhost` in your configuration:

```
db-servers-hosts = "127.0.0.1:3306,127.0.0.1:3307"
db-servers-credential = "root:password"
```

Ensure database user privileges are granted for `127.0.0.1` rather than `localhost`:

```
GRANT ALL PRIVILEGES ON *.* TO 'root'@'127.0.0.1' IDENTIFIED BY 'password';
```

Reference: `/pages/05.configuration/02.databases/docs.md:25`

14.1.2 What are the minimum database versions required?

Requirement: **replication-manager** requires GTID-enabled replication.

Supported versions: - MariaDB 10.0 or later - MySQL 5.6 or later (with GTID enabled) - Percona Server 5.6 or later (with GTID enabled)

Configuration: Ensure GTID is enabled on all database nodes:

```
gtid_mode = ON                    # MySQL
gtid_domain_id = 1               # MariaDB
enforce_gtid_consistency = ON    # MySQL
```

Reference: Installation documentation

14.1.3 Why isn't HAProxy showing statistics correctly?

Problem: HAProxy statistics not appearing in **replication-manager** dashboard or incorrect connection counts.

Cause: HAProxy versions earlier than 1.7 do not provide complete statistics information.

Solution: Upgrade to HAProxy 1.7 or later. Verify HAProxy statistics socket is accessible:

```
haproxy-servers = "127.0.0.1:3310"
haproxy-write-port = 3306
haproxy-read-port = 3307
```

Reference: `/pages/05.configuration/06.routing/01.haproxy/docs.md`

14.1.4 Why are my configuration changes not persisting after restart?

Problem: Changes made via the web UI or API are lost when **replication-manager** restarts.

Cause: Dynamic configuration saving is disabled by default.

Solution: Enable configuration persistence in the `[Default]` section of your config file:

```
[Default]
monitoring-save-config = true
```

With this enabled, changes are saved to `/var/lib/replication-manager/cluster_name/config.toml` (2.x) or `/root/.config/replication-manager/cluster_name/config.toml` (3.x).

Reference: `/pages/02.installation/02.configuration/docs.md:21`

14.1.5 How do I migrate from version 2.x to 3.x?

Major changes in 3.x:

Configuration location changes: - 2.x: `/etc/replication-manager/config.toml` - 3.x: `/root/.config/replication-m` (active config)

Docker volume requirements:

```
# Add third volume mount for 3.x
docker run -v /home/repman/etc:/etc/replication-manager:rw \
           -v /home/repman/data:/var/lib/replication-manager:rw \
           -v /home/repman/config:/root/.config/replication-manager:rw
```

Parameter renames (partial list): - `monitoring-config-rewrite` → `monitoring-save-config` - `hosts` → `db-servers-hosts` - `rpluser` → `replication-credential` - `prefmaster` → `db-servers-prefered-master`

Migration steps: 1. Backup configuration and data directories 2. Run parameter rename script on config files 3. Update Docker volume mounts if using containers 4. Install 3.x and verify configuration syntax 5. Test in non-production environment first

Reference: `/pages/02.installation/04.migration/docs.md`

14.1.6 What's the minimum configuration needed to start monitoring?

Version 3.x uses a two-part structure: `[Default]` section for global settings and separate cluster sections.

Minimal configuration for single cluster:

```
[Default]
monitoring-save-config = true
include = "/etc/replication-manager/cluster.d"

[cluster1]
title = "cluster1"
prov-orchestrator = "onpremise"
db-servers-hosts = "127.0.0.1:3306,127.0.0.1:3307"
db-servers-prefered-master = "127.0.0.1:3306"
db-servers-credential = "root:password"
replication-credential = "repl_user:repl_password"
```

Required cluster parameters: - `title`: Cluster name - `prov-orchestrator`: Provisioning orchestrator (typically "onpremise") - `db-servers-hosts`: Comma-separated database server list - `db-servers-prefered-master`: Preferred master for elections - `db-servers-credential`: Database admin credentials (user:password) - `replication-credential`: Replication user credentials (user:password)

Alternative (legacy style): For simple deployments, cluster parameters can be in `[Default]` section:

```
[Default]
title = "ClusterTest"
db-servers-hosts = "127.0.0.1:3306,127.0.0.1:3307"
db-servers-credential = "root:password"
replication-credential = "repl_user:repl_password"
```

`failover-mode = "manual"`

Reference: `/pages/02.installation/02.configuration/docs.md:27`

14.2 Replication & Synchronization

14.2.1 Does semi-sync replication guarantee no data loss after a master crash?

Short answer: No.

Detailed explanation: Semi-sync SYNC status does **not** guarantee the old master is replication-consistent with the cluster after a crash or shutdown.

Known issues: - MDEV-11855: Crash can leave extra transactions in binary log - MDEV-11853: Shutdown can leave uncommitted transactions

What semi-sync guarantees: No client applications have seen transactions that didn't reach a replica, but the master's binary log may contain additional events not yet replicated.

Impact: In heavy write scenarios, crashed masters often require re-provisioning from another node rather than rejoining the cluster.

Recommendation: Use `rpl_semi_sync_master_wait_point = AFTER_COMMIT` (default) to ensure client-visible transactions are safer, even though it may leave more transactions in the binary log after a crash.

Reference: `/pages/07.howto/01.replication-best-practice/docs.md:44`

14.2.2 Why does my switchover fail after long write inactivity?

Problem: Rejoining slaves during switchover fails when using `expire_logs_days` after extended periods without writes.

Cause: Binary logs are automatically purged based on `expire_logs_days`, which may remove logs needed for slave rejoin after the cluster has been idle.

Related bug: MDEV-10869

Solution: - Increase `expire_logs_days` value to retain logs longer - Use binary log retention based on space rather than time - Monitor binary log disk usage - Consider using `binlog_expire_logs_seconds` (MariaDB 10.6+) for finer control

Workaround: If switchover fails, you may need to re-provision affected slaves from the new master.

Reference: Current FAQ

14.2.3 What's the difference between AFTER_SYNC and AFTER_COMMIT?

Parameter: `rpl_semi_sync_master_wait_point`

AFTER_COMMIT (recommended): - Master commits transaction to storage - Master waits for slave acknowledgment - Master returns success to client - **Advantage**: Client-visible transactions are guaranteed replicated - **Disadvantage**: Slightly more transactions in binary log after crash

AFTER_SYNC: - Master syncs binary log to disk - Master waits for slave acknowledgment - Master commits to storage - Master returns success to client - **Advantage**: Fewer extra transactions in binary log after crash - **Disadvantage**: Clients may have seen transactions that are lost during failover

Recommendation: Use `AFTER_COMMIT` for safer client experience.

Reference: `/pages/07.howto/01.replication-best-practice/docs.md:50`

14.2.4 Can SUPER privileged users write during switchover on MariaDB?

Problem: Applications with SUPER privileges can write to a read-only master during switchover.

Cause: MariaDB does not have MySQL's `super_read_only` protection. The `READ_ONLY` flag does not block SUPER users from writing.

Related bug: MDEV-9458

Impact: During switchover: - Regular users are blocked by `READ_ONLY` - SUPER users can still write - Writes with SUPER privileges may pile up during `FLUSH TABLES WITH READ LOCK` - These writes de-queue after lock release, potentially causing inconsistency

Mitigation: - Delegate write protection to routing proxies (ProxySQL, MaxScale) - These proxies will not enable writes on a `READ_ONLY` slave - External scripts can manage routing changes under protection of FTWRL - **replication-manager** decreases `max_connections` during switchover to limit queued connections

Best practice: Don't grant SUPER privileges to application users.

Reference: Current FAQ

14.2.5 Why does MySQL hang during shutdown with GTID enabled?

Problem: MySQL server hangs during shutdown when using GTID with `autocommit=0` and `super_read_only=ON`.

Affected versions: - MySQL 5.7.0 to 5.7.24 - MySQL 8.0.0 to 8.0.13

Fixed in: - MySQL 5.7.25+ (Release Notes) - MySQL 8.0.14+ (Release Notes)

Cause: Transaction attempting to save GTIDs to `mysql.gtid_executed` table fails because `super_read_only=ON` prevents the update. With `autocommit=0`, the transaction never completes, blocking shutdown.

Solution: Upgrade to MySQL 5.7.25/8.0.14 or later.

Workaround (if upgrade not possible): Set `autocommit=1` or avoid `super_read_only` on slaves.

Bug reference: Bug #28183718

Reference: Current FAQ

14.2.6 What happens when semi-sync reaches timeout?

Problem: Semi-sync timeout causes workload changes and increased failover risk.

Behavior: When `rpl_semi_sync_master_timeout` (default: 10 seconds) is reached: 1. Master stops waiting for slave acknowledgment 2. Master switches to asynchronous replication 3. Workload is no longer constrained by network speed 4. **Sync status is lost** - failover becomes risky

Impact before timeout: Semi-sync slows workload to network replication speed, creating backpressure on writes.

Impact after timeout: - Crash or shutdown leads to potential data loss - Old master may require re-provisioning - Failover risk increases significantly

Monitoring: **replication-manager** tracks "In Sync" status and SLA metrics to determine when safe failover windows exist.

Reference: `/pages/07.howto/01.replication-best-practice/docs.md:46`

14.2.7 Why can't relay slaves reconnect after their master dies in multi-tier topology?

Problem: Relay slaves cannot automatically reconnect in multi-tier replication when their intermediate master fails.

Cause: **replication-manager** does not automatically manage relay node failures in multi-tier topologies.

Limitation: If you have:

`Master → Relay → Slave`

And the Relay node dies, the Slave cannot automatically reconnect to Master.

Workaround: Manually repoint slaves to the new topology after relay node failure.

Design consideration: Multi-tier topologies require additional operational procedures for relay node failures.

Reference: `/pages/05.configuration/05.replication/docs.md`

14.2.8 What does "server-id 1000" reserved mean?

Restriction: Do not use `server-id = 1000` on any database node in your cluster.

Reason: **replication-manager** reserves `server-id = 1000` for binlog server operations during crash recovery.

Impact: Using server-id 1000 in your cluster will cause: - Replication errors during backup operations - Binlog server conflicts - Crash recovery failures

Solution: Use any server-id except 1000. Common practice is sequential IDs: 3306, 3307, 3308, etc.

Reference: `/pages/05.configuration/03.failover/02.crash-recovery/docs.md`

14.3 Failover & Switchover

14.3.1 How does replication-manager prevent false positive failovers?

Multi-layer protection:

1. Multiple checks - Master failure is verified N times before acting

2. False positive detection: - Network flap detection - Heartbeat verification - Multiple monitoring endpoints - Quorum requirements with external arbitrator

3. Time-based protection: - `failover-time-limit`: Prevents repeated failovers within specified timeframe - Protects against flip-flop failures from same root cause

4. Default manual mode: - `failover-mode = "manual"` (default) - Sends alerts and waits for human intervention - User confirms failover via console, API, or CLI

5. Replication checks: - Valid slave must be available - Cluster configuration must be valid - Replication state checks (unless `check-replication-state = false`)

Best practice configuration:

```
failover-mode = "automatic"
failover-limit = 3
failover-time-limit = 10
```

```
failover-at-sync = false
failover-max-slave-delay = 30
```

Reference: `/pages/05.configuration/03.failover/01.false-positive-detetection/docs.md`

14.3.2 What happens when the entire cluster goes down?

Default behavior: **replication-manager** waits for the old master to recover.

Reason: First node to restart after total cluster failure could be a delayed slave, leading to significant data loss if promoted.

Configuration parameter: `failover-restart-unsafe`

When `failover-restart-unsafe = false` (default): - Prevents failover to first restarted slave - Waits for old master to show up - Prioritizes data safety over availability

When `failover-restart-unsafe = true`: - Allows failover to first restarted node - Favors availability when master cannot recover - Assumes DC crash brought down all nodes simultaneously (minimizing data skew)

Recommendation: - Use default (`false`) unless availability is critical - If using `true`, ensure automated slave startup procedures - In DC crash scenarios, start old master first when possible

Reference: `/pages/04.architecture/02.failover-workflow/docs.md:57`

14.3.3 Why did my failover get rejected?

Failover can be rejected for multiple reasons:

No valid slave available: - All slaves exceed `failover-max-slave-delay` (default: 30 seconds) - Highest-positioned slave doesn't match preferred master constraints - No slaves meet `db-servers-prefered-master` requirements - Candidate is in `db-servers-ignored-hosts` list

Failover limits reached: - `failover-limit` counter exceeded (default: 5) - Reset counter via console, HTTP, or API to re-enable

Time limit not met: - Previous failover within `failover-time-limit` window - Prevents flip-flop failures from same issue - Set to 0 for unlimited (not recommended)

Sync status requirement: - `failover-at-sync = true` but no slaves in SYNC status - Protects old master recovery at cost of availability

Cluster state invalid: - Replication configuration errors - Network partition detected - Arbitrator unreachable in multi-DC setup

Check status: Review logs and cluster state for specific rejection reason.

Reference: `/pages/04.architecture/02.failover-workflow/docs.md:20`

14.3.4 How long should I wait between failover attempts?

Parameter: `failover-time-limit`

Purpose: Prevents repeated failovers from the same root cause (flip-flop protection).

Recommended value: 10 seconds

```
failover-time-limit = 10
```

Behavior: If previous failover occurred within this time window, new failover is canceled.

Why this matters: - Hardware failure may affect multiple nodes sequentially - Network issues can cause cascading failures - Prevents promoting slaves that will immediately fail

Set to 0: Unlimited failovers (not recommended for production)

Reset counter: Use console/API to manually reset if legitimate failover needed within time window.

Reference: `/pages/05.configuration/03.failover/docs.md:33`

14.3.5 What does "failover-at-sync" mean for data safety?

Parameter: `failover-at-sync`

Purpose: Controls whether failover requires semi-sync SYNC status.

When `failover-at-sync = false` (default): - Failover proceeds even if semi-sync is not synchronized - Prioritizes availability over zero data loss - Some transactions may be lost

When `failover-at-sync = true`: - Failover only when at least one slave is in SYNC status - Prioritizes data consistency over availability - May prevent failover when most needed - Protects old master recovery

Recommendation for automatic failover: - **Balanced**: `failover-at-sync = false` with `failover-max-slave-delay = 30` - **Minimize data loss**: `failover-at-sync = true` with `failover-max-slave-delay = 0`

Trade-off: Strict sync requirements reduce availability during network issues or high load.

Reference: `/pages/04.architecture/02.failover-workflow/docs.md:39`

14.3.6 Why is my switchover stuck or timing out?

Problem: Switchover hangs or takes longer than expected.

Common causes:

1. FLUSH TABLES WITH READ LOCK timeout: - Parameter: `switchover-wait-kill` (default timeout) - Long-running queries prevent FTWRL from acquiring lock - Switchover waits for queries to complete

Solution: - Kill long-running queries manually - Adjust `switchover-wait-kill` timeout - Pre-check for long queries before switchover:

```
SELECT * FROM information_schema.processlist
WHERE command != 'Sleep' AND time > 10;
```

2. Slave lag: - Switchover waits for slaves to catch up - Check `SHOW SLAVE STATUS` lag on all slaves - Large transactions can delay catchup

3. Semi-sync timeout: - If semi-sync is slow or timing out - Network issues between master and slaves

4. Proxy reconfiguration delays: - HAProxy, ProxySQL, or MaxScale taking time to update - Check proxy logs and status

Monitoring: Enable higher log verbosity to debug:

```
log-level = 4
```

Reference: `/pages/05.configuration/04.switchover/docs.md`

14.3.7 When will replication-manager NOT perform automatic failover?

Failover is prevented when:

Cluster state issues: - [ ] No valid slave available - [ ] All slaves exceed `failover-max-slave-delay` - [ ] Cluster configuration is invalid - [ ] Network partition without arbitrator quorum

Configuration limits: - [ ] `failover-mode = "manual"` (default) - [ ] `failover-limit` reached (default: 5 failovers) - [ ] Previous failover within `failover-time-limit` window - [ ] `failover-at-sync = true` but no slave in SYNC status

Replication constraints: - [ ] All slaves with highest GTID position are in `db-servers-ignored-hosts` - [ ] No slaves match `db-servers-prefered-master` at highest position - [ ] `check-replication-state = true` and replication is broken

All cluster down: - [ ] `failover-restart-unsafe = false` (default) and old master not recovered yet

Manual override available: User can force failover via console/API by temporarily disabling checks.

Reference: `/pages/04.architecture/02.failover-workflow/docs.md`

14.4 Topology & Deployment

14.4.1 Why are two-node clusters not recommended?

Limitation: Two-node clusters lack fault tolerance for automatic failover decisions.

Problems with two nodes:

Split-brain risk: - Network partition isolates both nodes - Each node thinks the other is down - Both could become writable masters

No quorum: - Cannot determine which node should be master - Requires external arbitrator for reliable decisions

Single point of failure: - If slave fails, no failover targets - If master fails, only one promotion candidate - No redundancy for planned maintenance

Recommendation: Use minimum three nodes: - Provides N+2 redundancy - Allows one node failure with spare - Better quorum decisions - Supports maintenance windows

Alternative: Two-node with external arbitrator to break ties.

Reference: `/pages/05.configuration/05.replication/docs.md`

14.4.2 What are the limitations of Galera cluster support?

Supported but with constraints:

No serialized isolation: - Galera cluster doesn't support serialized transaction isolation - Increases deadlock probability vs traditional replication

Deadlock risk: - Multi-master writes across nodes can conflict - Applications must handle deadlock retries - More common than single-master topologies

Certification-based replication: - Different conflict detection model - Transactions can fail at commit time - Requires application-level retry logic

Recommendations: - Use optimistic locking strategies - Implement transaction retry logic - Consider single-writer patterns when possible - Monitor for certification failures

Reference: `/pages/04.architecture/03.topologies/06.multi-master-galera/docs.md`

14.4.3 How does multi-master prevent split-brain?

Required configuration for master-master (multi-master) topology:

Critical setting:

`read_only = 1`

Must be set in **MariaDB configuration file** (my.cnf), not just dynamically.

How it works: 1. Both nodes start in `read_only` mode 2. **replication-manager** promotes one node to writable master 3. Sets `read_only = 0` on active master only 4. Keeps `read_only = 1` on standby master

Without `read_only = 1` in config: - Both nodes could accept writes after restart - Split-brain condition - Data conflicts and replication breakage

Additional protection: - Routing proxies (HAProxy/ProxySQL) directed to single writer - Heartbeat table monitoring - External arbitrator for network partition scenarios

Reference: `/pages/05.configuration/05.replication/docs.md`

14.4.4 What happens if a relay slave crashes in multi-tier topology?

Problem: Relay node failures are not automatically managed.

Multi-tier example:

```
DC1: Master → Relay1
DC2: Relay1 → Slave1, Slave2
```

If Relay1 crashes: - Slave1 and Slave2 lose replication source - **replication-manager** does not auto-repoint to Master - Manual intervention required

Limitation: Designed for master failure, not intermediate relay failures.

Workaround options:

Option 1: Manually repoint slaves to master

```
CHANGE MASTER TO MASTER_HOST='master', ...
```

Option 2: Use scripts with `failover-post-script` to handle relay failures

Option 3: Avoid multi-tier topologies in critical paths

When multi-tier makes sense: - Cross-DC bandwidth optimization - Network topology constraints - Acceptable manual intervention for relay failures

Reference: `/pages/05.configuration/05.replication/docs.md`

14.5 Security & Configuration

14.5.1 How do I change the default API credentials?

Default credentials: `admin:repman` (INSECURE)

Security risk: Default credentials must be changed before production use.

Configuration parameter:

```
[Default]
api-credentials = "myuser:mypassword"
```

Effect: - All API and CLI access requires new credentials - Web UI login uses new credentials - CLI clients will prompt for password unless specified

CLI usage with custom credentials:

```
replication-manager-cli --user=myuser --password=mypassword status
```

Best practice: Use encrypted passwords (see password encryption question below).

Reference: `/pages/05.configuration/07.security/docs.md:31`

14.5.2 How do I encrypt passwords in configuration files?

Three-step process:

Step 1: Generate encryption key (as root)

```
replication-manager keygen
```

This creates an encryption key accessible only to root.

Step 2: Encrypt your password

```
replication-manager password secretpass
```

Output:

```
Encrypted password hash: 50711adb2ef2a959577edbda5cbe3d2ace844e750b20629a9bcb
```

Step 3: Use encrypted password in config

```
db-servers-credential = "root:50711adb2ef2a959577edbda5cbe3d2ace844e750b20629a9bcb"
replication-credential = "repl:50711adb2ef2a959577edbda5cbe3d2ace844e750b20629a9bcb"
```

Automatic decryption: When **replication-manager** starts and detects the encryption key, passwords are automatically decrypted.

Security: Encryption key is only readable by root, providing basic password obfuscation.

Reference: `/pages/05.configuration/07.security/docs.md:6`

14.5.3 How do I integrate with HashiCorp Vault?

Two modes available:

Mode 1: config_store_v2 (store credentials in Vault secret)

```
[mycluster]
vault-server-addr = "http://vault.example.com:8200"
vault-auth = "approle"
vault-role-id = "your-role-id"
vault-secret-id = "your-secret-id"
vault-mode = "config_store_v2"
vault-mount = "kv"
db-servers-credential = "applications/repman"
replication-credential = "applications/repman"
```

Create Vault secret:

```
vault kv put kv/applications/repman \
  db-servers-credential=root:password \
  replication-credential=repl:password
```

Mode 2: database_engine (automatic password rotation)

```
[mycluster]
vault-mode = "database_engine"
db-servers-credential = "database/static-creds/repman-monitor"
replication-credential = "database/static-creds/repman-replication"
```

Configure Vault database role:

```
vault write database/config/my-mysql-database \
    plugin_name=mysql-database-plugin \
    connection_url="{{username}}:{{password}}@tcp(127.0.0.1:3306)/" \
    allowed_roles="repman-monitor,repman-replication" \
    username="vaultuser" \
    password="vaultpass"

vault write database/static-roles/repman-monitor \
    db_name=my-mysql-database \
    username="repman" \
    rotation_period=600
```

Automatic rotation: Vault rotates passwords at specified interval; **replication-manager** fetches new passwords on authentication errors.

Reference: `/pages/05.configuration/07.security/docs.md:123`

14.5.4 Do I need to replace the self-signed certificates?

Yes, for production deployments.

Default behavior: **replication-manager** ships with self-signed certificates for HTTPS and API access.

Security risk: Self-signed certificates: - Can be MitM attacked - Generate browser warnings - Don't validate server identity

Configuration parameters:

```
[Default]
monitoring-ssl-cert = "/path/to/your/server.crt"
monitoring-ssl-key = "/path/to/your/server.key"
```

Generate proper certificates:

```
# RSA certificate
openssl genrsa -out server.key 2048
openssl req -new -x509 -sha256 -key server.key -out server.crt -days 3650

# ECDSA certificate (recommended)
openssl ecparam -genkey -name secp384r1 -out server.key
openssl req -new -x509 -sha256 -key server.key -out server.crt -days 3650
```

Best practice: Use certificates from your organization's PKI or a trusted CA like Let's Encrypt.

Reference: `/pages/05.configuration/07.security/docs.md:40`

14.6 Proxies & Routing

14.6.1 Why aren't ProxySQL users syncing automatically?

Problem: Database users not appearing in ProxySQL configuration.

Cause: User bootstrap feature not enabled or configured incorrectly.

Configuration parameter:

```
proxysql-bootstrap-users = true
```

Behavior when enabled: - **replication-manager** discovers users from database - Automatically pushes users to ProxySQL - Syncs on topology changes

Manual mode (`proxysql-bootstrap-users = false`): - You manage ProxySQL users manually - **replication-manager** doesn't modify user configuration - More control, more operational overhead

Troubleshooting: - Verify ProxySQL admin credentials configured correctly - Check `proxysql-servers` parameter points to admin interface - Review ProxySQL logs for connection errors

Reference: `/pages/05.configuration/06.routing/01.proxysql/docs.md`

14.6.2 What's the difference between ProxySQL bootstrap-users and manual configuration?

Bootstrap mode (`proxysql-bootstrap-users = true`): - **replication-manager** discovers database users automatically - Pushes users to ProxySQL configuration - Updates users on topology changes - Simplifies user management - Good for dynamic environments

Manual mode (`proxysql-bootstrap-users = false`): - You configure ProxySQL users yourself - **replication-manager** only manages host groups and routing - Full control over user attributes - Prevents accidental user changes - Better for static, audited environments

Recommendation: - Use bootstrap for development/testing - Use manual for production with strict change control

Reference: `/pages/05.configuration/06.routing/01.proxysql/docs.md`

14.6.3 Why does MaxScale have a monitoring delay?

Problem: MaxScale's internal monitor has built-in delays before detecting topology changes.

Cause: MaxScale uses polling intervals to detect master failures and slave status changes, typically 1-2 seconds.

Impact during failover: - MaxScale may route traffic to failed master briefly - Delay between **replication-manager** promotion and MaxScale awareness - Client connection errors during transition

Solution: MaxScale integration with shortcutting:

replication-manager can update MaxScale directly via its API, bypassing monitor delays.

Configuration:

```
maxscale-servers = "127.0.0.1:3306"
maxscale-write-port = 3306
maxscale-read-port = 3307
```

Benefit: Near-instant routing updates during switchover/failover.

Reference: `/pages/05.configuration/06.routing/02.maxscale/docs.md`

14.6.4 Which proxy should I use?

Decision matrix:

HAProxy: - Layer 4 TCP routing - Very fast, low overhead - No query inspection - No connection pooling - Best for: Simple routing, high performance

ProxySQL: - Layer 7 MySQL protocol - Query caching and rewriting - Connection pooling - Query routing based on patterns - Best for: Complex routing, read/write split, query optimization

MaxScale: - Layer 7 MySQL protocol - Advanced routing rules - Query firewall - Connection pooling - Best for: Enterprise features, fine-grained control

Consul: - Service discovery - DNS-based routing - Application-level failover - Best for: Microservices, dynamic environments

Recommendation: Start with HAProxy for simplicity, move to ProxySQL for advanced features.

Reference: Configuration documentation for each proxy type

14.7 Recovery & Data Loss

14.7.1 How do I recover a failed master?

Recovery methods depend on scenario:

Scenario 1: Master recoverable with GTID consistency - Old master GTID is subset of new master - **replication-manager** auto-rejoins as slave - No data loss, automatic process

Scenario 2: Master has extra transactions (diverged)

Method A: Flashback (MariaDB with flashback enabled)

```
autorejoin-flashback = true
```

- Rolls back diverged transactions
- Re-syncs with new master
- Diverged data is saved

Method B: mysqldump

```
autorejoin-mysqldump = true
```

- Dumps database from new master
- Restores to old master
- Slower but reliable

Method C: Physical backup

```
autorejoin-backup-binlog = true
autorejoin-semisync = false
```

- Uses physical backups (ZFS, LVM snapshots)
- Fastest for large databases

Method D: Manual re-provision - Provision old master from new master - Most reliable for complex scenarios

Crash information saved: Check `/var/lib/replication-manager/crash*Unixtime*/` for binary logs and election details.

Reference: `/pages/05.configuration/03.failover/02.crash-recovery/docs.md`

14.7.2 When should I use flashback vs mysqldump recovery?

Flashback recovery:

Requirements: - MariaDB (flashback not available in MySQL) - Binary logs in ROW format - `binlog_format = ROW` - Flashback enabled: `autorejoin-flashback = true`

Advantages: - Very fast (seconds to minutes) - No full data copy needed - Reverses diverged transactions

Disadvantages: - Only works for small divergence - Requires MariaDB specific features - May fail with DDL changes

Use when: Small transaction divergence, MariaDB environment, fast recovery priority

---

mysqldump recovery:

Requirements: - Any MySQL/MariaDB/Percona version - `autorejoin-mysqldump = true`

Advantages: - Works on any database version - Reliable and predictable - No special prerequisites

Disadvantages: - Slow for large databases (hours for TB+ databases) - Requires full data dump/restore - Locks tables during dump

Use when: Large divergence, MySQL (not MariaDB), guarantee consistency

---

Physical backup recovery:

Requirements: - ZFS, LVM, or storage snapshots - `autorejoin-backup-binlog = true`

Advantages: - Fastest for large databases - Minimal overhead - Block-level copy

Disadvantages: - Requires storage infrastructure - More complex setup

Use when: Very large databases, storage supports snapshots, fastest recovery critical

Reference: `/pages/05.configuration/03.failover/02.crash-recovery/docs.md`

14.7.3 Where can I find crash information after failover?

replication-manager records crash details in multiple locations:

Location 1: Crash directory

`/var/lib/replication-manager/crash*Unixtime*/`

Contains: - Binary logs from elected master at time of election - Replication state when node was still master - Useful for manual recovery and auditing

Location 2: Cluster state file

`/var/lib/replication-manager/cluster_name.json`

Contains:

```
{
  "crashes": [
    {
      "URL": "127.0.0.1:3310",
      "FailoverMasterLogFile": "bin.000001",
```

```
      "FailoverMasterLogPos": "459",
      "FailoverSemiSyncSlaveStatus": true,
      "FailoverIOGtid": [{"DomainID": 0, "ServerID": 3310, "SeqNo": 1}],
      "ElectedMasterURL": "127.0.0.1:3311"
    }
  ]
}
```

Location 3: API endpoint

```
GET /api/clusters/cluster_name/crashes
```

When crashes are cleared: When cluster topology returns to no ERROR state.

Reference: `/pages/07.howto/03.toubleshoot-crashes/docs.md:26`

14.8 Monitoring & Troubleshooting

14.8.1 How do I increase logging for debugging?

Increase log verbosity dynamically via API:

```
replication-manager-cli api \
  url=https://127.0.0.1:3000/api/clusters/cluster_name/settings/switch/verbosity
```

Or configure in config file:

```
[Default]
log-level = 4
```

Log levels: - 1: Errors only - 2: Warnings - 3: Info (default) - 4: Debug - 5+: Very verbose (trace level)

Per-module logging (3.1+):

```
log-level = 2                    # Global default
log-level-backup-stream = 4      # Debug backups
log-level-proxy = 1              # Minimal proxy logs
log-level-heartbeat = 4          # Debug heartbeat
```

Collecting debug information for support:

```
# Increase verbosity
replication-manager-cli api url=.../verbosity

# Reproduce issue

# Collect internal state
replication-manager-cli show > state.json

# Attach to support ticket:
# - /var/log/replication-manager.log
# - state.json
```

Reference: `/pages/07.howto/03.toubleshoot-crashes/docs.md:7`

14.8.2 Why is performance schema monitoring causing overhead?

Change in version 3.x: Performance Schema monitoring enabled by default.

New defaults in 3.x:

```
monitoring-performance-schema-mutex = true
monitoring-performance-schema-latch = true
monitoring-performance-schema-memory = true
```

Impact: Increased database load from performance schema queries, especially on: - High-transaction workloads - Many mutex/latch contentions - Memory-intensive operations

Solution if overhead is problematic:

Disable specific monitors:

```
monitoring-performance-schema-mutex = false
monitoring-performance-schema-latch = false
monitoring-performance-schema-memory = false
```

Or disable performance schema entirely on database:

```
[mysqld]
performance_schema = OFF
```

Trade-off: Disabling reduces monitoring visibility into database internals.

Recommendation: Monitor database CPU/load after upgrade; disable only if overhead is measurable.

Reference: `/pages/02.installation/04.migration/docs.md:156`

14.8.3 How do I check internal status for support tickets?

Collect internal status:

```
replication-manager-cli show
```

Output includes internal state of: - Settings - Clusters - Servers - Master - Slaves - Crashes - Alerts

Filter specific class:

```
replication-manager-cli show --get=servers
replication-manager-cli show --get=crashes
```

For support tickets, attach:

1. **Output of `show` command** (JSON format)
2. **Log file** with increased verbosity:

   ```
   /var/log/replication-manager.log
   ```

3. **Reproduce with debug logging**:

   ```
   # Enable verbose logging
   replication-manager-cli api url=.../verbosity
   # Reproduce issue
   # Collect logs
   ```

Submit to: https://github.com/signal18/replication-manager/issues

Reference: `/pages/07.howto/03.toubleshoot-crashes/docs.md:14`

14.9 Operational Best Practices

14.9.1 What are the recommended parallel replication settings?

MariaDB 10.1+ optimistic parallel replication:

```
slave_parallel_mode = optimistic
slave_domain_parallel_threads = 4    # Set to number of CPU cores
slave_parallel_threads = 4           # Set to number of CPU cores
expire_logs_days = 5
sync_binlog = 1
log_slave_updates = ON
```

Benefits: - In-order optimistic parallel replication - Replication can catch up with master in most scenarios - Combined with semi-sync for safety

Why this matters: - Keeps slaves synchronized - Reduces replication lag - Enables safer failover windows

Verification:

```
SHOW VARIABLES LIKE 'slave_parallel%';
```

Reference: `/pages/07.howto/01.replication-best-practice/docs.md:14`

14.9.2 What are the recommended semi-sync settings?

MariaDB semi-sync configuration:

```
plugin_load = "semisync_master.so;semisync_slave.so"
rpl_semi_sync_master = ON
rpl_semi_sync_slave = ON
loose_rpl_semi_sync_master_enabled = ON
loose_rpl_semi_sync_slave_enabled = ON
rpl_semi_sync_master_timeout = 10
rpl_semi_sync_master_wait_point = AFTER_COMMIT
```

Important notes:

Expected warning on slaves: - You will see "SemiSyncMaster Status switched OFF" warnings - This is normal - slaves don't act as semi-sync masters

Timeout value (10 seconds): - Workload slows to network speed until timeout - After timeout, switches to async (SYNC status lost) - Balance between safety and performance

Wait point: - Use `AFTER_COMMIT` (default) for client safety - Avoid `AFTER_SYNC` despite fewer binlog transactions after crash

Benefits: - Delays commit until one replica acknowledges - Provides "In Sync" status for SLA calculations - **replication-manager** uses for safe failover windows

Reference: `/pages/07.howto/01.replication-best-practice/docs.md:30`

14.9.3 Should I enable dynamic best practice enforcement?

Parameter: `monitoring-enforce-best-practices`

When enabled: **replication-manager** dynamically adjusts database settings to match best practices.

Warning: Dynamic changes are **lost on replication-manager restart** unless saved to config.

Recommendation:

DON'T rely on dynamic enforcement - instead: 1. Configure settings directly in database config files (my.cnf) 2. Use **replication-manager** enforcement as validation only 3. Make permanent changes to database configuration

Permanent settings example (my.cnf):

```
[mysqld]
sync_binlog = 1
innodb_flush_log_at_trx_commit = 1
slave_parallel_mode = optimistic
slave_parallel_threads = 4
rpl_semi_sync_master_enabled = ON
rpl_semi_sync_slave_enabled = ON
```

Use dynamic enforcement for: Testing and validation, not production operations.

Reference: `/pages/07.howto/02.enforce-best-practice/docs.md`

14.9.4 What backup strategy should I use?

Backup types available:

Logical backups: - **mysqldump**: Universal, slow for large databases - **mydumper**: Parallel logical backup, faster than mysqldump - Use for: Small/medium databases, cross-version restores

Physical backups: - **mariabackup** (formerly maria-backup): Hot backup for MariaDB - **xtrabackup**: Hot backup for MySQL/Percona - **restic**: Filesystem-level incremental backups - Use for: Large databases, faster restore

Snapshot backups: - **ZFS snapshots**: Instant point-in-time copies - **LVM snapshots**: Block-level snapshots - Use for: Very large databases, instant recovery

Configuration parameters:

```
# Storage location
backup-logical-type = "mysqldump"  # or "mydumper"
backup-physical-type = "mariabackup"
backup-disk-threshold-warn = 85
backup-disk-threshold-crit = 95

# Restic backups with auto-purge
backup-restic = true
backup-restic-purge-oldest-on-disk-space = true
backup-restic-purge-oldest-on-disk-threshold = 90
```

Recommendation: - Small databases (<100GB): mysqldump/mydumper - Large databases (100GB-1TB): mariabackup/xtrabackup - Very large (>1TB): ZFS/LVM snapshots + binlog streaming

Reference: `/pages/05.configuration/14.maintenance/02.backups/docs.md`